

LEARNING TO MODEL WHAT MATTERS
—
REPRESENTATIONS AND WORLD MODELS FOR EFFICIENT REINFORCEMENT
LEARNING

by

Claas Alexander Voelcker

A thesis submitted in conformity with the requirements
for the degree of Doctor of Philosophy

Department of Computer Science
University of Toronto

Learning to model what matters

—

Representations and world models for efficient reinforcement learning

Claas Alexander Voelcker

Doctor of Philosophy

Department of Computer Science

University of Toronto

2025

Abstract

Reinforcement learning (RL) is one of the most promising pathway towards building decision-making systems that can learn from their own successes and mistakes. However, despite their potential, RL agents often struggle to learn complex tasks, proving too inefficient, both in terms of samples and computational resources, and unstable in practice. To enable RL-based agents to live up to their potential, we need to address these limitation.

To this end, we take a close look as the mechanisms that lead to unstable and inefficient value function learning with neural networks. Learned value functions overestimate true returns during training, and this overestimation is linked to unstable learning in the feature representation layers of neural networks. To counteract this, we show the need for proper normalization of learned value approximations. Building on this insight, we then investigate model-based auxiliary tasks to stabilize feature learning further. We find that model-based self-prediction, in combination with value learning, leads to stable features.

Moving beyond feature learning, we investigate decision-aware model learning. We find that, similar to the issues encountered in representation learning, tying model updates to the value function can lead to unstable and even diverging model learning. This problem can be mitigated in observation-space models by using the value function gradient to measure its sensitivity with regard to model errors. We then move on to combine our insights into representation learning and model learning. We discuss the family of value-aware model learning algorithms and show how to extend its losses to account for learning with stochastic models. Finally, we show that combining all previous insights in a unified architecture can lead to stable and efficient value function learning.

Dedicated to my family and my husband-to-be

Acknowledgements

While a thesis is ostensibly the work of a single author, it is no secret that it is (almost) never written without the tireless support of many, many people. Even though the work itself is mine, it would not have been possible without advisors, mentors, co-authors, friends, and family.

Of course, the first and biggest thank you is addressed to my supervisors who have guided me along my path for the last five years. Without all of you, I would not be here! Amir-massoud Farahmand, thank you for making me a critical scientist. You encouraged me tirelessly to become precise and clear in my thinking, experimentation, and writing, and my work is all the better for it. Igor Gilitschenski, thank you for stepping up when I needed help. You were the person I could always turn to with all my (very German) frustrations and worries, and your calm pragmatism saved me more than once. In the same breath, I want to acknowledge the wonderful chair of my supervisory committee and mentor. Wil Cunningham, thank you for your raw enthusiasm, curiosity, and your playful approach to scientific exploration. One day, I truly believe you will know how the brain works!

Getting to a PhD program is in itself a long road, and so I want to thank the professors who taught me all the important foundations during my undergrad and master's. A special thanks of course goes to my Darmstadt thesis supervisor Kristian Kersting, and the two professors who mentored me and who trusted my abilities enough to write the reference letters that got me to Toronto, Jan Peters, and Johannes Fuernkranz.

The next round of thanks goes out to my wonderful collaborators in Toronto and beyond. To my coauthors Romina Abachi, Tyler Kastner, Victor Liao, Arash Ahmadian, Anastasiia Pedan, Jonas Guan, Eric Eaton, and last but definitely not least, Marcel Hussing, thanks for all of your amazing work! Maria and Taylor, I promise you you will be on this list soon enough, so I am including you in spirit already. To the undergraduate students who worked with me, Victor Liao, Arash Ahmadian, Yash Ramani, Allan Lin, Jasper Gerig, Anastasiia Pedan, Tehilla Helfenbaum, Ian Vyse, and Shabnam Tajik, thanks for putting up with my meandering ideas and good luck on all of your paths. Finally, even though we have also not yet officially co-authored a paper, a big thanks Eugene Vinitsky for becoming a great collaborator, teacher, and friend over the final stretch of my PhD.

A big thanks goes to Jonathan Thompson for hosting me at DeepMind, and to Heni Ben-Amor for wonderful coffee chats. And of course, a special *thank you* to Maria Attarian for being the best office mate!

The next group of people is the biggest part of the acknowledgements, my friends and lab mates. Roughly in order of appearance, I thank the Adage lab and visitors, Amin, Romina, Avery, Farnam, Yangchen, Mete, Tyler and Sierra, Amarildo, and Jongmin; the (late) PAIR

lab, Maria and Jon, Kate, Lily, Ziyi, Mayank, and Dylan; the TISL lab and visitors, Ashkan and Sarah, Sam, Yash, Umangi, Evgenii, Jasper, Mohammad, Kai, Justin, Alfred, Shuhong, Andrei, Toshi, Axel, Eugen, and Marcel; the CleverHans lab or extended Nicola mafia or potluck gang and visitors, Stefan, Mohammad, Avital, Sierra, Anvith, Patty, and David; friends from around Vector, especially Lukas, Taylor, Sep, and Shalev; representative for all the Vector staff, Abdur Mohammed and Jackie Zuborg, who were always quick to help with any problems; and finally, the great friends from all around DCS and the University of Toronto, especially Silvia and Gabriel.

Some friends have remained close after I moved to Toronto, and so a special thanks goes to Marcel, and the D120 and WiMi gang, Stefan and Maria, Julian and Gesine, Nadja, Benjamin and Steffi, Jannis, Anna and Philipp, Fabian and Steffi, Johny and Franzi, and Sial, Robyn, and Kay.

Collectively, I want to thank Queer in AI for their incredible work, and I send out all my love to those who find it harder to fight for the right to live their life in truth every day.

A special thanks goes to Corinne, for helping to keep me sane in Toronto, especially during the dark days of lock-downs, remote learning and teaching, and a long-distance relationship.

Of course, there are not enough words to express my thanks to my family, especially my mom and dad, Nikola and Hinrich, and my brother and sister-in-law, Till and Jasmine. Without your love, encouragement, and support I would have never managed to get this far.

Finally, for Heiko, there simply are no words to adequately express everything I owe you. Instead of words, you get me for the rest of your life.

Contents

Preface	1
1 Introduction	4
1.1 Stable and efficient value function learning	5
1.1.1 Learning to represent environments	5
1.1.2 Balancing decision-awareness and general-purpose learning	6
1.2 Thesis structure	8
2 Background	10
2.1 General mathematical notation and definitions	10
2.2 The Markov Decision Process	11
2.2.1 Occupancy distributions	13
2.2.2 State and action observations	14
2.3 Reinforcement Learning	15
2.3.1 Reinforcement Learning in Tabular Domains	15
2.3.2 Bellman optimality and Bellman operators	15
2.3.3 Parametric value function learning	17
2.3.4 Deep Reinforcement Learning in Continuous Action Spaces	22
2.4 Model-Based Reinforcement Learning	23
2.4.1 Training the model	24
2.4.2 Using the model	25
2.4.3 Abstract latent value models	28
2.5 Objective mismatch phenomenon	28
2.5.1 Value-aware model learning	29
2.6 Representation learning in RL	31
2.7 Decision-aware vs general-purpose learning	33
3 Instability of Value Function Learning	35
3.1 Preliminaries	37
3.1.1 Investigating large update-to-data ratios during priming	37

3.1.2	On the potential causes of divergence	38
3.1.3	Applying common regularization techniques	39
3.1.4	Divergence in practice	39
3.2	Towards high-UTD optimization without resetting	40
3.2.1	Limiting gradient explosion via unit-ball normalization	40
3.2.2	Evaluating feature output normalization during priming	41
3.3	Experimental evaluation	42
3.3.1	Feature normalization stabilizes high-UTD training	42
3.3.2	Other failure modes: Exploration limitations	43
3.3.3	Limit-testing feature normalization	44
3.4	Related work	45
3.5	Conclusion	46
4	Understanding Auxiliary Tasks for Value Function Learning	47
4.1	Introduction	47
4.2	Mathematical formalism for analysis	49
4.2.1	Linear networks as analytical models for training dynamics	49
4.3	Formalizing distractions and observation functions	51
4.3.1	Observation functions	52
4.3.2	Distracting state dynamics	52
4.4	Reconstruction and self-prediction losses	53
4.4.1	Case 1: Orthogonal state representations	53
4.4.2	Case 2: Observation function dependence	58
4.5	Understanding the effects on value function learning	60
4.5.1	Value functions in MDPs with distractions	61
4.5.2	Value function learning and auxiliary tasks	62
4.6	Empirical study	65
4.7	Conclusions	68
5	Value-gradient Aware Model Learning	70
5.1	Introduction	70
5.2	Value-Gradient Weighted Model Learning	72
5.2.1	Approximating IterVAML with the value function gradient	73
5.2.2	Bound between Value-Aware Model Learning and VaGraM	74
5.2.3	Preventing spurious local minima	75
5.2.4	Bounding the first-order Taylor loss	76
5.3	Experiment: Analyzing model learning	77
5.4	Experiment: Model-based Continuous Control	78
5.4.1	Hopper with reduced model capacity	79
5.4.2	Hopper with distracting dimensions	80

5.5	Related work	81
5.6	Conclusion	82
6	Calibrated Latent Models for Value Aware Model Learning	83
6.1	Introduction	83
6.2	Notation	84
6.3	The Value-Aware Model Learning framework	85
6.3.1	Iterative Value Aware Model Learning	86
6.3.2	The (m,b)-VAML Family	86
6.4	Decision-aware losses in stochastic environments	87
6.4.1	Calibration of surrogate loss functions	87
6.4.2	Model learning bias with stochastic models	88
6.4.3	Value learning bias in stochastic environments	92
6.4.4	Discussion of theoretical results	95
6.5	Calibration impact on finite state MDPs	95
6.6	Latent environment models and auxiliary losses	96
6.6.1	Deterministic models for stochastic environments	97
6.6.2	Auxiliary losses	100
6.7	Experimental evaluation	101
6.7.1	Experimental setup	101
6.7.2	Architecture design and (m,b)-VAML stability	103
6.7.3	Calibration experiments with latent-space models	105
6.8	Related work	107
6.9	Conclusions	108
7	Stabilizing Value Function Learning with Model-Generated Data	110
7.1	Introduction	110
7.2	Off-policy value function learning	112
7.3	Investigating the root cause of unstable Q learning	112
7.3.1	Action distribution shift can cause off-policy Q value divergence	113
7.3.2	Empirical Q value estimation with off-policy data	116
7.3.3	Prior attempts to combat misgeneralization and overestimation	117
7.4	Mitigation via model-generated synthetic data	119
7.4.1	Design choices and training setup	120
7.5	Experimental evaluation	121
7.5.1	Impact of using model-generated data	122
7.5.2	Performance and stability impact of resetting	124
7.5.3	Different quantities of model data	125
7.5.4	Further experiments and ablations	125
7.6	Related work	128

7.7	Conclusion	131
8	Conclusion	132
8.1	Future work	133
8.1.1	Decision-aware and general purpose learning	133
8.1.2	Explicit and implicit uses of model learning	134
8.1.3	Environment understanding	134
8.1.4	Multi-task learning	135
8.2	Final remarks	135
	Bibliography	136
A	Auxiliary Lemmata	156
A.1	Chapter 4: Helpful definitions and lemmata	156
A.1.1	Linear Algebra	156
A.1.2	Stochastic matrices	157
A.1.3	Ordinary differential equations	158
A.1.4	MDP representation and TD learning	159
A.2	Chapter 6: Helpful definitions and lemmata	161
A.2.1	Section 6.4 Lemmata	161
A.2.2	Propositions from Bertsekas & Shreve	164
B	Appendix – Instability of Value Function Learning	165
B.1	Implementation details and hyperparameters	165
B.2	Additional experimental results	167
B.2.1	Returns on all environments	167
B.2.2	Q-values on all environments	169
B.3	Unit norm gradient derivation	170
B.4	Open Problems and Limitations	171
C	Appendix – Understanding Auxiliary Tasks	173
C.1	Implementation details for the experiments	173
C.2	DMC results	175
C.3	Limitations	176
D	Appendix – Value-gradient Aware Model Learning	181
D.1	Full VAML for SAC	181
D.2	Additional experiments	182
D.2.1	Ablations	182
D.2.2	Performance in Mujoco benchmark suite	183
D.3	Experiment details	184

D.3.1	Architecture and hyperparameters	185
D.4	Deterministic vs probabilistic models	187
D.4.1	Ablation experiment with deterministic models	187
E	Appendix – Calibrated Models for VAML	189
E.1	Implementation Details – Section 6.5: Garnet	189
E.2	Implementation Details – Subsection 6.7.3: DM Control	189
E.2.1	Architecture	189
E.2.2	Environment	190
E.2.3	Additional results	191
F	Appendix – Value Function Learning with Model-Generated Data	192
F.1	Implementation	192
F.2	Further results	194
F.2.1	Q value overestimation	194
F.2.2	Humanoid results	195
F.2.3	TD-MPC2 ablation	195
F.2.4	MAD-TD, BRO, TD-MPC2 per env on the hard suite	196
F.2.5	Results across further DMC environments	198
F.2.6	Metaworld	201

List of Tables

6.1	Comparison of the major design choices in the (m, b) -VAML framework. Iter-VAML and MuZero can be used to update the model, while MuZero and Model-free TD learning can be used to update the value function. All model-based losses are uncalibrated when applied to stochastic model classes. In addition, MuZero suffers a bias when used for value function prediction which cannot be surmounted with an easy modification to the loss function. . . .	88
B.1	Shared hyperparameters between priming and high-UTD implementations .	166
B.2	Differing hyperparameters between priming and high-UTD implementations	166
C.1	Hyper-parameters the RL experiments	174
C.2	Network architecture for the MinAtar experiments.	174
C.3	Network architecture for the DMC experiments.	175
E.1	Hyperparameter of the deep learning experiments. Aside from the (m, b) -(C)VAML relevant hyperparameters, we keep all others consistent across environments and algorithms.	190
F.1	Network architecture for MAD-TD.	193
F.2	Hyperparameters. We adapted three parameters to the action repeat = 1 setting, as the magnitude of the reward changes.	193

List of Figures

1.1	An overview of the connections between chapters of the thesis. Chapters 3 and 4 discuss neural network-based representation learning, while Chapter 5 discusses model-based reinforcement learning. Chapters 6 and 7 combine ideas from both strands to establish latent value-aware models with auxiliary task learning. Chapter 3 introduces the problem of learning with high update ratios, which is also used in Chapter 5, and put into central focus again in Chapter 7. Solid lines show influence of algorithmic ideas, and dashed lines mark the problem setting of high update ratio learning.	8
2.1	A sketch of the Dyna framework. The RL agent collects data from the real environment and stores this in a replay buffer. This replay buffer is used to train a model of the environment, which acts as a copy of the environment from the perspective of the agent. Algorithms differ in the question how long model generated data is stored in the replay buffer, with some, such as MBPO, keeping the model generated experiences in a separate replay buffer for a few thousand steps, while others, like the MAD-TD algorithm we introduce in Chapter 7 using model-generated data only for a single gradient step before discarding it. While Janner et al. (2019) for example keeps model generated data for a few thousand update steps, the model-based algorithms presented in Chapter 6 and Chapter 7 do not store the data for multiple steps at all and generate it anew for each agent update.	26

2.2	A visual representation of the spectrum between decision-aware and general purpose learning. Several approaches presented in this thesis, such as Va-GraM (Chapter 5), CVAML (Chapter 6), and MAD-TD (Chapter 7) combine methods from both general purpose and decision-aware learning to achieve stable and efficient Reinforcement Learning (RL). OFN (Chapter 3) only consider model-free RL and can therefore be seen as a purely decision-aware method. Finally, while latent self-prediction and observation reconstruction are general purpose methods, Chapter 4 investigates how they can be used in synergy with TD learning. Based on this investigation, latent self-prediction is used to obtain a strong hybrid methods such as MAD-TD.	33
3.1	Return, in-distribution Q-values and Adam optimizer moments during priming for different lengths. Dotted lines correspond to end of priming. More priming steps lead to lower return and larger Q-values.	38
3.2	Priming with SAC and action regularization during priming. The latter lowers divergence.	38
3.3	Return and Q-values of priming runs with weight decay and dropout. Results indicate that both regularizers mitigate priming to some extent but not sufficiently.	38
3.4	In-distribution Q-values and critic loss of five SAC seeds on the humanoid-run task using $UTD = 32$. Values diverge at arbitrary time-points, not only during the beginning. Loss mirrors Q-value divergence.	39
3.5	A graphical representation of output feature normalization (OFN). Each feature vector is projected unto the unit sphere.	40
3.6	(Left) Return and (Right) Q-values comparing SGD result and OFN when priming for 100K steps. OFN obtains returns close to that of the well-trained SGD agent and learns an appropriate Q-value scale correctly.	41
3.7	L_2 norm of network weights per layer after priming for default and OFN architectures. OFN leads to smaller weights, especially for earlier layers. . .	41
3.8	Mean, interquartile mean (IQM), and median with 95% bootstrapped confidence intervals of standard SAC and OFN on the DMC15-500k Suite. OFN can maintain high performance even under large UTD. OFN with $UTD = 8$ achieves comparable performance to standard resetting with $UTD = 32$ across metrics.	42
3.9	Mean return of single-critic OFN, standard OFN and resetting; $UTD = 32$ on hopper-hop. Shaded regions are bootstrapped confidence intervals. . . .	42
3.10	Mean return on the dog DMC tasks, comparing OFN to SAC with resets and the model-based TD-MPC2. Shaded regions indicate bootstrapped confidence intervals. OFN outperforms SAC with resets, which is unable to learn and OFN with $UTD = 8$ and resetting is competitive with TD-MPC2. . . .	44

4.1	Diagram of the discussed loss functions and use cases. In latent self-prediction, we aim to predict the next state's <i>features</i> , computed with an embedding function $\phi(x')$ using the state's features $\phi(x)$ and a latent prediction model F . In observation reconstruction, the next state's <i>ground truth observations</i> x' are matched via the use of a decoder function $\psi(F(\phi(x)))$. In the <i>auxiliary task setup</i> , both the gradients from the feature learning loss and value function learning are propagated to the encoder, while in the <i>stand-alone scenario</i> , only the gradients from the feature learning loss are used to update Φ	49
4.2	Auxiliary task setup: Performance of all losses on the observation space as given without changes to the environment.	65
4.3	Stand-alone setup: Performance of all losses on the observation space as given without changes to the environment. The DQN baseline is using random features, which are not updated, to verify that learning features is indeed superior to a random feature baseline.	65
4.4	Distorted observation function with a random transformation.	66
4.5	Appending random noise channels.	67
4.6	Appending structured noise channels using the Freeway environment.	67
5.1	Visualization of discussed loss function with regards to a reference point marked with the white cross and the corresponding value function on the Pendulum environment. For the value function, darker color indicates a lower value. In the loss figures, darker color indicates how large the loss is if the model predicts $(\theta, \dot{\theta})$ instead of the reference sample marked in white. The VAML loss has a complex non-linear shape in the state space that follows iso-lines of the value function, while MSE and VaGraM are centered around the sample. For VaGraM, the rescaling of the MSE in the direction of high gradient along the θ axis is visible. Due to Equation 5.14, the scaling is aligned with the axis of the coordinate system and not rotated to fit the value function closer.	75
5.2	Evolution of the VAML loss over changing value functions on the Pendulum domain. Lines denote the mean and shaded areas show standard error over 8 model initialization and data set samples per model. In the linear setting, VAML achieves the lowest VAML error, while VaGraM is able to significantly outperform MSE. In the NN setting, VAML diverges rapidly, while VaGraM and MSE converge to approximately the same solution.	77

5.3	Performance of VaGraM and MLE models with reduced model size. The dotted lines correspond to the final performance reported for model-free SAC (grey, approx. 3200). Shaded area represents standard error over 16 repeated runs. VaGraM continues to solve the task almost unimpeded, while MLE is unable to even stabilize the Hopper when using a two layer neural network.	79
5.4	Performance of VaGraM and MLE models with distracting state dimensions. The dotted lines correspond to the final performance achieved by both algorithms on the Hopper task without distraction (grey, approx. 3200). Shaded area represents standard error over 16 repeated runs. VaGraM achieves significantly higher returns than the MLE baseline, especially in the most challenging setting with 15 distracting dimensions.	80
6.1	Sketch of (m, b) -VAML. The loss is computed from an <i>m-step model</i> and a $(m + b)$ -step environment trajectory. It is the difference between the estimated value of the <i>m-th model state</i> , and the b -step Bellman operator starting from the <i>m-th environment state</i>	85
6.2	Results for the Garnet experiments. In the top row, we show the mean squared error of the value function prediction using different latent sizes k , over three different temperatures. In the bottom row, we vary the temperature and show results for three values of k . Shaded regions are bootstrapped confidence intervals of the mean at 95% over 1000 independent problems. With the exception of deterministic problems (left), the CVAML loss reliably results in the lowest value prediction error.	95
6.3	Baseline comparison of the CVAML model with a purely general-purpose (only self-prediction loss) and model-free baseline. Across all tasks we see a strong performance benefit of the VAML-based model over the model-free and stochastic general-purpose baseline. On some dog tasks, a deterministic self-prediction model performs slightly better than the CVAML variant. However, on the humanoid tasks, performance of the CVAML loss is significantly better than any baseline. In aggregate, CVAML (with an auxiliary task) is a consistently strong loss.	103
6.4	Comparison of CVAML with an auxiliary task with purely decision-aware $(1, 0)$ -CVAML and $(1, 1)$ -CVAML models (no auxiliary self-prediction loss). We can see that while $(1, 0)$ -CVAML can learn non-trivial policies on the dog tasks, $(1, 1)$ -CVAML mostly fails to do so. In addition, both losses fail to learn non-trivial policies on the humanoid tasks.	104

6.5	Associated mean Q values over the replay buffer for CVAML with an auxiliary task and for purely decision-aware (1,0)-CVAML and (1,1)-CVAML models (no auxiliary self-prediction loss). The Q values are significantly overestimated for the (1,1)-CVAML loss without auxiliary task, explaining its weak performance (compare Figure 6.4).	105
6.6	Results for the latent-space model experiments. We show the aggregate metrics of the final performance on dog (left) and humanoid (right) environments, following Agarwal et al. (2021). Calibrating (1,1)-VAML leads to a clear improvement in performance in the humanoid environment, while for (1,0)-(C)VAML the difference is less distinct. This is consistent with our theoretical findings as learning a smaller variance model can be sufficient, but wrong value functions will be learned with a stochastic model when $b \geq 1$	106
6.7	Sample efficiency curve for both dog (left) and humanoid (right). Per-task normalized return is aggregated over 20 seeds per environment and 3 environments, with 95% bootstrapped confidence intervals shaded. In addition to a higher final return, we observe significantly earlier learning for the (0,1)-(C)VAML on the dog task.	107
6.8	Final performance aggregated across all tasks. We observe that we see a consistent improvement in performance when using (1,0) losses instead of (1,1) losses.	107
7.1	A visualization of the core issue we investigate. Even if a replay buffer contains good coverage for two policies (π_{old} and π_{new}) starting from $\rho = x_0$, this does not guarantee that it contains a transition for executing an action under the new policy on a state visited under the old. However, this state-action pair’s value estimate is used to update the value of state x_0 via Equation 7.1, without being grounded in an observed transition.	113
7.2	Left: the train , validation , and on-policy validation error of the Q function at UTD 1. Right: the Q values and return curves of TD3 agents across different UTD 1, 8, and 16.	117
7.3	Average loss values with and without model-generated data (compare Figure 7.2) at UTD 1.	122
7.4	Return curves for the dog tasks with differing UTD values. Across all environments, the return increases or remains stable with increasing UTD with MAD-TD. Without model data, the performance decreases when increasing the UTD. MPC is turned off in these runs to cleanly evaluate the impact of model data on critic learning.	123

7.5	Performance comparison on the hard tasks for MAD-TD, BroNet, and TD-MPC, with varying number of steps and action repeat settings. MAD-TD is on par with all baselines, has higher mean and IQM when trained for 2 million time steps and action repeat 2, and strongly outperforms TD-MPC2 and BroNet at 1 million time steps with action repeat 2.	123
7.6	Resetting evaluation of MAD-TD and BroNet. Lighter color denotes performance with reset, and darker without. While MAD-TD’s performance only increases slightly when adding resetting, BroNet is unable to achieve strong performance in and setting without resetting.	124
7.7	Average regret, mean over the hard suite. Lower regret corresponds to faster, more stable training. MAD-TD outperforms BroNet.	124
7.8	Return curves on the hard suite. We see that using substantially more data than 5% does not improve performance in a statistically significant way. . .	125
7.9	Aggregate statistics for differing values of α (amount of model data used) at UTD 8.	125
7.10	Return curves for the dog tasks when using on-policy, random and no model-generated data. When generating model-based data with random actions, performance of MAD-TD drops close to the model-free baseline, highlighting the importance of <i>on-policy</i> actions.	126
7.11	Performance curves for MAD-TD and Synther on the hard DMC tasks. Synther fails to achieve nontrivial results on most tasks, only outperforming a random policy on the humanoid walk and stand tasks.	127
7.12	Performance evaluation when reducing the model size of the latent model in MAD-TD. The performance predictably drops with decreasing hidden layer size, however only strongly decreasing the model size below 64 reduces the performance below that of the model-free ablation.	128
7.13	Magnitude of the difference between $Q(x, \pi(x))$ and $Q(x, \tilde{a})$, where $\tilde{a}$ is an adversarial perturbation of $\pi(x)$. Across the training why see higher perturbation magnitude for the runs without model correction data.	128
B.1	UTD8 Returns on Full DMC15-500K.	167
B.2	UTD32 Returns on Full DMC15-500K.	168
B.3	UTD8 Q-values on Full DMC15-500K. Resetting often works when Q-values diverge. ONF mitigates divergence.	169
B.4	UTD32 Q-values on Full DMC15-500K. Resetting often works when Q-values diverge. ONF mitigates divergence.	170
C.1	DMC: Auxiliary task scenario	176
C.2	DMC: Stand alone scenario	177
C.3	DMC: Auxiliary loss + reward prediction, not TD	177

C.4	DMC: Random noise distraction	178
C.5	DMC: Structured distraction	178
C.6	DMC: Observation space distortion	179
D.1	Comparison of VaGraM, MLE Model-based Policy Optimization (MBPO) baseline), IterVAML Farahmand (2018) and a value-weighting ablation on two simple continuous control tasks. The shaded area represents standard error estimated over 8 runs. While VaGraM, IterVAML and MBPO are able to achieve satisfactory performance on the Pendulum Swingup task, IterVAML fails to stabilize the more difficult Cartpole balancing task.	182
D.2	Comparison of VaGraM and MBPO on the Mujoco tasks presented in Janner et al. (2019). Lines mark average performance over 8 seeds and shaded areas mark the standard error of the mean. We see that VaGraM is able to perform on par with the MBPO implementation on the HalfCheetah and Walker tasks. It is unstable on the Ant task however.	183
D.3	Comparison of VaGraM and MBPO on the Mujoco tasks presented in Janner et al. (2019) using normalization (Bjorck et al., 2022). The instability on the Ant task is solved using the normalization, but the performance on HalfCheetah and Walker2d suffers.	184
D.4	Comparison of the empirical performance of deterministic and probabilistic MLE models vs VaGraM. Thick lines denote the mean and shaded area the standard error over 8 runs. In the limited capacity setting, the deterministic model is able to achieve significantly higher returns than the probabilistic baseline, however with stability issues during longer training. On the distracting benchmark, the deterministic model is not able to achieve any significant returns.	188
E.1	Model entropy (left) and loss value for the VAML-losses (right) aggregated across all environments. The shaded area represents a 95% stratified bootstrap interval. We see that the model entropy of the corrected (1,0)-VAML differs significantly, as we expect. However, this does not translate to a pronounced difference in the VAML error itself. Therefore, while the calibration term does indeed lead to a higher variance, this does not directly translate into an improvement in performance. In (1,1)-VAML, the model entropy does not differ significantly, however, without calibration, the VAML error seems to be growing slightly stronger (although not outside of overlapping confidence intervals). Given the large deviation in performance we observe in the main results for (1,1)-VAML, we conjecture that correcting the value function learning term is significantly more important for performance. . . .	191

F.1	Return curves and Q values with differing UTD values.	194
F.2	Return curves for the humanoid tasks when using on-policy (blue), random (green) and no model-generated data (orange). The observed performance impacts are comparable to the dog case.	195
F.3	Performance variation of the base MAD-TD model compared to TD-MPC2. Our changes only very few times lead to lower performance which is acceptable given the large reduction in computational cost.	196
F.4	Return curves with action repeat set to 2.	197
F.5	Return curves with action repeat set to 1.	197
F.7	Return curves for the impact of resetting on MAD-TD with and without MPC. Without MPC, resetting can still improve the performance, but with MPC, we see no significant benefits from resetting across environments except pendulum. The hopper results highlight the importance (and danger) of the reset interval, as seemingly the reset algorithm is not able to recover “in time” to improve performance.	199
F.8	Comparison of MAD-TD and TD-MPC2 across more environments of the DMC suite. We observe gains compared to TD-MPC2 in the hard tasks, especially in terms of early learning performance, while TD-MPC2 has advantages on the pendulum_swingup and acrobot_swingup tasks. These seem to be exploration and stability issues for which the longer model rollouts of TD-MPC2 seem to help. To prevent confusion, we highlight the choice of AR=2 for TD-MPC2.	200
F.9	Performance comparison on Metaworld between MAD-TD, BRO, and TD-MPC2. MAD-TD performs strongly on tasks which provide sufficient reward information to bootstrap the value function quickly, while learning more slowly on sparse reward tasks. This is consistent with the core goal of our algorithm, to stabilize and improve value function learning.	202
F.6	Return curves evaluating the impact of model-based data for critic learning and MPC. Overall, MPC and model-based critic learning both stabilize the learning process, as conjectured.	203

Preface

In this thesis, I summarize and present the results of my research conducted at the University of Toronto. As with all works of science, this work stands on the shoulders of giants and has been supported generously by my collaborators and advisors. As Machine Learning has become a collaborative discipline, much of my own work has been developed together with my advisors, colleagues at the University and abroad, and the amazing students whom I had the great fortune to mentor. This thesis is built on papers developed in collaboration with Amir-massoud Farahmand, Igor Gilitschenski, Animesh Garg, Eric Eaton, Marcel Hussing, Romina Abachi, Arash Ahmadian, Victor Liao, and Anastasiia Pedan. I acknowledge their great contributions to the work that led to this thesis.

This section clarifies the individual contributions in each chapter. With all papers, all authors were involved in editing the writing. The chapters in this thesis are lightly edited from the original papers to connect the individual contributions more clearly to the overarching narrative.

- **Chapter 3:** This chapter is based on Marcel Hussing, Claas Voelcker, Igor Gilitschenski, Amir-massoud Farahmand, and Eric Eaton (2024). “Dissecting Deep RL with High Update Ratios: Combatting Value Divergence”. In: *Reinforcement Learning Conference*. I contributed as joint first author and the core ideas of the work were developed in collaboration. Marcel Hussing provided the initial experiments on priming, the effects of optimizers, regularizers and conservative learning. The experiments on feature normalization were designed jointly. Writing was shared equally between Marcel Hussing and I.
- **Chapter 4:** This chapter is based on Claas Voelcker, Tyler Kastner, Igor Gilitschenski, and Amir-massoud Farahmand (2024c). “When does self-prediction help? Understanding Auxiliary Tasks in Reinforcement Learning”. In: *Reinforcement Learning Conference*. The writing, insights, proofs, and experiments are my contribution. The proof of [Proposition 4.5.2.1](#) is joint work with Tyler Kastner. [Lemma A.1.3.1](#) was suggested by Amin Rakhsha and fully written out by me.

- **Chapter 5:** This chapter is based on Claas Voelcker, Victor Liao, Animesh Garg, and Amir-massoud Farahmand (2022). “Value Gradient weighted Model-Based Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*. The writing, insights, core methodology, and experiments are my contribution, Victor Liao contributed to the experimental infrastructure and code.
- **Chapter 6:** This chapter is based on Claas Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-massoud Farahmand (2025b). “Calibrated value-aware loss functions with stochastic environment models”. In: *Proceedings of the International Conference on Machine Learning*, and an older version of the paper publicized as Claas Voelcker, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-massoud Farahmand (2024a). *λ -models: Effective Decision-Aware Reinforcement Learning with Latent Models*. The insights and most proofs are my contribution. The experimental architecture was first developed in collaboration with Romina Abachi, and Arash Ahmadian, and greatly expanded by Anastasiia Pedan. The final experiments were designed by me and implemented together with Anastasiia Pedan. [Lemma 6.6.1.1](#) was conjectured by me, the proof was thoroughly sketched out by Amir-massoud Farahmand, and finally written out and completed by me. [Lemma A.2.1.4](#) was suggested by Amin Rakhsha and fully written out by me.
- **Chapter 7:** This chapter is based on Claas Voelcker, Marcel Hussing, Eric Eaton, Amir-massoud Farahmand, and Igor Gilitschenski (2025a). “MAD-TD: Model-Augmented Data stabilizes High Update Ratio RL”. in: *Proceedings of the International Conference on Learning Representations*. I contributed as joint first author and the core ideas of the work were developed in collaboration. The initial insights of the paper were developed based on previous joint work (Hussing et al., 2024) together with Marcel Hussing as well as my own work in Voelcker et al. (2024a). The initial experiments on the impact of model data on on- and off-policy validation loss were contributed by me. All further experiments, the core architecture and experimental setup were developed jointly. Writing was shared equally between Marcel Hussing and I.

In addition to the papers listed above, I contributed to the following technical works on reinforcement learning during my time at the University of Toronto. These papers are not primary chapters in this thesis, and where their ideas are used, they are cited accordingly

- Romina Abachi, Claas Voelcker, Animesh Garg, and Amir-massoud Farahmand (2022). “VIPer: Iterative Value-Aware Model Learning on the Value Improvement Path”. In: *Decision Awareness in Reinforcement Learning Workshop ICML*: The paper proposes a latent model-based architecture that uses an IterVAML (Farahmand, 2018) style

loss over past value function estimates. Similar architectures were used in Voelcker et al. (2024a), Voelcker et al. (2025a), and Voelcker et al. (2025b) and the past value functions were discarded in favor of other, more computationally favorable auxiliary losses investigated in Voelcker et al. (2024c). I contributed to the idea and writing of the paper.

- Jonas Guan, Shon Eduard Verch, Claas Voelcker, Ethan C Jackson, Nicolas Papernot, and William A Cunningham (2024). “Temporal-Difference Learning Using Distributed Error Signals”. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*: The paper presents a novel architecture for training reinforcement learning agents without backpropagating TD learning signals through a neural architecture. As backpropagation is not a biologically plausible learning architecture, this work provides an important proof-of-feasibility for non-backpropagation based reinforcement learning. I served as a reinforcement learning expert on the paper and helped developed the experiments.
- Claas Voelcker, Marcel Hussing, and Eric Eaton (2024b). “Can we hop in general? A discussion of benchmark selection and design using the Hopper environment”. In: *Finding the Frame: An RLC Workshop for Examining Conceptual Frameworks*: This paper presents insights into the impact of environment selection on the empirical evaluation of reinforcement learning algorithms. The core ideas of the paper were developed jointly with Marcel Hussing and experimental evaluation was done jointly. The insights informed environment selection for [Chapter 6](#) and [Chapter 7](#).

All illustrations in this thesis are my original work, graphs and tables are taken from the acknowledged papers.

Chapter 1

Introduction

Current AI systems, such as large language model-based chat-bots and diffusion-based image generators, are primarily trained on fixed datasets. As a result, they cannot learn from interactions with the world, nor are they designed to engage with their environment in ways that facilitate ongoing learning. Similarly, most real-world autonomous systems, such as industrial robots, have to operate within tightly controlled settings so their behavior can be explicitly programmed by human engineers.

Developing truly intelligent and autonomous agents capable of acting across diverse and unstructured environments requires us to move past these limitations. To achieve this, we must enable artificial agents to learn from their interactions with the world in a self-directed manner, using their experiences to improve their capabilities. This goal has been perhaps most comprehensively pursued within the framework of reinforcement learning (Sutton and Barto, 2018).

The core idea behind reinforcement learning is that agents are able to improve based on their own experiences. Each interaction with the world generates a reward signal, which tells the agent how good their current action is. To improve, an agent seeks to explore the environment to maximize this reward. This creates the core loop of reinforcement learning: The agent acts in the world, obtains a reward and then seeks to repeat actions that lead to high reward and avoid actions which do not. Over time, through trial and error, this learning scheme leads to reward-maximizing behavioral policies which an agent can execute to complete the task that is specified by the reward.

The reinforcement learning approach to autonomous agents is a general recipe and has received much attention over the years, with breakthrough successes in playing games (Mnih et al., 2013; Silver et al., 2016), robotics (Hwangbo et al., 2019; Kaufmann et al., 2023; Smith et al., 2023), natural language processing and generation (Ouyang et al., 2022; Rafailov et al., 2023; Bi et al., 2024), and industrial applications (Degraeve et al., 2022). However,

despite its broad applicability, reinforcement learning is also often criticized for being unstable, hard to tune, and inefficient (Irpan, 2018; Henderson et al., 2018; Patterson et al., 2024b).

Many aspects leading to instability and inefficiency have only recently been put into focus. Recent work has identified culprits such as the failure to properly converge to good solutions (Kumar et al., 2021), lack of generalization and observational overfitting (Song et al., 2020; Kirk et al., 2023), and the diminishing ability of agents to learn consistently over many interaction steps (Nikishin et al., 2022b; Hussing et al., 2024). All of these reasons lead to a world in which, despite promising results, reinforcement learning has not yet achieved widespread adoption in many domains to which it is ostensibly well suited, such as robotics or scientific research, where discovering new ways of interacting with the complexity of the real world is necessary.

1.1 Stable and efficient value function learning

Can we make reinforcement learning stable and efficient in practice? To answer this question, we will put our focus on a central problem in reinforcement learning: value function estimation.

The value function is a central concept in reinforcement learning. Intuitively, it summarizes the expected future return of executing a policy starting from a given state. Value functions can be used to judge the impact of taking an action, to compare and improve policies, and to guide exploration. Given their usefulness, a lot of effort has been devoted to obtain good value function learning algorithms, especially using deep neural networks (Mnih et al., 2013; Van Hasselt, 2010; Van Hasselt et al., 2016a; Fujimoto et al., 2018b; Haarnoja et al., 2018; Hussing et al., 2024). However, stable and efficient value learning in online scenarios with complex function approximation, such as neural networks, remains an important unsolved problem, and many recent works show persistent problems with existing learning algorithms (Kumar et al., 2021; Nikishin et al., 2022b; Hussing et al., 2024). In this thesis we will show how the problems of the “deadly triad” (Sutton and Barto, 2018), especially bootstrapped value estimation and off-policy data, cause problems with learning stable features in Deep Reinforcement Learning agents. We present empirical and theoretical investigations into how these instabilities emerge, and propose algorithms to solve these issues.

1.1.1 Learning to represent environments

In order to learn values, an agent needs to filter and process their sensory input and learn about the structure of the environment, the consequences of their actions, and the rewards associated with them. To do so, they need to be able to extract and represent task-relevant information and distinguish it from irrelevant distractions. In deep learning, we call the

output of this process a *representation* of the environment. This can be any kind of data structure, most often a vector of real-valued numbers, that contains information about the environment and that are useable for machine learning algorithms. Going back to our initial example, an industrial robot can be tightly controlled and we can pre-specify what it will encounter in its limited environment and how to best represent it for decision making. But an autonomous agent needs to be able to perceive all of its surroundings and decide, based on current evidence, what information is necessary for its task and how to encode it. In empirical experiments (Hussing et al., 2024; Voelcker et al., 2024c; Voelcker et al., 2022) we show that failing to do so can lead to unstable value learning.

In supervised learning, neural networks have shown a remarkable ability to extract task-relevant information automatically from raw data, such as images, to solve prediction tasks (Goodfellow et al., 2016). However, in reinforcement learning, this has proven to be more difficult (Jaderberg et al., 2017; Igl et al., 2021; Lyle et al., 2021; Kumar et al., 2021; Nikishin et al., 2022b; Hussing et al., 2024). As the agent explores the environment gradually, it needs to simultaneously learn what information leads to good decisions, and to make good decisions. This creates a chicken-and-egg problem, as making good decisions requires an agent to be able to represent relevant information, but without knowing how to act, it is unclear what information is relevant (Igl et al., 2021; Voelcker et al., 2022). Failing to resolve this problem lead to agents getting stuck in suboptimal policies (Kumar et al., 2021; Nikishin et al., 2022b), or overfitting to irrelevant aspects of the environment (Song et al., 2020).

To understand how we can properly represent useful information in reinforcement learning, we will take a close look at two concepts which turn out to be intimately related: environment representations learned by neural networks during training, and world models which predict the behavior of the environment. Neural network representations are learned functions which map the sensory input of an agent, such as a camera feed or low-level joint information of a robot, to a vector that is useful for reinforcement learning (Ferns et al., 2004; Jaderberg et al., 2017; Abel, 2020; Le Lan et al., 2021). World models are functions that allow an agent to explicitly predict how its actions impact the environment. They can be used to improve value estimation or to plan good action sequences (Sutton, 1990; Janner et al., 2019; Hafner et al., 2020; Schrittwieser et al., 2020). As we will show, learning a world model provides an excellent way to obtain good representations, and good features simplify learning accurate world models.

1.1.2 Balancing decision-awareness and general-purpose learning

As mentioned, one important problem in representation learning is to be able to separate task-relevant information from distractions. To properly model task-relevant information, this thesis therefore analyzes *decision-aware* algorithms. We contrast these with *general-*

purpose algorithms, approaches that simply attempt to generate representations and models which can be used for any downstream task. As we will show, achieving stable and reliable learning requires leveraging and combining both.

Decision-awareness describes the idea that the components of a learning approach for decision making, such as representations and models, should account for the final decision task (Farahmand et al., 2017; Grimm et al., 2020; Abachi et al., 2020; Nikishin et al., 2022a). With this understanding, we can also group many model-free value learning approaches such as DQN (Mnih et al., 2013) or TD3 (Fujimoto et al., 2018b) under the umbrella of decision-aware methods. Similarly many supervised learning methods such as classifications are similarly decision-aware as they make use of an explicit task label.

In many cases, representations and world models can also be trained with *general-purpose* methods such as maximum likelihood estimation. These are unsupervised approaches which seek to simply represent the data through density estimation, compression, or projection. However, this can be inefficient or suboptimal, as resources are wasted on learning aspects of the data which are irrelevant to the decision-making task (Farahmand et al., 2017). Instead, a decision-aware algorithm should account for the decision-making task. In the context of a self-driving car for example, it should put a greater emphasis on certain parts of the observation space, such as a traffic light, instead of others, such as clouds moving in the background of a camera feed.

While this brief introduction seems to suggest that we should always use decision-aware approaches, experimental evidence complicates this conclusion. Even though decision-aware algorithms promise to be more resource efficient, training with general-purpose methods, for example by fitting a world model with a maximum likelihood objective, often turns out to be more stable in practice (Lovatto et al., 2020; Voelcker et al., 2022). In problems such as reinforcement learning, decision-aware algorithms need to continually adapt to the decision task as more information becomes available, and suffer from the aforementioned chicken-and-egg problem. General-purpose methods on the other hand have a single, stable training objective throughout the learning process. Especially in the beginning of training when available information about the decision task is limited, using a general-purpose method can lead to a more stable training algorithm. Therefore, at first glance, efficiency and stability, our two goals, seem to be somewhat at odds with one another.

Addressing this challenge is the contribution of this thesis. To this end, this thesis puts forth the following hypothesis which guides us through the works presented here:

Effectively combining available information on the decision task with general-purpose learning mechanisms addresses the issues of *instability* and *inefficiency* of value function learning and leads to effective reinforcement learning algorithms.

1.2 Thesis structure

The thesis structure is summarized in Figure 1.1. Before diving into the solutions to value learning instability, Chapter 2 presents a thorough overview of important definitions and algorithms in the field of reinforcement learning. This serves as the background for all further discussions in the thesis.

Chapter 3 presents insights into how instability emerges in value function learning. Experiments show that one of the major causes of unstable learning is diverging intermediate representations in the neural network architectures. To address this problem, we introduce an L_2 -norm-based normalization layer into the neural network architecture. This prevents the feature norm from growing over the course of training and leads to significant increases in learning stability.

Chapter 4 dives deeper into the question of stabilizing representations for value function learning. While several different methods have been proposed in the literature, it remains an open question which model-based auxiliary tasks can be used as general-purpose methods to stabilize learning without deteriorating the quality of the representations for the decision making tasks. This chapter presents both theoretical and empirical evidence that latent self-prediction models provide beneficial features for this goal.

Chapter 5 introduces a second perspective on learning stability and the tension between decision-awareness and general-purpose algorithms by focussing on model-based approaches. Here we investigate how the instability of value function learning can harm the training of decision-aware environment models. Empirical evidence is presented that suboptimal value function learning leads to a chicken-and-egg problem: without a good value function, we cannot train a good decision-aware model, but without a good decision-aware model, we cannot improve our value function. To escape this cycle, we suggest to augment a general purpose MSE loss with a

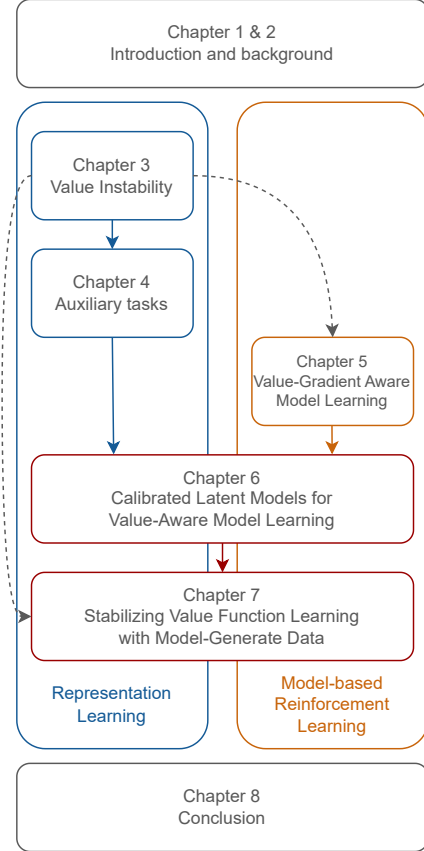


Figure 1.1: An overview of the connections between chapters of the thesis. Chapters 3 and 4 discuss neural network-based representation learning, while Chapter 5 discusses model-based reinforcement learning. Chapters 6 and 7 combine ideas from both strands to establish latent value-aware models with auxiliary task learning. Chapter 3 introduces the problem of learning with high update ratios, which is also used in Chapter 5, and put into central focus again in Chapter 7. Solid lines show influence of algorithmic ideas, and dashed lines mark the problem setting of high update ratio learning.

sensitivity score derived from the current value function estimate. This again highlights how combining both decision-aware and general purpose methods allows us to overcome the challenges of either.

Chapter 6 first shows how previous attempts at decision-aware model learning, including the IterVAML (Farahmand, 2018) and MuZero (Schrittwieser et al., 2020) losses can be unified into one family. We then discuss issues that arise in this framework when using previously established losses naively for learning value functions with stochastic environment models, and show how to mitigate this problem. The source of the problem is a subtle error term that appears when using the common decision-aware MuZero loss with stochastic models. Finally, this chapter discusses how latent model architectures, such as those in presented in Chapter 4, can be used to learn strong value-aware models and tame the issues presented in Chapter 5. With this, we fully unify the ideas of world model and representation learning.

Chapter 7 presents Model-augmented Data for TD learning (MAD-TD), an empirically strong algorithm that combines the ideas presented in this thesis. While previous chapters discuss the individual components of MAD-TD, here we put the focus back on solving the issue of instability established in Chapter 3, tying all of our discussions together. Using a latent value-aware model with well normalized features addresses a final important problem in value function learning: off-policy generalization. With a learned model, counterfactual questions of the form “What would happen if the agent took action a' in a previously visited state x ?” can be answered, which empirically has a strong stabilizing effect on value function learning.

Finally, this thesis concludes with Chapter 8 where we review the findings and discuss remaining questions and promising directions for future work.

Chapter 2

Background

Before we develop the core algorithms and proofs of the thesis, this chapter introduces the fundamentals of RL, Model Based Reinforcement Learning (MBRL), and representation learning. These fields form the basis of the presented research, and the methods and results are built on the established knowledge in these areas. The definitions and results presented here are well established in the literature and we therefore forgo individual references for each definition. Instead, readers are referred to appropriate reference work. The Markov Decision Process is discussed thoroughly in Puterman (1994), and a more general perspective on general continuous state- and action-spaces can be found in Bertsekas and Shreve (1978). For reinforcement learning, thorough introductions are presented in Sutton and Barto (2018) and Farahmand (2021). The presentation of the Markov Decision Process and Reinforcement Learning basics here roughly follows Farahmand (2011) and Farahmand (2021). For model-based reinforcement learning and representation learning, appropriate literature is cited directly as these fields are still in active development.

2.1 General mathematical notation and definitions

To define the concepts used in Machine Learning (ML) and especially in RL we require some mathematical language.

When talking about sets, we will generally use italic capital letters such as $\mathcal{X}$ or $\mathcal{A}$, except for canonical notation such as the symbol $\mathbb{R}$ used for the real numbers. We will use the notational shorthand $\{x\}_N$ and $\{x_1, \dots, x_N\}$ to denote sets containing elements x_1 to x_N , and $(x_1, \dots, x_N)$ or $(x)_N$ for sequences.

Functions are denoted following standard terminology as lower case letters, and operators use upper case letters. We will write $M(\mathcal{X})$ to denote the set of possible distributions over the set $\mathcal{X}$ equipped with an appropriate σ -algebra. Functions that map to $M(\mathcal{X})$ are called

probability kernels and play an important role in the definition of RL. For a probability kernel $\mathcal{P} : \mathcal{X} \rightarrow \mathcal{Y}$ we will use the standard notational shorthand $\mathcal{P}(y|x)$ to denote the probability (density) of y under the distribution $\mathcal{P}$.

Given a suitable probability (density) $p(x)$ for the probability space $(\Omega, \mathcal{F}, p)$, we will denote the expectation of a random variable $X : \Omega \rightarrow \mathbb{R}^n$ drawn from p as $\mathbb{E}_{X \sim p}[X] = \int_{\Omega} X(\omega) dp(\omega)$. This makes it clear from what distributions random variables are drawn when several distributions are involved such as in the case of the Kullback-Leibler Divergence (KL). In a discrete case ($|\mathcal{X} \times \mathcal{A}| < \infty$), the Lebesgue integral can be simplified to $\sum_{x \in \mathcal{X}} p(x)x$, and in the case of standard real vector-valued distributions such as the Gaussian, we will similarly write $\int p(x)x dx$ and forgo a full measure theoretic treatment. In these cases we will also not distinguish between random variables X and members of the underlying spaces $x \in \mathcal{X}$ and write x for simplicity.

2.2 The Markov Decision Process

At the heart of RL lies the **Markov Decision Process (MDP)** which serves as the core mathematical structure that formalizes the concept of learning in an environment.

Definition 2.2.0.1 (Markov Decision Process). *An MDP is a tuple $(\mathcal{X}, \mathcal{A}, \mathcal{P}, r, \gamma)$, where $\mathcal{X}$ is a set of states, $\mathcal{A}$ is a set of actions, $\mathcal{P} : \mathcal{X} \times \mathcal{A} \rightarrow M(\mathcal{X})$ is a transition probability kernel, $r : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$ is a reward function, and $\gamma \in [0, 1)$ is a discount factor.*

These mathematical objects describe a process in which an agent interacts with an environment over a sequence of discrete time steps. An agent is said to be in a state $x \in \mathcal{X}$ at time t , where it takes an action $a \in \mathcal{A}$. The state at $t = 0$ is sampled from a starting state distribution p_0 . Note that the state encompasses both the environment and the agent's internal state, such as memory or other information. There is no strict distinction between internal and external state in the formalism. The transition kernel is then used to sample a possible next state $x_{t+1} \in \mathcal{X}$ which the agent transitions to by drawing a sample from $\mathcal{P}(x'|x, a)$. At every timestep, after choosing an action a in state x , the agent receives the reward $r(x, a)$. In the case of discrete state-action sets, the next state is drawn from a discrete distributions. For some results, additional care is needed to treat continuous state-action spaces rigorously. In this thesis, such results are presented in [Chapter 6](#), where we draw on results from Bertsekas and Shreve (1978). Interested readers are referred there for a more in-depth discussion on continuous state-action spaces.

The word *Markov* in Markov Decision Process refers to the fact that the process is memoryless. This means that the next state distribution only depends on the current state and action, not on the past history of states. An agent acting in a *Markov* decision process therefore does not need to remember what happened in the past, all information is captured in the present state.

The agent's strategy for choosing actions is described by its policy, which describes the likelihood of the agent choosing different actions in state x .

Definition 2.2.0.2 (Policy). *A policy $\pi : \mathcal{X} \rightarrow M(\mathcal{A})$ is a function that maps from states to distributions over actions. Given an MDP $(\mathcal{X}, \mathcal{A}, \mathcal{P}, r, \gamma)$ and a policy π , we denote the policy-conditioned transition kernel as*

$$\mathcal{P}^\pi(x'|x) = \int_{\mathcal{A}} \pi(a|x) \mathcal{P}(x'|x, a) da.$$

Equipped with definitions for all standard components of the MDP and policies, we can now define trajectories and value functions.

Definition 2.2.0.3 (Trajectories). *Let $M = (\mathcal{X}, \mathcal{A}, \mathcal{P}, r, \gamma)$ be an MDP. We call a sequence of states and actions $\tau^n := (x_0, a_0, \dots, x_{n-1}, a_{n-1})$ a trajectory of length n . When n is omitted, we will assume a trajectory of infinite length. The likelihood of a trajectory that is generated by an agent following π in M is defined as*

$$p_{M,\pi}(\tau^n) = p_0(x_0) \pi(a_0|x_0) \prod_{i=1}^{n-1} \pi(a_i|x_i) \mathcal{P}(x_i|x_{i-1}, a_{i-1}).$$

When the starting state is fixed, we will write τ_x^n to signify that all probabilities are conditioned on x . The probability density is then defined as

$$p_{M,\pi}(\tau_{x_0}^n|x_0) = \pi(a_0|x_0) \prod_{i=1}^{n-1} \pi(a_i|x_i) \mathcal{P}(x_i|x_{i-1}, a_{i-1}).$$

In an analogous manner, we can also fix the starting state-action pair to obtain $\tau_{(x,a)}^n$. We denote the sum of discounted rewards associated with a trajectory as $J_\gamma(\tau) = \sum_{i=0}^{n-1} \gamma^i r(x_i, a_i)$.

In cases where we are dealing with both timestep indices and sample indices, we will write $x_i^{(t)}$ where t is the timestep and i is the sample index. Often, the choice of MDP and policy is clear from context and we will omit the subscripts $\pi, \mathcal{P}$ and γ from the notation to avoid clutter.

Trajectories allow us to easily define the value function of a policy.

Definition 2.2.0.4 (Value function). *Let $\mathcal{M} = (\mathcal{X}, \mathcal{A}, \mathcal{P}, r, \gamma)$ be an MDP. The policy value function $V^\pi : \mathcal{X} \rightarrow \mathbb{R}$ is equal to the expected discounted reward of infinite-length trajectories generated by an agent following policy π in $\mathcal{M}$,*

$$V^\pi(x) = \mathbb{E}_{\tau_x \sim p_{\mathcal{M},\pi}(\cdot|x)} [J_\gamma(\tau_x)].$$

The policy state-action value function $Q^\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$ is defined as

$$Q^\pi(x, a) = \mathbb{E}_{\tau_{(x,a)} \sim p_{\mathcal{M},\pi}(\cdot|x,a)} [J_\gamma(\tau_{(x,a)})].$$

This definition has an important caveat – it assumes that the expectation exists and is finite. If $\sum_{i=1}^n \gamma^i r_i$ does not converge as $n \rightarrow \infty$, for example because there exists a timestep t so that $r(x_i, a_i)/\gamma^i \geq 1$ for all $i \geq t$. To ensure this does not cause issues, we will generally assume that the reward is bounded so that for all state and action pairs $r_{\min} \leq r(x, a) \leq r_{\max}$. In this case the value will always be bounded between $(1 - \gamma)^{-1}r_{\min}$ and $(1 - \gamma)^{-1}r_{\max}$. However, there are canonical problems such as the LQR problem for which this does not hold. In these problems some policies do not induce a (finite) value function.

The goal of the agent in the standard MDP framework is to find an optimal policy $\pi^*(\cdot|x)$ from a set of possible policies Π which maximizes the future discounted reward when starting from state x . Finding such a policy is one of the core goals of *reinforcement learning*, which we discuss in [Section 2.3](#).

2.2.1 Occupancy distributions

It is often important to characterize the states visited under a policy without considering their temporal ordering. Sometimes we only care about how likely a state is to appear in any trajectory, not when it appears. We can formalize this notion by looking at n -step transition probabilities. These are marginals of the trajectory probabilities.

Definition 2.2.1.1 (N-step Transitions). *Let $\mathcal{M}$ be an MDP and π be a policy. The 1-step probability density $p_{\mathcal{M},\pi}^1(x, a|x_0)$ is defined as*

$$p_{\mathcal{M},\pi}^1(x, a|x_0, a_0) = \int_{\mathcal{A}} \pi(a|x) \mathcal{P}(x|x_0, a_0) d\pi(a_0|x_0).$$

The n -step transition is defined recursively as

$$p_{\mathcal{M},\pi}^n(x, a|x_0) = \int_{\mathcal{X}, \mathcal{A}} \pi(s|a) \mathcal{P}(x|x_{n-1}, a_{n-1}) dp_{\mathcal{M},\pi}^{n-1}(x_{n-1}, a_{n-1}|x_0).$$

Analogous to the trajectory case, $p_{\mathcal{M},\pi}^n(x, a|x_0, a_0)$ is the n -step probability resulting from the state-action pair (x_0, a_0) .

Summing up and discounting n -step transitions results in the discounted state-action occupancy measure.

Definition 2.2.1.2 (Discounted state-occupancy measure). *Let $\mathcal{M}$ be an MDP and π be a policy. The discounted state occupancy distribution $\rho_{\mathcal{M},\pi}(x, a|x_0, a_0)$ is the distribution constructed as*

$$\rho_{\mathcal{M},\pi}(x, a|x_0, a_0) = \frac{1}{1 - \gamma} \sum_{i=0}^{\infty} \gamma^i p_{\mathcal{M},\pi}^i(x, a|x_0, a_0).$$

Finally, in the case of *positive recurrent* transition kernels the sequence $p_{M,\pi}^n$ converges in distribution for $n \rightarrow \infty$. If this distribution is independent of the start state-action pair (which requires a positive recurrent and aperiodic transition kernel), we call it the stationary state-action distribution and write $\mu_{M,\pi}(x, a)$.

2.2.2 State and action observations

In the most general formulation of the MDP presented here, we do not place additional assumptions on the structure and relationships between the states and actions in $\mathcal{X}$ and $\mathcal{A}$. In practice, however, such structure exists for many interesting problems and can be exploited for learning. One important way in which structure is captured in practice is through state and an actions *observations*, which are crucial for the success of many RL algorithms. The observation of the state and action space can be seen as a mapping from the abstract state space to a concrete observation space, such as a vector of real numbers or an image.

A state observation function maps each state $x \in \mathcal{X}$ into an observation space $\mathcal{Y}$, often one that is equipped with structure such as $\mathbb{R}^n$. Most of the time we will not distinguish between a state (or action) and its observation. However, when we need to explicitly talk about such a function, we will denote it as $o : \mathcal{X} \rightarrow \mathcal{Y}$. Common hand-crafted observation functions map discrete states to one-hot vectors, or follow more complex schemes such as tiling activations.

If the state observation function is not invertible, the resulting problem is called a Partially Observable Markov Decision Process (POMDP). In this framework, an agent cannot discern from the observation alone in which state it currently is, and needs to use other information, such as the past history of observations. In general, this thesis will stick to the realm of fully observable MDPs and steer clear of POMDPs. Nonetheless, they are an important class of problems in their own right, especially for modelling real world phenomena in which full observability is rarely, if ever, given.

It is not uncommon for an MDP problem to have more than one way to represent states via observations. For example, in many robotic control tasks, the state can either be represented as a set of joint angles and velocities, or by a sequence of camera images from the robot, or even a combination thereof! Often, changing the observation space can make a problem much harder or simpler to learn in practice.

One simple example of this is the representation of angles, a common problem in many real-world applications such as robotics. An angle θ can be represented as a numerical value $\theta \in [0, 2\pi)$. However, this representations has a discontinuity from 2π to 0. For example, adding a small angle $\delta\theta$ to θ will generally result in a new angle $(\theta + \delta\theta) \bmod 2\pi$. If the same angle is instead represented as the vector $[\sin(\theta), \cos(\theta)]$, addition does not result in

a discontinuous functions. However, this has the drawback that not all two dimensional vectors denote valid angles, and angular addition is not a linear operation.

The problem of mapping from a state or its observation to a space that is beneficial for the goal of learning a good policy is called *representation learning*, and will be a major focus of this thesis. In general, we will assume that *observations* are mappings that are inherent to the problem and unchangeable from the perspective of the algorithm, while *representations* are mappings that the agent can change. However, this distinction is not always cleanly followed in the literature and observations are sometimes also referred to as representations.

2.3 Reinforcement Learning

Now that the problem of finding an optimal policy is established, we need algorithms to solve it. In this section, we review the basic concepts of RL that are used to obtain optimal policies. Given the scope of this thesis, we will focus solely on value driven methods that learn value functions and use these to improve upon the policy.

2.3.1 Reinforcement Learning in Tabular Domains

When the state and action sets are finite, all relevant components of the MDP problem can be represented by vectors and matrices. If the MDP has $n = |X|$ distinct states, the policy-conditioned transition kernel $\mathcal{P}^\pi$ becomes a matrix of shape $n \times n$, and the reward and state value function a vector of shape n .

Following standard Markov chain notation, any distribution over states is written as a row vector $\mathbf{x} = [p(x_1), \dots, p(x_n)]$ and the transition probability is given as

$$\left[\sum_{i=1}^n \mathcal{P}(x_0|x_i), \dots, \sum_{i=1}^n \mathcal{P}(x_n|x_i) \right] = \mathbf{x} \mathcal{P}^\pi.$$

The expected reward over a state distribution is also easily expressed as an inner product $\langle \mathbf{x}, r \rangle$. We will frequently make use of the fact that $V^\pi = \sum_{i=0}^{\infty} \gamma^i \mathcal{P}^i r = (I - \gamma \mathcal{P}^\pi)^{-1} r$. This again assumes that the infinite sum converges, as discussed previously. Beyond the results presented here which are foundational for all chapters, we present some additional results and lemmas, mostly taken from the literature in [Appendix A](#). These are specifically relevant for [Chapter 4](#) and [Chapter 6](#).

2.3.2 Bellman optimality and Bellman operators

When looking at the definition of the policy value function, we can quickly see that it is recursive. We can decompose the value function into the first step reward and the next

state's value

$$V^\pi(x) = \mathbb{E}_{a \sim \pi(\cdot|x)} [r(x, a) + \gamma \mathbb{E}_{x' \sim \mathcal{P}(\cdot|x, a)} [V^\pi(x')]] .$$

This decomposition allows us to use one of the central concepts of reinforcement learning and dynamic programming: the Bellman Optimality Principle:

An optimal policy has the property that whatever the initial state and initial decisions are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decisions. (Bellman, 1953)

Assuming that an agent will follow the current policy in the subsequent state, an agent can improve its policy based on the current value function by computing

$$\pi_{\text{new}}(x) = \arg \max_{a \in \mathcal{A}} [r(x, a) + \gamma \mathbb{E}_{x' \sim \mathcal{P}(\cdot|x, a)} [V^\pi(x')]] .$$

The corresponding value function then becomes

$$V^{\pi_{\text{new}}}(x) = \max_a [r(x, a) + \gamma \mathbb{E}_{x' \sim \mathcal{P}(\cdot|x, a)} [V^\pi(x')]] \geq V^\pi(x) .$$

By greedily picking the best action under the one step reward and the *current* policy's value function, the agent is guaranteed to receive at least the same future reward in each state as previously. This lays the foundation for a whole class of methods that all follow the same recipe: pick a policy, compute its value function, improve the policy based on the value function, and repeat. The standard examples of such algorithms are Policy Iteration (PI) and Value Iteration (VI), which require full knowledge of the transition kernel and reward.

Policy Iteration In policy iteration, we begin by fixing a policy and computing its value function. Doing this takes advantage of the recursive nature of the function by the way of a fixed point iteration. Given any appropriate function $Q : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$, we can define the Bellman operator as follows.

Definition 2.3.2.1 (On-policy Bellman Operator). *Let $\mathcal{M}$ be an MDP, π be a policy, and $Q : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$ any function mapping state-action pairs to real numbers. The Bellman Operator $\mathcal{T}^\pi$ is defined as*

$$[\mathcal{T}^\pi Q](x, a) = r(x, a) + \gamma \mathbb{E}_{x' \sim \mathcal{P}(\cdot|x, a)} [\mathbb{E}_{a' \sim \pi(x')} [Q(x', a')]]$$

for all states and actions in the MDP.

The true policy state-action value function Q^π is the fixed point of this operator $[\mathcal{T}^\pi Q^\pi] = Q^\pi$. When starting with any initial function Q , repeatedly applying the Bellman Operator

converges results in a fixed-point iteration that can be proven to converge to Q^π using Banach's Fixed Point Theorem.

With this Q function, we can define our new policy as $\pi^{\text{new}}(x) \leftarrow \arg \max_{a \in \mathcal{A}} Q(x, a)$ and then repeat the process. This can be proven to converge to the optimal policy π^* , by noticing that we have guaranteed improvement in π and that the process cannot get stuck in a local optimum. For a full proof, refer to Farahmand (2021).

Value Iteration Instead of waiting for our Bellman iteration to complete for each policy, we can instead greedily update the policy at every step. This leads to the so called Bellman optimality operator.

Definition 2.3.2.2 (Bellman Optimality Operator). *Let $\mathcal{M}$ be an MDP, π be a policy, and $V : \mathcal{X} \rightarrow \mathbb{R}$ any function mapping state-action pairs to real numbers. The Bellman Optimality Operator $\mathcal{T}^*$ is defined as*

$$[\mathcal{T}^*Q](x, a) = r(x, a) + \gamma \mathbb{E}_{x' \sim \mathcal{P}(\cdot|x, a)} \left[\max_{a' \in \mathcal{A}} Q(x', a') \right].$$

This again can be shown to converge directly to the Q function of the optimal policy.

2.3.3 Parametric value function learning

As the state-action space of an MDP grows larger, keeping an explicit record of each state's value in a long vector becomes less and less feasible. Memory requirements grow linearly with the size of the state-action space. In addition, we are not able to make use of additional structure in the state-spaces, such as meaningful observation functions. Therefore, it is generally useful to leverage approximations which are efficient to store and can make use of structure. In this thesis, we are foremost interested in parametric approximations.

Definition 2.3.3.1 (Parametric approximation). *A parametric approximation of a value function is defined by a k -dimensional vector $\omega \in \mathbb{R}^k$ and a function $V : \mathcal{X} \times \mathbb{R}^k \rightarrow \mathbb{R}$.*

In this thesis, we will mostly consider neural networks and use linear function approximations in Chapter 4 to develop a theoretical model of representation learning.

Once we have parameterized function, we cannot simply apply the Bellman operators in closed form as we did to obtain VI and PI in the tabular domain. To obtain the optimal weights of a parametric value function, we require a learning algorithm. While there is a huge variety of such algorithms, many currently popular ones revolve around the same core concept, approximate value or policy iteration. Approximate value iteration seeks a function in each iteration that is as close to $[\mathcal{T}V]$ as possible. To understand how to achieve this, we will quickly review some core concepts in machine learning and Empirical Risk Minimization (ERM).

Empirical Risk Minimization

In machine learning, the most common way to find the optimal parameters of a function is by minimizing a loss function. Given a parametric function $f_\theta(x) : \mathcal{X} \rightarrow \mathcal{Y}$ from some function class $\mathcal{F}_\theta$, and prediction targets y , the loss function $l : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ is computed to measure the error between the prediction and the target. Often, these loss functions are derived from distance functions or metrics such as the squared L_2 distance $l(y, f_\theta(x)) = \|y - f_\theta(x)\|^2$.

The goal of ERM is minimize the expected loss of our function estimation under some distribution P over data points x and targets y . Formally, we aim to find

$$f^* = \arg \min_{f_\theta \in \mathcal{F}_\theta} \int l(y, f_\theta(x)) dP(x, y).$$

However, we generally do not have access to the distribution μ in a way that allows us to freely sample unlimited data points (x, y) . Therefore, we instead assume access to a fixed dataset $D = \{(x_1, y_1) \dots, (x_N, y_N)\}$ which is i.i.d. with regard to P and define $\mathcal{L}(f_\theta, D) = \frac{1}{|D|} \sum_{i=1}^N l(y_i, f_\theta(x_i))$. Then the optimal function f^* according to the empirical measure defined via D is

$$\hat{f}^* = \arg \min_{f_\theta \in \mathcal{F}_\theta} \mathcal{L}(f_\theta, D).$$

We will generally use the notation $\mathbb{E}_D$ interchangeably with $\mathbb{E}_P$ throughout this thesis to signify an expectation taken across every possible dataset taken from P .

The study of when and how quickly the empirically optimal function $\hat{f}^*$ approaches the function that minimizes the expected loss over the underlying distribution f^* is an important area of research in machine learning. In this thesis, we will focus on examples of loss functions which are miscalibrated, which means that $\hat{f}^*$ does not converge to f^* as we increase the size of the dataset. In the next section, we will review the well-known case of the double sampling problem, and we will encounter this problem again in [Chapter 6](#) in the context of decision-aware model losses. We will also introduce the notion of miscalibration formally there.

Loss functions for Value Learning

The value function V^π could in principle be obtained from trajectories of the current policy, by computing

$$\mathcal{L}(\hat{V}, \{x, \tau_x\}_N) = \frac{1}{N} \sum_{i=1}^N \left(J(\tau_{x_i}) - \hat{V}(x_i) \right)^2.$$

Such an approach is called a Monte Carlo approach, as it relies on a Monte Carlo or sample-based approximation of the value function. However, the Monte Carlo approach does not take advantage of the Bellman decomposition of the value function.

If we use this structure, we can greatly improve the sample efficiency of our approach, as we do not need to collect full trajectories for each state we want to estimate the value function on. To do this, we need to define a loss function that depends on the value estimate in the target as well.

Bellman Residual Minimization and the Double Sampling Problem Naively, we might want to try and minimize a loss function of the form

$$\mathcal{L}_{\text{res}}(\hat{V}, \{x, r, x'\}_N) = \frac{1}{N} \sum_{i=1}^N \left(\hat{V}(x_i) - r_n - \gamma \hat{V}(x'_i) \right)^2.$$

This is called *Bellman Residual Minimization*, and we can show that it does not lead to a correct value function estimate. This result is well known in the reinforcement learning literature, see e.g. Sutton and Barto (2018, p. 299). We will briefly review a proof here as similar techniques will be relevant to show issues with similar loss functions in Chapter 6.

Proposition 2.3.3.1 (The Double Sampling Problem). *Let D be a dataset of (x_i, r_i, x'_i) tuples sampled from the stationary distribution of some MDP $\mathcal{M}$ with transition kernel $\mathcal{P}$ and fixed policy π . There exist function classes $\mathcal{V}$ that include V^π , the ground truth value function for $\mathcal{P}^\pi$, but for which*

$$V^\pi \notin \arg \min_{\hat{V} \in \mathcal{V}} \mathbb{E}_{x, r, x' \sim \mu} \left[\mathcal{L}_{\text{res}}(\hat{V}, \{x, r, x'\}) \right].$$

The proof is relatively straight forward. We need to show that the intended target, V^π , does not properly minimize the loss. To do this, we decompose the loss function as follows

$$\mathbb{E}_\mu \left[\mathcal{L}_{\text{res}}(\hat{V}, \{x, r, x'\}_N) \right] = \mathbb{E}_D \left[\frac{1}{N} \sum_{n=1}^N \left(\hat{V}(x_n) - r_n - \gamma \hat{V}(x'_n) \right)^2 \right] \quad (2.1)$$

$$= \mathbb{E}_\mu \left[\left(\hat{V}(x) - r - \gamma \hat{V}(x') \right)^2 \right] \quad (2.2)$$

$$= \mathbb{E}_\mu \left[\left(\hat{V}(x) - V^\pi(x) + V^\pi(x) - r - \gamma \hat{V}(x') \right)^2 \right] \quad (2.3)$$

$$= \mathbb{E}_\mu \left[\left(\hat{V}(x) - V^\pi(x) \right)^2 \right] + \mathbb{E}_D \left[\left(V^\pi(x) - r - \gamma \hat{V}(x') \right)^2 \right] \quad (2.4)$$

$$+ 2\mathbb{E}_\mu \left[\left(\hat{V}(x) - V^\pi(x) \right) \left(V^\pi(x) - r - \gamma \hat{V}(x') \right) \right] \quad (2.5)$$

If we substitute the true value function V^π for $\hat{V}$, it is easy to see that the first and last terms disappear. However, we are left with a variance like term

$$\mathbb{E}_D [\mathcal{L}_{\text{res}}(V^\pi, \{x, r, x'\}_N)] = \mathbb{E}_D \left[\frac{1}{N} \sum_{n=1}^N (V^\pi(x_n) - r_n - \gamma V^\pi(x'_n))^2 \right] \quad (2.6)$$

$$= \mathbb{E}_D \left[\left(\underbrace{V^\pi(x) - V^\pi(x)}_{=0} \right)^2 \right] + \mathbb{E}_D \left[(V^\pi(x) - r - \gamma V^\pi(x'))^2 \right] \quad (2.7)$$

$$- \mathbb{E}_D \left[\left(\underbrace{\hat{V}(x) - V^\pi(x)}_{=0} \right) (V^\pi(x) - r - \gamma \hat{V}(x')) \right] \quad (2.8)$$

$$= \mathbb{E}_D \left[(V^\pi(x) - r - \gamma V^\pi(x'))^2 \right] > 0. \quad (2.9)$$

Our formal statement requires a function that minimizes this loss better than the ground truth value function in our function class. We will discuss the existence of such functions and provide examples in [Chapter 6](#).

If we had two independent next state samples x'_i , we could construct an unbiased estimator of the target and obtain a correct loss. But in the standard RL setting, we assume strict sequential interaction. So in general we cannot rely on having two independent samples for each state whose value we want to estimate.

Target Network Updates Instead of naively minimizing the predicted difference between our current value function estimate, we only need a minimal change to make the loss function work. We cannot update the parameters of both the value estimate $\hat{V}$ and the bootstrapped Bellman estimate $R + \gamma \mathcal{P}^\pi \hat{V}$. So instead, we make use of an independent copy of the value function, $\hat{V}_{\text{target}}$, which is commonly called a target network. This target network is periodically updated towards the current value estimate $\hat{V}$. With the help of the target network, we can define

$$\mathcal{L}_{\text{TD}}(\hat{V}, \hat{V}_{\text{target}}, \{x_i, r_i, x'_i\}_n) = \frac{1}{n} \sum_{i=1}^n \left(\hat{V}(x_i) - \left[r + \gamma \hat{V}_{\text{target}}(x'_i) \right]_{\text{sg}} \right)^2$$

as our loss. The notation $[\cdot]_{\text{sg}}$ refers to a *stop gradient* operation, which clarifies that the loss function is not used to update the later part of the equation.

Different algorithms choose different approaches to obtain the target estimate $\hat{V}_{\text{target}}$. The simplest two variants are to either replace $\hat{V}_{\text{target}}$ at *every* update step, or instead to wait until $\hat{V}$ has converged to an optimum and update then. The later mimics the structure of the Bellman fixed-point iteration, but as it can take a lot of iterations for the value estimate

to converge fully, this can be computationally expensive. On the other hand, updating the target at every step can prove to be very unstable in practice. Instead of these extremes, most common algorithms either use a fixed, but large, update period, e.g. 1000 steps, (Mnih et al., 2013) or use a Polyak average of the parameters $\theta_{\text{target}} \leftarrow (1 - \alpha)\theta_{\text{target}} + \alpha\theta$ with a small value for α (Fujimoto et al., 2018a).

The introduction of a bootstrapped target that depends on the value estimate itself lies at the heart of (almost) all problems discussed in this thesis. With this design choice, we have introduced non-stationarity in the targets of our optimization problems which leads to unstable optimization. However, it is crucial to understand that we do not make this decision lightly. Relying on the Bellman backup instead of Monte-Carlo estimates makes many of our algorithms vastly more efficient. As our goals are stable *and efficient* value learning, we will therefore not avoid but embrace the use of bootstrapped target estimation.

Policy improvement After establishing a loss function that can be used to learn parametric value function approximations, we can apply (approximate) VI or PI to obtain policy updates. As in the tabular case, we can interleave policy update steps with value function approximation steps, or extract a policy from a fully converged value function. However, the former is used by almost all modern Deep Reinforcement Learning (DRL) algorithms, as waiting for the value function to converge can take a long time.

When the action space is tabular, a policy can be obtained from an approximate Q function by choosing $a \in \arg \max_{\mathcal{A}} Q(x, a)$. However, when the action space is continuous, we also have to parametrize the policy. In this thesis, we will mostly rely on the Deterministic Policy Gradient (DPG), which is introduced in [Subsection 2.3.4](#).

On- and off-policy learning In addition to a loss function, we also need a dataset to train on. Here, two major strategies dominate: on-policy and off-policy learning.

In *on-policy* learning, all data comes from a single policy, the one for which we are attempting to estimate the value. While this is generally much more stable, it also requires a large amount of samples, as we need to gather new data from scratch once we change the policy. In principle, old samples can be used if we apply a sampling correction term such as importance sampling. But even in this case, the importance weights quickly become too small for old samples to have an impact on the loss. As this thesis is interested in *efficient* learning, we instead focus on off-policy learning.

In *Off-policy* learning we reuse samples from old policies, which makes the approach more sample-efficient, but introduces several algorithmic challenges. Many algorithms can diverge when used with samples collected from different policies, even when their on-policy counterparts converge. The combination of bootstrapped value estimation, function approximation, and off-policy learning has therefore been termed as the *deadly triad* by Sutton

and Barto (2018). Nonetheless, all components of the deadly triad are useful or sometimes even necessary (in the case of function approximation) to scale RL to complex problems. While most chapters in this thesis deal with problems resulting from the deadly triad in one form or another, we will focus on off-policy learning specifically in Chapter 7.

While the textbook definition of the terms “on-policy” and “off-policy” refers to methods, we will in the following also use them in a slightly expanded way to refer to data. On-policy data is therefore collected with the target policy, while off-policy data is collected with one, or several, different behavior or data collection policies.

2.3.4 Deep Reinforcement Learning in Continuous Action Spaces

Current DRL methods use several additional mechanisms to ensure stable training of the RL agent’s value function and policy with neural networks. Here, we take a brief look at the Twin Delayed Deep Deterministic Policy Gradient (TD3) (Fujimoto et al., 2018a) and Soft Actor Critic (SAC) (Haarnoja et al., 2018) algorithms, which both build on Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2016), as they are used throughout the empirical work presented in this thesis.

Both TD3 and SAC are actor-critic algorithms, which interleave value function and policy updates. They are built for continuous state-action spaces and estimate state-action value functions. In addition, they use parameterized policies, as the policy update cannot be solved in closed form. Actor-critic algorithms therefore replace the closed form maximization with another gradient based update. In the case of TD3 and SAC, this is the DPG (Silver et al., 2014) which uses the fact that neural network based Q approximations are differentiable with regard to their inputs

$$\theta_\pi \leftarrow \theta_\pi + \alpha \nabla_{\theta_\pi} Q(s, \pi(x, \theta_\pi)). \quad (2.10)$$

While TD3 uses a deterministic policy $\pi : \mathcal{X} \rightarrow \mathcal{A}$, SAC uses a probabilistic Gaussian policy and computes the gradient using the re-parameterization trick

$$\nabla_{\theta} \mathbb{E}_{y \sim \mathcal{N}(\mu_{\theta}(x), \sigma_{\theta}(x))} [f(y)] \approx \frac{1}{N} \sum_{i=1}^N \left. \frac{d f(y)}{d y} \right|_{y=\mu_{\theta}(x)+\epsilon_i \sigma_{\theta}(x)} \left. \frac{d y}{d \theta} \right|_{y=\mu_{\theta}(x)+\epsilon_i \sigma_{\theta}(x)}, \quad (2.11)$$

where each ϵ_i is sampled independently from $\mathcal{N}(0, 1)$.

The main innovation of TD3 is the introduction of the twinned critic to counterbalance overestimation in the critic estimation. The overestimation problem stems from the fact that the actor is updated to maximize the critic’s prediction. If the critic estimate is noisy, the selected action’s value will in general be larger than the true value. Due to this, the

Bellman operator will propagate this overestimation into the Q function, which will inflate the Q values over time.

To counterbalance this, Fujimoto et al. (2018b) introduce two independent critic estimates Q_1 and Q_2 . The double critic’s loss function is

$$\mathcal{L}(Q_i|x, a, r, x', \pi) = \left(Q_i(x, a) - \left[r + \gamma \min_{j \in [1,2]} Q_j(x', \pi(x')) \right] \right)^2. \quad (2.12)$$

The actor update also maximizes this minimum over both critics

$$\mathcal{L}(\pi|x, Q_1, Q_2) = - \min_{j \in \{1,2\}} Q_j(x, \pi(x)). \quad (2.13)$$

With some assumptions on the error in both critics this max-min structure can be proven to prevent overestimation. However, as will be discussed in Chapter 3, these assumptions are not met in practice. Therefore, while this trick is helpful, by itself it is insufficient by itself to prevent overestimation. In addition to statistical error, the optimization process of the critic can be unstable and lead to overestimation.

SAC uses the double critic structure of TD3 and modifies the critic to optimize a maximum entropy objective. For this, the critic target is modified to include the stochastic policy’s entropy over the next state

$$\mathcal{L}(Q_i|x, a, r, x', \pi) = \left(Q_i(x, a) - \left[r + \gamma \min_{j \in [1,2]} Q_j(x', a') - \log \pi(a'|x') \right] \right)^2, \quad a' \sim \pi(\cdot|x'). \quad (2.14)$$

The policy objective function is similarly modified

$$\mathcal{L}(\pi|x, Q_1, Q_2) = - \min_{j \in [1,2]} Q_j(x, a) + \log \pi(a|x), \quad a \sim \pi(\cdot|x). \quad (2.15)$$

This maximum entropy modification can be interpreted in the framework of “control-as-inference” (Levine, 2018) and has been shown to be empirically more stable than TD3. However, it is also computationally slightly more expensive as it involves a sampling operation, and so several algorithms proposed in this thesis will use TD3 instead of SAC. Our insights are generalizable to most algorithms from the DDPG/TD3/SAC family.

2.4 Model-Based Reinforcement Learning

Many reinforcement learning algorithms compute their value functions and policies directly based on past data from the environment. In MBRL on the other hand, a predictive *model*¹

¹*Model* is a notoriously ambiguous term in machine learning. In the context of this thesis, the word *model* is used solely to describe an approximation of a transition function, reward function, or a similar

of the environment is trained and used in the RL algorithm. This model can be used to augment the learning, either by providing additional data as in the Dyna architecture (Sutton, 1990), or by providing gradient estimates (Hafner et al., 2020; Amos et al., 2021), or simply to enhance the representation learning as we will discuss in Chapter 4.

The most common kind of model used in MBRL is a *forward prediction model*. This is a parameterized function, such as a neural network, that takes a state-action pair as an input and predicts the next state. Simple extensions to these include multi-step models which use sequences of actions to predict short-horizon trajectories and stochastic models which parameterize distributions over next states.

Definition 2.4.0.1 (Types of environment models). *A probabilistic forward prediction model is a parameterized distribution $\hat{p}_\theta(x'|x, a)$ that approximates an MDP’s transition kernel.*

A deterministic forward prediction model is a function $f : \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{X}$ that directly maps to a next state.

A deterministic latent forward prediction model is a combination of an embedding function $\phi : \mathcal{X} \rightarrow \mathcal{Z}$ that maps from the state space $\mathcal{X}$ to a latent space $\mathcal{Z}$, and a latent dynamics model $f_{\mathcal{Z}} : \mathcal{Z} \times \mathcal{A} \rightarrow \mathcal{Z}$. Unless specified otherwise, we assume that $\mathcal{Z} = \mathbb{R}^k$, where k is the dimensionality of the latent space. Trajectories from a latent model are sampled conditioned on the embedding of an initial state x_0 .

All relevant definitions from Section 2.2 can be extended for environment models and we will generally use the notation $\hat{p}$ or $\hat{f}$ to differentiate a learned environment model from the ground truth.

Probabilistic latent environment models are also possible and are defined as one would expect via kernels over next latent states. We will revisit probabilistic latent dynamics models in the context of decision-aware learning in Chapter 6. There are several other types of models, such as inverse dynamics models which seek to predict actions given a state and its successor, but in this thesis, we will focus on the more common forward predictive models.

2.4.1 Training the model

Most commonly, models are trained by obtaining a dataset $\mathcal{D}$ of transition tuples consisting of states x , actions a , reward r , and next states x' . Often these datasets are obtained online during interaction with the environment, but they can also come from other offline sources such as human demonstrations.

object, while neural networks and other types of function approximators will be referred to as *function approximators* or simply *functions*.

Given a dataset, an approximate model $\hat{p}$ can be trained using a Maximum Likelihood Estimation (MLE) objective

$$\max_{\hat{p}} \frac{1}{N} \sum_{i=1}^N \log \hat{p}(x'_i | x_i, a_i). \quad (2.16)$$

For example, a Gaussian environment model, such as the ones used by Chua et al. (2018) and Janner et al. (2019) as well as in Chapter 5, uses the Gaussian negative log likelihood over a dataset as the loss function

$$\mathcal{L}_{\text{model}}(\mu_{\theta}, \Sigma_{\theta}; D) = -\frac{1}{N} \sum_{i=1}^N [\log \mathcal{N}(x'_i | \mu_{\theta}(x_i, a_i), \Sigma_{\theta}(x_i, a_i))]. \quad (2.17)$$

For deterministic models, a common objective is the Mean Squared Error (MSE)

$$\mathcal{L}_{\text{model}}(f_{\theta}; D) = \frac{1}{N} \sum_{i=1}^N (x'_i - f_{\theta}(x_i, a_i))^2. \quad (2.18)$$

These methods are commonly referred to as *reconstruction objectives* or *observation prediction* objectives (compare Chapter 4 for a thorough discussion), as the learned model predicts x' as it is provided by the environment or dataset. Later in this thesis, we will discuss *latent space models* more thoroughly as an alternative.

In addition to the next state, environment models are often trained to predict the reward as well, as the reward function is not generally assumed to be known. To do this, the model's output is extended by an additional scalar $\hat{r}_{\theta}(x)$ and we add $|r(x) - \hat{r}_{\theta}(x)|^2$ to the loss function. Unless noted otherwise, we will generally assume that the reward function is predicted using the model whenever the model is used.

2.4.2 Using the model

Of course learning a model is not sufficient on its own. If our ultimate goal is better agents, we also have to use the model in our RL algorithms. Here, we will very briefly survey common ways of using the model to improve different aspects of RL algorithms, which are also used in this thesis. However, many more ways of using a model have been proposed in the literature, and so interested readers are referred to the survey presented by Moerland et al. (2023).

Dyna

The Dyna framework (Sutton, 1990) is one of the most common ways of using an environment model in RL. The main idea is very simple: instead of only using the real

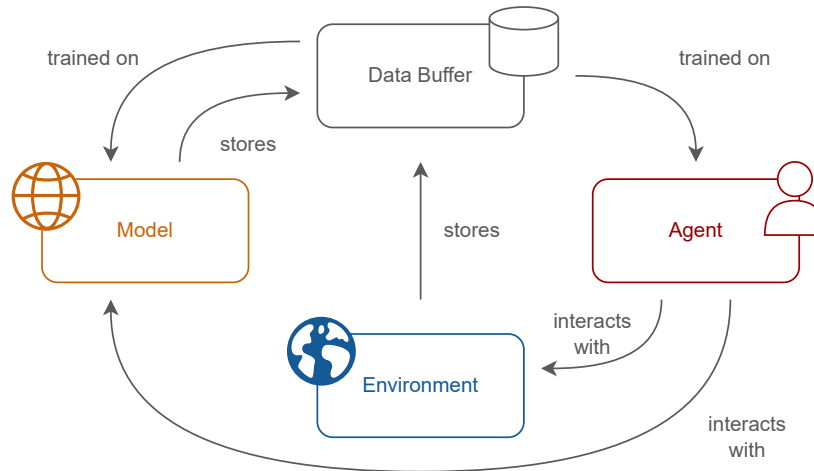


Figure 2.1: A sketch of the Dyna framework. The RL agent collects data from the real environment and stores this in a replay buffer. This replay buffer is used to train a model of the environment, which acts as a copy of the environment from the perspective of the agent. Algorithms differ in the question how long model generated data is stored in the replay buffer, with some, such as MBPO, keeping the model generated experiences in a separate replay buffer for a few thousand steps, while others, like the MAD-TD algorithm we introduce in [Chapter 7](#) using model-generated data only for a single gradient step before discarding it. While Janner et al. (2019) for example keeps model generated data for a few thousand update steps, the model-based algorithms presented in [Chapter 6](#) and [Chapter 7](#) do not store the data for multiple steps at all and generate it anew for each agent update.

environment for generating transition samples $\{x, a, r, x'\}$ we use the model as well. The concept is sketched out in [Figure 2.1](#). In this setup, the use of the model is mostly invisible to the RL algorithm which computes the value function and policy. This makes it very flexible, as we can freely vary the model training and RL algorithm without changing the overall framework.

There are several advantages to using the model in addition to the environment. The simplest is that we are able to generate synthetic data in addition to data from the real environment. If this data is similar enough to real data, this allows us to increase the sample efficiency of the RL training, by simply generating additional data. Whether this leads to strong performance improvements in applications however depends on how accurate we can make the model with limited data. If we need as many samples to obtain an accurate model as we need to learn a good value function or policy, the performance gains from Dyna might be limited.

An additional advantage of Dyna is that the model can serve as an oracle simulator. This means that we can get a next state x' for any x, a pair we query the model on. In the real environment we are instead forced into a sequential interaction where we cannot freely

choose the input state x . RL with an oracle generative model can be shown to be optimal in a minimax sense (Agarwal et al., 2020). However, the theoretical results rely on obtaining a maximum likelihood model with a well-specified model class. In practice, we are often not able to learn a sufficiently correct oracle model, and so again, gains in efficiency have to be weighed against model errors influencing the learned value and policy. We will discuss the impact of model errors on learning extensively in the next section.

As the Dyna framework is relatively flexible, many works in the literature propose algorithms which build on the general idea of learning a model and generating samples from it. We will review Dyna more closely in Chapter 5 and discuss related work. In Chapter 7, we will discuss how using Dyna and specifically the ability to query the model freely with visited states and on-policy actions can greatly stabilize value function learning.

Online Planning

While the Dyna framework focuses on improving the value and policies of the agent from stored experiences, a model can also be used to improve the policy as it is being executed. Depending on the structure of the MDP, proposed methods include random shooting (Chua et al., 2018), Monte-Carlo Tree Search (MCTS) (Silver et al., 2016; Schrittwieser et al., 2020), or Model Predictive Control (MPC) (Hansen et al., 2022; Hansen et al., 2024). While the details of the techniques vary, they all follow a similar base formula.

Starting from the current state x that the agent is in, the agent uses a search policy to query the world model for possible future trajectories. In the simplest case, random shooting, the agent simply collects n trajectories from the world model by adding random noise to its current policy, or drawing samples from a stochastic policy, and computes the Monte-Carlo estimate of the expected return with regard to the model. It then executes the most promising action according to these return estimates. MCTS and MPC can be seen as refinements of this simple techniques where the search procedure is refined by selectively searching over promising future states in the case of MCTS, or refining the initial action samples iteratively in the case of MPC.

In this thesis we will use MPC-based methods in Chapter 7 to improve upon a base policy using a learned model.

Other ways to use a model

Beyond learning with Dyna and planning, models can be incorporated into learning in a variety of other ways. Again, while a thorough survey of model-based reinforcement learning is out of scope for this thesis, we will highlight some additional ideas here as they are relevant for this thesis and related work.

Model-based representation learning: Instead of using the model directly as an oracle or simulator, a model learning loss can be used to improve the intermediate representations of a neural network-based agent. This technique is one example of learning with *auxiliary* tasks (Jaderberg et al., 2017). This idea will become an important aspect of this thesis, and will be the focus of Chapter 4. Therefore we forgo a lengthy introduction here and refer readers to the upcoming chapter.

First-order policy gradient estimation with a learned model: As neural networks have become the de-facto standard way to obtain an environment model in the current machine learning literature, we automatically obtain an additional useful algorithmic component from them: gradients. As most neural networks are end-to-end differentiable with regard to both their parameters and their inputs, we can use this fact to directly differentiate the model-based return estimate with regard to an action sequence or policy to improve learned policies by differentiating straight through a multi-step bootstrapped rollout. This technique is succinctly described in Amos et al. (2021) and used in works such as Hafner et al. (2020). Voelcker et al. (2022) discusses differentiable models in the context of decision-aware learning.

2.4.3 Abstract latent value models

Silver et al. (2017) introduced the idea of a purely abstract latent space model in which the transitions is aligned with the reward and value functions of the environment. Their work considers an uncontrolled setting, in which an action-independent Markov Reward Process (MRP) is modelled. The goal of their system is to learn the reward function of this process, which can be seen as doing policy evaluation in an MDP with a fixed policy. The core difference of their approach to other model-based reinforcement learning approaches is that they learn an abstract transition model where the states do not have a one-to-one correspondence to the environment observations.

Extending their work, Oh et al. (2017) show how to build an abstract model of a fully controllable MDP and highlight how such a model can be used for planning in latent space with a tree search approach such as MCTS (Schrittwieser et al., 2020) or beam search.

These abstract models will form the main backbone of our stable model-based architectures used in Chapter 6 and Chapter 7.

2.5 Objective mismatch phenomenon

One of the most important insights that form the basis of this thesis is the fact that model training objectives such as MLE are not aligned with the objective of *training a good model for RL* (Schneider, 1997; Joseph et al., 2013; Farahmand et al., 2017; Lambert et al., 2020).

As a simple example, assume a Dyna setting where a sample is collected from a deterministic model and has an error ϵ . A value function based method will use the model sample to compute a biased bootstrap target

$$Q_k(x, a) = r(x, a) + \gamma Q_{k-1}(f(x, a) + \epsilon). \quad (2.19)$$

The impact of the modelling error on the value function therefore depends both on the size of the error and the local behavior of the value function, and not only on the size of ϵ . We could colloquially say that not all errors are created equal.

As another illustrative example, we can imagine a value function that only depends on a subset of all state observation dimensions. In this case, a large error in an irrelevant dimension has no consequence on the obtained policy, yet a maximum likelihood loss for the model cannot properly capture this behavior without prior handcrafted features. Such cases are discussed in more detail in [Chapter 4](#) and [Chapter 5](#).

Intuitively, we would like to have an objective that bounds the difference in the value function estimate. We can motivate the use of an MLE-based loss function (e.g. the mean squared error for a Gaussian model with fixed variance) by an upper bound

$$\sup_{V \in \mathcal{F}} |\langle p - \hat{p}, V \rangle| \leq \|p - \hat{p}\|_1 \sup_{V \in \mathcal{F}} \|V\|_\infty \leq \sqrt{\text{KL}(p|\hat{p})} \sup_{V \in \mathcal{F}} \|V\|_\infty,$$

(taken from Farahmand et al. (2017)), where the second inequality is Pinsker’s inequality. However, this bound is loose and does not account for the geometry of the problem’s value function or any other knowledge that the agent has collected. In our example above a mean squared error would penalize deviations equally by their L_2 norm without accounting for the relevance of the dimensions. This problem was termed the *objective mismatch* by Lambert et al. (2020).

To refer to our goal of learning models which mitigate the mismatch, we will use the terms “decision-aware” and “value-aware” mostly interchangeably. Both terms were introduced by Farahmand et al. (2017). The former refers more generally to algorithms and models which account for the downstream decision task, while the latter refers more concretely to algorithms and models which account for errors in the value function target estimate. However, in this thesis our main focus is on value function learning, which means we mostly use decision-awareness in the more narrow sense of value-aware learning.

2.5.1 Value-aware model learning

To address the model mismatch, Farahmand et al. (2017) proposed *Value-aware Model Learning* (VAML), a loss function that captures the impact the model errors have on the

one-step value estimation accuracy. The core idea behind VAML is to penalize a model prediction by the resulting difference in a value function. Given a distribution over the state-actions space μ and a value function V , it is possible to define a value-aware loss function $\mathcal{L}_V(\hat{p}, p, \mu)$

$$\mathcal{L}_V(\hat{p}, p, \mu) = \int \mu(x, a) \left| \underbrace{\int p(x'|x, a) V(x') dx'}_{\text{environment value estimate}} - \underbrace{\int \hat{p}(x'|x, a) V(x') dx'}_{\text{model value estimate}} \right|^2 dx, a. \quad (2.20)$$

and its empirical approximation $\hat{\mathcal{L}}_V$ based on a dataset $D = (x_i, a_i, x'_i)_{i=1}^N$ of samples from μ and p

$$\hat{\mathcal{L}}_V(\hat{p}, D) = \frac{1}{|D|} \sum_{(x_i, a_i, x'_i) \in D} \left| V(x'_i) - \int \hat{p}(x'|x_i, a_i) V(x') dx' \right|^2. \quad (2.21)$$

The main problem of this approach is that it relies on the value function, which is not known a priori while learning the model. This leads to the "chicken-egg" problem of decision aware learning which will be the focus of [Chapter 5](#):

If a good model depends on knowing the correct decisions and good decisions need to be learned using the model, how can we learn a good model before knowing what the correct decision is?

In the original formulation by Farahmand et al. (2017), the value function is the supremum over a function space $\mathcal{V}$ to enable analysis

$$\mathcal{L}_{\text{VAML}}(\hat{p}, p, \mu) = \int \mu(x, a) \sup_{V \in \mathcal{V}} \left| \int p(x'|x, a) V(x') dx' - \int \hat{p}(x'|x, a) V(x') dx' \right|^2 dx, a. \quad (2.22)$$

While this evades the chicken-egg problem of decision-aware model learning and leads to tractable analysis in the case of linear value function spaces, finding a supremum for a function space parameterized by complex function approximators like neural networks is difficult. Furthermore, the supremum formulation is conservative and does not account for the fact that knowledge about the value function is gained over the course of exploration and optimization in an MBRL approach.

Instead of the supremum over a value function class, Farahmand (2018) introduced *Iterative Value-Aware Model Learning* (IterVAML) where the supremum is replaced with the current estimate of the value function. In each iteration, the value function is updated based on the model, and the model is trained using the loss function based on the last iteration's value function. The author presents error bounds for both steps of the iteration, but did not test

the algorithm to ascertain whether the presented error bounds are sufficient to guarantee a strong algorithm in practice. Furthermore, these work assume that both the Approximate Value Iteration and model learning procedure are conducted until they reach a small error at every step, which is often prohibitively expensive, or even impossible in case of neural network value function approximations. We will look at IterVAML more extensively in Chapter 5 and Chapter 6.

2.6 Representation learning in RL

The main goal of learning useful representations is to capture some desirable characteristic with regards to a prediction task in the latent space. This is often some form of smoothness or (Lipschitz)-continuity, or some optimal compression. Abel (2020) and Le Lan et al. (2021) provide excellent overviews of the general goals of representations in the context of RL.

One of the major components of the success of Deep Learning has been the surprising capability of neural networks to *learn* good representations for tasks such as classification (Bengio et al., 2013). One of the most prominent examples are the structured latent spaces obtained by the word2vec algorithm, where training to predict (embeddings of) words leads to structured spaces that allow for meaningful vector arithmetic (Mikolov et al., 2013; Goldberg and Levy, 2014).

The core challenges for representation learning in RL is that the tasks are fundamentally non-stationary in nature (Kumar et al., 2021; Nikishin et al., 2022b). As the agent explores the environment, value function and policy continually change, meaning that both the input distribution and the prediction targets change over time. This has led to the establishment of research into *learning dynamics* (Lyle et al., 2022b; Lyle et al., 2022a), *metric learning* (Ferns et al., 2011; Zhang et al., 2021; Le Lan et al., 2021; Kemertas and Jepson, 2022), *successor features* (Barreto et al., 2017; Borsa et al., 2019), and *auxiliary tasks* (Jaderberg et al., 2017; Bellemare et al., 2019; Lyle et al., 2021; Farebrother et al., 2023) to address these challenges.

Desirable properties of representations

Before discussing how representation learning in RL can be improved, it is necessary to characterize representations that lead to good performance in RL. As discussed above, in many classic RL approaches, the state space is taken to be a finite set of unordered states without any metric or distance between these except for the one induced by the transition function. Over such abstract states, one of the most commonly considered representations is a state aggregation. State aggregations are thoroughly discussed by Abel (2020), where

three core desiderata are established: good state aggregations should be easy to compute, enable efficient learning, and allow the selection of high-value policies.

Beyond state abstraction, the topology and metric of the representation space can also be considered. Le Lan et al. (2021) and Le Lan et al. (2022) discuss the capabilities of representations to enable generalization of learned value functions over the state space by analyzing the induced topology and metric space induced by different representation learning approaches. Their work allows us to consider the impact of the representation on the learning dynamics and generalization capabilities of different algorithms.

Ghosh and Bellemare (2020) extends the analysis of representation beyond fixed properties such as metric spaces to consider the impact of the given representation on the stability of the learning task. Using tools from linear dynamical system, they show that some proposed representations provide more stable learning dynamics for TD-learning as others.

General-purpose representation learning

To obtain optimal representations for reinforcement learning tasks in a given environment without a specific reward function, several different approaches have been proposed. In the nomenclature of this thesis, these are *general-purpose* representation learning methods. Several of the leading approaches are inspired from linear algebra and graph theory and seek to model fundamental aspects of the transition matrix.

Broadly speaking, the transition matrix $\mathcal{P}$ and the resolvent matrix $(I - \gamma\mathcal{P})^{-1}$ can be approximated by different decompositions such as eigenvalue decompositions, singular value decomposition, or Schur decomposition (Mahadevan, 2005; Mahadevan and Maggioni, 2007; Ghosh and Bellemare, 2020). These matrix approximations can then be used to obtain e.g. the largest eigenvectors which capture the long-term transition behavior of the underlying Markov chains. For example, the equality $V = (I - \gamma\mathcal{P})^{-1}r$ shows that if the resolvent matrix is well approximated, a representation can be obtained that can be used to approximate the value function of any reward. These decompositions and how they emerge from tractable prediction objectives will be discussed in detail in Chapter 4.

The goal of obtaining well-performing representations has also been addressed with empirically motivated approaches. Many of these seek to construct so called *auxiliary tasks* (Jaderberg et al., 2017), which are additional prediction objectives or loss functions which are added to the learning objectives of the RL agent. These generally do not take the reward or value prediction objectives into account. Such auxiliary tasks can be constructed using auto-encoder architectures (Jaderberg et al., 2017), contrastive learning (Laskin et al., 2020), or self-supervised objectives (Gelada et al., 2019; Schwarzer et al., 2021a; Schwarzer et al., 2021b; Tang et al., 2023). These methods tend to be especially important in cases with high-dimensional observations, i.e. as pixel based environments such as Atari games,

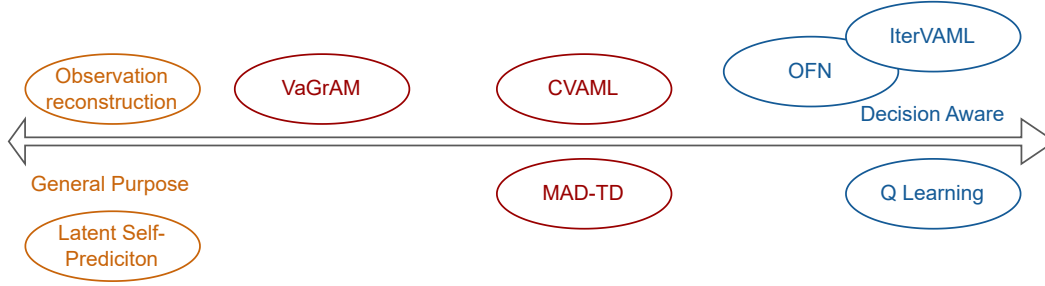


Figure 2.2: A visual representation of the spectrum between decision-aware and general purpose learning. Several approaches presented in this thesis, such as VaGrAM (Chapter 5), CVAML (Chapter 6), and MAD-TD (Chapter 7) combine methods from both general purpose and decision-aware learning to achieve stable and efficient RL. OFN (Chapter 3) only consider model-free RL and can therefore be seen as a purely decision-aware method. Finally, while latent self-prediction and observation reconstruction are general purpose methods, Chapter 4 investigates how they can be used in synergy with TD learning. Based on this investigation, latent self-prediction is used to obtain a strong hybrid methods such as MAD-TD.

or sparse rewards. The main distinction between these methods and MBRL is that the next state prediction is only used to improve the performance of an encoder function. Next state predictions are not directly used to improve the RL agents policy or value function estimation. We will analyze several types of auxiliary tasks in detail in Chapter 4.

Beyond state prediction tasks, several works have also shown the advantage of using value function prediction of auxiliary or randomly generated rewards as representation learning targets (Lyle et al., 2021; Farebrother et al., 2023). These commonly use the same approaches as off-policy value function learning, but instead of using the estimated value functions to derive a policy, the results are again discarded and only used for stabilization. As we will discuss in Chapter 4, many of these approaches lead to features which are closely related to features obtained from self-supervised predictions.

2.7 Decision-aware vs general-purpose learning

The major problem that motivates the work conducted in this thesis is that the current value function alone does not necessarily provide sufficient information to learn good representations or world models. As discussed in the introduction, reinforcement learning is fundamentally non-stationary in nature, which means that the current prediction task is not necessarily indicative of future prediction tasks.

To understand the problem intuitively, a sparse reward setting is instructive: when the task requires exploration, the value function is driven to predict 0 for all visited states until the agent has actually reached the goal and observed positive reward. A model or representation

can then collapse the whole state space into a single feature, as no value other than 0 needs to be predicted, and potentially completely prevent further exploration.

This phenomenon is alluded to by Tomar et al. (2023), who show that many techniques for value-based abstractions fail in hard-exploration scenarios. Similarly, Kemertas and Aumentado-Armstrong (2021) show that auxiliary training objectives strongly improve the performance of bisimulation-based representation learning, especially with sparse rewards. A related discussion in the the context of IterVAML will be presented in Chapter 5. Finally, Nikishin et al. (2022b) conjecture the existence of a *primacy bias*, the tendency of neural network-based value functions to overfit initial experience and to lose capacity to represent information obtained later in training. This notion of the primacy bias will be addressed in detail in Chapter 3 and Chapter 7, and we will see that the primacy bias can be related to the problems of learning representations and models from purely decision-aware objectives.

However, while decision-aware methods face challenges with stability, their great advantage lies in the fact that they can adapt to the task. While general-purpose methods are often more robust to shifts in the task and missing value information, they fail to be as efficient as the latter especially under resource constraints such as network size. Examples of this are presented in Chapter 4 and Chapter 5 where we see the benefits of using decision-aware algorithms when the observation space contains irrelevant distracting dimensions. Therefore, instead of exploring decision-awareness and general-purpose learning as opposing ideas, this thesis is instead looking to find synergy which allows us to take advantage of both paradigms. This idea is visualized in Figure 2.2, where the approaches presented in this thesis are roughly visualized along a spectrum between general-purpose and decision-aware learning.

Chapter 3

Instability of Value Function Learning

This chapter is based on Marcel Hussing, Claas Voelcker, Igor Gilitschenski, Amir-massoud Farahmand, and Eric Eaton (2024). “Dissecting Deep RL with High Update Ratios: Combatting Value Divergence”. In: *Reinforcement Learning Conference*.

To improve sample efficiency in off-policy, value-based RL contemporary work has begun increasing the number of gradient updates per collected environment step (Janner et al., 2019; Fedus et al., 2020; Chen et al., 2021; Hiraoka et al., 2022; Nikishin et al., 2022b; D’Oro et al., 2023; Schwarzer et al., 2023; Kim et al., 2023). However, as this update-to-data (UTD) ratio increases, value function learning seems to become unstable. This is perhaps surprising, as neural networks have shown themselves to be generally robust to the number of update steps in other areas such as supervised classification. In many cases, training for a very large number of steps actually increases the performance of neural networks, due to phenomena such as grokking (Power et al., 2022) or double descent (Belkin et al., 2019). As such, this phenomenon should raise our suspicion that training DRL agents causes issues with neural networks that are not observed in other domains.

Notably, a recent study conjectured the emergence of a *primacy bias* in deep actor critic algorithms, defined as “a tendency to overfit initial experiences that damages the rest of the learning process” (Nikishin et al., 2022b). This is a fairly broad explanation of the phenomenon, leaving room for investigation into *how* fitting early experiences causes sub-optimal learning behavior.

First approaches to tackle the learning failure challenges have been suggested, such as completely resetting networks periodically during the training process and then retraining

them using the contents of the replay buffer (Nikishin et al., 2022b; D’Oro et al., 2023). Resetting network parameters is a useful technique that, in some sense, can circumvent any ongoing optimization failures. Yet it seems likely that a more nuanced treatment of the various optimization challenges in deep RL might lead to more efficient training down the line, and provide more useful insights into how learning in DRL evolves more generally. Especially for achieving efficient learning, throwing away all learned parameters and starting from scratch periodically is counter-productive, for instance in scenarios where, keeping all previous experience is infeasible. As such, we set out to study the components of early training that impair learning more closely and examine whether high-UTD learning without resetting is possible.

To motivate our study, we repeat the priming experiment of Nikishin et al. (2022b), in which a network is updated for a large number of gradient steps on limited data. We show that during priming stages of training, value estimates diverge so far – and become so extreme – that it takes very long to unlearn them using new data. However, contrary to prior work, we find that it is not *impossible* to learn even after priming, it merely takes a long time and many samples to start achieving non-trivial returns. This sparks hope for our endeavor of smooth learning in high-UTD regimes. We show that preventing value function divergence allows learning to proceed. This suggests that the failure to learn does not stem from overfitting early data, which would result in correct value function on seen data, but rather from improperly fitting Q-values. We demonstrate that this divergence is most likely caused by prediction of out-of-distribution (OOD) actions that trigger large gradient updates, which destabilize feature learning. Empirically, we find that instead of settling into fixed representations, the norm of the learned representations in the intermediate layers of the critic neural network grows.

The identified behavior, although triggered by OOD action prediction, seems to be more than the well-known overestimation due to statistical bias (Thrun and Schwartz, 1993). Instead, we hypothesize that the problem is an optimization failure and focus on mitigating the unbounded growth of the feature norm with architectural changes. The main evidence for this hypothesis is that standard RL approaches to mitigating bias, such as minimization over two independent critic estimates (Fujimoto et al., 2018b), are insufficient. In addition, using pessimistic updates (Fujimoto et al., 2019; Fujimoto and Gu, 2021) or regularization (Krogh and Hertz, 1991; Srivastava et al., 2014) to treat the value divergence can lead to suboptimal learning behavior, which is why architectural improvements are preferable in many cases.

We show that a simple feature normalization method, unit-ball projection (Bjorck et al., 2022), is sufficient to prevent catastrophic overestimation due to divergence. Unit-ball projection decouples learning the scale of the values from the first layers of the network and moves it to the last linear layer. Empirically, this approach mitigates diverging Q-

values in the priming experiment. Even after a large amount of priming steps, the agent immediately starts to learn. In a set of experiments on the `dm_control` MuJoCo benchmarks (Tunyasuvunakool et al., 2020), we show that accounting for value divergence can achieve significant across-task performance improvements when using high update ratios. Moreover, we achieve non-trivial performance on the challenging dog tasks that are often only tackled using model-based approaches. We demonstrate comparable performance with the recently developed TD-MPC2 (Hansen et al., 2024), without using models or advanced policy search methods.

3.1 Preliminaries

To fully elucidate the role that increasing the update ratio plays in destabilizing value function training, ostensibly necessitating resets, we present a series of small-scale experiments.

3.1.1 Investigating large update-to-data ratios during priming

As mentioned, the definition of the primacy bias is broad. To obtain a more nuanced understanding, we set out to re-investigate the early stages of high-UTD training. To do so, we repeat the priming experiment conducted by Nikishin et al. (2022b).

We first collect 10,000 environment samples with a random policy. Then, using the SAC algorithm, we update the network during the priming phase. This means that we train the agent for a large number of gradient update steps without collecting additional data. After the priming phase, training continues as usual, interleaving gradient updates and environment interaction steps.

Prior results reported by Nikishin et al. (2022b) suggest that once the priming step has happened, agents lose their ability to learn. They suggest to reset the trained networks to recover the ability to improve. Here, we repeat this experiment using the simple Finger-spin task (Tunyasuvunakool et al., 2020) to study the root causes for this failure and to examine if there are ways to recover without resets. In this section, we report means over five random seeds with bootstrapped confidence intervals at 95% in shaded regions. Hyperparameters are kept consistent with previous work for ease of comparison.

We first ask whether there is a barrier as to how many update steps can be executed in the priming phase before the agent becomes unable to improve in the regular training phase. We test this by collecting an initial batch of 1,000 samples and varying the number of updates during priming from 25,000 to 50,000 and 100,000. The results are presented in Figure 3.1.

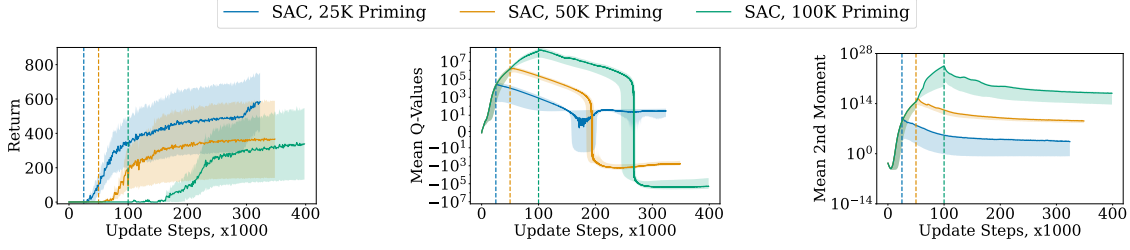


Figure 3.1: Return, in-distribution Q-values and Adam optimizer moments during priming for different lengths. Dotted lines correspond to end of priming. More priming steps lead to lower return and larger Q-values.

We make two key observations. First, lower amounts of priming are correlated with higher early performance. More precisely, it seems that many runs simply take longer before they start achieving non-zero returns as the number of priming steps increases. Second, during priming the scale of the average Q-value estimates on observed state-action pairs increases drastically. We find that the Q-values begin close to the achieved returns, but as priming goes on they eventually start to diverge drastically. Once the agent estimates very large Q-values, the final performance in terms of average returns deteriorates.

3.1.2 On the potential causes of divergence

We conjecture that Q-value divergence starts with overestimated values of OOD actions. This overestimation could cause the optimizer to continually increase Q-values via its momentum leading to divergence. To test this hypothesis, we add a conservative behavioral cloning (Pomerleau, 1988; Atkeson and Schaal, 1997) loss term to our actor that forces the policy to be close to replay buffer actions. Prior work employed this technique in offline RL to mitigate value overestimation (Fujimoto and Gu, 2021). More formally, our actor update is extended by the loss $\mathcal{L}_{c,\phi} = \min_{\phi} \mathbb{E}_{a \sim D, \hat{a} \sim \pi_{\phi}(x)} [\|a - \hat{a}\|_2]$. The results in Figure 3.2 indicate that the basis of the conjecture is corroborated as divergence is much smaller—but

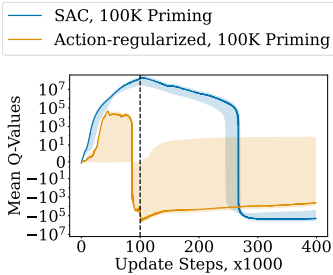


Figure 3.2: Priming with SAC and action regularization during priming. The latter lowers divergence.

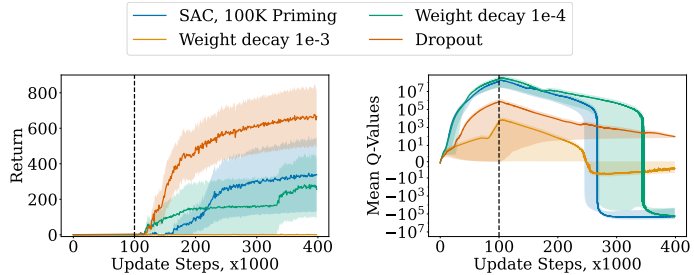


Figure 3.3: Return and Q-values of priming runs with weight decay and dropout. Results indicate that both regularizers mitigate priming to some extent but not sufficiently.

not mitigated completely—when values are learned on actions similar to seen ones. We will revisit this hypothesis in [Chapter 7](#). However, in practice we do not know when divergence sets in, which limits the applicability of this technique in realistic scenarios. Using it throughout all of training, rather than just during priming, impairs the learner’s ability to explore.

3.1.3 Applying common regularization techniques

Regularization is a common way to mitigate gradient explosion and is often used to address overestimation (Farebrother et al., 2018; Chen et al., 2021; Liu et al., 2021; Hiraoka et al., 2022; Li et al., 2023). We investigate the priming experiments under techniques such as using weight decay (Krogh and Hertz, 1991) or adding dropout (Srivastava et al., 2014) to our networks in [Figure 3.3](#).

Both weight decay as well as dropout can somewhat reduce the divergence during priming, but not to a sufficient degree to allow for fast learning afterwards. While we completely fail to attain strong performance with weight decay, dropout is able to recover a good amount of final return. However, both methods require tuning of a hyperparameter that trades off the regularization term with the main loss. This hyperparameter is environment-dependent and tuning it becomes infeasible for large UTD-ratios due to computational resource limitations. Still, the results imply that it is in fact possible to overcome the divergence in priming and continue to learn good policies.

3.1.4 Divergence in practice

One question that remains is whether we can find these divergence effects outside of the priming setup. We find that, while priming is an artificially constructed worst case, similar phenomena happen in regular training on standard benchmarks when increasing update ratios (see [Figure 3.4](#)). Further, the divergence is not limited to early stages of training as it happens at arbitrary points in time.

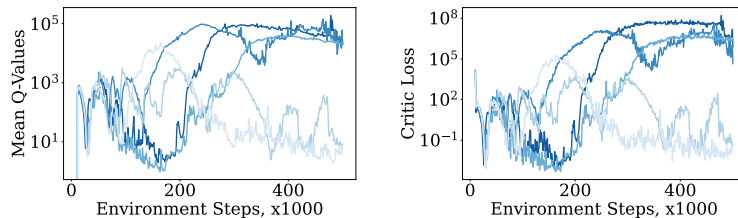


Figure 3.4: In-distribution Q-values and critic loss of five SAC seeds on the humanoid-run task using $UTD = 32$. Values diverge at arbitrary time-points, not only during the beginning. Loss mirrors Q-value divergence.

We therefore conjecture that divergence is not solely a consequence of the amount of experience gathered but rather happens due to state-action space coverage. We will revisit and test this conjecture in [Chapter 7](#). Note that the reported Q-values have been measured on the training data, not on a separate validation set. The respective critic losses become very large. All this points toward a failure to fit Q-values. This behavior also does not align with our common understanding of overfitting (Bishop, 2006), challenging the hypothesis that high-UTD learning fails merely due to large validation error (Li et al., 2023).

3.2 Towards high-UTD optimization without resetting

Regularization techniques such as those in [Section 3.1.3](#) can fail to alleviate divergence as they tend to operate across the whole network and lower the weights everywhere even if higher values are actually indicated by the data. They also require costly hyperparameter tuning. Thus, we turn towards network architecture changes to the commonly used fully-connected feed-forward networks that have proven useful in overcoming issues such as exploding gradients in other domains (Ba et al., 2016; Xu et al., 2019).

3.2.1 Limiting gradient explosion via unit-ball normalization

As discussed previously, the prediction of an unknown action might trigger the propagation of a large, harmful gradient. Further, the Q-values of our network ought to grow over time as they more closely approximate those of a good policy. If we predict incorrectly on one of these Q-values, a potentially very large loss is propagated. Gradients are magnified by multiplicative back-propagation via ReLU activations (Glorot et al., 2011) as well as momentum from Adam (Kingma and Ba, 2015). Note that all resulting issues arise in the early network layers. We hypothesize that we can address many of these problems by separating the scaling of the Q-values to the appropriate size from the earlier non-linear layers of the network and moving the Q-value scaling to the final linear layer.

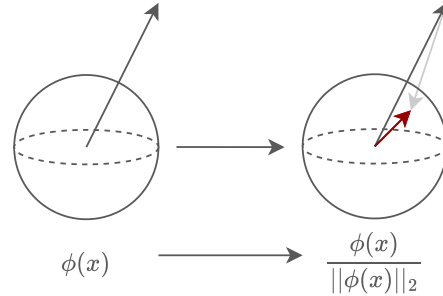


Figure 3.5: A graphical representation of output feature normalization (OFN). Each feature vector is projected unto the unit sphere.

One contender to achieve the value decoupling described in the previous paragraph is layer normalization (Ba et al., 2016), but one would have to disable scaling factors used in common implementations. Still, standard layer normalization would not guarantee small features everywhere. Instead, we use a stronger constraint and project the output features

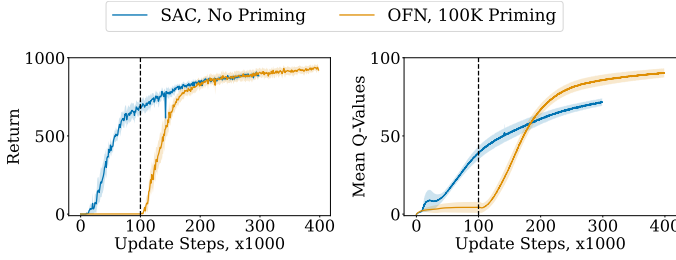


Figure 3.6: (Left) Return and (Right) Q-values comparing SGD result and OFN when priming for 100K steps. OFN obtains returns close to that of the well-trained SGD agent and learns an appropriate Q-value scale correctly.

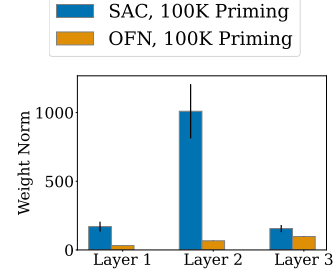


Figure 3.7: L_2 norm of network weights per layer after priming for default and OFN architectures. OFN leads to smaller weights, especially for earlier layers.

of the critic encoder onto the unit-ball using the function $f(\mathbf{z}) = \frac{\mathbf{z}}{\|\mathbf{z}\|_2}$ (Zhang and Sennrich, 2019), where $\|\cdot\|_2$ denotes the L_2 vector norm and $\mathbf{z}$ is the output of our encoder $\phi(x, a)$. This ensures that all values are strictly between 0 and 1 and the gradients will be tangent to the unit sphere. Note that this function’s gradient is not necessarily bounded to ensure small gradients for prior layers (see Appendix B.3), but we observe that empirically that gradient explosion does not occur. The unit-ball has previously been used to mitigate large action prediction in the actor (Wang et al., 2020) or to stabilize RL training in general (Bjorck et al., 2022). Notably, while Bjorck et al. (2022) discusses the reasons for using unit-ball regularization for policy learning, they present results for using unit-ball normalization for value learning without further discussion. Therefore our work here can be seen as a thorough analysis of a previously unsupported ad-hoc trick. For brevity, we will refer to this method as *output feature normalization* (OFN). We solely apply OFN to the critic, unlike Wang et al. (2020), since our goal is to mitigate value divergence. OFN is very simple and requires only a one-line change in implementation.

3.2.2 Evaluating feature output normalization during priming

To test the efficacy of the OFN-based approach, we repeat the priming experiment in Figure 3.6. We find that OFN achieves high reward and Q-value divergence during priming is fully mitigated. We therefore expect the average Q-values to be roughly at 10% of the undiscounted return which is roughly achieved by the OFN network. In addition, as shown in Figure 3.7, the weight norm of all layers of the networks are smaller, with the largest weights in the final layer.

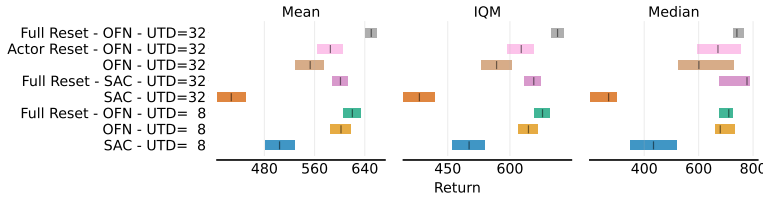


Figure 3.8: Mean, interquartile mean (IQM), and median with 95% bootstrapped confidence intervals of standard SAC and OFN on the DMC15-500k Suite. OFN can maintain high performance even under large UTD. OFN with UTD = 8 achieves comparable performance to standard resetting with UTD = 32 across metrics.

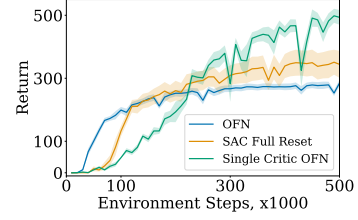


Figure 3.9: Mean return of single-critic OFN, standard OFN and resetting; UTD = 32 on hopper-hop. Shaded regions are bootstrapped confidence intervals.

3.3 Experimental evaluation

We evaluate our findings on the commonly used `dm_control` suite (Tunyasuvunakool et al., 2020). All results are averaged over ten random seeds.¹ We report evaluation returns similar to Nikishin et al. (2022b), which we record every 10,000 environment steps. We compare a standard two-layer fully-connected feed-forward network with ReLU (Nair and Hinton, 2010) activations, both with and without resetting, to the same network with OFN. The architecture is standard in many reference implementations. Architecture and the resetting protocol are taken from D’Oro et al. (2023) and hyperparameters are kept without new tuning to ensure comparability of the results. More details can be found in Appendix B.1.

To understand the efficacy of output normalization on real environments under high UTD ratios, we set out to answer multiple questions that will illuminate RL optimization failures:

Q1: Can we maintain learning without resetting neural networks?

Q2: Are there other failure modes beside Q-value divergence under high UTD ratios?

Q3: When resets alone fall short, can architectural changes enable better high-UTD training?

3.3.1 Feature normalization stabilizes high-UTD training

To answer **Q1**, we compare OFN and SAC with resets on the DMC15-500k benchmark with large update ratios of 8 and 32 as proposed by Nikishin et al. (2022b) and Schwarzer et al. (2023). We report mean, interquartile mean (IQM) and median as well as 95% bootstrapped

¹For comparison with TD-MPC2 (Hansen et al., 2024) we use data provided by their implementation, which only contains three seeds. As the goal is not to rank algorithmic performance but to give intuition about the relative strengths of adapting the network architecture, we believe that this is sufficient in this case.

confidence intervals aggregated over seeds and tasks, following Agarwal et al. (2021). The results are shown in Figure 3.8.

First, we observe that in both cases, $UTD = 8$ and $UTD = 32$, OFN can significantly improve over the non-resetting baseline across all metrics. The value estimates that diverge seem to have been handled properly (see Appendix B.2.2); learning is maintained. We note that our approach with $UTD = 8$ achieves mean and IQM performance comparable to that of standard resetting with $UTD = 32$. In median and quantile performance, all $UTD = 32$ overlap, highlighting that outliers contribute to the performance measurement. Note that the overall performance drops slightly for the OFN-based approach when going from $UTD = 8$ to $UTD = 32$. We conjecture that there are other learning problems such as exploration that have not been treated by alleviating value divergence. However, these do not lead to complete failure to learn but rather slightly slower convergence.

3.3.2 Other failure modes: Exploration limitations

To validate the hypothesis of other failures and answer **Q2**, we run two additional experiments. First, our current focus is on failures of the critic; our proposed mitigation does not address any further failures that might stem from the actor. To understand the actor’s contribution to the performance increases from resetting, we test the OFN-based architecture again and simply reset the actor. For comparison, we also run a second experiment in which we reset all learned parameters, including the critic. This settings are referred to as “Actor Reset” and “Full Reset” respectively in Figure 3.8.

The results in Figure 3.8 indicate that actor resetting can account for a meaningful portion of OFN’s performance decline when going from $UTD = 8$ to $UTD = 32$. The actor-reset results are within variance of the full-resetting standard baseline. Further, we observe that there is still some additional benefit to resetting the critic as well. This does not invalidate the premise of our hypothesis, value divergence might not be the *only* cause of problems in the high UTD case. We have provided evidence that it is a *major* contributor. Resetting both networks of OFN with $UTD = 32$ outperforms all other baselines on mean and IQM comparisons.

To explain the remaining efficacy of critic resets, we examine the hopper-hop environment where standard SAC with resets outperforms OFN. In RL with function approximation, one might not only encounter over- but also under-estimation (Wu et al., 2020; Lan et al., 2020; Saglam et al., 2021). We believe that hopper is sensitive to pessimism, and periodically resetting the networks might partially and temporarily counteract the inherent pessimism of the dual critic setup.

To corroborate this conjecture, we repeated some experiments with a single critic. As OFN handles divergence it might not require a minimization over two critics (Fujimoto

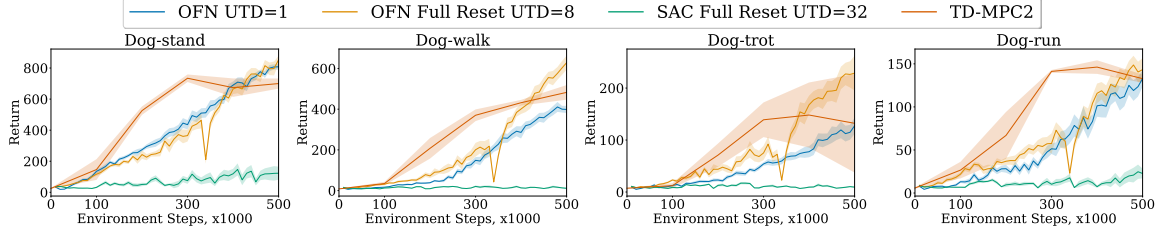


Figure 3.10: Mean return on the dog DMC tasks, comparing OFN to SAC with resets and the model-based TD-MPC2. Shaded regions indicate bootstrapped confidence intervals. OFN outperforms SAC with resets, which is unable to learn and OFN with $UTD = 8$ and resetting is competitive with TD-MPC2.

et al., 2018b). We compare OFN using a single critic and 32 updates per environment step to standard SAC and OFN in Figure 3.9. With a single critic, OFN does not get stuck in a local minimum and outperforms full resetting. Note that this is only true in few environments, leading us to believe that the effects of high-update training are at least in part MDP-dependent. In some environments we observe unstable learning with a single critic, which highlights that the bias countered by the double critic optimization and the overestimation from optimization we study are likely orthogonal phenomena that both need to be addressed.

3.3.3 Limit-testing feature normalization

To answer **Q3**, we move to a set of training environments which are considered especially hard for model-free approaches, namely the dog tasks of the DMC suite. Standard SAC can generally not obtain any reasonable reward and, due to their complex dynamics, these tasks are often tackled using model-based approaches such as TD-MPC2 (Hansen et al., 2024) with complicated update procedures and carefully tuned network architectures. We evaluate SAC and OFN on the dog tasks and compare against TD-MPC2 in Figure 3.10.

First, observe that resetting without OFN does not result in only minor performance increases over the course of training. However, OFN with $UTD = 1$ can already obtain very good performance across all tasks, indicating that a major problem for SAC in these high-dimensional tasks is value divergence. When increasing the update ratio to 8 and adding resetting, we improve the performance of the OFN agent even further and can match the reported results of the strong model-based TD-MPC2 baseline.

We have already seen that resetting can take care of multiple optimization failures. However, these experiments also indicate that resetting is not a panacea as it is only effective when the initially learned policy can obtain some reward before being overwritten. This seems intuitive since resetting to a policy that cannot gather any useful data should not help. These results highlight that the early training dynamics of RL are highly important when

it comes to training on complex environments and fitting early data correctly and quickly is crucial for success.

This also opens up the question why resetting in the humanoid environments in the previous sections can yield success even though very little reward is observed. Besides greater divergence due to larger observation spaces in the dog MDPs, we suspect that this might be related to the complexity of exploration. The ability of a random policy to obtain non-trivial reward and information about the environment has been shown to be a crucial factor in explaining the success of DRL methods in discrete environments (Laidlaw et al., 2023), and similar phenomena might be in effect here.

3.4 Related work

Our work closely examines previous work on the primacy bias and the related resetting technique (Anderson, 1992; Nikishin et al., 2022b; D’Oro et al., 2023; Schwarzer et al., 2023). Going beyond, overestimation and feature learning challenges are a widely studied phenomenon in the literature.

Combatting overestimation Overestimation in off-policy value function learning is a well-established problem. Discussions in the RL literature date back far before the prime times of deep learning (Thrun and Schwartz, 1993; Precup et al., 2001). With the rise of deep learning, researchers have tried to address the overestimation bias via algorithmic interventions such as combining multiple Q-learning predictors to achieve underestimation (Van Hasselt, 2010; Van Hasselt et al., 2016a; Zhang et al., 2017; Lan et al., 2020), using averages over previous Q-values for variance reduction (Anschel et al., 2017), or general error term correction (Lee et al., 2013; Fox et al., 2016). In the context of actor-critic methods, the twinned critic minimization approach of Fujimoto et al. (2018b) has become a de-facto standard. Most of these approaches are either computationally expensive, such as training Q function ensembles, or break down under very high update ratios, such as in the case of the pessimistic twinned Q learning trick. To regulate the overestimation-pessimism balance more carefully, several authors have attempted to use larger ensembles of independent Q-value estimates (Lee et al., 2021; Peer et al., 2021; Chen et al., 2021; Hiraoka et al., 2022). Ensembling ideas were also combined with ideas from distributional RL (Bellemare et al., 2017) to combat overestimation (Kuznetsov et al., 2020). Instead of addressing the statistical bias in deep RL, our study focuses on the problems inherent to neural networks and gradient based optimization for value function estimation. Work from offline-to-online RL has demonstrated that standard layer normalization can bound value estimates during offline training and mitigate extrapolation while still allowing for exploration afterwards (Ball et al., 2023). Layer normalization has subsequently been used to achieve generally strong results in offline RL (Tarasov et al., 2023). Our work is also related to a recent contribution

using batch-normalization for increased computational efficiency by Bhatt et al. (2024) who focus on decreasing update ratios rather than increasing them. A concurrent work by Nauman et al. (2024a) provides a large scale study on different regularization techniques to combat overestimation. This work also demonstrates the efficacy of SAC on the dog tasks when properly regularized but it does not highlight the effects of Q-value divergence from exploding gradients as a key challenge for this set of environments.

Combating plasticity loss Another aspect of the primacy bias is the tendency of neural networks to lose their capacity for learning over time (Igl et al., 2021), sometimes termed *plasticity loss* (Lyle et al., 2022a; Abbas et al., 2023). Recent work mitigates plasticity loss using feature rank maximization (Kumar et al., 2021), regularization (Lyle et al., 2023), or learning a second copied network (Nikishin et al., 2024). Some of the loss stems from neurons falling dormant over time (Sokar et al., 2023). A concurrent, closely related work by Lyle et al. (2024) disentangles the causes for plasticity loss further. They use layer normalization to prevent some of these causes, which is closely related to our unit-ball normalization. Our work differs in that we focus on the setting of high update ratios and use stronger constraints to mitigate value divergence rather than plasticity loss.

3.5 Conclusion

By dissecting the effects underlying the primacy bias, we have identified a crucial challenge: *value divergence*. While the main focus in studying increased Q-value has been on the statistical bias inherent in off-policy sampling, we show that Q-value divergence can arise due to problems inherent to neural network optimization. This optimization-caused divergence can be mitigated using the unit-ball normalization approach, which shines on the `dm_control` benchmark with its simplicity and efficacy. With this result, we challenge the assumption that failure to learn in high-UTD settings primarily stems from *overfitting* early data by showing that combating value divergence is competitive with resetting networks. This offers a starting point towards explaining the challenges of high-UTD training in more detail and opens the path towards even more performant and sample efficient RL in the future.

This chapter shines a light on the important role that the implicit representations learned from end-to-end critic optimization play in value function learning. In essence, model-free value learning is a purely *decision-aware* method, as it solely focuses on extracting good features for the current value prediction target. But due to the deep entanglement between optimization, exploration, and non-stationary data, we can see that relying solely on end-to-end learning for value estimation is a potentially perilous endeavour. While simple regularization can already alleviate some of the issues with value learning, the next chapter takes an in-depth look at more explicit ways of constructing good representative features.

Chapter 4

Understanding Auxiliary Tasks for Value Function Learning

This chapter is based on Claas Voelcker, Tyler Kastner, Igor Gilitschenski, and Amir-massoud Farahmand (2024c). “When does self-prediction help? Understanding Auxiliary Tasks in Reinforcement Learning”. In: *Reinforcement Learning Conference*.

4.1 Introduction

As the previous chapter shows, end-to-end critic training can cause problems with learning good intermediate neural network representations. While regularization can alleviate some of the optimization failures, it only addresses issues of divergence, and does not ensure that relevant information is properly represented in a decision-aware fashion. To achieve this, we now take a look at *auxiliary tasks* which directly ensure that transition information is well represented in learned features.

Since the emergence of deep learning, techniques for deep supervised learning have been successfully incorporated into reinforcement learning (RL) agents (Mnih et al., 2013; Lillicrap et al., 2016). However, as we have seen, the RL setting contains additional complications such as non-stationary optimization target and the reliance on bootstrapping. These hurdles generally add instability to the RL training process, and as discussed, *the failure to learn good features* is a central problem in deep RL (Kumar et al., 2021; Lyle et al., 2022a; Nikishin et al., 2022b; Hushing et al., 2024).

To mitigate this failure, one common approach is to add auxiliary tasks to the learning objective (Jaderberg et al., 2017). Popular examples include predicting next state observations (Jaderberg et al., 2017) and predicting functions of the next state, or latent self-prediction,

(Schwarzer et al., 2021a; Ni et al., 2024). To understand the performance of these approaches, recent literature (Tang et al., 2023; Le Lan et al., 2023) considers the *learning dynamics* of auxiliary task learning in simple linear surrogate models (Saxe et al., 2014). Findings in the literature show that observation reconstruction should provide better features than latent self-prediction (Behzadian et al., 2019; Tang et al., 2023). However, this causes a theory-practice gap as empirical work has found that latent self-prediction outperforms observation reconstruction across many benchmarks (Schwarzer et al., 2021a; Ni et al., 2024). This suggests that the assumptions made in Behzadian et al. (2019) and Tang et al. (2023) might not match the setting that is commonly investigated in the empirical literature.

To address this gap, we pose two questions: (a) *How do auxiliary losses behave when combined with a TD loss?* Tang et al. (2023), Le Lan et al. (2022), and Tang and Munos (2023) have studied the learning dynamics of auxiliary tasks alone, evaluating their performance without addressing the interaction between the auxiliary task and the main goal, to learn a (correct) value function. (b) *How can we describe the behavior of auxiliary losses in the presence of distractions (states and transition dynamics irrelevant for the reward) and observation functions (different ways to measure the underlying state)?* MDP structures like distractions have been hypothesized to lead to differing performance between different auxiliary tasks (Ni et al., 2024), but to our knowledge no theoretical study has been established.

Section 4.2 we briefly review notation and provide a formal model of the studied representation losses. In Section 4.3, we present a formalization of distractions and observation functions. We use the framework of *factored MDPs* (Boutilier et al., 2000) with Kronecker products (Mahadevan, 2009) to represent a common class of distractions. To model observation functions, we use *linear reparametrization* as a tractable way to go beyond one-hot representations.

In Section 4.4 and Section 4.5 we analyze the features learned with observation prediction and latent self-prediction alone and in combination with TD learning. We also show how these stationary features change with the introduction of distractions and observation functions. From this analysis we find that latent self-prediction is a strong *auxiliary task*, while observation prediction is a strong feature learning method when used *alone*. The differences are highlighted in Figure 4.1. This bridges one of the biggest gaps between previous analysis of learning dynamics and empirical results.

In Section 4.6 we test the predictions derived from our theoretical framework by evaluating feature learning losses in the MinAtar suite (Young and Tian, 2019)¹. We design ablations

¹All code for our experiments is available at https://github.com/adaptive-agents-lab/understanding_auxiliary_tasks.

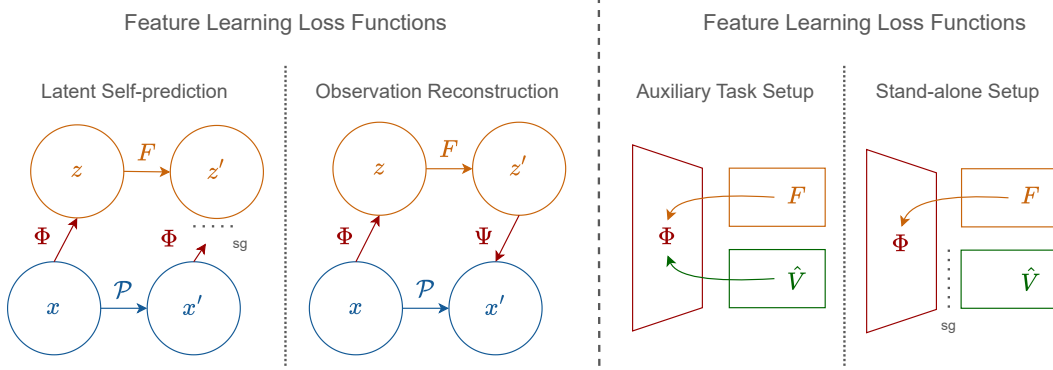


Figure 4.1: Diagram of the discussed loss functions and use cases. In latent self-prediction, we aim to predict the next state’s *features*, computed with an embedding function $\phi(x')$ using the state’s features $\phi(x)$ and a latent prediction model F . In observation reconstruction, the next state’s *ground truth observations* x' are matched via the use of a decoder function $\psi(F(\phi(x)))$. In the *auxiliary task setup*, both the gradients from the feature learning loss and value function learning are propagated to the encoder, while in the *stand-alone scenario*, only the gradients from the feature learning loss are used to update Φ .

that mirror both our formalization and previous approaches to test distraction robustness in empirical environments. The theory partially predicts the performance differences in the test suite, validating that the insights we obtain from the simple linear surrogate models used for analysis are useful for practitioners. However, as we have an incomplete understanding of the underlying benchmark problems themselves (Voelcker et al., 2024b), more research is needed to fully characterize the interaction of the environment dynamics with the stabilizing auxiliary tasks.

4.2 Mathematical formalism for analysis

In this chapter, we will focus on learning in finite state-action problems. However, our approximations will not be tabular, as discussed in Chapter 2, but two layer linear networks. This allows us to discuss the mechanisms underlying representation learning in a tractable setting. We will generally assume that our states are represented by one-hot vectors.

4.2.1 Linear networks as analytical models for training dynamics

Rigorously analyzing the effect of different loss functions on neural networks is challenging due to non-linearities in the networks, shifting data distributions, and policy updates. Therefore, we have to resort to studying simplified models to obtain quantitative and qualitative results, and only consider policy evaluation in our analysis. Studying feature learning dynamics using linear networks was popularized by Saxe et al. (2014) and has proven to be a valuable tool to analyze diverse objectives such as TD learning (Tang and Munos, 2023;

Le Lan et al., 2023), latent self-prediction (Tian et al., 2021; Tang et al., 2023), and linear auto-encoders (Pretorius et al., 2018; Bao et al., 2020).

We rewrite the feature learning algorithms using two to three matrices in lieu of more complex functions. The embedding function or encoder $\Phi \in \mathbb{R}^{n \times k}$ projects the one-hot encoded state x into a k dimensional embedding space, $F \in \mathbb{R}^{k \times k}$ is a *latent model* or transition function mapping the embedded state to the next state’s latent embedding. $\hat{V} \in \mathbb{R}^k$ and $\hat{r} \in \mathbb{R}^k$ *value and reward functions*, which operate in the latent space. Finally, for the reconstruction loss, $\Psi \in \mathbb{R}^{k \times n}$ is a *decoder* which maps from the latent space back to the ground truth observation space.

Furthermore, we use a simplifying assumption on the dataset and the embedding function throughout this chapter.

Assumption 4.2.1.1. *Assume that Φ has maximal rank at initialization. Assume that the dataset of states $\mathcal{D}$ is uniform i.i.d. over the state space and fixed throughout learning.*

While uniform i.i.d. is a potentially optimistic assumption, we make it here to avoid dealing with questions of exploration and unvisited states. Instead, we could have also assumed that the states are distributed according to the stationary distribution under the fixed policy. However, as the results presented in Ghosh and Bellemare (2020) (discussed in detail in Section 4.5) show, latent self-prediction based features guarantee stable convergence of the TD loss independent of the sampling distribution (as long as all states have non-zero probability in the dataset).

With these assumptions, and the fact that we represent state as one hot vector, we can use the fact that the covariance of the state features becomes

$$\mathbb{E}_{x \sim \mathcal{D}}[xx^T] = I.$$

This simplifies notation throughout this chapter.

Using our set of assumptions, we study four important loss functions for RL: observation reconstruction, where the aim is to fit the next state observation $x^\top \Phi F \Psi \approx x'$, latent reconstruction $x^\top \Phi F \approx x' \Phi$, where the aim is to predict learned features of the next state, and TD learning $x^\top \Phi \hat{V} \approx x^\top (r^\pi + \gamma x'^\top \phi \hat{V})$. To clarify the differences, we show a diagram explaining the losses and training setups in Figure 4.1. Following common notation, $[\cdot]_{\text{sg}}$ signifies a *stop-gradient* operation; no gradient is taken with regard to terms in the parenthesis.

Formally, these are written as

$$\text{Reconstruction: } L_{\text{rec}}(\Phi, F, \Psi) = \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| x^\top \Phi F \Psi - x^\top P^\pi \right\|_2^2 \right], \quad (4.1)$$

$$\begin{aligned}
\text{Latent self-prediction:} \quad L_{\text{lat}}(\Phi, F) &= \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| x^\top \Phi F - \left[x^\top P^\pi \Phi \right]_{\text{sg}} \right\|_2^2 \right], & (4.2) \\
\text{TD Learning:} \quad L_{\text{td}}(\Phi, \hat{V}) &= \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| x^\top \Phi \hat{V} - \left[x^\top \left(r^\pi + \gamma P^\pi \Phi \hat{V} \right) \right]_{\text{sg}} \right\|_2^2 \right]. & (4.3)
\end{aligned}$$

Following the nomenclature of this thesis, we refer the first two losses as *general-purpose*, and TD-learning as *decision-aware* as it depends on information specific to the decision problem. To analyze the discrete-time learning dynamics, we study the analogous continuous-time gradient flow, which allows us to use the toolkit of dynamical systems theory. Writing α_Φ and α_F for the representation and model learning rates, i.e., the latent self-prediction dynamics are

$$\frac{d}{dt} \Phi_t = -\alpha_\Phi \nabla_{\Phi_t} L_{\text{lat}}(\Phi_t, F_t) = -2\alpha_\Phi (\Phi_t F_t - P^\pi \Phi_t) F_t^\top, \quad (4.4)$$

$$\frac{d}{dt} F_t = -\alpha_F \nabla_{F_t} L_{\text{lat}}(\Phi_t, F_t) = -2\alpha_F \Phi_t^\top (\Phi_t F_t - P^\pi \Phi_t). \quad (4.5)$$

We primarily consider the *two-timescale* regime, i.e. the assumption that $\alpha_F \rightarrow \infty$ (Tang et al., 2023). Intuitively, this describes a learning setup in which the latent model is learned “much faster” than the latent mapping. This guarantees the non-collapse property for Φ , which means that the inner product $\Phi^\top \Phi = C$ is constant throughout training (Tang et al., 2023). In addition, as we are operating in the gradient flow limit, we will set $2\alpha_\Phi = 1$ throughout this chapter. This results in the following dynamics for self-predictive learning:

$$F_t^* = \left(\Phi_t^\top \Phi_t \right)^{-1} \Phi_t^\top P^\pi \Phi_t, \quad \frac{d}{dt} \Phi_t = \left(I - \Phi_t \left(\Phi_t^\top \Phi_t \right)^{-1} \Phi_t^\top \right) P^\pi \Phi_t F_t^{*\top}, \quad (4.6)$$

where our assumption on the initialization of Φ and the non-collapse property guarantee that the inverse exists $(\Phi_t^\top \Phi_t)^{-1}$.

4.3 Formalizing distractions and observation functions

To bridge the theory-practice gap in feature learning, we formalize two structures of MDPs commonly found in Deep RL benchmarks that have, to the best of our knowledge, not appeared in work analyzing feature learning: observation functions and distractions. To allow a close comparison with previous work, our changes to the formalism are minimal on purpose, while still highlighting the important role these changes play in different loss functions.

4.3.1 Observation functions

Previous literature (Tang et al., 2023; Tang and Munos, 2023; Le Lan et al., 2022) has eschewed the underlying observation of states in their analysis of representation dynamics. The assumption that x is represented by a one-hot vector, which we have made explicit above, implies that the features for the underlying states can be learnt independently. With a one-hot representations, the features of x are simply $x^\top \Phi = \Phi[i]$, the i -th row of Φ . A more realistic setting, which we focus on in Subsection 4.4.2, is considering observation functions acting on the underlying states. This allows for representing systems where some states have correlated observations, which may be helpful *or* harmful for the RL problem.

We briefly state this for reference later.

Assumption 4.3.1.1. *Let an observation function for a finite state space MDP be an invertible matrix $\mathcal{O} \in \mathbb{R}^{n \times n}$. Assume that the reward is not affected by this introduction.*

Formalization: To introduce an observation function while remaining in the regime of analyzing linear networks and finite state problems, each one of n states is mapped to a unique n -dimensional observation vector by an invertible *observation matrix* $\mathcal{O} \in \mathbb{R}^{n \times n}$.²

This change from one-hot vectors to arbitrary vectors allows us to account for similarities between different state observations. For example, if two states have almost identical observation vectors, they will be mapped to similar points in the latent representation space unless the features directly counteract this. We study the impact of changing the observations with a linear reparameterization in Subsection 4.4.2 by replacing x with $\bar{x} = \mathcal{O}^\top x$.

4.3.2 Distracting state dynamics

In addition to observation functions, another common problem that many reinforcement learning algorithms face are distractions. While distractions have been a focus of empirical work studying the relative efficacy of different auxiliary tasks (Ni et al., 2024), a simple formalism whose effect on learning dynamics can be analyzed has not been presented. We propose to model an MDP with distractions using factored MDPs (Boutillier et al., 2000).

Definition 4.3.2.1 (A factored MDP model of a distraction). *Let $M = (\mathcal{M}, P_M, R_M)$, $N = (\mathcal{N}, P_N, R_N)$ be a pair of Markov decision processes. The product process $M \otimes N$ is a MDP with state space $\mathcal{M} \times \mathcal{N}$, transition kernel $P_M \otimes P_N$ (where $\otimes$ signifies the Kronecker product), and reward function $R_M \otimes \mathbf{1} + \mathbf{1} \otimes R_N^\top$.*

If $R_N = 0$, we refer to N as a distracting process, as it does not contribute to the reward.

²We could also consider projection into higher dimensional spaces $\mathcal{O} \in \mathbb{R}^{n \times d}$ with $d > n$, without violating the Markov assumption, but this leads to additional complications (working with pseudo-inverses instead of inverses) which do not contribute meaningfully to the insights in this work.

This process models a common occurrence: two non-interacting processes unfold simultaneously, with the states being a combination of the two. Such a process can model some well-studied forms of distraction, such as the background distractions used in Stone et al. (2021) or the random observation dimension used in Nikishin et al. (2022a) and the next chapter. In this case, the foreground process M is assumed to carry the reward information, while the reward vector of the background process N is 0. We review important properties of the Kronecker product in Subsection A.1.1.

Note that our formalizations of observation functions and distracting processes is distinct from the assumptions in *linear MDPs* (Jin et al., 2020). Concretely we do not assume that the processes are low-rank compressible, just that they are factorizable.

4.4 Reconstruction and self-prediction losses

In this section and the next, we present an analysis of the stability conditions of reconstruction and self-prediction losses with linear networks. Using this analysis, we obtain several *insights*, qualitative predictions about how we expect the studied losses to behave in more complicated scenarios. These *insights* present the basis for our empirical comparison in Section 4.6.

4.4.1 Case 1: Orthogonal state representations

Tang et al. (2023) show that for MDPs with symmetric transition kernels ($\mathcal{P}^\top = \mathcal{P}$), latent self-prediction converges to subspaces spanned by eigenvectors of P^π . We extend this result in the following sense: if P^π has positive real eigenvalues, invariant sub-spaces which are not spanned by the top- k eigenvectors are unstable for gradient descent.³ Note that the resulting features are identical to those obtained from the random reward assumption described in Le Lan et al. (2023) (albeit under slightly different technical conditions), which highlights the close connection between the self-predictive approach and bootstrapped generalized value function learning. This furthermore suggests that using random rewards as auxiliary objectives (Farebrother et al., 2023) should result in similar features to those obtained with self-prediction.

Assumption 4.4.1.1. *Assume a two-timescale scenario and that F is initialized with full rank. Hence the non-collapse property $(\Phi_t^\top \Phi_t = \Phi_0^\top \Phi_0)$ (Tang et al., 2023) holds.*

When referring to eigenvectors and singular vectors, we mean the *right* vectors of the corresponding matrices unless stated otherwise. We now present our first theoretical result.

Proposition 4.4.1.1 (Stationary points of latent self-prediction). *Assume Assumption 4.2.1.1 and Assumption 4.3.1.1 hold. Furthermore, suppose P^π is real-diagonalizable. If the columns*

³The assumption of real *positive* eigenvalues is both more and less restrictive than the symmetry assumption made by Tang et al. (2023). An extension of our result to negative eigenvalues is presented below, together with an extended comparison to the results in Tang et al. (2023) and Le Lan et al. (2023).

of Φ_t span an invariant subspace of P^π , Φ_t is a stationary point of the dynamical system. Furthermore, if P^π is real-diagonalizable with positive eigenvalues, all invariant subspaces not spanned by the top- k eigenvectors sorted by eigenvalue are asymptotically unstable for gradient descent.

Proof. At any stationary point the gradient $\frac{d}{dt}\Phi_t$ must be equal to 0, which from (4.6) means that we must have $\left(\Phi_t(\Phi_t^\top\Phi)^{-1}\Phi_t^\top - I\right)P^\pi\Phi_t\Phi_t^\top P^\pi\Phi_t(\Phi_t^\top\Phi)^{-1} = 0$. We will denote stationary points with Φ^* .

Assume that the column vectors of Φ^* spans an invariant subspace of P^π . This implies that there exists a full rank matrix A so that $P^\pi\Phi^* = \Phi^*A$. Then

$$\left(\Phi^* \left(\Phi^{*\top}\Phi^*\right)^{-1} \Phi^{*\top} - I\right) P^\pi\Phi^*F^* = \left(\Phi^* \underbrace{\left(\Phi^{*\top}\Phi^*\right)^{-1} \Phi^{*\top}\Phi^* - \Phi^*}_{=I}\right) AF^* = 0. \quad (4.7)$$

This shows that $\frac{d\Phi}{dt} = 0$, which means that Φ^* is indeed a stationary point, which proves the first part of the proposition.

There are additional critical points of the differential equation, as discussed by Tang et al. (2023). In the analysis of stability, we first show the case of critical points corresponding to the claim in the proposition. We then briefly discuss other cases after the proof.

Case 1: Φ_t spans an invariant subspace of P^π Invariant subspaces correspond to subspaces spanned by right eigenvectors of P^π .

We write P for P^π to reduce notational clutter. Let $e_1, \dots, e_k$ be the eigenvectors corresponding to the k largest eigenvalues of P . Let Φ^* correspond to any set of k eigenvectors of P .

To show that all non top- k eigenspaces are asymptotically unstable critical points of the differential equation defined by the gradient flow of Φ . To show this, we aim to show that there exists an eigenvector of the Jacobian with an eigenvalue larger than 0. For this, we construct the directional derivative at the critical point. The directional derivative is the Jacobian vector product, which allows us to circumvent the need to work with higher order tensor derivatives. We then proceed to show that there exists a direction which corresponds to the eigenvector of the Jacobian with positive eigenvalue. This concludes the proof. This technique closely follows the one used by Le Lan et al. (2023).

Assume $\text{span}\{\Phi^*\} \neq \text{span}\{e_1, \dots, e_k\}$. This implies that there exists at least one eigenvector $e_j \in \{e_1, \dots, e_k\}$ and $e_j \notin \text{span}\{\Phi^*\}$, with corresponding eigenvalues λ_j .

Let D_Δ be the directional derivative of $\frac{d}{dt}\Phi|_{\Phi=\Phi^*}$ in the direction Δ . We construct the directional derivative using the product rule (terms colored for ease of reading),

$$D_\Delta \frac{d}{dt}\Phi|_{\Phi=\Phi^*} = -D_\Delta \left((\Phi F^* - P\Phi) F^{*\top} \right) |_{\Phi=\Phi^*} \quad (4.8)$$

$$= -\textcolor{red}{D}_\Delta (\textcolor{red}{\Phi F^*} - P\Phi) |_{\Phi=\Phi^*} F^{*\top} - (\Phi^* F^* - P\Phi^*) \textcolor{red}{D}_\Delta F^* |_{\Phi=\Phi^*}^\top. \quad (4.9)$$

For the directional derivative, we only consider directions that are orthogonal to Φ^* , so $\Phi^{*\top} \Delta = 0$. Then $\underbrace{P\Phi^* = \Phi^* A}_{\text{subspace condition}} \implies \Delta^\top P\Phi^* = 0$. For the derivative with regard to F^* , we have

$$\begin{aligned} \textcolor{red}{D}_\Delta F^* |_{\Phi=\Phi^*} &= D_\Delta \left(\Phi^\top \Phi \right)^{-1} \Phi^\top P\Phi \\ &= \left(D_\Delta \left(\Phi^\top \Phi \right)^{-1} \right) \Phi^{*\top} P\Phi^* + \left(\Phi^{*\top} \Phi^* \right)^{-1} \left(D_\Delta \Phi^\top P\Phi \right) \\ &= - \underbrace{\left(\Delta^\top \Phi^* + \Phi^{*\top} \Delta \right)}_{=0} \left(\Phi^{*\top} \Phi^* \right)^{-2} \Phi^{*\top} P\Phi^* + \left(\Phi^{*\top} \Phi^* \right)^{-1} \left(\underbrace{\Delta^\top P\Phi^*}_{=0} + \Phi^{*\top} P\Delta \right) \\ &= \left(\Phi^{*\top} \Phi^* \right)^{-1} \left(\Phi^{*\top} P\Delta \right). \end{aligned} \quad (4.10)$$

Therefore, the first term in the second line is dropped, as well as the first term of the final summand.

Note that $F^* = \left(\Phi^{*\top} \Phi^* \right)^{-1} \Phi^{*\top} P^\pi \Phi^* = \underbrace{\left(\Phi^{*\top} \Phi^* \right)^{-1} \Phi^{*\top} \Phi^*}_{=I} \text{diag}(\Lambda_i) = \text{diag}(\Lambda_i)$, where Λ_i

is the set of eigenvalues corresponding to the eigenvectors in Φ^* and $\text{diag}(\Lambda_i)$ is the diagonal matrix of eigenvalues corresponding to those eigenvectors spanned by Φ^* .

This allows us to compute the remaining derivative,

$$\textcolor{red}{D}_\Delta (\textcolor{red}{\Phi F^*} - P\Phi) |_{\Phi=\Phi^*} \text{diag}(\Lambda_i) \quad (4.11)$$

$$= (\Delta \text{diag}(\Lambda_i) + \Phi^* \textcolor{red}{D}_\Delta F^* - P\Delta) \text{diag}(\Lambda_i) \quad (4.12)$$

$$= \left(\Delta \text{diag}(\Lambda_i) + \Phi^* \left(\Phi^{*\top} \Phi^* \right)^{-1} \Phi^{*\top} P\Delta - P\Delta \right) \text{diag}(\Lambda_i), \quad (4.13)$$

where we use the fact that $D_\Delta P\Phi^* = PD_\Delta \Phi^* = P\Delta$.

Finally, as $\Phi^* F^* = \Phi^* \text{diag}(\Lambda_i) = P^\pi \Phi^*$, we obtain

$$D_\Delta \frac{d}{dt}\Phi|_{\Phi=\Phi^*} = - \left(\Delta \text{diag}(\Lambda_i) + \Phi^* \left(\Phi^{*\top} \Phi^* \right)^{-1} \Phi^{*\top} P\Delta - P\Delta \right) \text{diag}(\Lambda_i) \quad (4.14)$$

$$- \underbrace{(\Phi^* F^* - P\Phi^*)}_{=0 \text{ see Equation 4.7}} \left(\Phi^{*\top} P\Delta \right)^\top. \quad (4.15)$$

By the definition of the directional derivative as the Jacobian-vector product, we can now assert

$$\left(\frac{d}{d\Phi} \frac{d}{dt} \Phi|_{\Phi=\Phi^*} \right) \Delta = - \left(\Delta \text{diag}(\Lambda_i) + \left(\Phi^* \left(\Phi^{*\top} \Phi^* \right)^{-1} \Phi^{*\top} - I \right) P \Delta \right) \text{diag}(\Lambda_i). \quad (4.16)$$

What remains to be shown is that there exist a direction which corresponds to a positive eigenvalue of the Jacobian of the dynamics.

Choose $\Delta = v_j u^\top$. Let v_j be an eigenvector not in the span of Φ^* but in the top- k eigenvectors. Let λ_j be the corresponding eigenvalue. By our assumption before, there exist at least one eigenvalue $\lambda_i \in \Lambda_i$ so that $\lambda_j > \lambda_i$.

Note that $\left(\Phi^* \left(\Phi^{*\top} \Phi^* \right)^{-1} \Phi^{*\top} - I \right) P v_j = \left(\Phi \left(\Phi^{*\top} \Phi^* \right)^{-1} \underbrace{\Phi^{*\top} v_j}_{0 \text{ by construction}} - I v_j \right) \lambda_j = -\lambda_j v_j$

and therefore $\left(\Phi^* \left(\Phi^{*\top} \Phi^* \right)^{-1} \Phi^{*\top} - I \right) P \Delta = -\Delta \lambda_j I$.

To simplify notation, we will write Λ_i for $\text{diag}(\Lambda_i)$ from now on as there is no risk of confusion.

$$- \left(\Delta \Lambda_i + \left(\Phi^* \left(\Phi^{*\top} \Phi^* \right)^{-1} \Phi^{*\top} - I \right) P \Delta \right) \Lambda_i = - \Delta (\Lambda_i - \lambda_j I) \Lambda_i \quad (4.17)$$

$$= - \Delta (\Lambda_i^2 - \lambda_j \Lambda_i) \quad (4.18)$$

$$= \Delta (\lambda_j \Lambda_i - \Lambda_i^2). \quad (4.19)$$

We can now choose u so that it is any eigenvector of $(\lambda_j \Lambda_i - \Lambda_i^2)$. As this is a diagonal matrix, it is easy to see that if $\lambda_j > \lambda_i$ for any λ_i , the matrix will have a positive eigenvalue, meaning there exists a direction in which the critical point is unstable. $\square$

Case 2: Non-invariant subspace cases: Not all critical points lie in invariant subspaces. One such an alternative critical point is the case of $\Phi^\top P \Phi = 0$, and more generally, for each set of column vectors ϕ_i in Φ , they need to either be mapped to an invariant or an orthogonal subspace by P^π to be stable. In the orthogonal case, the Jacobian at the critical point becomes 0, meaning no conclusion about stability can be drawn from this analysis. We do however conjecture that the non-invariant subspace critical points are also saddle-points or unstable solutions of the ODE, following the experimental evidence presented in Tang et al. (2023).

Negative eigenvalues: In case the matrix has negative eigenvalues, the stability conditions in the final step of the proof change. The matrix $\lambda_j \Lambda_i - \Lambda_i^2$ will not have negative eigenvalues

if λ_i is negative but λ_j is positive. The ranking of stable points follows this slightly un-intuitive ordering: all negative eigenvalues sorted by absolute value followed by all positive eigenvalues sorted by absolute value.

Our proof implies that even without the assumptions of symmetry of P^π required in Tang et al. (2023), the dynamics of latent self-prediction will tend to converge to invariant subspaces spanned by eigenvectors with large eigenvalues as other invariant subspaces are unstable. This is important as these are most important for representing potential reward functions in the environment, at least without additional assumptions on reward structure, then smaller eigenvectors (Le Lan et al., 2023).

We can contrast this with the features learned by a reconstruction loss. We write $\text{span}(A)$ for both the span of the column vectors of A or for the span of a set of vectors A , depending on context.

Proposition 4.4.1.2 (Stationary points of reconstruction). *Assume [Assumption 4.2.1.1](#) and [Assumption 4.3.1.1](#) hold. Write $(u_1, \dots, u_n), (v_1, \dots, v_n)$ for the left and right singular vectors of P^π sorted in descending order by singular value. Any stationary point (Φ^*, F^*, Ψ^*) of L_{rec} under the two timescale scenario satisfies $\text{span}(\Phi^*) = \text{span}(\{u_1, \dots, u_k\})$, $\text{span}(\Psi^{*\top}) = \text{span}(\{v_1, \dots, v_k\})$.*

Proof. We first show that under the two timescale scenario, F is stationary and therefore does not change the span of the critical points.

Due to the assumption of the two-timescale scenario, we compute Ψ^* by solving the linear regression problem,

$$\frac{d}{d\Psi^*} \|\Phi F \Psi^* - P\|_F^2 = F^\top \Phi^\top (\Phi F \Psi^* - P) \quad (4.20)$$

$$0 = F^\top \Phi^\top (\Phi F \Psi^* - P) \quad (4.21)$$

$$\Leftrightarrow B^* = \left(F^\top \Phi^\top \Phi F\right)^{-1} F^\top \Phi^\top P = F^{-1} \left(\Phi^\top \Phi\right)^{-1} \Phi^\top P. \quad (4.22)$$

Plugging this solution back into the original equation,

$$\|\Phi F \Psi^* - P\|_F^2 = \|\Phi F F^{-1} \left(\Phi^\top \Phi\right)^{-1} \Phi^\top P - P\|_F^2 \quad (4.23)$$

$$= \|\Phi \left(\Phi^\top \Phi\right)^{-1} \Phi^\top P - P\|_F^2, \quad (4.24)$$

we notice that F cancels. Therefore, the optimality conditions for A follow from the Eckart-Young theorem, as presented in [Lemma A.1.1.3](#). $\square$

Features of this form have been studied extensively and the convergence properties of linear auto-encoders are well understood (Baldi and Hornik, 1989; Pretorius et al., 2018; Bao et al., 2020).

Proposition 4.4.1.1 and Proposition 4.4.1.2 together show that there is a subtle but important difference between latent self-prediction and observation reconstruction: the features will converge to invariant subspaces in the former case, and to singular space in the latter case. Note that if P^π is a symmetric matrix, then the singular spaces and the invariant subspace spanned by top- k eigenvectors coincide and latent self-prediction and reconstruction converge to the same features (Tang et al., 2023).

Behzadian et al. (2019) show that top k singular vectors are optimal low-rank linear features when making no assumptions on the reward, meaning observation reconstruction should lead to the best features when considering the supremum over any bounded reward. If eigenvectors and singular vectors differ, Behzadian et al. (2019) and Le Lan et al. (2023) both highlight that singular vectors often lead to better performing features.

So far our results line up with previously presented theoretical works and do not shed additional light on the observed theory-practice gap. However, the assumption (or rather lack of assumptions) on the MDP, the reward, and the observation structure will be crucial to understanding these results more deeply in the next sections.

Insight 4.4.1.1 (Optimality of observation prediction). *The features learned by observation prediction are in general superior to those of latent self-prediction, when using solely one of these as the loss function, and if no additional assumptions on the structure of the reward are made.*

4.4.2 Case 2: Observation function dependence

Recall that the gradient dynamics presented in Equation 4.6 and analyzed in Proposition 4.4.1.1 and Proposition 4.4.1.2 rely on the assumption that $\mathbb{E}[xx^\top] = I$. We now use the idea of an observation matrix $\mathcal{O}$ to account for problems in which the observation space creates similarities between features. To do this, we simply replace every occurrence of $x^\top$ in the losses presented in Section 4.2 with $x^\top \mathcal{O}$. We assume that x is a one-hot vector as discussed before and the coverage is still uniform (Assumption 4.2.1.1 still holds), so all correlation between states arise as $\mathbb{E}[\mathcal{O}^\top xx^\top \mathcal{O}] = \mathcal{O}^\top \mathcal{O}$.

This results in the following loss functions

$$L_{\text{rec}}(\Phi, F, \Psi) = \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| x^\top \mathcal{O} \Phi F \Psi - x^\top P^\pi \mathcal{O} \right\|_2^2 \right] = \left\| \mathcal{O} \Phi F \Psi - P^\pi \mathcal{O} \right\|_2^2, \quad (4.25)$$

$$L_{\text{lat}}(\Phi, F) = \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| x^\top \mathcal{O} \Phi F - \left[x^\top P^\pi \mathcal{O} \Phi \right]_{\text{sg}} \right\|_2^2 \right] = \left\| \mathcal{O} \Phi F - [P^\pi \mathcal{O} \Phi]_{\text{sg}} \right\|_2^2 \quad (4.26)$$

$$L_{\text{td}}(\Phi, F) = \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| x^\top \mathcal{O} \Phi \hat{V} - \left[x^\top \left(r + \gamma P^\pi \mathcal{O} \Phi \hat{V} \right) \right]_{\text{sg}} \right\|_2^2 \right] \quad (4.27)$$

$$= \left\| \mathcal{O} \Phi \hat{V} - \left[\left(r + \gamma P^\pi \mathcal{O} \Phi \hat{V} \right) \right]_{\text{sg}} \right\|_2^2. \quad (4.28)$$

It is important to note that for latent self-prediction and Temporal Difference (TD) this rewriting leads to a linear basis change of Φ compared to the original loss, as each occurrence of $\mathcal{O}$ is multiplied by Φ . The only loss for which this is not the case is the reconstruction approach.

Proposition 4.4.2.1. *Assume Assumption 4.2.1.1, Assumption 4.3.1.1, and Assumption 4.4.1.1 hold. Let $\{\Phi_{\text{lat}}^*\}$ and $\{\Phi_{\text{td}}^*\}$ be the set of critical points of L_{lat} or L_{td} respectively. Then $\mathcal{O}^{-1}\Phi_{\text{lat/td}}^*$ are stationary points for the reparameterized losses $L_{\text{lat}}^\mathcal{O}$ and $L_{\text{td}}^\mathcal{O}$. Furthermore, if $\Phi_{\text{lat/td}}^*$ is an asymptotically stable point of $L_{\text{lat/td}}$ that has a Jacobian with all negative eigenvalues, $\mathcal{O}^{-1}\Phi_{\text{lat/td}}^*$ is an asymptotically stable point of $L_{\text{lat/td}}^\mathcal{O}$.*

Proof. We first write out all losses with the observation matrix $\mathcal{O}$. The reward function is assumed to not change under the introduction of $\mathcal{O}$, therefore we do not multiply $\mathcal{O}$ to $x^\top r$.

Note that in the cases of L_{lat} and L_{TD} , all occurrences of $\mathcal{O}$ are multiplied by Φ . Therefore the corresponding gradient flows are reparameterizations as defined in Lemma A.1.3.1, and the proposition follows directly. $\square$

Note that while the stationary points and asymptotic stability conditions of the gradient flow might be unaffected by the introduction of observation distortions, the same might not be true for the dynamics of descent with finite step sizes. The numerical conditioning of the involved matrices change depending on $\mathcal{O}$ and so the impact of discretization due to finite step sizes changes the resulting dynamical system.

Proposition 4.4.2.2. *Assume Assumption 4.2.1.1, Assumption 4.3.1.1, and Assumption 4.4.1.1 hold. Let $(u_1, \dots, u_n)$, $(v_1, \dots, v_n)$ be the left and right singular vectors of $\mathcal{O}^{-1}P^\pi\mathcal{O}$. Any stationary point (Φ^*, F^*, Ψ^*) of $L_{\text{rec}}^\mathcal{O}$ satisfies $\text{span}(\Phi^*) = \text{span}(\{u_1, \dots, u_k\})$, $\text{span}(\Psi^{*\top}) = \text{span}(\{v_1, \dots, v_k\})$.*

Proof. As before, note that $L_{\text{rec}}^\mathcal{O}$ is of the form $\|\mathcal{O}AXB - P\mathcal{O}\|_F^2$, with $A \in \mathbb{R}^{n \times k}$, $X \in \mathbb{R}^{k \times k}$, and $B \in \mathbb{R}^{k \times n}$.

Due to the assumption of the two-timescale scenario, we compute Ψ^* by solving the linear regression problem,

$$\frac{d}{d\psi} \|\mathcal{O} \Phi F \Psi - P\mathcal{O}\|_F^2 = F^\top \Phi^\top \mathcal{O}^\top (\mathcal{O} \Phi F \Psi - P) \quad (4.29)$$

$$0 = F^\top \Phi^\top \mathcal{O}^\top (\mathcal{O} \Phi F \Psi^* - P) = 0 \quad (4.30)$$

$$\Leftrightarrow \Psi^* = \left(F^\top \Phi^\top \mathcal{O}^\top \mathcal{O} \Phi F \right)^{-1} F^\top \Phi^\top \mathcal{O}^\top P. \quad (4.31)$$

Substituting into $L_{\text{rec}}^\mathcal{O}$, we obtain

$$\|\mathcal{O} \Phi F \Psi - P \mathcal{O}\|_F^2 = \|\mathcal{O} \Phi F \left(F^\top \Phi^\top \mathcal{O}^\top \mathcal{O} \Phi F \right)^{-1} F^\top \Phi^\top \mathcal{O}^\top P - P \mathcal{O}\|_F^2 \quad (4.32)$$

$$= \|\mathcal{O} \Phi \left(\Phi^\top \mathcal{O}^\top \mathcal{O} \Phi \right)^{-1} \Phi^\top \mathcal{O}^\top P - P \mathcal{O}\|_F^2, \quad (4.33)$$

which implies that F is stationary.

We note that $\|\mathcal{O} \Phi \Psi - P \mathcal{O}\|_F^2$ is the reduced rank regression problem solved in [Lemma A.1.1.3](#) for which the solution are the top- k left and right singular vectors of $\mathcal{O}^{-1} P \mathcal{O}$. $\square$

The singular value decomposition of $\mathcal{O}^{-1} P^\pi \mathcal{O}$ will in general not have a clearly interpretable relationship to that of P^π and r , so the optimality result obtained by Behzadian et al. (2019) do not hold in this case. However this does not mean that different observation functions will always harm the ability of the reconstruction loss to obtain good features. Consider for example an observation transformation that maps states directly to value and reward function. This would clearly be an example of a helpful observation transformation. However, in general we conjecture that arbitrarily changing the observation function will harm the reconstruction loss approach.

Insight 4.4.2.1 (Observation dependence of autoencoder models). *Due to the invariance properties of latent self-prediction, we expect the performance of latent self-prediction to suffer less than the performance of observation reconstruction when perturbing the observation space arbitrarily.*

4.5 Understanding the effects on value function learning

The optimality of a representation for value estimation depends non-trivially on the structure of the reward structure of the MDP. Previous works (Behzadian et al., 2019; Bellemare et al., 2019; Le Lan et al., 2022) attempted to reason about the optimality of representations without relying on the reward structure, by arguing that certain subspaces (such as the span of top- k eigenvectors or top- k singular vectors) are optimal assuming unknown rewards.

Instead, here we take the value function structure into account. Furthermore, we argue that the top- k eigenspaces (resp. singular spaces) are not always optimal if the agent is optimizing a particular reward.

4.5.1 Value functions in MDPs with distractions

The optimality of features can be measured in how close the projection of the true value function onto their span is to the ground truth, i.e. in the L_2 norm. This raises the question under what conditions the top k eigenvectors or singular vectors would not span the value function well. For this, we turn to our notion of an MDP with distractions.

As our results show convergence to top singular and eigenvectors, we will now construct an example of a process in which the top- k eigenvalues will contain redundant copies of the basis function of the reward. This happens if the distracting process has larger eigenvalue than the reward-relevant process. In this case, the projection of the reward on to the top- k eigenvectors of the joint process will be identical to a vector where all entries contain the average reward. This however clearly does not provide a good basis for the reward function. Intuitively, the top- k eigenvectors only represent the distracting process, not the reward-relevant one.

Proposition 4.5.1.1 (Suboptimality of top k eigenspaces with distractions). *Assume an MDP with distraction composed of two independent processes $\mathcal{M}$ and $\mathcal{N}$ according to Definition 4.3.2.1. Let $v_1, \dots, v_m$ and $u_1, \dots, u_n$ be the eigenvectors of $\mathcal{M}$ and $\mathcal{N}$ respectively, with associated eigenvalues λ_j and μ_i ordered by norm. Assume $\forall i < k : \mu_i > \lambda_2$ and $r_{\mathcal{N}} = 0$. Let U_k be an orthonormal basis for the top- k eigenspace of $\mathcal{M} \otimes \mathcal{N}$. Then $\text{span}(U_k) = \text{span}(\{\mathbf{1} \otimes v_i | \forall i \leq k\})$ and $\text{proj}_{U_k}(r_{\mathcal{M}} \otimes \mathbf{1}) = (\sum_{i=1}^m r_i/n) \mathbf{1}_{n \cdot m}$.*

Proof. We first note that $\text{span}(V_k) = \text{span}(\{v_i \otimes u_1 | \forall i \leq k\})$ follows directly from Lemma A.1.1.1 and Lemma A.1.1.2. The eigenvector $u_1 = \mathbf{1}$ as M is a stochastic matrix.

As V_k is an orthogonal basis, write the projection operation as

$$V_k V_k^\top (r_{\mathcal{M}} \otimes \mathbf{1}) = \frac{1}{m} (\mathbf{1} \otimes \text{orth}(V)) \left(\mathbf{1} \otimes \text{orth}(V)^\top \right) (r_{\mathcal{M}} \otimes \mathbf{1}) \quad (4.34)$$

$$= \frac{1}{m} \left((\mathbf{1} \otimes \mathbf{1}) \otimes (\text{orth}(V) \text{orth}(V)^\top) \right) (r_{\mathcal{M}} \otimes \mathbf{1}) \quad (4.35)$$

$$= \frac{1}{m} (\mathbf{1} \otimes \mathbf{1}) r_{\mathcal{M}} \otimes (\text{orth}(V) \text{orth}(V)^\top) \mathbf{1} \quad (4.36)$$

$$= \sum_{i=1}^m \frac{r_i}{m} \mathbf{1}. \quad (4.37)$$

□

This means that when distractions are involved, the top- k eigenvectors do not span the reward function well. By Lemma A.1.2.2, which shows how the eigenvectors form a basis of the value function based on the reward, this directly implies suboptimal value function approximation. Note that our assumptions here are very strong as we consider a *worst case*

distraction for clarity, but the problem emerges whenever the distracting process has several large eigenvalues, as these will lead to redundant representations in the basis.

4.5.2 Value function learning and auxiliary tasks

So far, we have only looked at the features obtained by training Φ with the model-based prediction task. However, in many empirically strong algorithms, we use both an auxiliary loss and a TD loss and propagate the gradients jointly into the embedding function. Therefore, we now analyse the combined stabilizign plus TD loss scenario.

We begin by formalizing the reward function structure we will analyze. Let us write $w_1, \dots, w_n$ for the eigenvectors of P^π . We will assume that r^π has a low-dimensional structure in the following sense:

Assumption 4.5.2.1. *There exist $i_1, \dots, i_m \in \{1, \dots, |\mathcal{X}|\}$ such that $r^\pi \in \text{span}(w_{i_1}, \dots, w_{i_m})$, and $m \leq k$. Let furthermore $\{w_{i_1}, \dots, w_{i_m}\}$ be a minimal basis in the sense that $w_{i_n} w_{i_n}^\top r^\pi \neq 0$ for all n .*

We now write the summed losses

$$L_{\text{rec+td}}(\Phi, F, \psi, \bar{r}) = L_{\text{rec}}(\Phi, F, \psi) + L_{\text{td}}(\Phi, \bar{r})$$

and

$$L_{\text{lat+td}}(\Phi, F, \bar{r}) = L_{\text{lat}}(\Phi, F) + L_{\text{td}}(\Phi, \bar{r}).$$

To obtain our result, we require a lemma about the stationary points of the TD gradient flow. This proof closely follows related statements by Ghosh and Bellemare (2020), Tang et al. (2023), and Le Lan et al. (2022). We present the argument here for clarity with all assumptions necessary for our work.

Lemma 4.5.2.1 (Critical points of L_{TD}). *Let Assumption 4.2.1.1, Assumption 4.4.1.1, and Assumption 4.5.2.1 hold. Assume $\Phi^* \in \mathbb{R}^{n \times k}$ is an orthonormal invariant representation for P^π in the sense that $\Phi^{*\top} \Phi^* = I$ and $\text{span}(\Phi^*) = \text{span}(P^\pi \Phi^*)$. Let furthermore $r^\pi \in \text{span}(\Phi^*)$ and let V be the corresponding value function. Then Φ^* is a critical point of L_{TD} and for the corresponding weights $\hat{V}^*$ we have $\Phi^* \hat{V}^* = V$.*

Proof. By Lemma A.1.4.2, Lemma A.1.4.3, and Lemma A.1.4.4 and the stated assumptions the weights $\hat{V}$ converge. Therefore, we can analyze the induced dynamical system with $\hat{V}^*$.

Find $\hat{V}^*$ as the stationary point of $\nabla_{\hat{V}} L_{\text{TD}}$

$$\nabla_{\hat{V}} \left\| \Phi^* \hat{V} - \left[r^\pi + \gamma P \Phi^* \hat{V} \right]_{\text{sg}} \right\|_2^2 \Big|_{\hat{V}=\hat{V}^*} = \Phi^{*\top} \left(\Phi^* \hat{V}^* - \left[r^\pi + \gamma P \Phi^* \hat{V}^* \right] \right) = 0 \quad (4.38)$$

$$\Leftrightarrow \Phi^{*\top} \Phi^* \hat{V}^* - \Phi^{*T} r^\pi - \gamma \Phi^{*\top} \Phi^* B \hat{V}^* = (\Phi^{*\top} \Phi^* - \gamma \Phi^{*\top} P^\pi \Phi^*) \hat{V}^* - \Phi^{*\top} r^\pi = 0 \quad (4.39)$$

$$\Leftrightarrow \hat{V}^* = (\Phi^{*\top} \Phi^* - \gamma \Phi^{*\top} P^\pi \Phi^*)^{-1} \Phi^{*\top} r^\pi = (I - \gamma \Phi^{*\top} P^\pi \Phi^*)^{-1} \Phi^{*\top} r^\pi \quad (4.40)$$

The invertibility of $(I - \gamma \Phi^{*\top} P^\pi \Phi^*) = \Phi^{*T} D(I - \gamma P^\pi) \Phi^* = A_\Phi^*$ is guaranteed as all eigenvectors are nonzero.

We now show that Φ^* is a stationary point by showing that

$$\nabla_\Phi L_{\text{TD}}|_{\Phi=\Phi^*} = 0.$$

We note that as Φ^* spans an invariant subspace of P^π there exists an invertible matrix B so that $P^\pi \Phi^* = \Phi^* B$. Therefore $(I - \gamma \Phi^{*\top} P^\pi \Phi^*) = (I - \gamma B)$.

$$\nabla_\Phi L_{\text{TD}}|_{\Phi=\Phi^*} = \nabla_\Phi \left\| \Phi \hat{V}^* - \left[r^\pi + \gamma P \Phi \hat{V}^* \right]_{\text{sg}} \right\|_2^2 \Big|_{\Phi=\Phi^*} \quad (4.41)$$

$$= \left(\Phi \hat{V}^* - r^\pi - \gamma P \Phi \hat{V}^* \right) (\hat{V}^*)^\top \Big|_{\Phi=\Phi^*} \quad (4.42)$$

$$= \left(\Phi^* (I - \gamma B) \hat{V}^* - r^\pi \right) (\hat{V}^*)^\top \quad (\text{substitute first occurrence of } \hat{V}^*) \quad (4.43)$$

$$= \left(\Phi^* \Phi^{*\top} r^\pi - r^\pi \right) (\hat{V}^*)^\top \quad (4.44)$$

$$= \underbrace{(r^\pi - r^\pi)}_{=0} (\hat{V}^*)^\top = 0 \quad (4.45)$$

The final equality is due to the fact that the columns of Φ^* are orthonormal, which means that $\Phi^* \Phi^{*\top}$ is an orthogonal projection onto the span of Φ^* . To verify, note that

$$\left(\Phi^* \Phi^{*\top} \right)^2 = \left(\Phi^* \underbrace{\Phi^{*\top} \Phi^*}_{=I} \Phi^{*\top} \right) = \left(\Phi^* \Phi^{*\top} \right).$$

Furthermore, by [Assumption 4.5.2.1](#), $r^\pi \in \text{span}(\Phi^*)$, which means $\Phi^* \Phi^{*\top} r^\pi = r^\pi$ for an orthogonal projection $\Phi^* \Phi^{*\top}$. Moreover, by [Lemma A.1.4.6](#), $\Phi^* \hat{V}^* = V$, which concludes the proof. $\square$

It is easy to see that the stationarity point of TD learning are a subset of those of latent self-prediction. We can formalize this relationship to show that adding a stabilizing self-prediction loss to a TD loss does not change the quality of the value approximation at its stationary point.

Proposition 4.5.2.1. *Suppose that P^π is real diagonalizable, and that [Assumption 4.2.1.1](#), [Assumption 4.4.1.1](#), and [Assumption 4.5.2.1](#) hold. There exists a non-trivial critical point Φ^* of the two-timescale TD loss L_{td} such that $\text{span}(r^\pi) \subseteq \text{span}(\Phi^*)$. Furthermore, Φ^* is a*

critical point of the two-timescale joint loss L_{td+lat} . Therefore combining L_{TD} and L_{Lat} does not exclude the existence of a stationary point with 0 value function approximation error.

Proof. By Assumption 4.5.2.1 there exists a set of k vectors $\phi_1, \dots, \phi_k$ such that $r^\pi \in \text{span}(\phi_1, \dots, \phi_k)$ and $\phi_1, \dots, \phi_k$ span an invariant subspace of P^π . We can orthogonalize this set of vectors by applying the Gram Schmidt procedure to the eigenvectors $(w_{i_1}, \dots, w_{i_m})$. By Lemma A.1.4.1 the matrix $\Phi^* \in \mathbb{R}^{n \times k}$ whose columns are $\phi_1, \dots, \phi_k$ is a critical point for L_{lat} and spans an invariant subspace of P^π . In addition, by Lemma A.1.2.2, Φ forms a complete basis for the value function V^π . By Lemma 4.5.2.1, Φ is also a critical point of L_{TD} , with 0 value function approximation error. As Φ is a critical point for both L_{TD} and L_{Lat} , it is a critical point of $L_{TD} + L_{Lat}$. $\square$

We can now contrast this with the reconstruction case. In this case, our assumption that the top- k singular vectors do not agree with the reward-spanning eigenvectors will lead to a mismatch between the losses.

Proposition 4.5.2.2. *Let Assumption 4.2.1.1, Assumption 4.3.1.1, and Assumption 4.5.2.1 hold. If the reward spanning eigenvectors do not lie within the span of the top- k singular vectors, $\text{span}(w_{i_1}, \dots, w_{i_m}) \not\subseteq \text{span}(u_1, \dots, u_k)$, the critical points of the two-timescale joint loss L_{td+rec} are guaranteed to not be minimizers of the value function approximation error.*

Proof. Following from Assumption 4.5.2.1 and Lemma A.1.2.2, we have that any critical point Φ^* of L_{td} with perfect value function approximation fulfils $\text{span}(r^\pi) \subseteq \text{span}(\Phi^*)$, as without this condition, Φ would not have all necessary basis vectors to represent V^π . Under our low-dimensionality assumption $r^\pi \in \text{span}(w_{i_1}, \dots, w_{i_m})$, this condition implies that $\text{span}(w_{i_1}, \dots, w_{i_m}) \subseteq \text{span}(\Phi^*)$. From Proposition 4.4.1.2 we know that any critical point Φ^* of L_{rec} satisfies $\text{span}(\Phi^*) = \text{span}(\{u_1, \dots, u_k\})$, where $u_1, \dots, u_k$ are the top k left singular vectors of P^π . Under the assumption that $\text{span}(w_{i_1}, \dots, w_{i_m}) \not\subseteq \text{span}(u_1, \dots, u_k)$ these conditions cannot happen simultaneously, and hence no critical point of the joint loss achieves perfect value function reconstruction. $\square$

Contrasting these propositions suggests that when combined with TD learning losses, latent self-prediction can be a more helpful auxiliary task. This does not hold for the reconstruction loss, where we can construct cases in which the joint loss leads to worse TD error.

Insight 4.5.2.1 (Latent self-prediction as an auxiliary task). *For good performance across a wide variety of tasks, latent self-prediction needs to be combined with TD learning as an auxiliary task. It is a preferable auxiliary task to observation prediction in most scenarios, but especially in scenarios with distracting processes.*

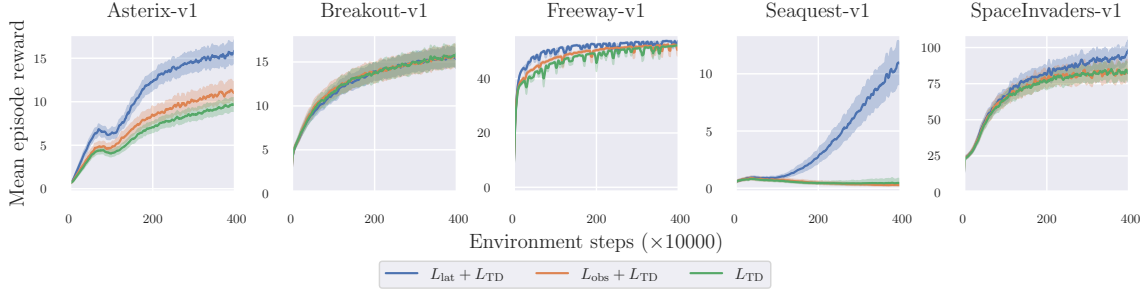


Figure 4.2: Auxiliary task setup: Performance of all losses on the observation space as given without changes to the environment.

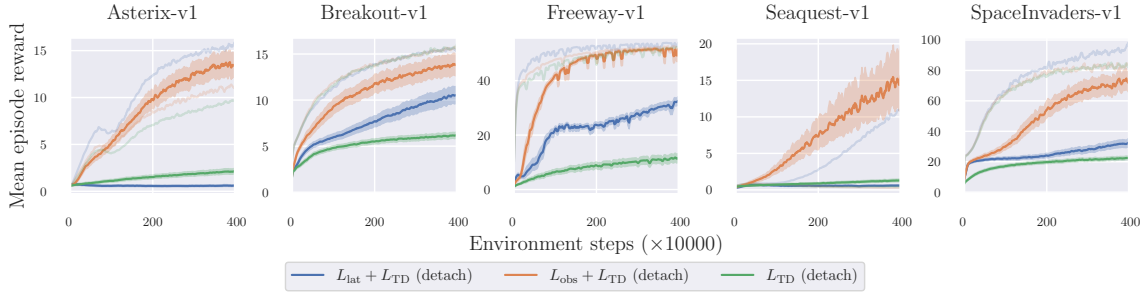


Figure 4.3: Stand-alone setup: Performance of all losses on the observation space as given without changes to the environment. The DQN baseline is using random features, which are not updated, to verify that learning features is indeed superior to a random feature baseline.

4.6 Empirical study

We aim to empirically verify the statements marked as “*Insights*” throughout this chapter: superiority of observation prediction as a standalone feature learning loss (Insight 4.4.1.1), impact of the observation function on the different loss functions (Insight 4.4.2.1), and the relative strength of latent-self prediction as an auxiliary loss compared to reconstruction (Insight 4.5.2.1). As our theory addresses the simplified setting of policy evaluation with linear models, we seek to test if the insights transfer to the more common setting of control with neural networks. Across all experiments, we report mean performance over 30 seeds and shaded 95 bootstrapped confidence interval.

To test these hypotheses, we use the MinAtar suite of five Atari inspired video games (Young and Tian, 2019) and the DMC 15 suite (Tunyasuvunakool et al., 2020).⁴ Both are small enough to perform thorough investigations, while providing non-trivial observation spaces and dynamics. Detailed information about the implementation and hyperparameters can be found in Section C.1.

⁴DMC experiments are presented in Section C.2.

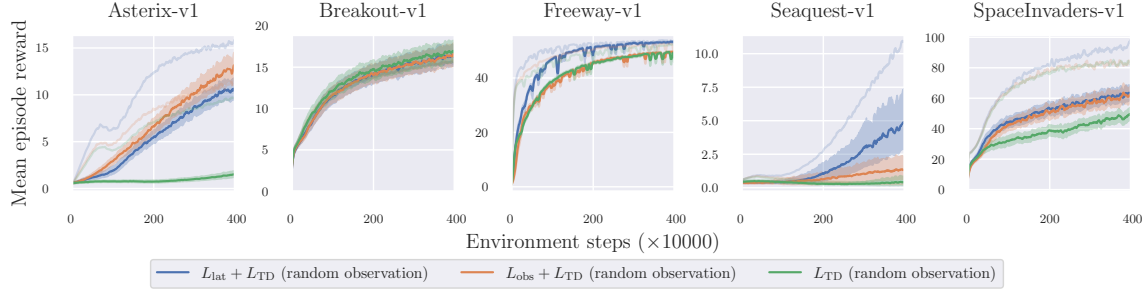


Figure 4.4: Distorted observation function with a random transformation.

Auxiliary task learning vs general purpose feature learning (Figure 4.2 and Figure 4.3): First, we compare both the auxiliary task and stand-alone feature learning scenarios. As expected from prior work (Jaderberg et al., 2017; Schwarzer et al., 2021a; Farebrother et al., 2023), in all cases using an auxiliary loss performs no worse (and often better) than vanilla DQN. We find that as expected from Insight 4.5.2.1, latent self-prediction is a stronger auxiliary loss function than observation reconstruction in three out of five environments. However, when using the general-purposes losses alone, we clearly see observation reconstruction performing significantly better than latent self-prediction, which fails to learn any relevant features in several cases. This verifies Insight 4.4.1.1.

Curiously, in the case of the Seaquest environment, we find that using observation prediction alone outperforms using it as an auxiliary task strongly, and performs on par with the auxiliary task variant of the latent self prediction loss. Seaquest also has the sparsest reward structure in the test suite, which can make it a challenging environment for DQN. In this case, features based purely on the observed transition might allow for better policy learning.

This highlights that no algorithm is clearly superior in all settings and the reward and observation structure is very relevant for the performance of each loss.

Observation space distortions (Figure 4.4): To test the impact of changing the observation function, we sample a random binary matrix and multiply it to the flattened observation vector. We then reshape the observation to the original shape again.

All algorithms show themselves to be strongly impacted by this random observation distortions, which suggests that our claim of invariance of self-prediction relies too strongly on the linear gradient-flow limit. This can in part be explained by the use of a convolutional layer in the standard baseline implementation of DQN which we adapted. However, we still find that at least on two environments (Seaquest and Freeway), the latent self-prediction auxiliary task is able to recover more of the original performance than either observation prediction or DQN. The DQN baseline seems to suffer the most from the introduction of

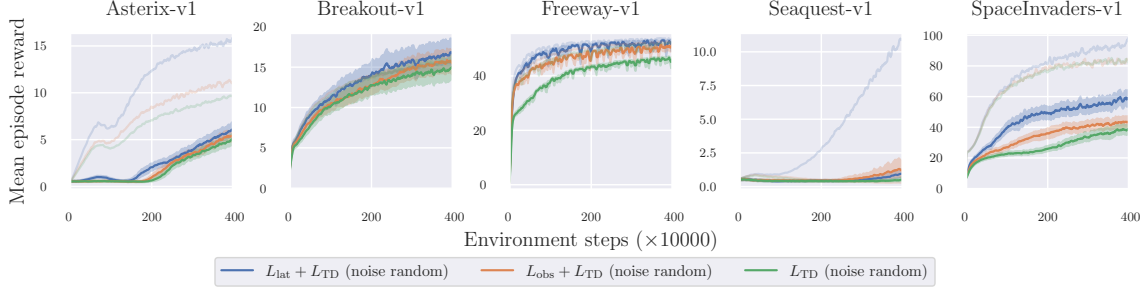


Figure 4.5: Appending random noise channels.

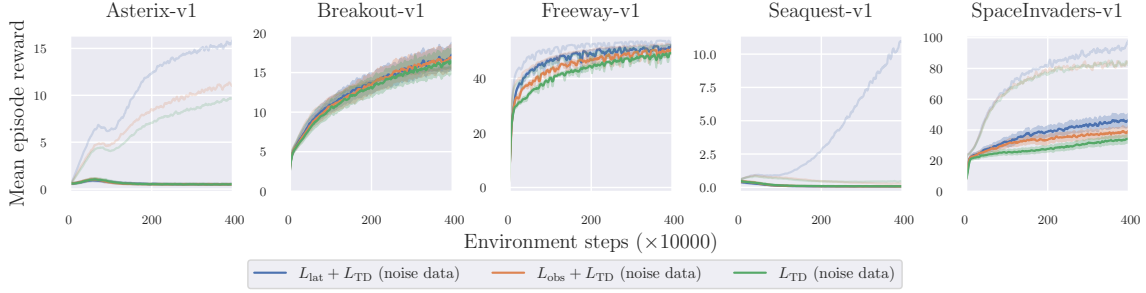


Figure 4.6: Appending structured noise channels using the Freeway environment.

the observation change, which suggests that correlations of the existing observation space play an important role in learning correct value function prediction.

Overall, we find that [Insight 4.4.2.1](#) does not fully translate to the more complex test setting. In part, this may be explained by the fact that the original observation spaces of the test environments already violate our assumptions for the one-hot encoding. In addition, introducing linear correlation might not impact non-linear model learning in the same way it would impact linear models. This highlights the need for more in-depth research on the interplay between given observation space and feature learning.

Distractions (Figure 4.5 and Figure 4.6): As our results are dependent on the spectral structure of the environment, different distraction models can be assumed to have differing impact on the efficacy of the tested losses. This behavior is dependent on the structure of the noise. If the distraction does not strongly change the top- k singular or eigenspaces, it will be less problematic for the auxiliary tasks, especially for observation reconstruction. Testing the impact of different distraction models on the top- k spaces is out of scope for this work, but we conjecture that fully random noise has less structure than distractions following clear patterns.

Therefore, we consider two simple distraction models in our experiments. The distractions are concatenated to the original observations along the channel dimension. First, we use random noise sampled independently for each state from a Bernoulli distribution. As there

is no predictable structure in this noise, we expect all algorithms to be able to deal with this distraction better. Second, we choose one of the environments at random (Freeway-v1) and concatenate two copies to the observation space of each environment. The dynamics are obtained by sampling a random action at each timestep independent from the policy and stepping the distraction environment with it.

We find a small advantage in some environments to using the latent self-prediction loss and using random noise, and no clear advantage from any algorithm in the structured noise case. Structured noise poses a much larger challenge to most algorithms, completely preventing learning in several cases. This partially validates that not only the presence or absence of noise matters, but also how it changes relevant quantities, e.g. the eigenvalues of the transition kernel.

In the continuous control experiments presented in [Section C.2](#), we find that the self-prediction loss performs generally better than in the MinAtar suite. As the observation and reward structure differs between these two benchmark suite, this observation lends more credibility to our claims that the observation structure impacts the performance of algorithms.

4.7 Conclusions

When choosing an RL approach to use, practitioners are overwhelmed with a variety of loss function choices, without clear indication which one will be preferable in what scenario. In our work, we introduce analytically tractable notions of distractions and observation functions. With these we predict the performance of latent self-prediction and observation reconstruction as stand-alone feature learning methods and auxiliary tasks, and study our theoretical insights empirically. Our evaluation lends credibility to the use of simple surrogate models to obtain practically relevant insights into algorithmic performance. However, in several cases we also find deviations between our predictions and more complex benchmarks. Therefore, while we claim that our theoretical models have the ability to guide the choice of algorithms for applied settings, there is still a sizeable gap between theory and practice that remains to be bridged in future work.

We also note that our experiments showed substantial differences in behavior of auxiliary losses both within and across benchmarks and different noise distractions. Previous work that studied the effect of distraction (Nikishin et al., 2022a; Voelcker et al., 2022; Ni et al., 2024) did not discuss their distraction model in detail. In light of our results, we suggest that empirical research should be careful about the choice of benchmark and experimental setup and discuss the implications of the empirical setup explicitly.

On the empirical side the surprising effectiveness of the observation prediction loss on the Seaquest environment highlights the fact that even within benchmark suites, differences in the reward functions and observation models can lead to differing rankings between algorithms. This further highlights the necessity of studying the structure of MDPs and to design algorithms that are robust to different structures, or adapt to them automatically.

Overall, we can conclude that model-based auxiliary tasks can be very helpful to structure latent representations in a way that complements the task of value function learning. More importantly for the overall project of this thesis, we now have our first concrete example of an algorithm that leverages both a general-purpose method, predicting latent features, and a decision-aware method, in this case TD learning, and shows that there exists a synergy between these methods for stabilizing feature learning!

However, we have so far only used the model learning to stabilize the feature space. Instead, we can also use the learned model itself to improve value function learning by generating additional data from the model in a Dyna style algorithm. In the next chapter we will take a look at such model-based methods. We will first investigate the idea of decision-aware learning in observation space models. In the following chapter, we will show that combining latent self-prediction with decision-aware model learning greatly improves the stability of the IterVAML paradigm.

Chapter 5

Value-gradient Aware Model Learning

This chapter is based on Claas Voelcker, Victor Liao, Animesh Garg, and Amir-massoud Farahmand (2022). “Value Gradient weighted Model-Based Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.

5.1 Introduction

While our previous chapters have focussed on representation learning, we will now shift our focus to decision-aware model learning. Model-based RL is an enticing paradigm for scenarios in which samples of the true environment are difficult or expensive to obtain, such as computationally intensive simulators, real-world robots, or environments involving humans, since the model can be used to generalize the policy to unseen regions of the state space. The approach has received a lot of attention and significant progress has been made in the field (Sutton, 1990; Deisenroth and Rasmussen, 2011; Levine and Koltun, 2013; Hafner et al., 2020; Moerland et al., 2023; Schrittwieser et al., 2020).

One of the core problems of model-based policy learning methods, however, is that the accuracy of the model directly influences the quality of the learned policy or plan (Schneider, 1997; Kearns and Singh, 2002; Ross and Bagnell, 2012; Talvitie, 2017; Luo et al., 2019; Janner et al., 2019). Model errors tend to accumulate over time, and therefore long-term planning under approximate models can lead to suboptimal policies compared to the performance achievable with model-free approaches. This is especially prevalent in settings with complex dynamics, such as robotic environments with discontinuities, which can be hard to model with common function approximation methods. These model approximation errors

are nearly impossible to avoid with current methods due to limits of function approximation in model and value learning algorithms, which cannot fully capture the full distribution over dynamics functions perfectly, and the use of finite datasets.

Hence, it is important that a model is accurate *where it counts* for the planning procedure, by modelling dimensions and data points that have a higher impact on the planning. But this objective is not captured in most current MBRL methods, which generally use MLE-based losses to train a parametric model of the environment without involving information from the planning process. In [Chapter 2](#) we reviewed this problem under the term *objective mismatch* (Lambert et al., 2020), but the problems has been investigated in several earlier works as well (Joseph et al., 2013). Several recent papers have investigated the objective mismatch (Abachi et al., 2020; Zhang et al., 2021; Ayoub et al., 2020; Grimm et al., 2020; Grimm et al., 2021; Nikishin et al., 2022a), but currently theoretical investigation and understanding of possible approaches do not perform well when applied to complex deep learning based approaches (Lovatto et al., 2020) or the proposed approaches rely on heuristics which might not be applicable generally (Nair et al., 2020).

Summary of Contributions. In this chapter, we present the *Value-Gradient Weighted Model Learning* (VaGraM) which rescales the mean squared error loss function with gradient information from the current value function estimate. We first discuss the disadvantages of the Value-Aware Model Learning framework (Farahmand et al., 2017; Farahmand, 2018) by conducting an empirical analysis of the optimization behavior, and form two hypotheses for the lack of empirical performance gain despite theoretical intuition: (a) the theory does not account for the complicated learning behavior of value functions discussed in the preceding two chapters and (b) it also does not address how to counter problems that arise when the state space is yet insufficiently explored in early stages of the model training. Our experiments show, qualitatively and quantitatively, that the VaGraM loss impacts the resulting state and value prediction accuracy, and that it solves the optimization problems of previously published approaches. Beyond pedagogical domains, we show that VaGraM performs on par with a current state-of-the art MBRL algorithms in more complex continuous control domains, while improving robustness to irrelevant dimensions in the state-space and smaller model sizes.

The VaGraM algorithm again follows our overarching idea of combining a general-purpose objective, mean-square error minimization, and a decision-aware one, reweighing by value function sensitivity. Instead of using shared network layers, here we demonstrate that we can also modify loss functions to directly incorporate both idea.

5.2 Value-Gradient Weighted Model Learning

One of the main drawbacks of model-based reinforcement learning is the fact that model errors propagate and compound when the model is used for planning (Schneider, 1997; Kearns and Singh, 2002; Talvitie, 2017). As discussed in Section 2.5, one attempt at addressing the objective mismatch is the IterVAML loss (Farahmand, 2018)

$$\mathcal{L}_V(\hat{p}, p, \mu) = \int \mu(x, a) \left| \underbrace{\int p(x'|x, a) V(x') dx'}_{\text{environment value estimate}} - \underbrace{\int \hat{p}(x'|x, a) V(x') dx'}_{\text{model value estimate}} \right|^2 d(x, a). \quad (5.1)$$

Farahmand (2018) presents error bounds for both model and value learning steps of the iteration, but did not test the algorithm to ascertain whether the presented error bounds are sufficient to guarantee a strong algorithm in practice. As our analysis in the previous two chapters shows, fitting value functions accurately enough to meet the assumptions of Farahmand (2018) is difficult when dealing with neural networks and partially covered state-action spaces. As we discuss now, IterVAML provides an intuitive fix to the model-mismatch problem, yet faces two key issues which lead to empirical ineffectiveness Lovatto et al. (2020). These phenomena are investigated and verified in detail in Section 5.3.

Value function evaluation outside of the empirical state-action distribution IterVAML suffers when randomly initialized models predict next states that are far away from the current data distribution or if the optimization procedure leads the model’s prediction outside of the covered state space. Since the value function has only been trained on the current data distribution, it will not have meaningful values at points outside of its training set. Nonetheless, these points can still achieve small value prediction errors if, due to the optimization process, the value function outside the training distribution happens to have the same value at the model prediction as the environment sample. We therefore require that our value-aware loss function should not directly depend on the value function at the model prediction, since these might be potentially meaningless.

Suboptimal local minima The predictions of the model can converge to a solution that is far away from the environment sample in the state space if the prediction’s and the ground truth’s values are equal. In this case, we find that the model-based value prediction often performs poorly after updating the value function as two different states are not guaranteed to have the same values after updating. We expect that the updated model loss forces the model prediction to a new solution, but due to the non-convex nature of the VAML loss in the state space, the model can get stuck or even diverge. This is especially prevalent when the previous minimum is situated outside of the empirically covered state space. A stable value-aware loss function should therefore have only one minimum in the state space

that lies within the empirical state distribution. The full loss function will likely still admit additional local minima in the parameter space due to the non-linear nature of the model itself, but the global optimum should coincide with the true model and the loss function should be convex in the state space.

Note that several of these issues are related to the fact that previous empirical works using the IterVAML loss have used observation-space models. As we discussed in the previous chapters, stabilizing value learning by regularizing the learned latent-space features alleviates many issues around divergence and uninformative values. In this chapter, we will also constrain ourselves to observation-space models, to understand the issues facing value-aware learning in detail. In [Chapter 6](#) we will show how combining IterVAML style losses with latent self-prediction can lead to strong value aware learning.

5.2.1 Approximating IterVAML with the value function gradient

From our insights, we obtain two criteria: a stable loss should not depend on the value function at states not captured in the dataset, and it should not admit local minima beyond an MLE solution. To derive a loss function that fulfils these requirements, we start from the assumption that the difference between the model prediction and the environment next states x' are small. This is implicitly required by many MBRL approaches, since an MLE model cannot be used to estimate the next state's value otherwise. We also assume that the model has small transition noise, akin to the model assumptions underlying MSE regression, otherwise the difference between a model sample and the next state sample might be large. Under this assumption, the IterVAML loss can be approximated by a Taylor expansion of the value function, where we denote the expansion of V around a reference point x' as $\hat{V}_{x'}$ and obtain $\hat{V}_{x'}(x) \approx V(x') + (\nabla_x V(x)|_{x'})^\top (x - x')$. Using this expansion at the next state sample $x'_i \in \mathcal{D}$ collected from the environment for each tuple independently instead of the original value function, the VAML error can be stated

$$\hat{\mathcal{L}}_{\hat{V}} = \frac{1}{|\mathcal{D}|} \sum_{\{x_i, a_i, x'_i\} \in \mathcal{D}} \left(V(x'_i) - \int \hat{P}_\theta(x'|x_i, a_i) (V(x'_i) + (\nabla_x V(x)|_{x'_i})^\top (x' - x'_i)) dx' \right)^2 \quad (5.2)$$

$$= \frac{1}{|\mathcal{D}|} \sum_{\{x_i, a_i, x'_i\} \in \mathcal{D}} \left(\int \hat{P}_\theta(x'|x_i, a_i) \left((\nabla_x V(x)|_{x'_i})^\top (x' - x'_i) \right) dx' \right)^2. \quad (5.3)$$

This objective function crucially does not depend on the value function at unknown state samples, all x'_i are in the dataset the value function is trained on, which solves the first of our major problems with the VAML paradigm.

We can simplify the objective above even further if we restrict ourselves to deterministic models of the form $\hat{x}'_i = f_\theta(x, a)$. Since VAML requires the expectation of the value function under the model and the environment to be equal, we can exchange the probabilistic model

with a deterministic one as long as we assume that the mean value function under the true environment is close to the empirical estimate of the value function from a single sample. We explore the prerequisites and consequences of this assumption further in [Chapter 6](#). The model loss is now

$$\sum_i \left((\nabla_x V(x)|_{x'_i})^\top (f_\theta(x_i, a_i) - x'_i) \right)^2. \quad (5.4)$$

We can see that the objective is similar to a mean squared error regression with a vector that defines the local geometry of the objective function. This vector can be interpreted as a measure of sensitivity of the value function at each data point and dimension. In regions where the value function changes significantly, the regression pushes the model to be very accurate.

5.2.2 Bound between Value-Aware Model Learning and VaGraM

The error in Taylor approximation $\mathcal{R}(V, x', f_\theta(s, a))$ is bounded by $\frac{M}{2}(x')^2 \|x' - f_\theta(x, a)\|^2$ with M dependent on the Hessian of the value function. Plugging this into the VAML loss and assuming worst case approximation errors, we obtain an upper bound on the VAML error

$$\mathbb{E}_{x', x, a \sim D} \left[(V(f_\theta(x, a)) - V(x'_0))^2 \right] \quad (5.5)$$

$$= \mathbb{E}_{x', x, a \sim D} \left[((\nabla_x V(x)|_{x'})^\top (f_\theta(x, a) - x') + \mathcal{R}(V, x', f_\theta(x, a)))^2 \right] \quad (5.6)$$

$$\leq \mathbb{E}_{x', x, a \sim D} \left[(|(\nabla_x V(x)|_{x'})^\top (f_\theta(x, a) - x')| + |\mathcal{R}(V, x'_0, f_\theta(x, a))|)^2 \right] \quad (5.7)$$

$$\leq \mathbb{E}_{x', x, a \sim D} \left[\left(|(\nabla_x V(x)|_{x'})^\top (f_\theta(x, a) - x')| + \frac{M(x')}{2} \|x' - f_\theta(x, a)\|^2 \right)^2 \right] \quad (5.8)$$

$$\leq 2 \cdot \mathbb{E}_{x', x, a \sim D} \left[((\nabla_x V(x)|_{x'})^\top (f_\theta(x, a) - x'))^2 \right] + 2 \cdot \mathbb{E}_{x', x, a \sim D} \left[\frac{M(x')^2}{4} \|x' - f_\theta(x, a)\|^4 \right]. \quad (5.9)$$

The parameter $M(x')$ depends on the Hessian at each state x' . However, computing the Hessian of the value function is an expensive operation, and therefore we found it infeasible to compute this term in practice. Alternatively, we could attempt to find a hyperparameter $M \geq M(x')$ for all x' and bound the loss this way. However, we can only tune this parameter approximately and did not achieve strong performance as we are forced to choose very conservative bounds. In this case, we are effectively only optimizing the polynomial term which is akin to performing mean squared regression (except for using the fourth power instead of the second).

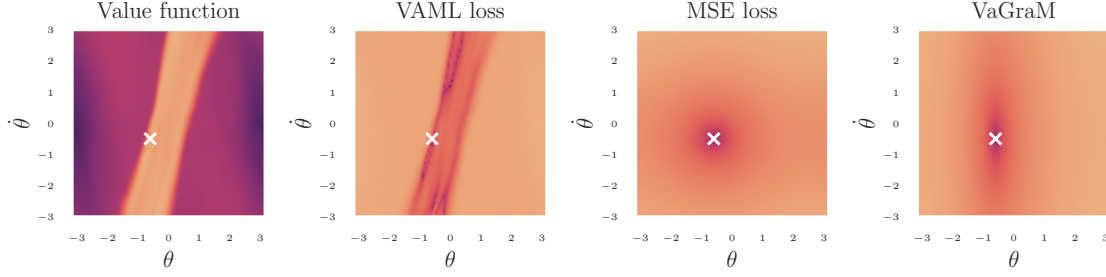


Figure 5.1: Visualization of discussed loss function with regards to a reference point marked with the white cross and the corresponding value function on the Pendulum environment. For the value function, darker color indicates a lower value. In the loss figures, darker color indicates how large the loss is if the model predicts $(\theta, \dot{\theta})$ instead of the reference sample marked in white. The VAML loss has a complex non-linear shape in the state space that follows iso-lines of the value function, while MSE and VaGraM are centered around the sample. For VaGraM, the rescaling of the MSE in the direction of high gradient along the θ axis is visible. Due to Equation 5.14, the scaling is aligned with the axis of the coordinate system and not rotated to fit the value function closer.

5.2.3 Preventing spurious local minima

The formulation above retains one problem: Equation 5.4 does not constrain the solution for each (x, a, x') tuple sufficiently. For each (x, a, x') tuple, the loss function only requires that the difference between the model and environment sample be orthogonal to the gradient of the value function, which describes a hyperplane of solutions. These predictions can lie arbitrarily far away from the environment sample, which breaks the assumption underlying the Taylor approximation that the model prediction is within a small region of the expanded state point. This can be made more clear by looking at the loss for a single datapoint. Writing $\dim(\mathcal{X})$ for the dimensionality of the state space, we obtain

$$\min_{\theta} ((\nabla_x V(x)|_{x'})^\top f_{\theta}(x, a))^2 \quad (5.10)$$

$$= \min_{\theta} \left(\sum_{n=0}^{\dim(\mathcal{X})} (\nabla_x V(x)|_{x'})_n \cdot f_{\theta}(x, a)_n \right)^2. \quad (5.11)$$

Assuming that f is flexible and can predict any next state x' , the optimal solution is obtained from an under-determined linear system of equations

$$\sum_{n=0}^{\dim(\mathcal{X})} (\nabla_x V(s)|_{x'})_n \cdot f_{\theta}(x, a)_n = 0 \quad (5.12)$$

This system admits far more solutions than either the corresponding IterVAML loss or a mean squared error, and many of them will achieve arbitrary large value prediction errors. In fact, the equation describes a hyperplane of minimal solutions consisting of every weight

vector that is orthogonal to the gradient of the value function at the reference sample, with $\dim(\mathcal{X}) - 1$ free variables. Therefore we need to enforce the closeness of the model prediction and the environment sample, since the Taylor approximation is only approximately valid in a close ball around the reference sample.

Again, one way to achieve this closeness is by adding the second order Taylor term, which results in an additional MSE loss term. However, as discussed, finding a good M is expensive in practice. Therefore, we approach the solution to this problem as outlined below.

5.2.4 Bounding the first-order Taylor loss

To prevent these suboptimal solutions and achieve our second design goal, we consider an upper bound on the value-gradient loss by applying a corollary of the Cauchy-Schwarz inequality $(\sum_{i=1}^n x_i)^2 \leq n \sum x_i^2$ to change the square of the sum with a sum of squares. We denote the diagonal matrix with vector a on the diagonal as $\text{diag}(a)$ and rephrase the sum as a vector-matrix multiplication

$$\sum_{\{x_i, a_i, x'_i\} \in \mathcal{D}} \left((\nabla_x V(x)|_{x'_i})^\top (f_\theta(x_i, a_i) - x'_i) \right)^2 \quad (5.13)$$

$$\leq \dim(\mathcal{X}) \sum_{\{x_i, a_i, x'_i\} \in \mathcal{D}} \left((f_\theta(x_i, a_i) - x'_i)^\top \text{diag}(\nabla_x V(x)|_{x=x'_i})^2 (f_\theta(x_i, a_i) - x'_i) \right). \quad (5.14)$$

This reformulation is equivalent to a mean squared error loss function with a per-sample-dimension diagonal scaling matrix. Because the scaling matrix is positive semi-definite by design, each summand in the loss is a quadratic function with a single solution as long as the derivative of the value function does not become zero in any component. Therefore this upper bound assures our second requirement: the loss function does not admit spurious local minima.¹

To give an intuitive insight into all the discussed loss functions, we visualized each one for a pedagogical environment, the Pendulum stabilization task. The resulting loss curves can be seen in Figure 5.1. The VAML loss has a complicated shape that depends on the exact values of the value function while both MSE and our proposal have a paraboloid shape. Compared to MSE, our proposed loss function is rescaled to account for the larger gradient of the value function in the θ axis.

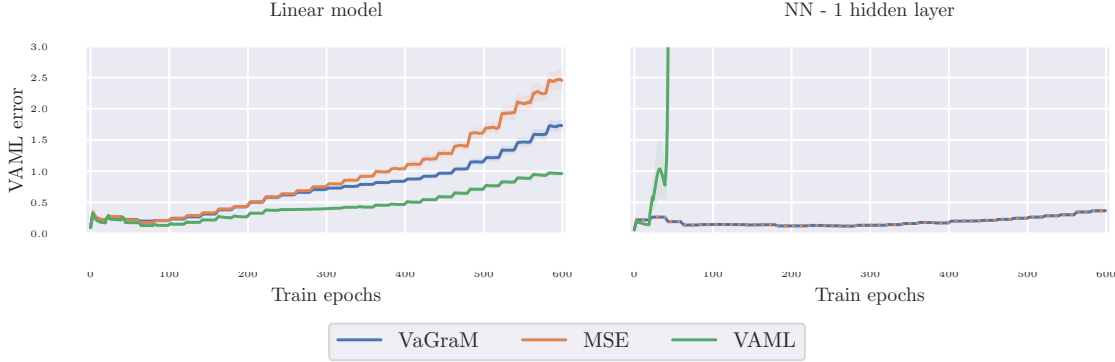


Figure 5.2: **Evolution of the VAML loss over changing value functions on the Pendulum domain.** Lines denote the mean and shaded areas show standard error over 8 model initialization and data set samples per model. In the linear setting, VAML achieves the lowest VAML error, while VaGraM is able to significantly outperform MSE. In the NN setting, VAML diverges rapidly, while VaGraM and MSE converge to approximately the same solution.

5.3 Experiment: Analyzing model learning

We compare the performance of VaGraM, with both MSE and VAML on a pedagogical environment with a small state space and smooth dynamics to gain qualitative insight into the loss surfaces. We use the Pendulum environment, a canonical control problem in which an under-actuated pendulum must be swung and stabilized to an upright position. We use the implementation provided by Brockman et al. (2016). To learn the policy and its value function, we use the SAC algorithm (Haarnoja et al., 2018). The original IterVAML analysis assumed that the value function was obtained using approximate value iteration (AVI) (Gordon, 1995; Ernst et al., 2005; Farahmand et al., 2010). We use SAC instead of a full AVI for stability in large scale experiments and discuss a proper extension of the VAML loss to SAC in Section D.1. We find that the difference in loss is negligible and therefore use SAC together with VAML throughout our experiments. More information on the implementation and hyperparameters of all of our experiments can be found in Section D.3.

To simplify the setup of the evaluation, we decided to investigate the model losses without model-based value function learning. This allows us to focus solely on the loss functions, without taking into account the inter-dependency between model and value function updates. Instead of the model-based loop, we used the SAC algorithm in a model-free setup to estimate the value function. We saved the intermediate value functions after each epoch of training, corresponding to 200 environment steps, and optimized the models using stochas-

¹We note that state dimensions are ignored for points in which components of the value function become zero, potentially leading to spurious minima as before solutions, but in practice this rarely happens for more than a few points.

tic gradient descent on the respective loss function, updating the value function used for the loss every 1000 model training steps. As the MLE loss, we used the mean squared error which assumes a Gaussian model with fixed variance.

To compare the optimization, we used two architectures, a linear model without feature transformations and a neural network with a single hidden layer and 16 neurons. We obtained a dataset of states sampled uniformly over the whole state space and used the environment transition function to compute ground truth next state samples. Finally, we evaluated each models VAML error with regards to the current value function on a held out dataset and plotted the results in Figure 5.2.

Dependency on value function estimates. The first cause for lacking empirical performance with VAML that we discussed in Section 5.2 was that the algorithm can predict successor states with incorrect value function as they lie outside of the data distribution.

In our experiment we find that a linear regression model remains stable under all three loss functions, VaGraM, MSE and VAML. But when using a flexible function approximation, the VAML loss converges in the first iteration with the given value function, but then rapidly diverges once the value function is updated. When investigating the mean squared error of the VAML solution, we find that the model finds a stable minimum of the VAML loss outside of the reachable state space of the pendulum. This confirms our hypothesis that flexible VAML models can find solutions outside of the empirical state space distribution, which are unstable once we update the value function. VaGraM remains stable even with flexible function approximation and achieves a lower VAML error than the MSE baseline when using a model with insufficient capacity to represent the dynamics.

Single solution convergence. In the experiment, we see that the MSE and VaGraM models converge to a similar solution when using a neural network. This leads us to the conclusion that our loss function really only admits a single solution and that this solution coincides with the mean square error prediction when the function approximation has sufficient capacity to model the dynamics function with high precision. On the other hand, the VAML network converges to solutions that are far away from the environment sample measured in the L_2 norm and it cannot recover from these spurious minima due to the complex optimization landscape.

5.4 Experiment: Model-based Continuous Control

Due to the limited complexity of the pendulum environment, the quantitative differences between the mean squared error and VaGraM are at times insignificant in this setting. The dynamics function of the environment can be approximated sufficiently well with a simple neural network and one hidden layer.

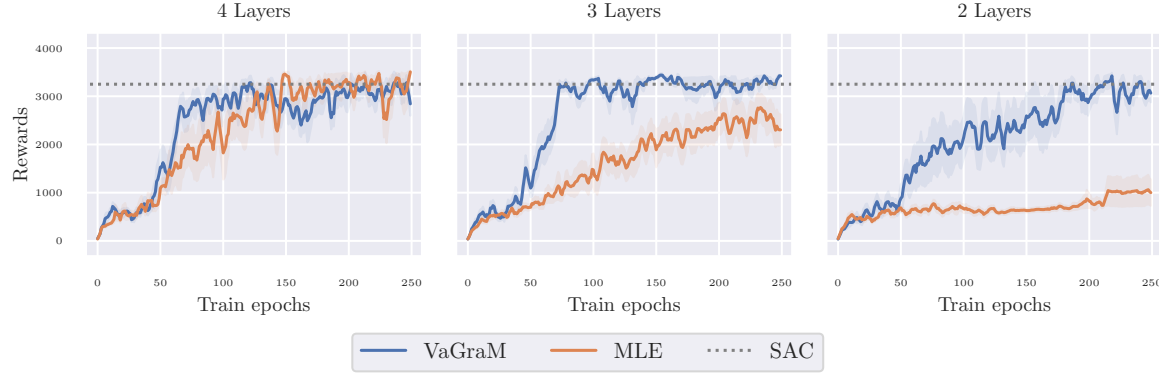


Figure 5.3: **Performance of VaGraM and MLE models with reduced model size.** The dotted lines correspond to the final performance reported for model-free SAC (grey, approx. 3200). Shaded area represents standard error over 16 repeated runs. VaGraM continues to solve the task almost unimpeded, while MLE is unable to even stabilize the Hopper when using a two layer neural network.

The underlying theory supporting VAML states that a value-aware loss is preferable to a mean squared error loss in a setting where the capacity of the model is too small to fully approximate the problem or the state space contains dimensions that are irrelevant for the control problem. To test whether our loss function is superior to a maximum likelihood approach in these cases, we used the Hopper environment from the OpenAI gym benchmark (Brockman et al., 2016). As a deep learning based Dyna algorithm, we chose MBPO (Janner et al., 2019) and ran all of our experiments using the implementation provided by Pineda et al. (2021). We kept the structure of the MBPO algorithm and models and replaced the model loss function with VaGraM.

5.4.1 Hopper with reduced model capacity

In the first experiment, we decreased the network size of the used neural network ensemble. Janner et al. (2019) use fully connected neural networks with four hidden layers and 200 neurons per layer. To test the performance of the algorithm under smaller models, we ran tests with two and three layer networks and 64 neurons per hidden layer. The results are shown in Figure 5.3. As before, when using a sufficiently powerful function approximation, we see no difference between the maximum likelihood approach and VaGraM, suggesting that the networks are flexible enough to capture the true environment’s dynamics sufficiently close for planning. But when reducing the model size, the maximum likelihood models quickly lose performance, completely failing to even stabilize the Hopper for a short period in the smallest setting, while VaGraM retains almost its original performance.

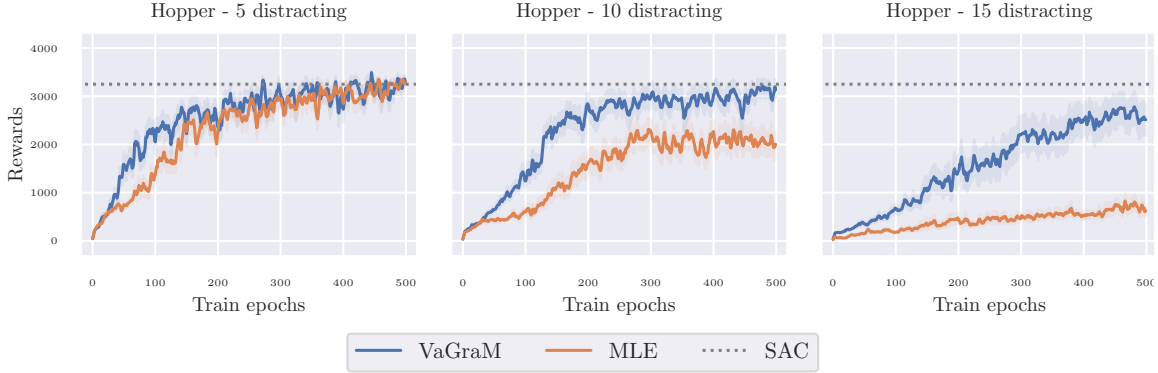


Figure 5.4: **Performance of VaGraM and MLE models with distracting state dimensions.** The dotted lines correspond to the final performance achieved by both algorithms on the Hopper task without distraction (grey, approx. 3200). Shaded area represents standard error over 16 repeated runs. VaGraM achieves significantly higher returns than the MLE baseline, especially in the most challenging setting with 15 distracting dimensions.

5.4.2 Hopper with distracting dimensions

To show that VaGraM is able to achieve good performance in a setting where there are additional dynamics in the environment that do not contribute to the control problem, we appended distracting dimensions to the Hopper observations. This mirrors the experimental setup used in Chapter 4. These are independent of the original environment state space and reward function, and evolve under non-linear and discontinuous dynamics (details in Section D.3). A setting with distracting dimensions is known to pose difficulty for model-based control algorithms (Stone et al., 2021) and neural networks struggle to model non-linear, discontinuous dynamics, so with an increasing number of dimensions the task becomes harder.

The results of this experiment are shown in Figure 5.4. When using five distracting dimensions, both models are able to capture the dynamics sufficiently well to achieve comparable reward to the original environment. When increasing the number of dimensions, the performance of the MLE model deteriorates, as more and more of its capacity is used to model the added dynamics. VaGraM continues to be able to achieve reward even under the presence of distracting dimensions, since the gradient of the value function with regards to the state dimensions which are irrelevant to the control problem becomes small over training. Still, the performance of VaGraM also suffers with increasing dimensions: when adding 20 distracting dimensions, neither algorithm is able to stabilize the Hopper consistently. In this case, the value function approximation cannot differentiate sufficiently between the relevant and irrelevant dimensions with the amount of environment samples provided.

In summary, we find that VaGraM is able to deal with challenging distractions and reduced model capacity significantly better than a MLE baseline. This validates that our algorithm

is really value-aware and can use the value function information to improve the performance of a model-based controller in settings where the model is unable to fully represent the environment.

5.5 Related work

Several authors have noted on the problem of learning models that align with the goal of obtaining a good policy. The proposed approaches fall into three broad categories: value-function or policy dependency, representation learning, and data resampling.

Inspired by VAML, Abachi et al. (2020) present a method that seeks to align the policy gradients under a learned model with the gradients under the true environment. Similar to our proposal, D’Oro et al. (2020) also proposed to reweigh samples in a log likelihood loss, but used policy search as the reinforcement learning approach and did not account for individual state dimensions. Nikishin et al. (2022a) show an approach to directly optimizing the policy performance on the real environment by learning a model with implicit differentiation. Asadi et al. (2018) show that the original VAML loss coincides with a Wasserstein distance in the model space under the assumption that the value function is Lipschitz smooth. We find that all of these approaches suffer from similar scaling issues as VAML and have not been shown to lead to strong empirical performance outside of toy settings.

Grimm et al. (2020) characterize the space of possible solutions to the model learning problem under different restrictions given by value functions and policies, and propose to learn models that are value-equivalent, similar to Farahmand et al. (2017). In a follow-up work Grimm et al. (2021) expand on this idea and show that their principle can be used to improve the MuZero algorithm (Schrittwieser et al., 2020). However, these works do not discuss the optimization challenges in actually finding such value-equivalent models, they mostly characterize the space under different value functions and policies, and present an orthogonal research direction to this chapter.

An alternative to characterizing the problem in the state space of the MDP are representation learning approaches, which seek to map the original states to a latent representation that is more amenable to control. Such approaches include Value Prediction Networks (Oh et al., 2017), Embed-to-Control (Watter et al., 2015) and related proposals (Levine et al., 2020; Cui et al., 2021). Zhang et al. (2021), Kemertas and Aumentado-Armstrong (2021), and Kemertas and Jepson (2022) build on the idea of bisimulation metrics (Ferns et al., 2004; Ferns et al., 2011) which seeks to characterize the difference between MDPs by finding a state mapping that is reward-invariant under policy and value function. In the following chapters, we show how to extend IterVAML and related approaches to include state-representation learning.

Lambert et al. (2020) hypothesize that the objective mismatch could be solved by reweighing the training buffer for model learning to prioritize data points with high value. These are more likely to matter for obtaining an optimal policy. A similar proposal was evaluated empirically by Nair et al. (2020). Contrary to our work however, this technique cannot account for the differing impact of the state space dimensions and scaling, since the data points are weighted as a whole.

5.6 Conclusion

This chapter presents a core issue with using decision-aware methods such as VAML and IterVAML. It shows that prior work does not account for two important optimization phenomena that appear when learning models with empirical value function approximations. To mitigate this issue, we introduced an attempt at getting the best of both decision-aware and general purpose learning, Value-Gradient Weighted Model Learning (VaGraM). VaGraM is a novel loss function to train models that model a dynamics function *where it matters* for the control problem. We highlighted how VaGraM counters the issues of pure decision-aware learning and show the increased stability of the training procedure when using our loss in a pedagogical environment.

On the Mujoco benchmark, VaGraM performs on par with maximum likelihood estimation when using large neural network models. However, introducing additional complications to the problem results in drastic performance impacts for MLE based models, which highlights the necessity for value function aware losses in challenging environments and settings in which sufficient model capacity cannot be guaranteed. In these cases, value-awareness can greatly increase the performance of Dyna algorithms by focusing the model learning procedure on relevant aspects of the state space.

While VaGraM is an important step towards our goal of unifying general purpose and decision-aware learning, it does not take into account the importance of feature learning. As we saw in the previous chapter, latent self-prediction objectives can lead to features which are aligned with the goal of value prediction. In the next chapter we will see that these losses can also serve as auxiliary losses to stabilize the problems with IterVAML that we observe in this chapter.

Chapter 6

Calibrated Latent Models for Value Aware Model Learning

This chapter is based on Claas Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-massoud Farahmand (2025b). “Calibrated value-aware loss functions with stochastic environment models”. In: *Proceedings of the International Conference on Machine Learning* and Claas Voelcker, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-massoud Farahmand (2024a). *λ -models: Effective Decision-Aware Reinforcement Learning with Latent Models*.

6.1 Introduction

As discussed in the previous chapter, the paradigm of *value-aware model learning* (VAML) (Farahmand et al., 2017) and *value-equivalence* (Grimm et al., 2020; Grimm et al., 2021) addresses this by training models that lead to accurate value estimation. Prominent value-aware model learning approaches are MuZero (Schrittwieser et al., 2020) and IterVAML (Farahmand, 2018). The MuZero loss has been shown to perform well in discrete (Schrittwieser et al., 2020; Ye et al., 2021) and continuous control tasks (Hansen et al., 2022; Hansen et al., 2024), but has received little theoretical investigation. On the other hand, IterVAML is a theoretically motivated algorithm but not commonly used in empirical work. In the previous chapter however we saw that empirically, IterVAML approaches tend to be unstable when used with observation-space models. However, as we discuss previous experiments with IterVAML and MuZero differ significantly in terms of the neural network architectures used in practice.

In this chapter we show that MuZero and IterVAML can be unified in a family of losses, which we term (m, b) -Value-Aware Model Losses ((m, b) -VAML). The name stresses the two core hyperparameters: the model rollout steps, m , and steps used to estimate the bootstrapped value function target, b .

The (m, b) -VAML losses are used as surrogate losses in place of other value- or model learning losses. Therefore, it is important to ask whether they are calibrated (Steinwart and Christmann, 2008). A calibrated loss does not lead to suboptimal minima when the function class includes optimal functions for the original target loss.

Research question: This chapter has three parts, each with a theoretical and empirical section. We answer two questions about the (m, b) -VAML family:

- (a) What variants of the (m, b) -VAML losses are well-calibrated to recover correct models and value functions?
- (b) How do deterministic latent-model architectures impact the stability of (m, b) -VAML algorithms?
- (c) Do we observe problems with uncalibrated losses when using stochastic architectures?

Contributions: As our main theoretical contribution, we mathematically analyze the family of (m, b) -VAML algorithms. We prove that all sample-based members of this loss family are uncalibrated when used with a stochastic environment model. To counter this issue, we derive a novel loss variant.

In the second part, we address issues arising from the way current algorithms in the (m, b) -VAML family are commonly implemented. We prove that a stochastic model class is not necessary to learn a single-step decision-equivalent model in stochastic environments. This validates the practice of primarily using deterministic models in empirical work (Oh et al., 2017; Schrittwieser et al., 2020; Hansen et al., 2022). Empirically we find that using stochastic models can still lead to improved performance, although this is environment-dependent.

6.2 Notation

Reinforcement Learning: We consider a standard as presented in Section 2.2. As a brief reminder, we review the notation. The policy-conditioned transition kernel is $\mathcal{P}^\pi(x'|x) = \int \mathcal{P}(x'|x, a)\pi(a|x)da$. The value is the unique fixed point of the Bellman operator

$$[\mathcal{T}_{\mathcal{P}^\pi} V](x) = \mathbb{E}_\pi [r(x_0, a_0) + \gamma \mathbb{E}_{\mathcal{P}^\pi} [V(x_1)] | x_0 = x].$$

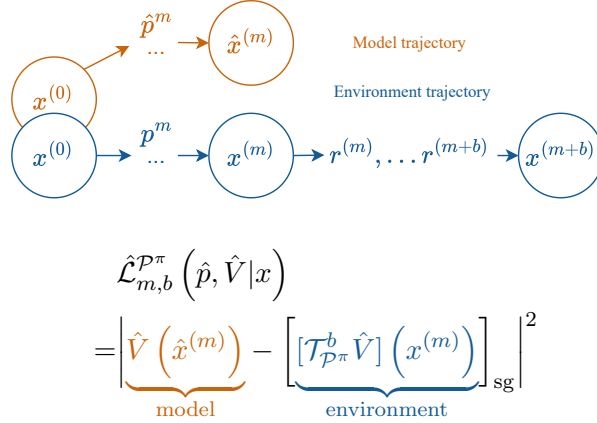


Figure 6.1: Sketch of (m, b) -VAML. The loss is computed from an m -step model and a $(m + b)$ -step environment trajectory. It is the difference between the estimated value of the m -th model state, and the b -step Bellman operator starting from the m -th environment state.

This operator can be extended to a multi-step version as

$$[\mathcal{T}_{\mathcal{P}^\pi}^b V](x) = \mathbb{E}_{\pi, \mathcal{P}} \left[\sum_{n=0}^{b-1} \gamma^n r(x_n, a_n) + \gamma^b V(x_b) \middle| x_0 = x \right].$$

We also define $[\mathcal{T}_{\mathcal{P}^\pi}^0 V](x) = V(x)$.

Model-based RL: In this chapter, we focus on value learning with model data, which is commonly referred to as Dyna (Sutton, 1990). We use $\hat{p}$ to refer to a stochastic model, and $\hat{f}$ for deterministic models. When a model is used to predict the observation x' from x, a (such as the model used in MBPO (Janner et al., 2019)) we call it an *observation-space models*. Alternatively, *latent-space models* of the form $\hat{p}(z'|z, a)$ are used, where $z \in \mathcal{Z}$ is a representation of a state $x \in \mathcal{X}$ given by $\phi : \mathcal{X} \rightarrow \mathcal{Z}$. Observation-space models predict next states in the representation of the environment, while latent-space models reconstruct learned features of these states.

The notation $x^{(n)}$ refers to the n -th step in a rollout starting in state x in the environment. We will use $\hat{x}^{(n)}$ to refer to samples from the n -th step model prediction and write $\mathbb{E}_{\hat{p}^{(m)}}[\cdot]$ as a shorthand for $\mathbb{E}_{x^{(m)} \sim \hat{p}(\cdot|x)}[\cdot]$.

6.3 The Value-Aware Model Learning framework

The losses of the decision-aware learning framework share the goal of finding models that provide good value function estimates. Instead of simply learning a model using maximum likelihood estimation, the losses are based on differences in value prediction.

6.3.1 Iterative Value Aware Model Learning

While IterVAML has been discussed in Section 2.5 and Chapter 5, we will now focus on its relationship to other loss functions. To achieve this, we reintroduce the IterVAML loss with a small modification, an m -step and k -sample variant. The loss function introduced previously can be seen as a single-step and expectation-based variant of this more general loss.

This more general IterVAML still computes the difference between the expected value under the model and samples in the environment

$$\begin{aligned}\mathcal{L}_{\text{IterVAML},m}^{\mathcal{P}^\pi}(\hat{p}, V|x) &= \left| \mathbb{E}_{\hat{p}^m} [V(\hat{x}^{(m)})] - \left[\mathbb{E}_{\mathcal{P}^\pi} [V(x^{(m)})] \right]_{\text{sg}} \right|^2 \\ \mathcal{L}_{\text{IterVAML},m}^{\mathcal{P}^\pi}(\hat{p}, V|x) &\approx \hat{\mathcal{L}}_{\text{IterVAML},m}^K(\hat{p}, V|x, x^{(m)}) = \left| \frac{1}{K} \sum_{k=1}^K [V(\hat{x}_k^{(m)})] - [V(x^{(m)})]_{\text{sg}} \right|^2.\end{aligned}\tag{6.1}$$

We will use $\mathcal{L}_{\text{IterVAML},m}$ to refer to the expectation-based version, and $\hat{\mathcal{L}}_{\text{IterVAML},m}^k$ to refer to the sampling-based version. To reduce notational complexity, we drop the action dependence in all following propositions; all results hold without loss of generality for the action-conditioned case as well. The expectation-based version of the IterVAML loss has an important relationship to the error of computing the model’s Bellman operator compared to the true environments Bellman operator.

Proposition 6.3.1.1. *Farahmand (2018) Let $\mathcal{P}^\pi$ be the policy-conditioned transition kernel of an MDP and let $V : \mathcal{X} \rightarrow \mathbb{R}$ be a function. Let $\hat{p}(\cdot|x)$ be a model so that $\mathcal{L}_{\text{IterVAML},1}^{\mathcal{P}^\pi}(\hat{p}, V|x) = 0$. Then $[\mathcal{T}_{\mathcal{P}^\pi} V](x) = [\mathcal{T}_{\hat{p}} V](x)$.*

Proof. Assume $\hat{p}$ fulfills $\mathcal{L}_{\text{IterVAML},1}^{\mathcal{P}^\pi}(\hat{p}, \hat{V}|x) = 0$. Then $\mathbb{E}_{\hat{p}} [V(\hat{x}^{(1)})] = \mathbb{E}_{\mathcal{P}^\pi} [V(x^{(1)})]$. By definition of the Bellman operator, we have

$$[\mathcal{T}^{\mathcal{P}^\pi} V](x) = r(x) + \gamma \mathbb{E}_{x^{(1)} \sim \mathcal{P}^\pi(\cdot|x)} [V(x^{(1)})] = r(x) + \gamma \mathbb{E}_{x^{(1)} \sim \hat{p}(\cdot|x)} [V(x^{(1)})] = [\mathcal{T}^{\hat{p}} V](x).$$

□

Intuitively, a model achieving 0 loss can be used instead of the ground truth environment when computing the Bellman operator. The set of models that achieve 0 IterVAML error is equal to the set of value-equivalent models for the value estimate $\hat{V}$ (Grimm et al., 2021).

6.3.2 The (m,b)-VAML Family

The MuZero loss (Schrittwieser et al., 2020) was introduced to unify the value function and model learning components of an MBRL algorithm. It can be interpreted as a variation

of the IterVAML loss that uses a single sample estimate of the Bellman Operator and a bootstrapped target estimate. To unify the losses, we present them in a single equation with two important hyperparameters: m , the number of steps in the trajectory that the loss is computed over, and b , the number of steps for the multi-step Bellman operator. We refer to this unified family of losses as (m, b) -Value Aware Model Losses (VAML)

$$\hat{\mathcal{L}}_{m,b}^{\mathcal{P}^\pi}(\hat{p}, \hat{V}|V_{\text{tar}}, x, x^{(m)}) = \left| \hat{V}(\hat{x}^{(m)}) - [\mathcal{T}_{\mathcal{P}^\pi}^b V_{\text{tar}}](x^{(m)}) \right|_{\text{sg}}^2, \quad (6.2)$$

where V_{tar} is a target network (Mnih et al., 2013). We denote the stop-gradient operation with $[\cdot]_{\text{sg}}$. Note that samples from the real environment are used to approximate $[\mathcal{T}_{\mathcal{P}^\pi}^b \hat{V}]$. The loss function and the relation of the model and environments rollout are visualized in Figure 6.1.

Several works use variations of this loss: The original MuZero algorithm (Schrittwieser et al., 2020) and follow-up work (Ye et al., 2021; Antonoglou et al., 2022) use $m \geq 1$ and $b \geq 1$. In continuous control, $m \geq 1, b = 1$ has been used in the TD-MPC line of work (Hansen et al., 2022; Hansen et al., 2024). When using only a single sample $k = 1$, the sample-based IterVAML loss is equal to $\hat{\mathcal{L}}_{m,0}^{\mathcal{P}^\pi}$. Finally, regular model-free TD learning corresponds to $m = 0, b \geq 1$.

6.4 Decision-aware losses in stochastic environments

The goal of learning in MBRL is to recover an (approximately) optimal model and to learn a correct value function. As we have shown, minimizing the IterVAML loss perfectly results in a model that leads to a correct Bellman Operator. However, in practice, the inner expectation of the IterVAML loss has to be replaced by a sampling-based approximation. In addition, $(m, b \geq 1)$ -VAML, MuZero, is used to update the value function directly in addition to the model. We show when this leads to learning correct models and value functions asymptotically. An overview of our conclusions can be found in Table 6.1. Proofs are found in Section A.2.

6.4.1 Calibration of surrogate loss functions

Formally, we ask whether the surrogate (m, b) -VAML is *calibrated*. Intuitively, a calibrated surrogate loss does not select a suboptimal function for the target loss. Formally, we require that the surrogate loss is not perfectly minimized by a function that does not perfectly minimize the target loss. Therefore, we call losses that do not have this property *minimum-uncalibrated* losses, or simply *uncalibrated* outside of formal statements. We borrow the concept from Steinwart and Christmann (2008), however, we use a slightly more restricted definition of *uncalibrated* here. This means that all losses which we consider uncalibrated

Model	Env	IterVAML ($m \geq 1, b = 0$)	MuZero ($m \geq 1, b \geq 1$)	Model-free TD ($m = 0, b \geq 1$)
Det. model,	det. env	calibrated	calibrated	calibrated
Stoch. model,	det. env	uncalibrated (resolvable)	uncalibrated (for VF)	calibrated
Det. model,	stoch. env	calibrated	calibrated	calibrated
Stoch. model,	stoch. env	uncalibrated (resolvable)	uncalibrated (for VF)	calibrated
Can update model		✓	✓	X
Can update VF		X	✓ (for det. model)	✓

Table 6.1: Comparison of the major design choices in the (m, b) -VAML framework. IterVAML and MuZero can be used to update the model, while MuZero and Model-free TD learning can be used to update the value function. All model-based losses are uncalibrated when applied to stochastic model classes. In addition, MuZero suffers a bias when used for value function prediction which cannot be surmounted with an easy modification to the loss function.

are also uncalibrated in the definition given in Steinwart and Christmann (2008), but not vice versa.

Definition 6.4.1.1 (Minimum-uncalibrated surrogate losses). *Let $\mathcal{L}_{\text{tar}}(f, x, x^{(m)})$ be a loss function defined over samples from an MDP. Let $\hat{\mathcal{L}}_{\text{sur}}$ be a surrogate function for the loss. Let $\mathcal{F}^*$ be a set of (perfect) minima of $\mathcal{L}_{\text{tar}}$ so that $\mathcal{L}_{\text{tar}}(f^*, x, x^{(m)}) = 0$ for all $f^* \in \mathcal{F}^*$. A surrogate function is minimum-uncalibrated for the target loss if there exists an MDP, a function class $\mathcal{F}$ with $\mathcal{F}^* \subseteq \mathcal{F}$, and a state x so that*

$$\arg \min_{f \in \mathcal{F}} \mathbb{E}_{x^{(m)} \sim \mathcal{P}^\pi(\cdot|x)} [\mathcal{L}_{\text{sur}}(f, x, x^{(m)})] \not\subseteq \mathcal{F}^*.$$

A prominent example of an minimum-uncalibrated loss is Bellman Residual Minimization (BRM), which results in the *double sampling issue*. We reviewed this issue and a proof in Section 2.3 and will see how related issues arise here. Recall that while in BRM the target loss is minimized by the ground-truth value function, BRM actually chooses a function that minimizes an additional variance term.

We will show that some issues introduced by naively using (m, b) -VAML can be fixed with a tractable modification to the loss. We call such cases *resolvable*.

6.4.2 Model learning bias with stochastic models

Most prior works use (m, b) -VAML with deterministic models. Exceptions are Voelcker et al. (2022) and Antonoglou et al. (2022), but neither changes the loss functions to account for the model parametrization. For a stochastic model class, $(m, 0)$ -VAML is uncalibrated for the target loss $\mathcal{L}_{\text{IterVAML}}$.

To show this for a general class of MDPs, we require the existence of at least one MPD with transition function that fulfils the following condition.

Assumption 6.4.2.1. *Let p be a probability distribution over $\mathcal{X}$, and $f : \mathcal{X} \rightarrow \mathbb{R}$, so that no x exists for which $f(x) = \mathbb{E}_p[f(x)]$.*

The existence of such an MDP is trivial to show. Take, for example, a two state single action MDP with a transition matrix and function of

$$\mathcal{P} = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}, f = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Take p in the assumption to be $\mathcal{P}(\cdot|x_1)$. In this case, $\mathbb{E}_p[f(x)] = 0.5$, but no state x exists so that $f(x) = 0.5$. Therefore, the assumption is justified by example.

This is an important assumption as it guarantees that there is no distribution q with 0 variance such that $\mathbb{E}_q[f(x)] = \mathbb{E}_p[f(x)]$, which would result in a “trivial” deterministic IterVAML error. This excludes a corner case of our proof, as fully deterministic environments and models do not lead to an uncalibrated IterVAML loss. The assumption is used to prove the existence of a “better” solution under the surrogate sample-based loss that does not minimize the IterVAML target loss.

We will revisit this assumption in [Subsection 6.6.1](#), where we will relax the assumption on f to show that if we can embed our states x into a continuous representation space (under some additional technical conditions) a deterministic model with 0 IterVAML loss always exists.

We begin with analyzing the loss for $m \geq 1$ and $b = 0$.

Proposition 6.4.2.1. *Let $\hat{\mathcal{L}}_{m \geq 1, 0}^{\mathcal{P}^\pi}(\hat{p}, V|x, x^{(m)})$ be a surrogate loss for $\mathcal{L}_{\text{IterVAML}, m}^{\mathcal{P}^\pi}(\hat{p}, V|x)$. Let $\mathcal{P}^*$ be the set of all distributions p for which $\mathbb{E}_p[V(\hat{x}^{(m)})] = \mathbb{E}_{\mathcal{P}^\pi}[V(x^{(m)})]$. There exist an MDP and a class of distributions $\mathbb{P}$ with $\mathcal{P}^\pi \in \mathcal{P}^* \subseteq \mathbb{P}$ so that*

$$\text{Arg min}_{\hat{p} \in \mathbb{P}} \mathbb{E}_{\hat{p}} \left[\hat{\mathcal{L}}_{m \geq 1, 0}^{\mathcal{P}^\pi}(\hat{p}, V|x, x^{(m)}) \right] \not\subseteq \mathcal{P}^*.$$

Therefore, $\hat{\mathcal{L}}_{\text{model}}$ is minimum-uncalibrated.

Proof. Expanding the empirical IterVAML loss with k samples, obtain

$$\mathbb{E}_{\mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{m \geq 1, 0}^{\mathcal{P}^\pi}(\hat{p}, V|x, x^{(m)}) \right] \tag{6.3}$$

$$= \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(V(\hat{x}^{(m)}) - V(x^{(m)}) \right)^2 \right] \tag{6.4}$$

$$= \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(V(\hat{x}^{(m)}) - \mathbb{E}_{\hat{p}}[V(\hat{x}^{(m)})] + \mathbb{E}_{\hat{p}}[V(\hat{x}^{(m)})] - V(x^{(m)}) \right)^2 \right] \tag{6.5}$$

$$= \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(V(\hat{x}^{(m)}) - \mathbb{E}_{\hat{p}}[V(\hat{x}^{(m)})] \right)^2 \right] + \tag{6.6}$$

$$\begin{aligned}
& \underbrace{2 \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(V(\hat{x}^{(m)}) - \mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] \right) \left(\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} [V(\hat{x}^{(m)})] - V(x^{(m)}) \right) \right]}_{=0} + \\
& \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] - V(x^{(m)}) \right)^2 \right] \\
& = \underbrace{\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(V(\hat{x}^{(m)}) - \mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] \right)^2 \right]}_{=\text{Var}} \\
& + \underbrace{\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] - V(x^{(m)}) \right)^2 \right]}_{(2)}
\end{aligned} \tag{6.7}$$

Equation 6.7 is 0 since $\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} [V(\hat{x}^{(m)}) - \mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})]] = 0$, and samples from $\hat{p}$ and $\mathcal{P}^\pi$ are independent.

The first term is the variance of the value under the model. The final term can again be decomposed as

$$(2) = \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] - V(x^{(m)}) \right)^2 \right] \tag{6.8}$$

$$= \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] - \mathbb{E}_{\mathcal{P}^\pi} [V(x^{(m)})] + \mathbb{E}_{\mathcal{P}^\pi} [V(x^{(m)})] - V(x^{(m)}) \right)^2 \right] \tag{6.9}$$

$$\begin{aligned}
& = \underbrace{\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] - \mathbb{E}_{\mathcal{P}^\pi} [V(x^{(m)})] \right)^2 \right]}_{=\text{IterVAML}} \\
& + \underbrace{\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\mathcal{P}^\pi} [V(x^{(m)})] - V(x^{(m)}) \right)^2 \right]}_{\text{independent of } \hat{p}}.
\end{aligned} \tag{6.10}$$

The middle term of the binomial expansion again vanishes as $\mathbb{E}_{\mathcal{P}^\pi} [\mathbb{E}_{\mathcal{P}^\pi} [V(x^{(m)})] - V(x^{(m)})] = 0$, and samples from $\hat{p}$ and $\mathcal{P}^\pi$ are independent.

When we drop the term that is independent of the model, the loss is composed of the target loss, and an additional variance term.

$$\mathbb{E}_{\mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{i \geq 1, 0}^{\mathcal{P}^\pi} (\hat{p}, V | x, x^{(m)}) \right] \tag{6.11}$$

$$\begin{aligned}
& = \underbrace{\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(V(\hat{x}^{(m)}) - \mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] \right)^2 \right]}_{=\text{Var}} \\
& + \underbrace{\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\hat{p}} [V(\hat{x}^{(m)})] - \mathbb{E}_{\mathcal{P}^\pi} [V(x^{(m)})] \right)^2 \right]}_{=\text{IterVAML}}
\end{aligned} \tag{6.12}$$

We prove in Lemma A.2.1.3 that there exist problems in which the ground truth model does not minimize this loss. Let $\mathcal{P}^\pi(\cdot|x)$ be a distribution so that Lemma A.2.1.3 holds. Then there exists a $\hat{p}$ with

$$\hat{\mathcal{L}}_{m \geq 1, 0}^{\mathcal{P}^\pi}(\hat{p}, V|x, x^{(m)}) < \hat{\mathcal{L}}_{m \geq 1, 0}^{\mathcal{P}^\pi}(\mathcal{P}^\pi, V|x, x^{(m)}) \text{ and } \mathbb{E}_{\hat{p}}[V(\hat{x}^{(m)})] \neq \mathbb{E}_{\mathcal{P}^\pi}[V(x^{(m)})]. \quad \square$$

If we use the k sample version of the empirical IterVAML loss instead, we obtain the following decomposition

$$\mathbb{E}_{\mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{m \geq 1, 0}^{\mathcal{P}^\pi}(\hat{p}, V|x, x^{(m)}) \right] \quad (6.13)$$

$$\begin{aligned} &= \frac{1}{k} \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\underbrace{\left(V(\hat{x}^{(m)}) - \mathbb{E}_{\hat{p}}[V(\hat{x}^{(m)})] \right)^2}_{=\frac{1}{k} \text{Var}} \right] \\ &\quad + \underbrace{\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\hat{p}}[V(\hat{x}^{(m)})] - V(x^{(m)}) \right)^2 \right]}_{\text{IterVAML}}. \end{aligned} \quad (6.14)$$

The only difference when using more samples is that the variance term is scaled by the factor $\frac{1}{k}$, as the variance of the mean estimator $\sum_{i=1}^k V(x_i^{(m)})$ decreases. Consequently, for larger values of k , the condition in Lemma A.2.1.2 for g becomes stricter and the calibration error decreases. As $k \rightarrow \infty$, the condition becomes unfulfillable. This is also intuitive, as

$$\frac{1}{k} \sum_{i=1}^k V(x_i^{(m)}) \rightarrow \mathbb{E}_{\hat{p}}[V(x^{(m)})]$$

as $k \rightarrow \infty$ almost surely (assuming standard conditions hold).

When using samples from a stochastic model to compute the loss function in Equation 6.2, we are left with a variance error term that is closely related to the double-sampling problem

$$\mathbb{E}_{\hat{p}}[\hat{\mathcal{L}}_{1, 0}^{\mathcal{P}^\pi}(\hat{p}, V|x)] = \mathcal{L}_{\text{IterVAML}, 1}^{\mathcal{P}^\pi}(\hat{p}, V|x, x') + \text{Var}_{\hat{p}}(V(\hat{x})). \quad (6.15)$$

In the classic double-sampling problem, we cannot correct the variance term as we do not have oracle access to the environment. Here the issue is our model, and we can generate multiple samples from it. Therefore, we can estimate this variance term from samples and correct the loss. This correction is reminiscent of Antos et al. (2008) but is simpler to obtain as we only need model samples.

To obtain the correction, we define $\hat{\mu}_{\hat{p}}^{m, k} = \frac{1}{k} \sum_{i=1}^k V(\hat{x}_i^{(m)})$, the empirical estimator of the expected return in Equation 6.1. The variance of this estimator can be estimated as $\widehat{\text{Var}}_{\hat{p}}^{m, k} = \frac{1}{k} \sum_{i=1}^k (V(\hat{x}_i^{(m)}) - \hat{\mu}_{\hat{p}}^{m, k})^2$. With this we can define a new loss which can be

computed with at least two samples from the model

$$\hat{\mathcal{L}}_{\text{CVAML},m}^k = \hat{\mathcal{L}}_{\text{IterVAML},m}^k - \widehat{\text{Var}}_{\hat{p}}^{i,k}.$$

We refer to the loss as *Corrected VAML* (CVAML).

Proposition 6.4.2.2. *Let $\hat{\mathcal{L}}_{\text{CVAML},m}^k = \hat{\mathcal{L}}_{\text{IterVAML},m}^k - \widehat{\text{Var}}_{\hat{p}}^{i,k}$. The variance-corrected loss $\hat{\mathcal{L}}_{\text{CVAML},m}^k(\hat{P}, V|x, x^{(m)})$ is a calibrated surrogate loss for $\mathcal{L}_{\text{IterVAML},m}^{\mathcal{P}^\pi}(\hat{P}, V|x, x^{(m)})$.*

Proof. Reusing the previous derivation in the proof of Proposition 6.4.2.1, Equation 6.12, we obtain

$$\mathbb{E}_{\mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{\text{var},m}^k(\hat{P}, V|x, x^{(m)}) \right] \quad (6.16)$$

$$= \mathbb{E}_{\mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{i \geq 1,0}^{\mathcal{P}^\pi}(\hat{p}, V|x, x^{(m)}) \right] - \underbrace{\frac{1}{k} \mathbb{E}_{\hat{x}, \mathcal{P}^\pi} \left[\left(V(\hat{x}^{(m)}) - \mathbb{E}_{\hat{p}} \left[V(\hat{x}^{(m)}) \right] \right)^2 \right]}_{=\text{Var}} \quad (6.17)$$

$$= \underbrace{\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\mathbb{E}_{\hat{p}} \left[V(\hat{x}^{(m)}) \right] - \mathbb{E}_{\mathcal{P}^\pi} \left[V(x^{(m)}) \right] \right)^2 \right]}_{\text{IterVAML}}. \quad (6.18)$$

Therefore, the surrogate loss and target loss are equivalent in expectation. As the target loss and the surrogate loss are equivalent in expectation, all minimizer of the expected CVAML loss minimize the target loss, which fulfills the requirement for calibration. $\square$

Analogous to the IterVAML case, the sample-based $(m, b \geq 1)$ -VAML loss is an uncalibrated loss for learning a model. To address this issue, we will introduce a corresponding variance correction term.

6.4.3 Value learning bias in stochastic environments

We now show this issue also affects the value function learning with the MuZero loss. The problem lies in the use of the bootstrapped Bellman target together with a multi-step value rollout. In an MDP, the values of two states x and y are not guaranteed to be equal (or even particularly close) just because they share an ancestor state, unless we make assumptions about the variance of the value function over successor states. However, the MuZero loss still minimizes the difference in value functions between these two states.

We show that the MuZero loss therefore is not guaranteed to recover the correct value function, even when we have a perfect (stochastic) model. To formalize the issue, we compare the solution found with (m, b) -VAML with the regular TD loss function

$$\begin{aligned} \mathcal{L}_{\text{TD}}(V|V_{\text{tar}}, x^{(m)}, x^{(m+1)}, r^{(m)}) \\ = \left(V(x^{(m)}) - \left[r^{(m)} + \gamma V_{\text{tar}}(x^{(m+1)}) \right] \right)^2. \end{aligned} \quad (6.19)$$

Note that the issue arises because we are estimating values of successor states with the MuZero loss. Therefore, while the loss depends on the state x , all computation is actually conducted wrt the sampled successor states $x^{(m)}$.

Proposition 6.4.3.1. *Let $\mathcal{L}_{\text{TD}}(V|V_{\text{tar}}, x^{(m)})$ be the target loss, and let $\hat{\mathcal{L}}_{m,1}^{\mathcal{P}^\pi}(\mathcal{P}^\pi, V|V_{\text{tar}}, x, x^{(m)}, r^{(m)})$ be the surrogate loss. There exists a set of functions $\mathcal{V}$ for any V_{tar} that is not a constant function, for which two conditions hold:*

1. *The set is complete, meaning that $[\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}] \in \mathcal{V}$ for some target function V ,*
2. *and,*

$$\text{Arg min}_{\hat{V} \in \mathcal{V}} \mathbb{E}_{\mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{m,1}^{\mathcal{P}^\pi}(\mathcal{P}^\pi, \hat{V}|V_{\text{tar}}, x, x^{(m)}) \right] \not\subseteq [\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}].$$

Therefore, $\hat{\mathcal{L}}_{m,1}^{\mathcal{P}^\pi}$ is minimum-uncalibrated.

Proof. By assumption, let $\hat{p}$ in the MuZero loss be the true transition kernel p . Expand the MuZero loss by $[\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}](\hat{x}^{(m)})$ as before and take its expectation:

$$\mathbb{E}_{\mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{m,b}^{\mathcal{P}^\pi}(\mathcal{P}^\pi, V|V_{\text{tar}}, x, x^{(m)}) \right] \quad (6.20)$$

$$= \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left[\hat{V}(\hat{x}^{(m)}) - \left[r(x^{(m)}) + \gamma V_{\text{tar}}(x^{(m+1)}) \right] \right]^2 \right] \quad (6.21)$$

$$= \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\hat{V}(\hat{x}^{(m)}) - [\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}](\hat{x}^{(m)}) \right)^2 \right] + \quad (6.22)$$

$$2 \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\hat{V}(\hat{x}^{(m)}) - [\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}](\hat{x}^{(m)}) \right) \cdot \right. \quad (6.23)$$

$$\left. \left([\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}](\hat{x}^{(m)}) - \left[r(x^{(m)}) + \gamma V_{\text{tar}}(x^{(m+1)}) \right] \right) \right] + \quad (6.24)$$

$$\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left([\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}](\hat{x}^{(m)}) - \left[r(x^{(m)}) + \gamma V_{\text{tar}}(x^{(m+1)}) \right] \right)^2 \right] \quad (6.25)$$

We aim to study the minimizer of this term. The first term (Equation 6.22) is the bootstrapped Bellman residual with a target V_{tar} . The third term (Equation 6.25) is independent of $\hat{V}$, so we can drop it when analyzing the minimization problem.

The second term (Equation 6.23 and Equation 6.24) simplifies to

$$\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\hat{V}(\hat{x}^{(m)}) \left([\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}](\hat{x}^{(m)}) - \left[r(x^{(m)}) + \gamma V_{\text{tar}}(x^{(m+1)}) \right] \right) \right] \quad (6.26)$$

as the remainder is independent of $\hat{V}$ again.

This remaining term however is not independent of $\hat{V}$ and not equal to 0 either. Instead, it decomposes into a variance-like term, using the conditional independence of $\hat{x}^{(1)}$ and $x^{(1)}$

given $x^{(0)}$:

$$\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\hat{V} \left(\hat{x}^{(m)} \right) \left(\left[\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}} \right] \left(\hat{x}^{(m)} \right) - \left[r \left(x^{(m)} \right) + \gamma V_{\text{tar}} \left(x^{(m+1)} \right) \right] \right) \right] \quad (6.27)$$

$$= \mathbb{E}_{\hat{p}} \left[\hat{V} \left(\hat{x}^{(m)} \right) \left[\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}} \right] \left(\hat{x}^{(m)} \right) \right] - \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\hat{V} \left(\hat{x}^{(1)} \right) \left[r \left(x^{(m)} \right) + \gamma V_{\text{tar}} \left(x^{(m+1)} \right) \right] \right] \quad (6.28)$$

$$= \mathbb{E}_{\hat{p}} \left[\hat{V} \left(\hat{x}^{(m)} \right) \left[\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}} \right] \left(\hat{x}^{(m)} \right) \right] - \mathbb{E}_{\hat{p}} \left[\hat{V} \left(\hat{x}^{(m)} \right) \right] \mathbb{E}_{\mathcal{P}^\pi} \left[\left[r \left(x^{(m)} \right) + \gamma V_{\text{tar}} \left(x^{(m+1)} \right) \right] \right]. \quad (6.29)$$

Combining this with Equation 6.22, we obtain

$$\mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{m,b}^{\mathcal{P}^\pi}(\mathcal{P}^\pi, V | V_{\text{tar}}, x, x^{(m)}) \right] \quad (6.30)$$

$$= \mathbb{E}_{\hat{p}, \mathcal{P}^\pi} \left[\left(\hat{V} \left(\hat{x}^{(m)} \right) - \left(\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}} \right) \left(\hat{x}^{(m)} \right) \right)^2 \right] + \quad (6.31)$$

$$\mathbb{E}_{\hat{p}} \left[\hat{V} \left(\hat{x}^{(m)} \right) \left[\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}} \right] \left(\hat{x}^{(m)} \right) \right] - \mathbb{E}_{\hat{p}} \left[\hat{V} \left(\hat{x}^{(m)} \right) \right] \mathbb{E}_{\mathcal{P}^\pi} \left[\left[r \left(x^{(m)} \right) + \gamma V_{\text{tar}} \left(x^{(m+1)} \right) \right] \right]. \quad (6.32)$$

The first summand is the Bellman residual, for which the only minimizer is $\mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}}$. However, by Lemma A.2.1.4, we can construct a function class so that

$$\text{Arg min } \mathbb{E}_{\mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{m,b}^{\mathcal{P}^\pi}(\mathcal{P}^\pi, V | V_{\text{tar}}, x, x^{(m)}) \right] \not\subseteq \{ \mathcal{T}_{\mathcal{P}^\pi} V_{\text{tar}} \}.$$

□

While our proof only discusses the case $b = 1$, the same issue also appears with larger b . In many cases, the problem will be exacerbated by longer rollout horizons m and b , as the variance of all functions involved grows with the time horizon.

The problem in the loss function again depends on the variance of the value function with regard to the model. To overcome this issue, we will introduce another variance correction term, similar to $\hat{\mathcal{L}}_{\text{CVAML}}$. This retains the advantage that the MuZero loss is used for both model and value updates. However, as we average over the value prediction from the model, we only guarantee that

$$\mathbb{E}_{\hat{p}} \left[V(\hat{x}^{(m)}) \right] \approx \mathbb{E}_{\mathcal{P}^\pi} \left[\left[\mathcal{T}_{\mathcal{P}^\pi}^b V_{\text{tar}} \right] (x^{(m)}) \right], \quad (6.33)$$

not that the values for each state are correct. However, these are necessary for accurate planning or policy improvement. Therefore, it is still important to use a loss variant with $m = 0$ for learning accurate values for each state.

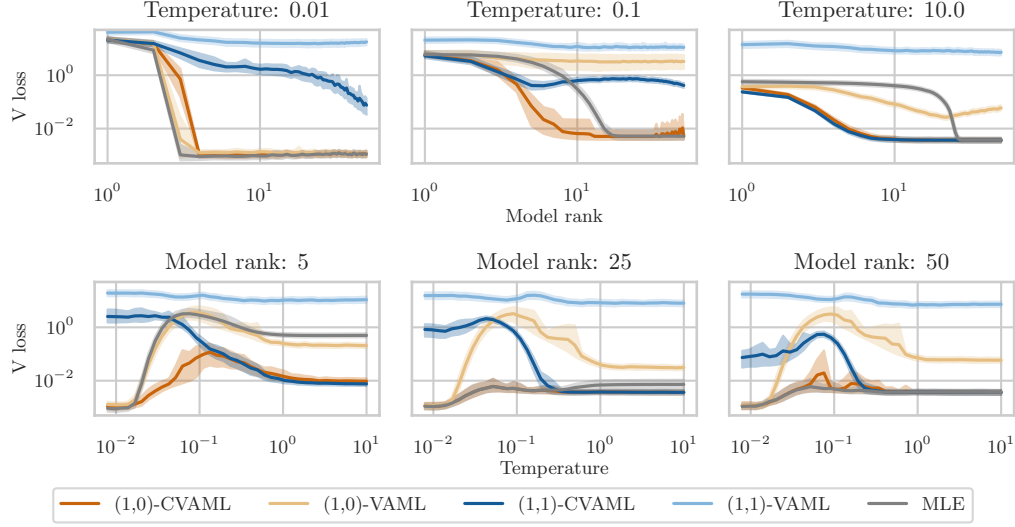


Figure 6.2: Results for the Garnet experiments. In the top row, we show the mean squared error of the value function prediction using different latent sizes k , over three different temperatures. In the bottom row, we vary the temperature and show results for three values of k . Shaded regions are bootstrapped confidence intervals of the mean at 95% over 1000 independent problems. With the exception of deterministic problems (left), the CVAML loss reliably results in the lowest value prediction error.

6.4.4 Discussion of theoretical results

While the problem resulting from the use of stochastic models is resolvable, the MuZero loss is still insufficient for learning *per state* value function with stochastic models. In addition, we observe issues with learning accurate value functions with the $(m \geq 1, b \geq 1)$ -VAML loss even after correctly calibrating it. If the corrected loss is used to update the model, and a standard model-free or model-based value estimate is used as the value function learning target, the correct value function can be learned.

Insight 6.4.4.1 (Calibrated losses). *To obtain a calibrated loss, we propose using $\hat{\mathcal{L}}_{\text{CVAML}}$ to update the model and $\hat{\mathcal{L}}_{0,j}^{\hat{P}}$, or $\hat{\mathcal{L}}_{0,j}^{\mathcal{P}^\pi}$, to update the value function.*

6.5 Calibration impact on finite state MDPs

To test our findings, we run the (m, b) -VAML losses on small, finite state Garnet problems (Bhatnagar et al., 2007). Every problem is generated by sampling m successor states for each state x_i , and parameterizing the transitions with weights $\omega_{i,j}$ sampled uniformly iid for all successor states x_j . The transition distribution is given by

$$p(x'_i | x_j) = \frac{e^{\omega_{i,j}/\tau}}{\sum_i e^{\omega_{i,j}/\tau}}.$$

Varying the temperature τ we can interpolate between a deterministic transition and a m uniform one.

The models are parameterized with two learnable matrices $\phi \in \mathbb{R}^{n \times k}$ and $\psi \in \mathbb{R}^{n \times k}$ so that $\hat{p}_k(x_i|x_j) = \text{softmax}(\hat{\omega}_{i,j}) = \text{softmax}(\phi_i^\top \psi_j)$. By varying k we create a low-rank constraint. As Farahmand et al. (2017) shows, (m, b) -VAML should be used when the model has insufficient capacity to represent the environment. As a baseline, we use the model $\hat{p}_{\text{KL},k} \in \arg \min_{\hat{p}} \text{KL}(p||\hat{p})$.

We focus solely on value estimation in the Garnet MDPs with a fixed policy to simplify the experimental setup. We also use the ground truth reward function.

Results: The numerical results are graphed in Figure 6.2. For the near-deterministic ground truth environment, we see no benefit from using a calibrated (m, b) -VAML. We find that the algorithms are able to exploit the non-linear softmax function to achieve very accurate models in deterministic environments. However, even small amounts of stochasticity prevent this solution.

As the stochasticity increases, we see an advantage for the calibrated losses. $(1, 1)$ -VAML is not able to achieve any good results even when the underlying environment is deterministic. This corroborates the theoretical finding that the error depends on the model’s stochasticity and not on the environment. As we initialize the models with high entropy, $(1, 1)$ -VAML is unable to learn a high variance value.

Calibrated $(1, 1)$ -VAML still struggles in environments with low stochasticity. We find that due to the variance term over value functions, the model is prone to get stuck in local minima as it reduces the value function difference per state faster than the model predictive variance. This suggests that a $(m \geq 1, 0)$ -CVAML loss is preferable with stochastic environments.

6.6 Latent environment models and auxiliary losses

Prior literature (Lovatto et al., 2020), and indeed our own discussion in Chapter 5 have found the IterVAML loss to be unstable. However, we have shown that IterVAML and MuZero are closely related within the (m, b) -VAML family, and while the former has been criticized for instability, the later does not have this reputation. The main difference between prior empirical investigations of IterVAML and MuZero indeed has less to do with the inherent (dis)advantages of the loss functions, and can instead be explained by architectural choices. In practice, most implementations of MuZero-based models use deterministic latent model structures (Schrittwieser et al., 2020; Ye et al., 2021; Hansen et al., 2022; Antonoglou et al., 2022), while our investigation of IterVAML in the previous chapter, as well as similar experiments in Lovatto et al. (2020), were conducted with observation-space models.

This raises two important questions: (b) Is the use of *deterministic* models sufficient for modelling transitions for RL, especially in the case of stochastic environments, and (b) can the empirical performance differences between different (m, b) -VAML losses be understood (mostly) in light of this architectural choice? We will first address the former question since this can be answered with a formal, mathematical analysis.

6.6.1 Deterministic models for stochastic environments

In general, deterministic function approximations cannot capture the full transition distribution in stochastic environments. However, it is an open question whether a deterministic *latent* model can be sufficient for learning a value-equivalent model, as conjectured by Oh et al. (2017).

We can now answer this question in the affirmative, albeit with some technical caveats: there exist a measurable deterministic model for stochastic environments that perfectly minimizes the $(1, 0)$ -VAML loss. Showing the existence of a deterministic value-aware model relies on the continuity of the transition kernel and involved functions ϕ and V . These are standard assumptions that are necessary to prove the measurability of many relevant functions in continuous state-action spaces. For formal definitions, especially the definitions on the continuous stochastic kernels, see Bertsekas and Shreve (1978, p.134).

The proposition relies on the existence of a deterministic mapping, which we prove here as a lemma.

Lemma 6.6.1.1 (Deterministic Representation Lemma). *Let $\mathcal{X}$ be a compact, connected, metrizable space. Let p be a continuous kernel from $\mathcal{X}$ to probability measures over $\mathcal{X}$. Let $\mathcal{Z}$ be a metrizable space. Consider a bijective mapping $\phi : \mathcal{X} \rightarrow \mathcal{Z}$ and any $V : \mathcal{Z} \rightarrow \mathbb{R}$. Assume that they are both continuous. Denote $V_{\mathcal{X}} = V \circ \phi$.*

Then there exists a measurable function $f^ : \mathcal{Z} \rightarrow \mathcal{X}$ such that we have $V(f^*(\phi(x))) = \mathbb{E}_p[V_{\mathcal{X}}(x')|x]$ for all $x \in \mathcal{X}$.*

Proof. Since ϕ is a bijective continuous function over a compact space and maps to a Hausdorff space (by assumption $\mathcal{Z}$ is metrizable, which implies it is a Hausdorff space), it is a homeomorphism. The image of $\mathcal{X}$ under ϕ , $\mathcal{Z}_{\mathcal{X}}$ is then connected and compact. Since $\mathcal{X}$ is metrizable and compact and ϕ is a homeomorphism, $\mathcal{Z}_{\mathcal{X}}$ is metrizable and compact. Let $\theta_{V, \mathcal{X}}(x) = \mathbb{E}_{x' \sim p(\cdot|x)}[V(x')]$. Then, $\theta_{V, \mathcal{X}}$ is continuous (Proposition A.2.2.1). Define $\theta_{V, \mathcal{Z}} = \theta_{V, \mathcal{X}} \circ \phi$. Since ϕ is a homeomorphism, ϕ^{-1} is continuous. The function $\theta_{V, \mathcal{Z}}$ can be represented as a composition of continuous functions $\theta_{V, \mathcal{Z}} = \theta_{V, \mathcal{X}} \circ \phi^{-1}$ and is therefore continuous.

As $\mathcal{Z}_{\mathcal{X}}$ is compact, the continuous function V takes a maximum and minimum over the set $\mathcal{Z}_{\mathcal{X}}$. This follows from the compactness of $\mathcal{Z}_{\mathcal{X}}$ and the extreme value theorem. Furthermore

$V_{\min} \leq \theta_{V,\mathcal{Z}}(z) \leq V_{\max}$ for every $z \in \mathcal{Z}_{\mathcal{X}}$. By the intermediate value theorem over compact, connected spaces, and the continuity of V , for every value $V_{\min} \leq v \leq V_{\max}$, there exists a $z \in \mathcal{Z}_{\mathcal{X}}$ so that $V(z) = v$.

Let $h : \mathcal{Z}_{\mathcal{X}} \times \mathcal{Z}_{\mathcal{X}} \rightarrow \mathbb{R}$ be the function $h(z, z') = |\theta_{V,\mathcal{Z}}(z) - V(z')|^2$. As h is a composition of continuous functions, it is itself continuous. Let $h^*(z) = \min_{z' \in \mathcal{Z}_{\mathcal{X}}} h(z, z')$. For any $z \in \mathcal{Z}_{\mathcal{X}}$, by the intermediate value argument, there exists z' such that $V(z') = V(z)$. Therefore $h^*(z)$ can be minimized perfectly for all $z \in \mathcal{Z}_{\mathcal{X}}$.

Since $\mathcal{Z}_{\mathcal{X}}$ is compact, h is defined over a compact subset of $\mathcal{Z}$. By Proposition A.2.2.2, there exists a measurable function $f^*(z)$ so that $\min_{z'} h(z, z') = h(z, f^*(z)) = 0$. Therefore, the function f^* has the property that $V(f^*(z)) = \mathbb{E}_p[V(z')|z]$, as this minimizes the function h .

Now consider any $x \in \mathcal{X}$ and its corresponding $z = \phi(x)$. As

$$h(z, f^*(z)) = |\theta_{V,\mathcal{Z}}(z) - V(f^*(z))|^2 = 0$$

for any $z \in \mathcal{Z}_{\mathcal{X}}$,

$$V(f^*(\phi(x))) = \theta_{v,\mathcal{Z}}(z) = \mathbb{E}_p[V_{\mathcal{X}}(x')|x]$$

as desired. □

Equipped with this lemma, we can now prove the main result.

Proposition 6.6.1.1. *Let $\mathcal{X}$ be a compact, connected, metrizable space. Let p be a continuous kernel from $\mathcal{X}$ to probability measures over $\mathcal{X}$. Let $\mathcal{Z}$ be a metrizable space. Consider a bijective latent mapping $\phi : \mathcal{X} \rightarrow \mathcal{Z}$ and any $V : \mathcal{Z} \rightarrow \mathbb{R}$. Assume that they are both continuous. Denote $V_{\mathcal{X}} = V \circ \phi$.*

Then there exists a measurable function $f^ : \mathcal{Z} \rightarrow \mathcal{Z}$ such that we have $V(f^*(\phi(x))) = \mathbb{E}_p[V_{\mathcal{X}}(x^{(1)})]$ for all $x \in \mathcal{X}$.*

Furthermore, the same f^ is a minimizer of the expected IterVAML loss over any distribution $x \sim \rho$*

$$f^* \in \arg \min_{\hat{f}} \mathbb{E}_{\rho, \mathcal{P}^{\pi}} \left[\hat{\mathcal{L}}_{\text{IterVAML},1}(\hat{f}, V_{\mathcal{X}}|V_{\mathcal{X}}, x) \right].$$

Proof. As we only deal with single steps here, we use $x^{(1)}$ and x' interchangeably as the latter is simpler and more common nomenclature. The statement of the proposition itself is written with the $x^{(1)}$ notation to remain consistent with the main body of the paper.

The existence of f^* follows under the stated assumptions (compact, connected, and metrizable state space, metrizable latent space, continuity of all involved functions) from [Lemma 6.6.1.1](#).

First, expand the equation to obtain:

$$\mathbb{E}_{\rho, \mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{\text{IterVAML},1}(f, V|V_{\mathcal{X}}x, x') \right] \quad (6.34)$$

$$= \mathbb{E}_{\rho, \mathcal{P}^\pi} \left[\left[V(f(\phi(x))) - V_{\mathcal{X}}(x') \right]^2 \right] \quad (6.35)$$

$$= \mathbb{E}_{\rho, \mathcal{P}^\pi} \left[\left(V(f(\phi(x))) - \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x^{(1)})] + \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] - V_{\mathcal{X}}(x^{(n)}) \right)^2 \right]. \quad (6.36)$$

After expanding the square, we obtain three terms:

$$\mathbb{E}_{\rho, \hat{\rho}, \mathcal{P}^\pi} \left[\hat{\mathcal{L}}_{\text{IterVAML},1}(f, V|V_{\mathcal{X}}x, x') \right] \quad (6.37)$$

$$= \mathbb{E}_{\rho, \hat{\rho}, \mathcal{P}^\pi} \left[\left| V(f(\phi(x))) - \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] \right|^2 \right] \quad (6.38)$$

$$+ 2\mathbb{E}_{\mathcal{P}^\pi} \left[\left[V(f(\phi(x))) - \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] \right] \left[\mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] - V_{\mathcal{X}}(x') \right] \right] \quad (6.39)$$

$$+ \mathbb{E}_{\rho, \mathcal{P}^\pi} \left[\left| \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] - V_{\mathcal{X}}(x') \right|^2 \right] \quad (6.40)$$

Apply the tower property to the inner term to obtain:

$$2\mathbb{E}_{\rho, \mathcal{P}^\pi} \left[\left[V(f(\phi(x))) - \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] \right] \left[\mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] - V_{\mathcal{X}}(x') \right] \right] \quad (6.41)$$

$$= 2\mathbb{E}_{\rho, \mathcal{P}^\pi} \left[\left[V(f(\phi(x))) - \mathbb{E} [V_{\mathcal{X}}(x')] \right] \underbrace{\mathbb{E}_{\mathcal{P}^\pi} [\mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] - V_{\mathcal{X}}(x') | x']}_{=0} \right] = 0. \quad (6.42)$$

Since the statement we are proving only applies to the minimum of the IterVAML loss, we will work with the arg min of the loss function above. The resulting equation contains a term dependent on f and one independent of f :

$$\arg \min_f \mathbb{E}_{\rho, \mathcal{P}^\pi} \left[\left| V(f(\phi(x))) - \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] \right|^2 \right] + \mathbb{E}_{\mathcal{P}^\pi} \left[\left| \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] - V_{\mathcal{X}}(x') \right|^2 \right] \quad (6.43)$$

$$= \arg \min_f \mathbb{E}_{\rho, \mathcal{P}^\pi} \left[\left| V(f(\phi(x))) - \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')] \right|^2 \right]. \quad (6.44)$$

Finally, it is easy to notice that $V(f^*(\phi(x))) = \mathbb{E}_{\mathcal{P}^\pi} [V_{\mathcal{X}}(x')]$ for all x by the definition of f^* . Since the lemma holds for all x , we do not need to specify the distribution ρ further. Therefore f^* minimizes the final loss term and, due to that, the IterVAML loss.

□

We can conclude that given a sufficiently flexible function class $\mathcal{F}$, (m, b) -VAML can recover an optimal deterministic model for value function prediction. Note that our conditions solely ensure the *existence* of a measurable function; the learning problem might still be very challenging.

6.6.2 Auxiliary losses

While the original MuZero algorithm only uses the MuZero loss to update the latent embedding, dynamics model, and value function estimation, several more recent works add auxiliary stabilizing losses (Ye et al., 2021; Hafner et al., 2021; Hansen et al., 2024; Voelcker et al., 2025a). These allow the model to learn meaningful transitions even before the value function is properly approximated, which helps e.g. in sparse-reward environments.

Most prior works use some form of Bootstrap-Your-Own-Latent (BYOL) (Grill et al., 2020) loss, which minimizes the difference between the next states encoding $\phi(x^{(m)})$ and the model prediction $f^m(x^{(0)})$. We saw in the previous chapter that such losses can greatly aid in stabilizing value learning. In addition, latent-self prediction losses like BYOL show a strong synergy with value function learning. Therefore it stands to reason that using them in an (m, b) -VAML setup can also yield strong benefits.

As a reminder, when the difference between next state embedding and model prediction is measured by the L_2 norm, the model loss is

$$\hat{\mathcal{L}}_{\text{model}, m}^{\mathcal{P}^\pi}(\hat{f}, \phi, \hat{V}|x) = \underbrace{\hat{\mathcal{L}}_{m, b}^{\mathcal{P}^\pi}(\hat{f}, \phi, \hat{V}|x)}_{(m, b)\text{-VAML}} + \lambda \underbrace{\left| \hat{f}^m(\phi(x)) - [\phi(x^{(m)})]_{\text{sg}} \right|^2}_{\text{auxiliary}}. \quad (6.45)$$

The introduction of a stabilizing loss poses a challenge for the result in [Proposition 6.6.1.1](#). For example, a pre-trained LLM backbone might be used, which is naturally stochastic due to the sampling strategies used to query it. Therefore, it is still important to have a calibrated surrogate loss for stochastic models, as many use cases make stochastic models attractive.

For a flexible enough model and embedding function class, the perfect BYOL-based model f_{aux}^* would predict $\mathbb{E}_{\mathcal{P}^\pi}[\phi(x')|x]$. However, f_{aux}^* only coincides with the optimal model under the VAML loss iff

$$\mathbb{E}_{\mathcal{P}^\pi}[\hat{V}(\phi(x^{(1)}))] = \hat{V}\left(\mathbb{E}_{\mathcal{P}^\pi}[\phi(x^{(1)})]\right). \quad (6.46)$$

This is the case if and only if the value function is linear in the embedding features ϕ , which is also referred to as a linear expectation model (Wan et al., 2019). However, learning an

embedding in which the value function is linear can be difficult in practice and may not lead to stable model predictions in complex, high-dimensional environments.

An exception to the use of deterministic models is the architecture proposed by Antonoglou et al. (2022). However, their model and auxiliary loss rely on a biased straight-through gradient estimation. This introduces complications for finding the optima of the loss function and model class.

6.7 Experimental evaluation

To summarize our theoretical results, we can state the following

- (m, b) -VAML losses are uncalibrated, but we can adapt the losses to calibrate them.
- If latent model architectures are used, a perfect, deterministic $(1, 0)$ -VAML model exists, but might not be easily learnable.
- Auxiliary losses can be used to stabilize learning (m, b) -VAML models in practice, but might lead to issues when used with deterministic models and stochastic environments.

However, it is unclear how large the impact of all these theoretical results is in realistic environments. Therefore, we are now faced with several empirical questions, which we will answer with experiments on a subset of DMC environments (Tunyasuvunakool et al., 2020) encompassing various tasks within the *humanoid* and *dog* domains. These two domains are the most challenging in the DMC suite, and the standard comparison for current methods (Voelcker et al., 2025a; Nauman et al., 2024b).

1. Do IterVAML and MuZero perform equivalently when used with latent model architectures?
2. Do (m, b) -VAML algorithms improve when combined with auxiliary losses?
3. Can we observe the effects of calibration on deterministic environments?

6.7.1 Experimental setup

To conduct our experiments we use two neural network architectures for the model, a stochastic and a deterministic latent model, while keeping the rest of the training setup consistent for a clean comparison. All experiments are conducted with latent model architectures composed of an encoder, a latent dynamics model, and a value and policy function head. All networks are simple fully connected feed-forward networks whose architectures are identical to those used in the DMC experiments in Chapter 4. For hyperparameters and further architectural details, see Section E.1. For the stochastic case, the latent models are multivariate, diagonal Gaussian distributions where mean and variance are parameterized

by the latent network (Chua et al., 2018; Janner et al., 2019; Paster et al., 2021). In the deterministic case we simply use a feed-forward network.

Model roll-outs are produced by sequentially sampling latent states from the model conditional on the initial state and an action sequence using the reparametrization trick.

$$\hat{p}(\hat{z}'|z, a) = \hat{\mu}(z, a) + \hat{\sigma}(z, a) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (6.47)$$

The auxiliary loss for stochastic models is the negative log-likelihood of the next states' latent representations under the current dynamics model,

$$\mathcal{L}_{\text{aux},m}(\hat{p}, \varphi | x, a^{(0:m-1)}, x^{(m)}) = -\log \hat{p}^m \left(\varphi(x^{(m)}) | \varphi(x), a^{(0:m-1)} \right), \quad (6.48)$$

where $a^{(0:m-1)}$ is a sequence of actions of length m starting from state $x^{(0)}$.

In the case of deterministic models, we use the MSE between the next states' representations and the model's predictions,

$$\mathcal{L}_{\text{aux},m}(\hat{p}, \varphi | x, a^{(0:m-1)}, x^{(m)}) = \left\| \hat{p}^m \left(\cdot | \varphi(x), a^{(0:m-1)} \right) - \varphi(x^{(m)}) \right\|^2. \quad (6.49)$$

For training the policy and value function, we follow a data-mixing protocol. We use both model-generated on-policy, and real environment data from the replay buffer to train a value and policy head using the (m, b) -VAML loss with a twinned critic parametrization similar to TD3 (Fujimoto et al., 2018b). This means the model influences the value function both through the encoder and through on-policy model-generated data. The impact of the data-mixing strategy will be discussed in further detail in Chapter 7.

The actions for environment interactions are obtained from the actor and the environment model using MPC (Hansen et al., 2022): initialized at the action produced by the actor for the current state, this algorithm iteratively refines the action to maximize the expected return. In total our setup uses the model for training the shared encoder, generating data for value and policy improvement, and for online model-based search.

For (m, b) -CVAML, the calibration term is computed by sampling four independent trajectories from the model ($k = 4$) and calculating the means and variances of next-state values produced by the critic across the different samples. We will refer to the uncalibrated losses as (m, b) -VAML for which the variance term is not added, and we choose $k = 1$ to show the most pronounced effect.

We plot aggregated final performance over 20 random seeds per environment-loss pair with 95% CI, estimated with stratified percentile bootstrap (Patterson et al., 2024b). Aggrega-

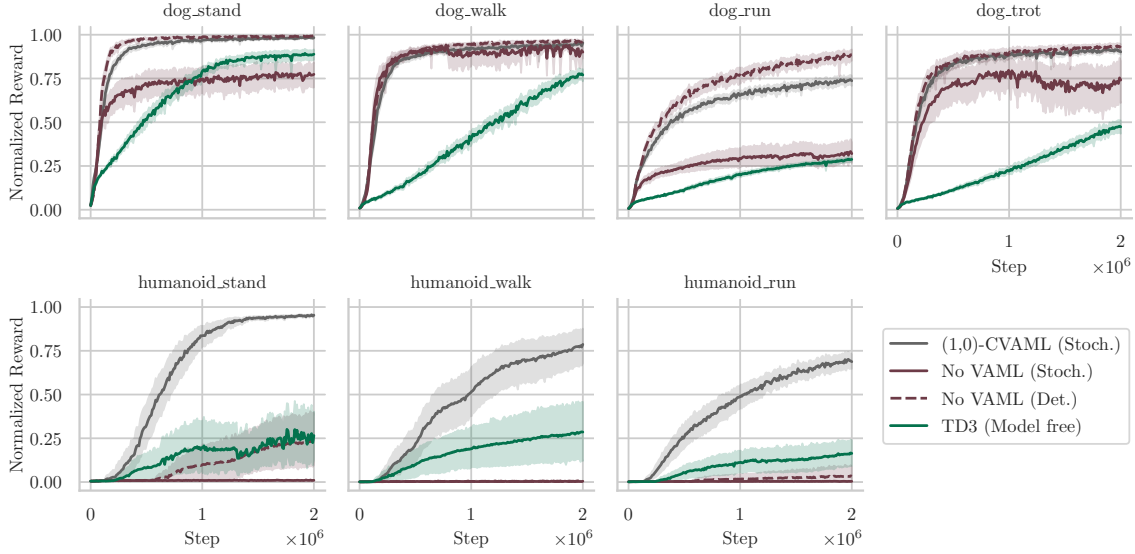


Figure 6.3: Baseline comparison of the CVAML model with a purely general-purpose (only self-prediction loss) and model-free baseline. Across all tasks we see a strong performance benefit of the VAML-based model over the model-free and stochastic general-purpose baseline. On some dog tasks, a deterministic self-prediction model performs slightly better than the CVAML variant. However, on the humanoid tasks, performance of the CVAML loss is significantly better than any baseline. In aggregate, CVAML (with an auxiliary task) is a consistently strong loss.

tions of final performance over several environments are visualized using the RLiability library (Agarwal et al., 2021).

6.7.2 Architecture design and (m,b)-VAML stability

We first study the impact of the latent design and auxiliary loss across all tasks. In Figure 6.3 we compare the (1,0)-CVAML (with auxiliary self-prediction loss) with both a model-free TD3 baseline, and with general-purpose models using only the self-prediction loss. The TD3 baseline is regularized with layer norms to counterbalance the effects described in Chapter 3. We found the layer norms recommended by Hansen et al. (2024) to be slightly more performant with deep models at low UTD than OFN, so we use these here and in the next chapter.

While the dog tasks show small improvements when removing the CVAML loss for deterministic environments, all other models and tasks show a strong drop in performance. In several cases, using a self-prediction loss even underperforms the model-free baseline.

In Figure 6.4 we show the results of training (1,0)-CVAML and (1,1)-CVAML respectively without auxiliary losses. Comparing the results to the discussion in Chapter 5, we see that the combination of regularized architecture and latent model structure by itself, even

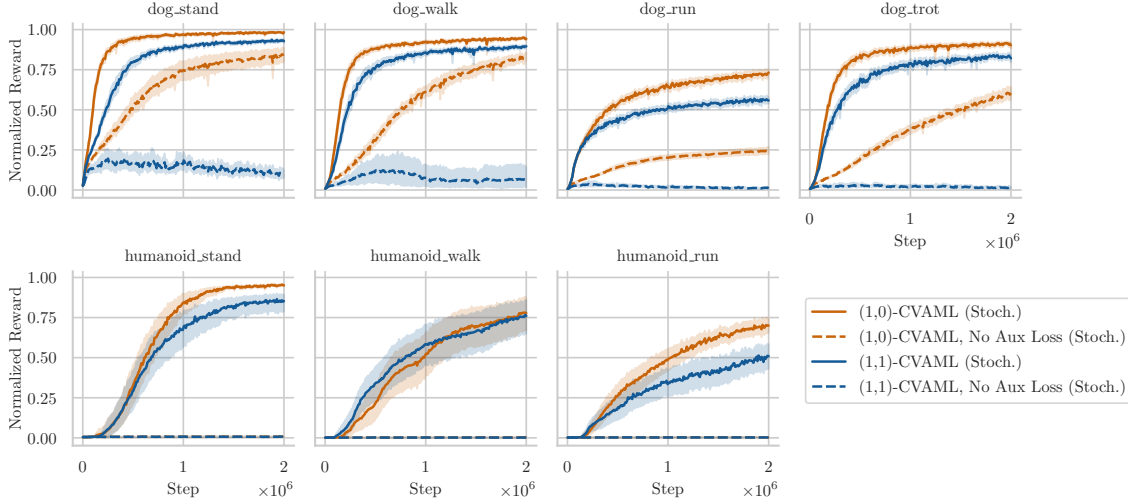


Figure 6.4: Comparison of CVAML with an auxiliary task with purely decision-aware $(1, 0)$ -CVAML and $(1, 1)$ -CVAML models (no auxiliary self-prediction loss). We can see that while $(1, 0)$ -CVAML can learn non-trivial policies on the dog tasks, $(1, 1)$ -CVAML mostly fails to do so. In addition, both losses fail to learn non-trivial policies on the humanoid tasks.

without auxiliary self-prediction, can lead to stable and performant models on the dog tasks. However, on the humanoid tasks all models fail to learn without auxiliary stabilizing loss, and on the dog tasks the auxiliary task greatly increases the performance. These results are perhaps surprising, given the results in the previous chapter and the relatively strong performance of MuZero in the literature.

Looking at the associated Q values in Figure 6.5 allows us to understand the difference between previous results and this one. Training with the MuZero loss leads to greatly inflated Q values, while the IterVAML-style loss (in combination with the latent model architecture and appropriate data mixing) does not exhibit strong value divergence on the dog task. As we are using a stochastic model architecture, $(1, 1)$ also faces the additional difficulty of being trained to predict average bootstrapped Q values for future states. In addition, here we are not training in a high-UTD regime in these experiment, while MBPO (Janner et al., 2019) and by extension VaGrAM and previous investigations into IterVAML (Lovatto et al., 2020) train with a rather aggressive UTD value of 20. Finally, the strong performance of MuZero without auxiliary losses has been attested primarily in the Arcade Learning Environment (Schrittwieser et al., 2020). As our experiments in Chapter 4 show, Atari games and continuous control tasks can show different behavior with auxiliary tasks, with the later benefitting much more strongly from their addition.

However, on the humanoid task we nonetheless see no meaningful learning, although the Q values are not diverging with either loss. This corroborates the hypothesis of Chapter 4 further: in the humanoid environment, initial exploration is uninformative and all Q functions

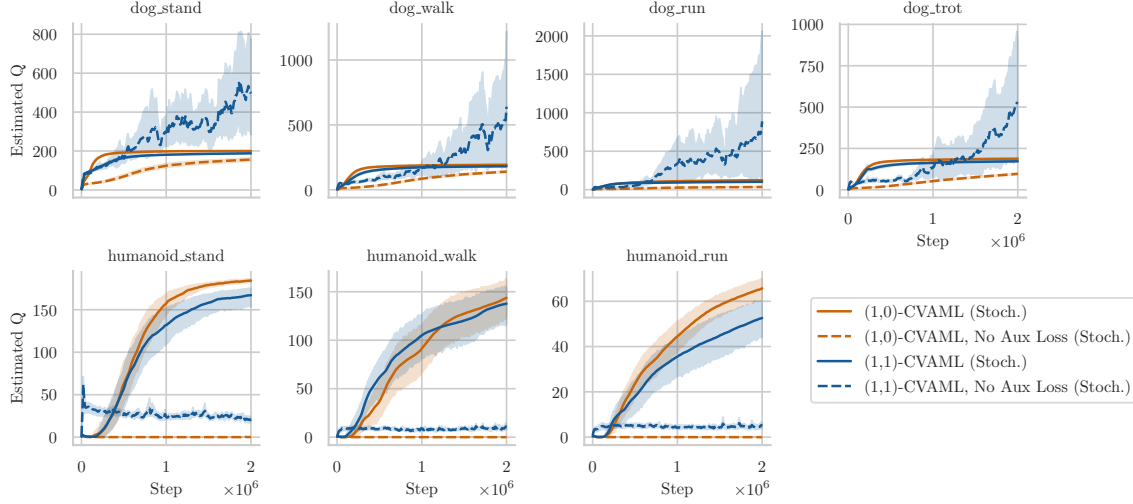


Figure 6.5: Associated mean Q values over the replay buffer for CVAML with an auxiliary task and for purely decision-aware (1,0)-CVAML and (1,1)-CVAML models (no auxiliary self-prediction loss). The Q values are significantly overestimated for the (1,1)-CVAML loss without auxiliary task, explaining its weak performance (compare Figure 6.4).

are close to 0 across all seen state-action pairs. A decision-aware loss by itself is therefore not able to learn a model that meaningfully enables the algorithm to learn meaningful state representations nonetheless.

When combined with an auxiliary loss, both loss variants learn non-trivial policies across all tasks, as the figures show. In this combination, (1,0)-CVAML is consistently better or at least as good as (1,1)-CVAML.

6.7.3 Calibration experiments with latent-space models

We show aggregated final performance across both DMC domains, dog and humanoid, in Figure 6.6. Figure 6.7 shows aggregated learning curves for each domain.

The difference in performance is most noticeable between the (1,1)-VAML and -CVAML in the humanoid task, as the uncalibrated loss impacts both model and value function learning in this case. This effect is less prominent with (1,0)-CVAML, where only the model learning is affected.

In the humanoid domain, we observe a performance improvement when using the calibration with the MuZero-style loss. In the dog domain, confidence intervals overlap although there is a small trend in the mean. However, most current algorithms on the dog domain use categorical value representations (Farebrother et al., 2024), for which further theoretical analysis is necessary before calibration can be addressed.

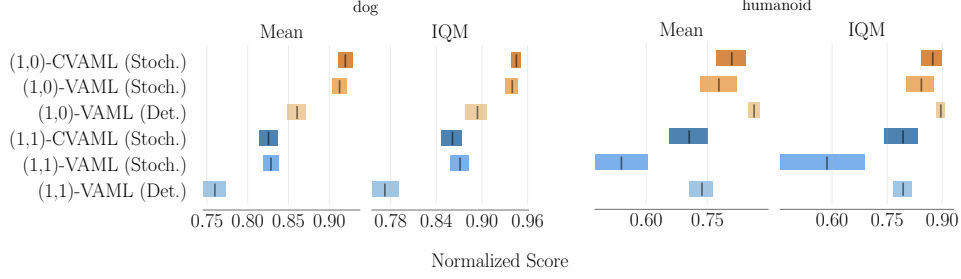


Figure 6.6: Results for the latent-space model experiments. We show the aggregate metrics of the final performance on dog (left) and humanoid (right) environments, following Agarwal et al. (2021). Calibrating (1,1)-VAML leads to a clear improvement in performance in the humanoid environment, while for (1,0)-(C)VAML the difference is less distinct. This is consistent with our theoretical findings as learning a smaller variance model can be sufficient, but wrong value functions will be learned with a stochastic model when $b \geq 1$.

In addition to the calibration effect, we observe that probabilistic models outperform deterministic ones in the dog domain, even though the simulator is deterministic. This is in line with the claims in some prior work (Chua et al., 2018; Janner et al., 2019), as stochastic models can reduce the tendency of the critic to exploit model errors. However, this advantage seems to be domain-specific and is not replicated in the humanoid suite.

Finally, we observe a consistent advantage of (1,0)-updates over the (1,1)-updates. Figure 6.8 shows that averaged across all environments we obtain a clear gap between all variants of the (1,0)-VAML loss over the (1,1)-VAML loss. In combination with the results from the previous section, we can therefore conclude that (1,0)-VAML losses can indeed lead to strong performance. Revisiting our empirical questions, we observe the following:

1. Do IterVAML and MuZero perform equivalently when used with latent model architectures? No, in fact, without additional stabilizing auxiliary loss, the IterVAML-style loss performs better than the MuZero-style loss.
2. Do (m,b) -VAML algorithms improve when combined with auxiliary losses? Yes, all variants of the (C)VAML family greatly increase in performance with a latent self-prediction auxiliary loss, and in some environments, such as the humanoid tasks, the additional auxiliary loss is necessary to achieve non-trivial performance.
3. Can we observe the effects of calibration on deterministic environments? Yes, we can observe the effects of calibration even in deterministic environments when using stochastic models. Especially with the (1,1)-(C)VAML loss, which is affected by calibration issues both in the model and value learning components, we observe a clear benefit from calibration. For the (1,0)-(C)VAML loss, calibration has a less clear impact. This is consistent with the fact that in deterministic environments, a deterministic latent model is sufficient to provide good value estimates. Therefore,

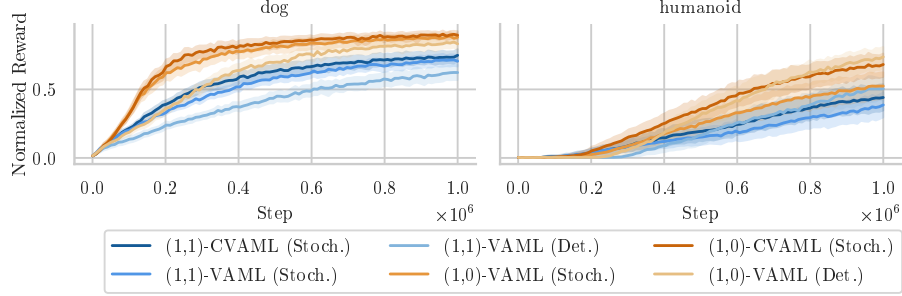


Figure 6.7: Sample efficiency curve for both dog (left) and humanoid (right). Per-task normalized return is aggregated over 20 seeds per environment and 3 environments, with 95% bootstrapped confidence intervals shaded. In addition to a higher final return, we observe significantly earlier learning for the (0,1)-(C)VAML on the dog task.

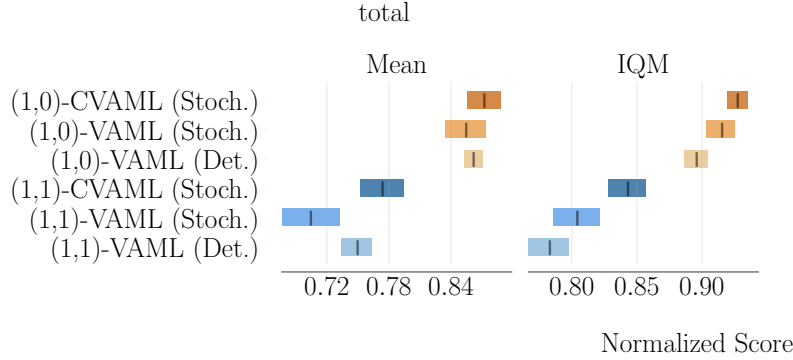


Figure 6.8: Final performance aggregated across all tasks. We observe that we see a consistent improvement in performance when using (1,0) losses instead of (1,1) losses.

the variance minimizing effect of the uncalibrated loss is less impactful on the final performance.

Insight 6.7.3.1 (Remarks for practitioners). *While deterministic models are theoretically sufficient for learning value-equivalent models, we observe benefits in some benchmarks from the use of stochastic models. Using calibrated losses is empirically especially important for MuZero-style model and value updates.*

6.8 Related work

VAML and MuZero: Farahmand (2018) established IterVAML based on earlier work (Farahmand et al., 2017). Several extensions have been proposed, such as a VAML-regularized MSE loss (Voelcker et al., 2022) and a policy-gradient aware loss (Abachi et al., 2020). Combining IterVAML with latent spaces was first explored by Abachi et al. (2022), but no experimental results were provided. MuZero (Schrittwieser et al., 2020; Ye et al., 2021) is built based on earlier works which introduce the ideas of learning a latent

model jointly with the value function (Silver et al., 2017; Oh et al., 2017). However, none of these works investigate the calibration of the loss function. Antonoglou et al. (2022) propose an extension to MuZero in stochastic environments but focus on the model architecture, not the value function loss. Hansen et al. (2022) and Hansen et al. (2024) adapted the MuZero loss to continuous control environments but did not extend their formulation to stochastic variants. Grimm et al. (2020) and Grimm et al. (2021) consider how the set of value equivalent models relates to value functions. They are the first to show the close connection between the notions of value-awareness and MuZero.

Other decision-aware algorithms: Several other works propose decision-aware model learning algorithms that do not directly minimize a value function difference. D’Oro et al. (2020) weigh the samples used for model learning by their impact on the policy gradient. Nikishin et al. (2022a) uses implicit differentiation to obtain a loss for the model function with regard to the policy performance measure. To achieve the same goal, Eysenbach et al. (2022a) and Ghugare et al. (2023) choose a variational formulation. Modhe et al. (2021) proposes to compute the advantage function resulting from different models instead of using the value function. Ayoub et al. (2020) presents an algorithm for on selecting models based on their ability to predict value function estimates and provide regret bounds with this algorithm.

Learning with suboptimal models: Several works have focused on the broader goal of using models with errors without addressing the loss functions of the model. Among these, some attempt to correct models using information obtained during exploration (Joseph et al., 2013; Talvitie, 2017; Modi et al., 2020; Rakhsha et al., 2022; Rakhsha et al., 2024), or to limit interaction with wrong models (Buckman et al., 2018; Janner et al., 2019; Abbas et al., 2020). Several of these techniques can be applied together with corrected (m, b) -VAML to improve the model and value function learning further. Finally, we do not focus on exploration, but Guo et al. (2022) show how auxiliary losses can be used not only to stabilize learning but also to improve exploration.

6.9 Conclusions

We theoretically analyze commonly used value-aware losses such as the MuZero and IterVAML loss and show that they are *uncalibrated* surrogate losses. When using (m, b) -VAML losses, such as the popular IterVAML and MuZero algorithms, with stochastic environment models, the loss learns low variance models, even if those do not recover the correct value function. Building on our proofs, we propose a novel variant of the loss to stabilize learning with stochastic environments and evaluate its efficacy in practice.

Our experiments further show that the calibration of the (m, b) -VAML losses is important for obtaining strong learning with stochastic environment models. In addition, while pre-

vious works showed that IterVAML losses can be unstable in practice (Lovatto et al., 2020; Voelcker et al., 2022), we find that this can be overcome by adopting the latent model architecture used in Schrittwieser et al. (2020) and the auxiliary losses established by Li et al. (2023) and Hansen et al. (2022). When combined with a suitable value learning procedure, IterVAML performs on par with or better than MuZero in continuous control tasks. Finally, while (m, b) -VAML losses have mostly been used with deterministic environments, our work enables the community to use stochastic models with a calibrated loss and shows the potential merits of this approach in a number of environments.

In the final chapter, we will take a more detailed look at how combining real and model-generated data stabilizes learning.

Chapter 7

Stabilizing Value Function Learning with Model-Generated Data

This chapter is based on Claas Voelcker, Marcel Hussing, Eric Eaton, Amir-massoud Farahmand, and Igor Gilitschenski (2025a). “MAD-TD: Model-Augmented Data stabilizes High Update Ratio RL”. in: *Proceedings of the International Conference on Learning Representations*.

7.1 Introduction

Instead of solely relying on data gathered by a target policy, *off-policy* RL aims to leverage experience gathered by past policies (Sutton and Barto, 2018) to fit the value function of a target policy. Ideally, given a fixed dataset of experiences, training over several iterations should help extract important information from past data. However, as we have seen in [Chapter 3](#), simply increasing the number of gradient update steps, the *replay ratio* or *update-to-data (UTD) ratio*, can lead to highly unstable learning (Nikishin et al., 2022b; D’Oro et al., 2023; Hussing et al., 2024; Nauman et al., 2024b).

Prior work has stabilized the learning by using double Q minimization to reduce overestimation (Fujimoto et al., 2018b), training ensemble methods to improve value estimation (Chen et al., 2021; Hiraoka et al., 2022), introducing architectural regularization ([Chapter 3](#), Hussing et al. (2024) and Nauman et al. (2024b)), or resetting networks periodically throughout the learning process (D’Oro et al., 2023; Schwarzer et al., 2023; Nauman et al., 2024b). However, while useful, pessimistic underestimation and architectural regularization are insufficient by themselves to decisively combat the problem, and so most methods re-

sort to either network resets or ensembles. In our resetting experiments in Subsection 3.3.3 we see that in difficult tasks, resetting still improves policy performance. Critic ensembles can be expensive to train, and resetting has several important limitations: in real systems, re-executing a random policy can be expensive or unsafe, the resetting interval needs to be carefully tuned (Hussing et al., 2024), and when storing a full reset buffer is infeasible, resetting loses important information.

We narrow in on a key issue contributing to unstable training: *wrong value function estimation on unobserved on-policy actions* (Thrun and Schwartz, 1993; Tsitsiklis and Van Roy, 1996). Off-policy RL uses the values of states sampled under old policies with actions from the target policy to update the value function. However, these state-action pairs themselves are not in the replay buffer, as the actions are from the current policy, and hence their value estimate is not directly improved by training. We call these *unobserved on-policy* actions to signal that they are not included in the replay buffer (so *unobserved* from the point of view of the training) yet stem from the current target policy. Consequently, a learned function which achieves low error on seen data is not guaranteed to generalize well to actions that *differ* from past actions. This problem is related to overfitting (Li et al., 2023) and contributes to overestimation (Thrun and Schwartz, 1993; Van Hasselt, 2010; Fujimoto et al., 2018b). However, overfitting assumes that train and test set are drawn from the same distribution, while we focus on the distribution shift between data collection and target policy. Previous work has investigated the difficulty of off-policy learning due to this shift (Maei et al., 2009; Sutton et al., 2016; Van Hasselt, 2010; Fujimoto et al., 2018b), yet there are no tractable mitigation strategies that work well in the high UTD regime with deep RL.

To corroborate our hypothesis that generalization to unobserved actions is a major obstacle for training at high UTDs, we examine the behavior of value functions on on-policy transitions. Our experiments reveal that value functions generalize significantly worse to unobserved on-policy action transitions than to validation data from the same distribution as the training set. Building on this, we propose to improve on-policy value estimation by using *model-generated on-policy data*.

Previous investigations into model-based deep RL have focused on learning values fully in model roll-outs (Buckman et al., 2018; Janner et al., 2019; Hafner et al., 2020; Ghugare et al., 2023) and the associated challenges (Zhao et al., 2023; Hansen et al., 2024). In contrast, we show that mixing a small amount of model-generated on-policy data with real off-policy replay data is sufficient to achieve stable learning in the high UTD regime. Our method, Model-Augmented Data for TD learning (MAD-TD), mitigates the generalization issues of the value function in the hardest tasks of the DeepMind control (DMC) benchmark (Tunyasuvunakool et al., 2020) and achieves strong and stable high UTD learning without resetting or redundant ensemble learning.

The main contributions of this work are twofold: First, we empirically show the existence of misgeneralization from off-policy value estimation to on-policy predictions. We connect this issue to the challenge of stable learning with high update ratios and highlight how increasing the update ratio increases Q function overestimation. Second, we provide a new method called MAD-TD that improves the value function accuracy on unobserved on-policy actions with model-generated data and stabilizes training at high update ratios. This method proves to have equivalent performance to or outperform previous strong baselines.

7.2 Off-policy value function learning

Many algorithms attempt to simplify the direct policy optimization problem by first learning a policy value function Q^π , which is defined via a recursive equation

$$Q^\pi(x, a) = r(x, a) + \gamma \mathbb{E}_{x' \sim \mathcal{P}(\cdot|x, a), a' \sim \pi(\cdot|x')} [Q^\pi(x', a')] \quad .$$

The policy can then be incrementally improved by picking $\pi_{k+1}(x) \in \arg \max_{a \in \mathcal{A}} Q^{\pi_k}(x, a)$ at every time step k . In practice, Q^π and π are often parameterized as neural networks and learned from data. To increase the sample efficiency of the algorithm, it is common to store *all* collected interaction data independent of the collection policy in a replay buffer $\mathcal{D} = \{(x_t, a_t, r_t, x_{t+1})_{t=0}^T\}$. As the Q-value only depends on the policy via the policy evaluation at the next state, it is possible to estimate Q-values from past interaction data by minimizing the fitted Q-learning objective

$$\mathcal{L}(\hat{Q}|\mathcal{D}, \pi) = \frac{1}{|\mathcal{D}|} \sum_{t=1}^T \left| \hat{Q}(x_t, a_t) - \left[r_t + \gamma \hat{Q}(x_{t+1}, a') \right]_{\text{sg}} \right|^2 \quad \text{with } a' \sim \pi(\cdot|x_{t+1}) \quad . \quad (7.1)$$

Here $[\cdot]_{\text{sg}}$ denotes the stop gradient operation introduced to avoid the double sampling bias and all data contained in the replay buffer is colored [blue](#). However, the Q value at the next state x_{t+1} is evaluated with an action a' that is *not* guaranteed to be in the replay memory, as the target policy can be different from the policy used to gather the sample. Hence, we require the Q value to generalize to potentially unseen actions. We provide a visualization of this issue in [Figure 7.1](#).

7.3 Investigating the root cause of unstable Q learning

Minimizing [Equation 7.1](#) finds the policy Q function over a replay buffer with sufficient coverage of all states and actions that this policy visits. However, in most continuous control RL algorithms (Lillicrap et al., 2016; Haarnoja et al., 2018; Fujimoto et al., 2018b), this update is interleaved with policy update steps. The data in $\mathcal{D}$ then necessarily becomes *off-policy* as training progresses.

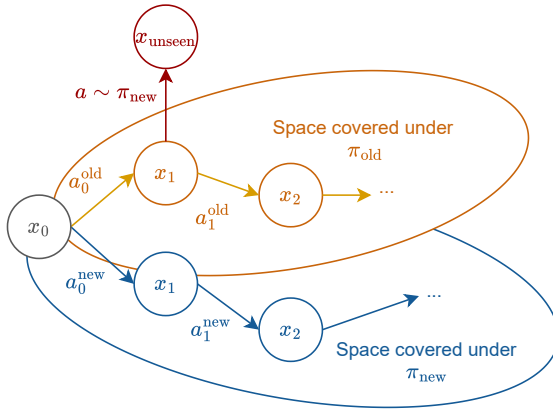


Figure 7.1: A visualization of the core issue we investigate. Even if a replay buffer contains good coverage for two policies (π_{old} and π_{new}) starting from $\rho = x_0$, this does not guarantee that it contains a transition for executing an action under the new policy on a state visited under the old. However, this state-action pair’s value estimate is used to update the value of state x_0 via Equation 7.1, without being grounded in an observed transition.

This means that the number of actor and critic optimization steps needs to be balanced with gathering new data. Obtaining new on-policy data is vital to continually improve policy performance (Ostrovski et al., 2021), but performing more update steps before gathering new data ensures that the existing data has been used effectively to improve the policy. The *replay ratio* (Fedus et al., 2020) or *update-to-data (UTD) ratio* (Nikishin et al., 2022b), which governs the number of gradient steps per environment step, is therefore a vital hyperparameter.

Naively training with high UTD ratios can lead to collapse in off-policy deep RL (Nikishin et al., 2022b). In Chapter 3 we showed that this correlates with a strong increase in the norm of unnormalized neural network weights. However, we did not describe a root cause of this overestimation.

We now conjecture that one of the major causes of the instability of high UTD off-policy learning are wrong Q values on *unobserved actions*. It is a well-known problem that off-policy TD learning is not guaranteed to converge, even when only considering state-value learning (Baird, 1995; Tsitsiklis and Van Roy, 1996; Sutton et al., 2016; Ghosh and Belle-mare, 2020). To differentiate the problem from *overfitting* to the training distribution, we use the term *misgeneralization* to highlight the importance of the distribution shift in causing the issue. Our experiments in Subsection 7.3.2 show that generalization to on-policy actions is more difficult than generalization to a validation dataset that follows the training distribution, and that higher UTDs exacerbate the issue.

7.3.1 Action distribution shift can cause off-policy Q value divergence

To highlight the role that on-policy actions play in stabilizing of Q value learning, we show an analysis the stability of Q learning with linear features. The core ideas follow Sutton et al. (2016) and are also explored in Tsitsiklis and Van Roy (1996) and Sutton (1988). We assume

that the Q function is parameterized with fixed features and weights as $Q(x, a) = \phi(x, a)^\top \theta$. Furthermore, we assume that the feature matrix Φ which is constructed by concatenating all state-action features, has full rank. Let X and A be the sizes of the state and action space respectively. Let $P \in \mathbb{R}^{X \cdot A \times X}$ be the matrix of transition probabilities from state-action pairs to states. A policy can then be expressed as a mapping $\Pi \in \mathbb{R}^{X \times X \cdot A}$ from states to the likelihood of taking each action. We use $R \in \mathbb{R}^{X \cdot A}$ to denote the vector of rewards. Finally, $D^\pi \in \mathbb{R}^{X \cdot A \times X \cdot A}$ is a matrix where the main diagonal contains the discounted state-action occupancies of P^π starting from ρ . If we assume access to a mixed replay buffer $\mathcal{D} = \bigcup \{\mathcal{D}^{\pi_1}, \dots, \mathcal{D}^{\pi_n}\}$ gathered with different policies, the population Q learning loss for a target policy Π can be written as

$$L(\theta) = \sum_{i=1}^n \left[D^{\pi_i} \left(\Phi^\top \theta - [R + \gamma P \Pi \Phi^\top \theta]_{\text{sg}} \right)^2 \right] . \quad (7.2)$$

The stability of learning with this loss can be analyzed using the gradient flow (compare Chapter 4). To derive the gradient flow, we compute the gradient of the loss function with regard to the parameters θ . As the loss has a relatively simple quadratic form and the derivative is a linear transformation, it decomposes nicely as

$$\nabla_\theta L(\theta) = 2\Phi \sum_{i=1}^n D^{\pi_i} \left(\Phi^\top \theta - R - \gamma P \Pi \Phi^\top \theta \right) \quad (7.3)$$

$$= 2\Phi \sum_{i=1}^n D^{\pi_i} \left((I - \gamma P \Pi) \Phi^\top \theta - R \right) \quad (7.4)$$

$$= 2\Phi \sum_{i=1}^n D^{\pi_i} (I - \gamma P^\pi) \Phi^\top \theta - 2\Phi \sum_{i=1}^n D^{\pi_i} R . \quad (7.5)$$

Using the equation for the gradient flow $\dot{\theta} = -\frac{\eta}{2} \nabla_\theta L(\theta)$ with learning rate $\frac{\eta}{2}$, we obtain

$$\dot{\theta} = -\eta \Phi \underbrace{\sum_{i=1}^n D^{\pi_i} (I - \gamma P^\pi) \Phi^\top \theta}_{\text{Keymatrix } A} + \eta \Phi \sum_{i=1}^n D^{\pi_i} R . \quad (7.6)$$

This gradient flow is guaranteed to be asymptotically stable (meaning it will not diverge around the stationary point θ^*) if the key matrix $A = \sum_{i=1}^n D^{\pi_i} (I - \gamma P \Pi)$ is positive definite (see Sutton (1988) for a more thorough explanation).

While the proof by Sutton (1988), which we use as a reference, discusses the stationary distribution of the Markov chain P^π , we define our loss in terms of a discounted state-action occupancy. We therefore briefly prove an auxiliary result to extend the analysis

to the case of discounted state occupancy probabilities. Note that when we talk about positive-definiteness, we use the definition from Sutton (1988) which applies to potentially non-symmetric matrices, and merely requires that $u^\top Xu > 0$ for all vectors u .

Lemma 7.3.1.1. *Let P be a stochastic matrix. Define the discounted state occupancy distribution μ of P for some starting state distribution ρ and some discount factor $\gamma \in [0, 1)$ as*

$$\mu^\top = (1 - \gamma) \sum_{n=0}^{\infty} \gamma^n \rho^\top P^n.$$

Let D be a diagonal matrix whose entries correspond to the discounted state occupancy distribution. Then the matrix $D(I - \gamma P)$ is positive definite.

Proof. First, note that

$$(1 - \gamma)\rho^\top + \gamma\mu^\top P = \mu^\top$$

by the definition of μ and the properties of the infinite sum. Therefore,

$$\mu^\top P = \frac{1}{\gamma} (\mu - (1 - \gamma)\rho) \quad .$$

Sutton (1988) asserts that a matrix A is positive definite iff $A + A^\top$ is positive definite. Furthermore, if the diagonal entries of a symmetric matrix are positive and all its off-diagonal entries are negative, then it suffices to show that the row and column sums of matrix are positive, as this is a sufficient condition to show that the matrix is positive diagonal-dominant.

For

$$D(I - \gamma P) + (I - \gamma P^\top)D^\top$$

the off-diagonal terms are clearly non-positive as D is diagonal. On the main diagonal, we have $2(\mu_i - \gamma p(i|i)\mu_i)$ which is positive as $p(\mu_i|\mu_i) \leq 1$. It now suffices to show that the row and column sums of $D(I - \gamma P)$ are positive. For the row sum, we can make use of the fact that P is a stochastic matrix, so

$$D(I - \gamma P)\mathbf{1} = D(\mathbf{1} - \gamma\mathbf{1}) \geq \mathbf{1} \quad .$$

For the column sum, we make use of the fact that $\mathbf{1}D = \mu$. Then

$$\mu(I - \gamma P) = \mu - \gamma \frac{1}{\gamma} (\mu - (1 - \gamma)\rho) = (1 - \gamma)\rho \geq \mathbf{1} \quad .$$

As ρ is a probability vector the final inequality holds for all $\gamma \in [0, 1)$.

All conditions presented by Sutton (1988) hold, and therefore we have $D(I - \gamma P)$ is positive definite. $\square$

Using this result, we can decompose the key matrix and see that the positive definiteness depends on the difference in policy between the replay buffer and the target policy

$$\sum_{i=1}^n D^{\pi_i} (I - \gamma P \Pi) = \underbrace{\sum_{i=1}^n D^{\pi_i} (I - \gamma P \Pi_i)}_{\text{positive definite}} + \underbrace{\gamma \sum_{i=1}^n D^{\pi_i} P (\Pi_i - \Pi)}_{\text{no guarantees}} . \quad (7.7)$$

In general, we can provide no guarantees for the **second term** outside of the on-policy case ($\Pi_i = \Pi$) where it becomes 0. The stability depends on the difference between the target policy and the data-collection policies. If the target policy takes actions which are not well covered under the training policies, the remainder can be non positive definite. The case we have discussed previously, where actions miss completely, is an extreme version of this, but even if we have samples of all state-action pairs, the distribution shift between the behavior and target policy can lead to divergence.

When using features, the eigenvalue conditions on the key matrix are only sufficient, not necessary, as the features can allow for sufficient generalization between observed and unobserved state-action pairs. In deep RL, the features ϕ are updated alongside with the weights, making it hard to provide definitive mathematical statements on stability. With good function approximation, we could hope that the learned value function generalizes correctly to unseen actions. In the next section we investigate this for a non-trivial task from the DMC suite and highlight that, while the value function does not diverge irrecoverably, good generalization is not guaranteed either.

7.3.2 Empirical Q value estimation with off-policy data

In environments with large state-action space, ensuring coverage is difficult. To investigate whether learning is stable nonetheless, we train a model-free TD3 agent on the *dog walk* environment (Tunyasuvunakool et al., 2020). The architecture is presented in detail in Subsection 7.4.1, and is regularized to prevent catastrophic divergence¹, uses latent self-prediction to stabilize feature learning (compare Chapter 4), and uses clipped double Q learning (Fujimoto et al., 2018b). This means it uses the most common techniques which are designed to prevent misgeneralization and overestimation.

While training a TD3 agent (Fujimoto et al., 2018b), we save transitions in a validation buffer with a 5% probability. At regular intervals we compute the critic loss on this val-

¹Here we use the strategy proposed by Lavoie et al. (2023) instead of the projection proposed in Chapter 3, as this proved to be amenable to more stable model learning losses using cross-entropy minimization instead of mean squared error for the latent model loss.

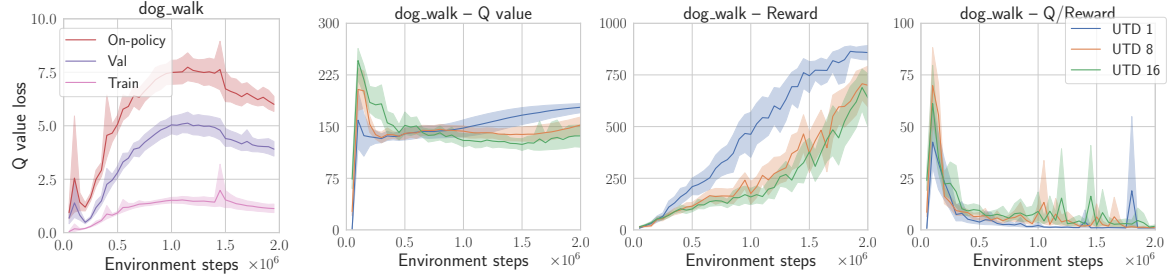


Figure 7.2: Left: the **train**, **validation**, and **on-policy** validation error of the Q function at UTD 1. Right: the Q values and return curves of TD3 agents across different UTD 1, 8, and 16.

validation set. In addition, we reset our simulator to each validation state and sample an action from the target policy. We then simulate the ground truth on-policy transition and compute the loss over these. This allows us to test how well our value function generalizes to target policy state-action pairs (as depicted in Figure 7.1).

The results are presented in Figure 7.2 and show a gap both between the train and validation sets, as well as the validation and the on-policy sets. While we use the on-policy state-actions to update the Q value, these estimates are not actually consistent with the environment. Furthermore, the Q value overestimation grows with increasing UTDs. This phenomenon was previously discussed in the context of over-training on limited data in Chapter 3.

The experiments show that the problem outlined in Subsection 7.3.1 is not merely a mathematical curiosity, but that Q value generalization to out-of-replay-distribution actions is difficult in practice, and becomes more difficult with increasing update ratios. Even though full divergence is not observed as new data is continually added to the replay buffer, it takes a long time for the effects of severe early overestimation to dissipate.

7.3.3 Prior attempts to combat misgeneralization and overestimation

Prior strategies that deal with misgeneralization can be grouped into four major directions: architectural regularization to prevent divergence of the value function, pessimism or ensemble learning to combat overestimation, networks resets to restart learning, and synthetic data generation. While all of these interventions help to some degree, they each either do not solve the problem in full or cause additional issues.

Architectural regularization Architecture changes (Hussing et al., 2024; Nauman et al., 2024a; Nauman et al., 2024b; Lyle et al., 2024) and auxiliary feature learning losses (Schwarzer et al., 2021a; Zhao et al., 2023; Ni et al., 2024; Voelcker et al., 2024c) are largely reliable interventions, and have shown to provide improvements without much drawbacks in prior work. However, as our experiment presented in Subsection 3.3.3 and Subsection 7.3.2

highlight, by themselves they can mitigate catastrophic overestimation and divergence, but do not guarantee proper generalization.

Pessimism and ensembles To combat overestimation directly, the most prominent approach in continuous action spaces is Clipped Double Q Learning (Fujimoto et al., 2018b). Here, a Q value estimate is obtained from two independent estimates $\hat{Q}_1$ and $\hat{Q}_2$. If the error of the two critic estimators is assumed to be independent noise on the true critic estimate then using the minimum over both estimates is guaranteed to underestimate the true critic value in expectation. However, in complex settings this assumption on the error of the critic estimates may not hold.

Ensembles (Lan et al., 2020; Chen et al., 2021; Hiraoka et al., 2022; Farebrother et al., 2023) or online tuning of the rate of pessimism (Moskovitz et al., 2021) have been proposed to obtain tighter lower bounds on the Q value. However, these strategies can be expensive as redundant models or hyperparameter tuning are needed. As a simpler strategy, recent works have also employed clipping to obtain an upper bound of the Q function to prevent divergence (Fujimoto et al., 2024).

Resetting Finally, network resets been shown to mitigate training problems (Nikishin et al., 2022b; D’Oro et al., 2023; Schwarzer et al., 2023; Nauman et al., 2024b) in high UTD regimes. However, in cases where the agent fails to explore any useful parts of the state space within the reset interval, restarting the learning process will not improve performance (Hussing et al., 2024). This makes tuning the resetting interval both important and potentially difficult and no tuning recipes have been presented. Resetting is also a potentially hazardous strategy in real-world applications, where re-executing a random policy might be costly or infeasible due to safety constraints. Finally, it heavily relies on the assumption that all past interaction data can be kept in the replay buffer.

Data generation Lu et al. (2024) attempts to combat failures of high UTD learning by supplementing a replay buffer with data generated from a trained diffusion model. This idea is inspired by the hypothesis that failure to learn in high-UTD settings is caused by a lack of data (Nikishin et al., 2022b). The method, SynthER, improves learning accuracy on simple tasks in the DMC benchmark. However, we demonstrate that simply adding more data is insufficient to combat misgeneralization by comparing SynthER to MAD-TD in the next section.

All of these strategies are somewhat able to alleviate the problem of out-of-distribution value estimation, yet none of them directly address the issue at the root. In the next chapter, we present an alternate approach that aims to directly regularize the action value estimates under the target policy.

7.4 Mitigation via model-generated synthetic data

As value functions misgeneralize due to lack of sufficient on-policy data, we propose to obtain synthetic data from a learned model instead. However, model-based RL can also cause problems such as compounding world model errors and optimistic exploitation of errors in the learned model. By using both real and model-generated data, we can trade-off these issues: on-policy data improves the value function and limits the impact of off-policy distribution shifts, while using only a limited number of model-generated samples prevents model errors from deteriorating the value estimates.

Our approach builds on the TD3 algorithm (Fujimoto et al., 2018b) and the TD-MPC2 architecture and model (Hansen et al., 2024). Our critic is updated with both model-based and real data following the Dyna framework (Sutton, 1990). More precisely, we replace a small fraction α of samples $\{x, a, r, x'\}$ in each batch with samples from a learned model $\hat{p}$ starting from the same state $\{x, \pi(x), \hat{r}, \hat{x}'\}$ with $\hat{r}, \hat{x}' \sim \hat{p}(\cdot|x, \pi(x))$. In our experiments, α is set to merely 5%. We term this approach Model-Augmented Data for TD learning (MAD-TD).

Model vs Q function generalization We expect that a learned models will yield better generalization than the Q function for two reasons. First, the policy is updated each step to find an action that maximizes the value function. This means we are effectively conducting an adversarial search for overestimated values.

To see this, note that the goal of our policy optimization is to find $\max_a Q(x, a)$. A potential adversarial objective for a regression problem would attempt to find δ so that

$$\begin{aligned} & \max(Q(x, a + \delta) - Q(x, a))^2 \\ \text{s.t. } & \|\delta\|_2 \leq \epsilon \end{aligned}$$

Note that while adversarial examples are constrained to not deviate far from the reference sample, our off-policy policy improvement objectives are not restricted in a similar fashion. Indeed, Fujimoto et al. (2018b) references this phenomenon while discussing the clipped Q learning trick, but as we have seen throughout several chapter now, clipping alone cannot prevent persistent Q function overestimation. However, we conjecture that the additional samples provided in model-based reinforcement learning can smooth out the Q landscape with regards to a , so that finding a policy that strongly and erroneously overestimates Q becomes harder. We provide evidence for this conjecture in Subsection 7.5.4.

The model’s reward and state estimation error on the other hand are independent of this process. We test the adversarial robustness of our model-augmented value functions in Subsection 7.5.4. Second, our experiment shows that value functions primarily diverge at the beginning of training. In these cases, coverage is low and on-policy state-action pairs

are often not available. Obtaining a slightly wrong, yet converging value estimate can then be more useful than a diverging one. Even as more data is gathered, new policies might not revisit old states with a high likelihood. Therefore even as training continues we expect the model data to provide some benefit.

7.4.1 Design choices and training setup

Our model is based on the successful TD-MPC2 model (Hansen et al., 2024) combined with the deterministic actor-critic algorithm TD3 (Fujimoto et al., 2018b). We aim to reduce the complexity of TD-MPC2 to the minimal necessary components to achieve strong learning in the DM Control suite, and thus forgo added exploration noise, SAC, ensembled critics, and longer model rollout for training or policy search. We outline several design choices here and refer to Section F.1 for more detail.

Encoder: Like TD-MPC2, we parameterize the state with a learned encoder $\phi : \mathcal{X} \rightarrow \mathcal{Z}$ with a SimNorm nonlinearity (Lavoie et al., 2023). This transformation groups a latent vector into groups of k entries and applies a softmax transformation over each group. This bounds the norm of the features, which has been shown aid with stable training (Hussing et al., 2024; Nauman et al., 2024a). Note that while we observed benefits from using a stochastic network on some tasks in Chapter 6, we chose deterministic encoders here for consistency with the baseline.

Critic representation and loss: We use the HL-Gauss transformation to represent the Q function (Farebrother et al., 2024). The critic loss is the cross-entropy between the estimated Q function’s categorical representation and the bootstrapped TD estimate. To stabilize learning, we initialize the critic network towards predicting 0 for all states. We find that this stabilizes the initial update steps in which almost no data is available.

Model loss: The world model predicts the next state latent representation and the observed reward from a given encoded state $\phi(x)$ and action a . The loss has three terms: the cross-entropy loss over the SimNorm representation of the encoded next state, the MSE between the reward predictions, and the cross-entropy between the next state critic estimate and the predicted state’s critic estimate. Following the evidence presented in Chapter 6, we use the latent VAML loss instead of the MuZero loss used by Hansen et al. (2024). In addition, since we have a categorical Q representation, we use a cross-entropy loss to compute the VAML term instead of a squared loss.

Full loss: With these changes, the full model and Q learning loss can be written as follows. As we use the HL-Gauss representation (Farebrother et al., 2024) for the critic, the loss is the cross-entropy between the estimated Q function’s categorical representation Q_{rep}

and the bootstrapped TD estimate,

$$\mathcal{L}_Q = \sum_{i=1}^m \text{TD}(\hat{Q}_{\text{rep}})_i \log \hat{Q}_{\text{rep}_i} ,$$

where the indices i denote the positions of the categorical vector representation used by HL-Gauss. This is the same loss that is used for the two-hot encoding in Hansen et al. (2024), the only difference is the target encoding function. For more details, see Farebrother et al. (2024).

We use a latent encoder $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{Z}$ that maps into the simnorm space, the space of n - k -dimensional simplices (Lavoie et al., 2023). Writing $\hat{p}$ for the learned world model and $\hat{r}, \hat{x}' \sim \hat{p}(|x, a)$ for reward and next latent-state samples, the loss for our model and encoder is

$$\mathcal{L}_{\text{model}}(x, a, r, x') = \mathcal{L}_{\text{rew}}(x, a, r) + \mathcal{L}_{\text{forward}}(x, a, x') + \mathcal{L}_Q(x, a, r, x') \quad (7.8)$$

$$\mathcal{L}_{\text{rew}}(x, a, r) = (r - \hat{r})^2 \quad (7.9)$$

$$\mathcal{L}_{\text{forward}} = - \sum_{i=1}^{n \cdot k} \phi(x')_i \log \hat{x}'_i , \quad (7.10)$$

where the index i is again element-wise across the simplex representation used for the latent state. Note that we propagate the critic learning gradients into the encoder only for the real data and not the model generated one to prevent instability.

Training: We train the architecture by interleaving one environment step with one round of updates with a varying number of gradient steps governed by the UTD parameter. For each update step, a new mini-batch is sampled independently from a replay buffer of previously collected experience. We found that varying the number of update steps only for the critic and actor while keeping the update ratio for the model and encoder updates at 1 leads to significantly more stable learning.

Run-time policy improvement with MPC: Following the approach outlined by Hansen et al. (2022), the learned model can also be used at planning time to obtain a better policy. Using the model for MPC at planning time exploits the same benefit of models as the critic learning improvement: we obtain a model-corrected estimate of the value function and choose our policy accordingly. As we only train our model for one step, we also conduct the MPC rollout for one step into the future.

7.5 Experimental evaluation

We conduct all of our experiments on the DeepMind Control suite (Tunyasuvunakool et al., 2020). Following Nauman et al. (2024b)’s recommendations we focus our main comparisons

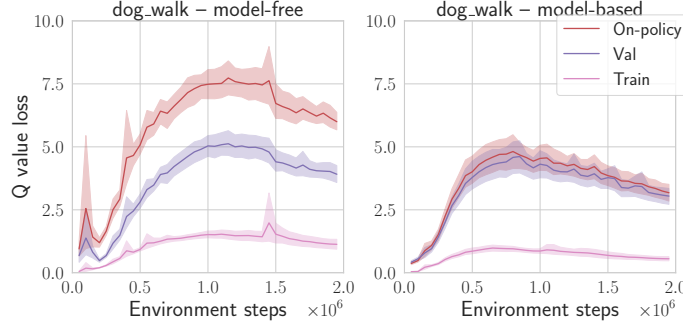


Figure 7.3: Average loss values with and without model-generated data (compare Figure 7.2) at UTD 1.

and ablations on the two hardest settings, the *humanoid* and *dog* environments (which we will refer to as the *hard suite*). Implementation details can be found in Section F.1. Unless stated otherwise we evaluate MAD-TD with a UTD of 8 and use the same hyperparameters across all tasks.

Note that even though we refer to training MAD-TD without using model data for the critic as “model-free”, the algorithm still benefits from the model through feature learning which has proven to be a strong regularization technique in high UTD settings (Schwarzer et al., 2023). All main result curves are aggregated across 10 seeds per task and we plot mean and bootstrapped confidence intervals for the mean at the 95% certainty interval. For aggregated plots, we use the library provided by Agarwal et al. (2021). Results on more environments are presented in Section F.2.

7.5.1 Impact of using model-generated data

We first repeat the experiment presented in Subsection 7.3.2 and show the results in Figure 7.3. Using model-based data closes the gap between on-policy and validation loss. We also observe that the initial Q overestimation disappears, which is consistent across all hard environments (see Subsection F.2.1). This provides evidence that we are indeed able to overcome the unseen action challenge.

Performance with and without model data at varying UTD ratios: In Figure 7.4 we present the impact of using model-based data across different UTD ratios. Humanoid results are found in Subsection F.2.2. As is directly evident, across the dog tasks, we observe stagnating or deteriorating performance when increasing the update ratio, consistent with reports in prior work. However, when using a small fixed amount of model-generated data, this trend is reversed across all tested environments, with performance improving or at least remaining consistent. We find that with model-based data, training is stable across a range of UTDs, even beyond those tested in recent high UTD work (Nauman et al., 2024b). We also note that we observe only limited benefits from increasing the UTD ratio when properly

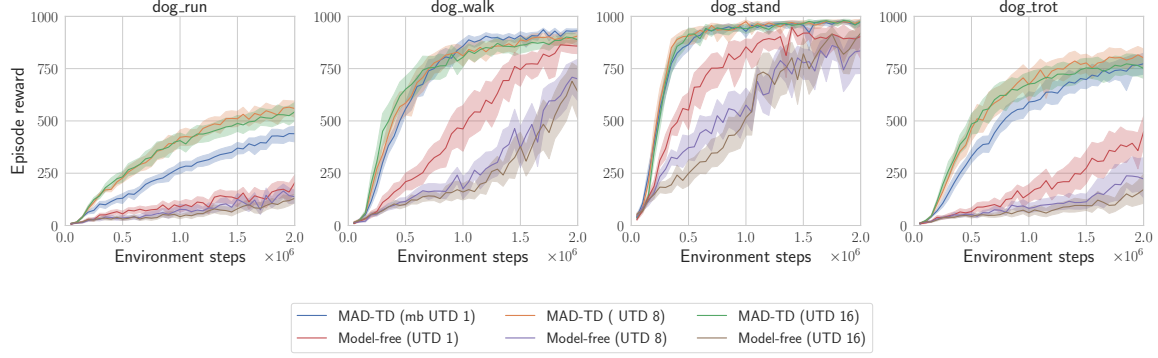


Figure 7.4: Return curves for the dog tasks with differing UTD values. Across all environments, the return increases or remains stable with increasing UTD with MAD-TD. Without model data, the performance decreases when increasing the UTD. MPC is turned off in these runs to cleanly evaluate the impact of model data on critic learning.

mitigating *misgeneralization*, except for the highly challenging dog run task, all other tasks perform on par with each other.

Comparison with baselines: As our method combines model-free and model-based updates, we compare our method against both TD-MPC2 (Hansen et al., 2024), a strong model-based baseline, and BroNet (Nauman et al., 2024b), a recent algorithm proposed for high UTD learning. Since Nauman et al. (2024b) and Hansen et al. (2024) trained with differing numbers of action repeats, and we found that the performance does not cleanly translate between these regimes, we present our method both with an action repeat value of 1 and 2. Some hyperparameters are adapted to the AR=1 setting (compare Table F.2). The results are presented in aggregate in Figure 7.5, with per environment curves show in Subsection F.2.4. We find that our method performs on par or above previous methods, and strikingly it is able to achieve higher returns faster than both TD-MPC2 and BroNet.

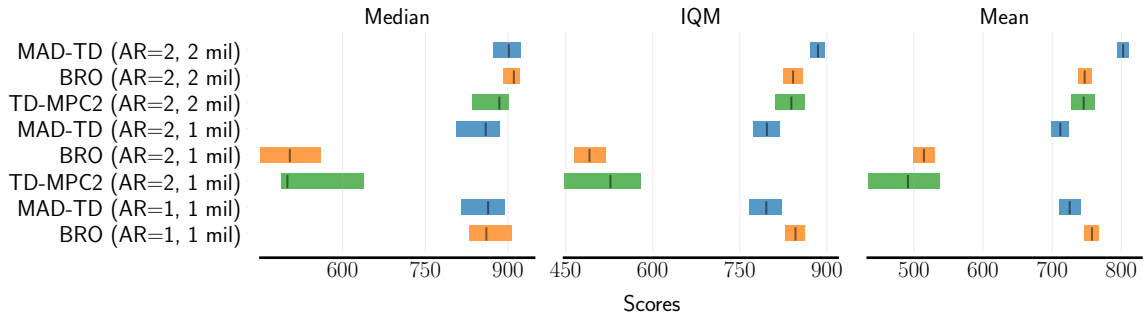


Figure 7.5: Performance comparison on the hard tasks for MAD-TD, BroNet, and TD-MPC, with varying number of steps and action repeat settings. MAD-TD is on par with all baselines, has higher mean and IQM when trained for 2 million time steps and action repeat 2, and strongly outperforms TD-MPC2 and BroNet at 1 million time steps with action repeat 2.

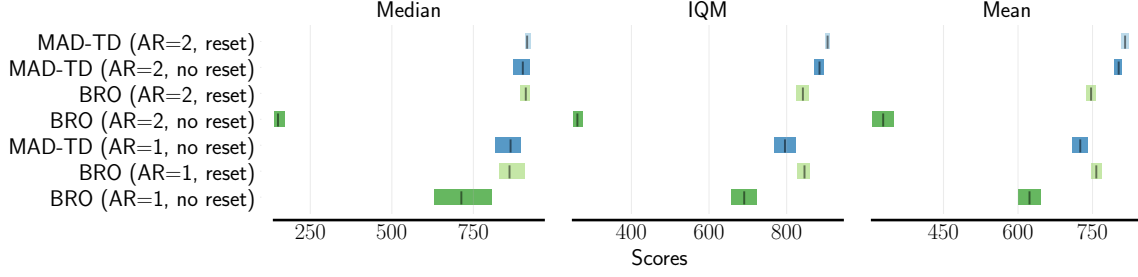


Figure 7.6: Resetting evaluation of MAD-TD and BroNet. Lighter color denotes performance with reset, and darker without. While MAD-TD’s performance only increases slightly when adding resetting, BroNet is unable to achieve strong performance in and setting with-out resetting.

7.5.2 Performance and stability impact of resetting

Resetting comparison: To investigate whether our technique benefits from more stable training, we set up a comparison in which we test the effects of resetting on our method. Figure 7.6 presents aggregate results comparing our approach and BroNet, both with and without resetting. Across all tasks we find that resetting barely improves MAD-TDs performance with the tested hyperparameters and update steps. However, the BroNet algorithm is not able to achieve reliable performance without resets. Overall, these results highlight that our model substantially improves the problems related to incorrect generalization of the value function, and that these are likely a major cause of the failure of high UTD learning in the DMC tasks.

Continued training: To highlight the pitfalls of resets, we employ a common RL metric the per timestep average regret

$$\overline{\text{Reg}}(T) = \frac{1}{T} \sum_{t=0}^{T-1} (\mathcal{R}^* - \mathcal{R}_t) ,$$

where $\mathcal{R}_t$ denotes the cumulative return in episode t and $\mathcal{R}^*$ the optimal return. We use the maximum return any of the algorithms achieved $\hat{\mathcal{R}}^*$ as a lower bound on the optimal return $\mathcal{R}^*$. Regret quantifies how much better the algorithm could have performed throughout training. In other words, in situations where continued learning is crucial, such as many safety critical applications, regret might be a better measure of performance. It captures not only how good the final policy is, but also how well the algorithm adapts over time, and minimizes mistakes. We present a comparison of MAD-TD and the resetting-based BroNet in Figure 7.7 using an action repeat of 1. The results show, even though both algorithms

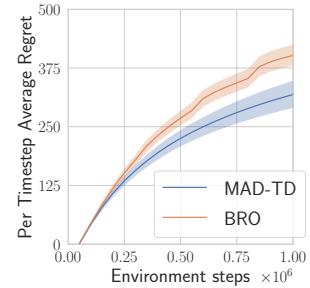


Figure 7.7: Average regret, mean over the hard suite. Lower regret corresponds to faster, more stable training. MAD-TD outperforms BroNet.

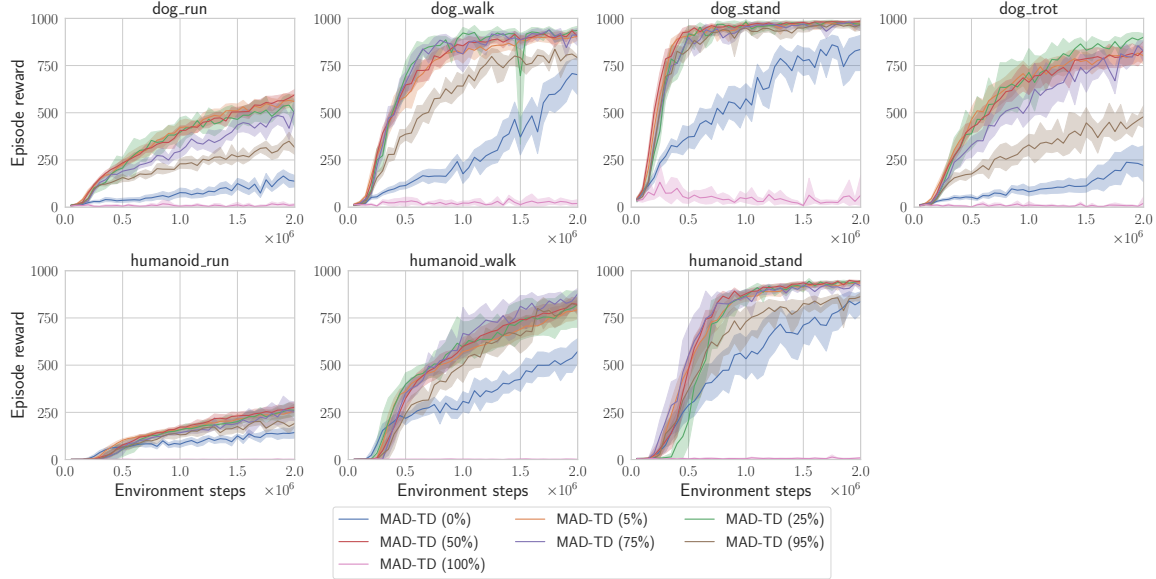


Figure 7.8: Return curves on the hard suite. We see that using substantially more data than 5% does not improve performance in a statistically significant way.

are close in their final return, their training behavior differs vastly. MAD-TD has lower regret showcasing its strength in continued deployment.

7.5.3 Different quantities of model data

We evaluate using more model data to update our value functions and provide the results in Figure 7.8. Aggregated scores are presented in Figure 7.9. We observe that the majority of gain is obtained when using limited amount of model data, and larger amounts only provided limited gains in some humanoid runs. When using high amounts of model generated data, we observe deteriorating performance, which implies that the agent learns to exploit the model instead of solving the real task. This observation is consistent with similar observation about model exploration in prior work Zhao et al. (2023).

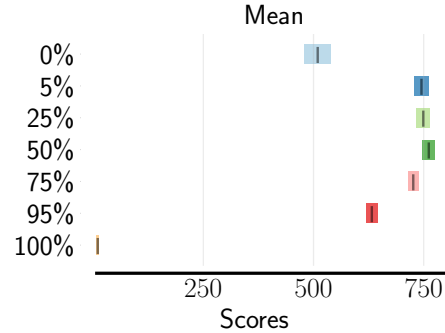


Figure 7.9: Aggregate statistics for differing values of α (amount of model data used) at UTD 8.

7.5.4 Further experiments and ablations

To further test our approach, we present two additional experiments on the *hard suite*: changing the action selection for the model data generation, and reducing the model perfor-

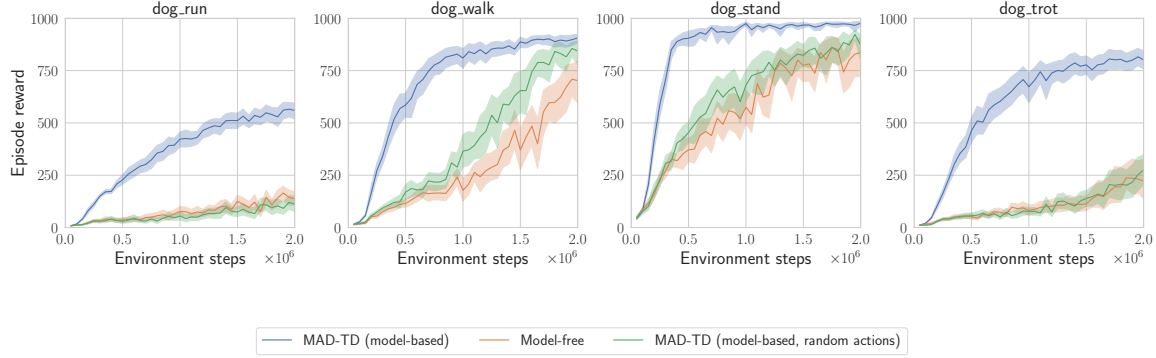


Figure 7.10: Return curves for the dog tasks when using **on-policy**, **random** and **no model-generated** data. When generating model-based data with random actions, performance of MAD-TD drops close to the model-free baseline, highlighting the importance of *on-policy* actions.

mance. In addition, we investigate the impact of using model based data on the smoothness of the learned value function.

Off-policy action selection in the model: To verify that the improvement in performance is due to the off-policy correction provided by the model, we repeat the *hard suite* experiments with a UTD of 8 and 5% model data, but we choose actions randomly from a uniform distribution across the action space. The results are presented in Figure 7.10. They highlight that random state-action pairs do not provide the necessary correction and the performance deteriorates to that of the model-free baseline.

SynthER A related approach to obtain additional data is the diffusion-based method proposed by Lu et al. (2024). In this work, the replay buffer data is augmented with additional samples obtained from a diffusion model that is trained on the replay buffer. The underlying hypothesis of SynthER is that the failure of high-UTD learning stems mostly from a lack of diverse data in the replay buffer. They demonstrate on the easier DM Control tasks that simply adding data from a generative model can be beneficial to learning. This is opposed to our hypothesis, which claims that high-UTD learning is difficult specifically due to the lack of off-policy action corrections.

We present a comparison of our method and SynthER on the hard DMC tasks. Results can be found in Figure 7.11.

As is evident from the lack of strong performance of SynthER, merely increasing the amount of generated data is insufficient to combat the failure of learning at high UTD. We find that the Q values of the SynthER agents quickly diverge on all tasks in which it is unable to learn. This strengthens our hypothesis that for hard tasks, off-policy action correction is vital to achieve strong results.

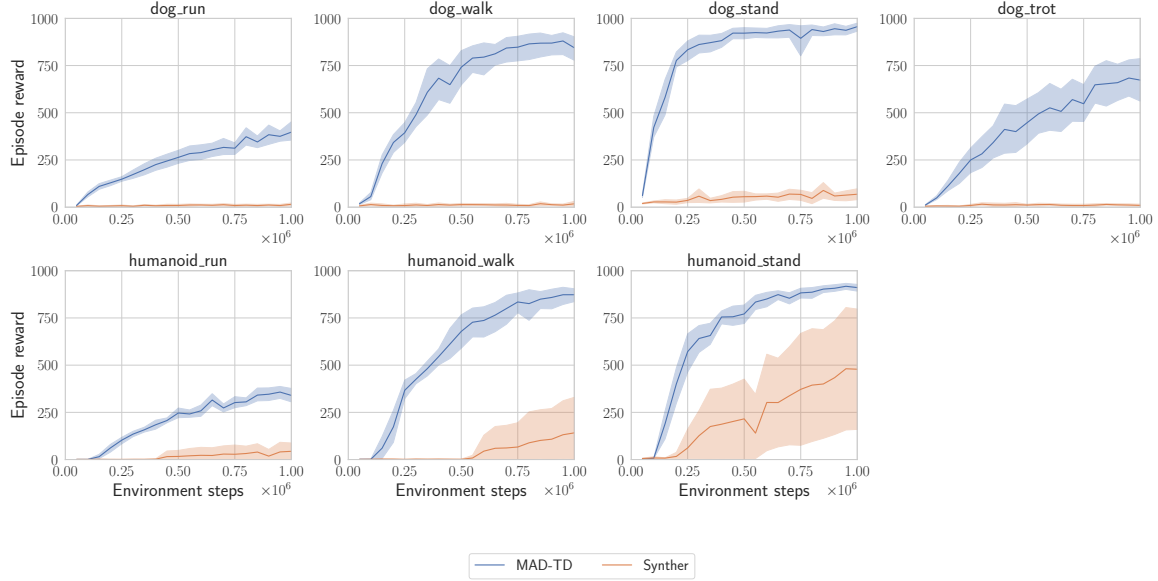


Figure 7.11: Performance curves for MAD-TD and Synther on the hard DMC tasks. Synther fails to achieve nontrivial results on most tasks, only outperforming a random policy on the humanoid walk and stand tasks.

Smaller model networks: To study the effect of the modelling capacity on our method, we ablate the size of the latent model by reducing the network size across the hard suite. The results are presented in Figure 7.12. We see that reducing the network size has an immediate and monotonic impact on the performance of our approach, suggesting that the model learning accuracy and prediction capacity is indeed vital for our approach to function well. However, even with small models of 64 hidden units, we still see some benefits from training with the model predicted data.

Perturbation robustness of the model-corrected value function: In the introduction of our method, we conjectured that one of the problems of training in actor-critic learning is that the actor is conducting a quasi adversarial search for overestimated values on the learned critic.² To understand how easy it is to find strongly overestimated Q values with gradient-based policy search procedure used in methods like TD3 (Fujimoto et al., 2018b) or SAC (Haarnoja et al., 2018), we conduct a search for adversarial action examples on our replay buffer. We used the iterated projected gradient method (Madry et al., 2018) to find actions which lead to strongly increasing Q values close to the replay buffer reference actions across training on the humanoid environments at a UTD of 1 with and without model data.

²Quasi because the actor is not constrained to find an action close to the replay buffer sample.

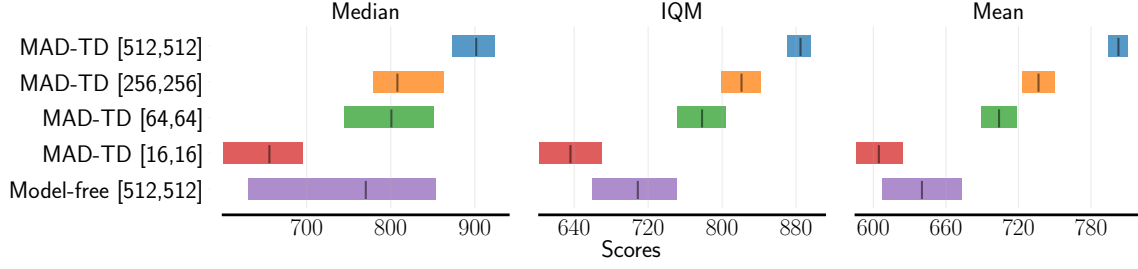


Figure 7.12: Performance evaluation when reducing the model size of the latent model in MAD-TD. The performance predictably drops with decreasing hidden layer size, however only strongly decreasing the model size below 64 reduces the performance below that of the model-free ablation.

Results are presented in Figure 7.13. Across the humanoid tasks we find that not using any model data leads to value functions with higher oscillations, either across the whole training run in `humanoid_run`, or in the middle of training like in `stand` and `walk`.

7.6 Related work

Beyond mitigating value function overestimation and unstable learning (see Subsection 7.3.3), other works have approached the difficulty of off-policy learning and high update ratios from other perspectives. Here, we survey further related papers which do not provide direct background for this work, but are nonetheless relevant as either alternative approaches or possible enhancements.

Other ways of incorporating off-policy data Having access to more diverse data has been shown to be beneficial for reinforcement learning, when this data is carefully used to mitigate the problems resulting from off-policy training. Ball et al. (2023) show that a large

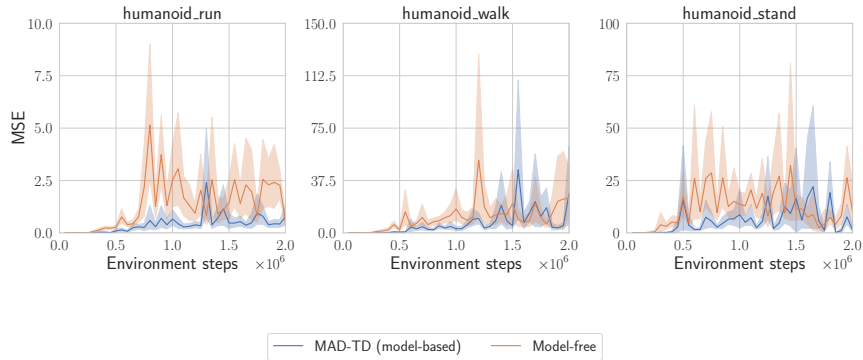


Figure 7.13: Magnitude of the difference between $Q(x, \pi(x))$ and $Q(x, \tilde{a})$, where $\tilde{a}$ is an adversarial perturbation of $\pi(x)$. Across the training why see higher perturbation magnitude for the runs without model correction data.

offline replay buffer can be used to improve training by sampling online training batches both from online data and offline data, and labelling the offline transitions with a reward of 0. Agarwal et al. (2022) and Tirumala et al. (2024) also highlight that previously collected replay buffers can be used to improve training performance on agents. In this work, we focus on the online setting where we do not have access to a replay buffer of previously collected transitions. These ideas however can easily be combined by e.g. training a model from an available larger offline data buffer.

In the online off-policy regime, Fujimoto et al. (2024) recently proposed TD7, which incorporates similar architectural choices to MAD-TD. They use a self-predictive encoder to learn good state representations, but concatenate them with the state and action representation provided by the environment to limit loss of information. This design choice proved to be beneficial but would require learning an observation-space next-state prediction, which is difficult in practice, especially in high dimensional environments. To address the policy distribution shift, TD7 does not update the actor at every timestep but instead collects several full trajectories with a fixed policy and then conducts update steps afterwards. However, this interval still needs to be balanced as a hyperparameter. TD7 was not evaluated on DMC, which is why we do not present a comparison.

Model-based reinforcement learning As surveying model-based reinforcement learning is a rather sizeable task, we refer readers to the survey by Moerland et al. (2023) for reference. Decision-aware latent models such as the one Hansen et al. (2024) and we use have been studied specifically in several different variants. Silver et al. (2017) proposes a latent model that is trained with TD learning, which provides the basis for the MuZero algorithm (Schrittwieser et al., 2020). The addition of a latent self-prediction loss was first proposed by Li et al. (2023) to stabilize learning problems with the TD learning loss. This interplay was further studied by Ni et al. (2024) and Voelcker et al. (2024c) in recent works.

From a theoretical angle, decision-aware losses similar to those used in MuZero were first studied by Farahmand et al. (2017) and Farahmand (2018). Grimm et al. (2020) and Grimm et al. (2021) further study the loss landscape and minimizers of such losses, while Kastner et al. (2023) studied the extension of the loss to distributional settings.

A more thorough overview on the topic of decision-aware learning can be found by Wei et al. (2024).

Offline reinforcement learning In the context of batch reinforcement learning or offline RL (Lange et al., 2012; Fujimoto et al., 2019), the action distribution shift is a known phenomenon. The main counter to the problem however does not rely on closing the generalization gap, but on explicit pessimistic regularization (Jin et al., 2021). Such pessimistic

regularization has been shown to be highly detrimental in online RL, as it removes the capability for the agent to explore its environment efficiently (D’Oro et al., 2023; Hussing et al., 2024). In offline RL, authors have explored the capability of models to provide some improvements to generalization (Yu et al., 2020). However, in online RL the community has mostly relied on the hope that additional optimistic exploration based on the value function will close the generalization gap without explicit interventions. We show that this is not the case.

Loss of plasticity A phenomenon that has been observed in both the continual learning and RL communities is the tendency for neural network based agents to lose their ability to learn over time (see Parisi et al. (2019) for a survey of continual learning and Igl et al. (2021) for a discussion of the phenomenon in RL). Sometimes the phenomenon is referred to as plasticity loss (Lyle et al., 2022a; Abbas et al., 2023; Dohare et al., 2024). As highlighted before, we do not find strong evidence for the primacy bias or loss of plasticity during our experiments on the DMC suite.

However, that does not imply that the phenomenon does not exist. In fact, we believe that resolving stability issues such as those presented in our paper will help us to better isolate other nuanced issues such as plasticity loss more clearly. Previous studies have identified and combated plasticity loss using feature rank maximization (Kumar et al., 2021), regularization (Lyle et al., 2023), additional neural network copies (Nikishin et al., 2024), minimizing dormant neurons (Sokar et al., 2023; Xu et al., 2024), various neural network architecture changes (Lee et al., 2023), slow and fast network updates (Lee et al., 2024) or weight clipping (Elsayed et al., 2024).

It is unclear how many improvements obtained by these changes can be explained by divergence effects (Chapter 3) or stability issues such as those established in our work as there seems to be a non-zero overlap in techniques that combat either. Nauman et al. (2024a) have argued that many RL training problems can be difficult to disentangle from the plasticity loss phenomenon.

It is also not unlikely that the training dynamics of the state-based dense-reward tasks on the DMC suite are more benign than those found in Atari games. Many works on plasticity loss have studied sparse image-based control tasks with pure Q learning approaches, such as DQN on the Atari benchmark Sokar et al. (2023) and Lee et al. (2024). The problem may be more prevalent when replay buffers cannot be maintained in full and the RL setting becomes a true continual learning problem.

Other stability perspectives Our work studies the stability of losses during training. We highlight that forgoing resetting decreases regret as the executed policies are more stable in the sense that they are not reset at regular intervals. We also highlight that

model-generated data can somewhat improve the stability of policies against adversarial attacks. However, there are other notions of *stability* that should be considered relevant and that are orthogonal to our work. Here we will give a non extensive overview into the different directions that exist as a starting point for the reader. For instance from a theoretical perspective, stability can be formulated as differential privacy (Vietri et al., 2020) or algorithmic replicability to obtain identical policies (Eaton et al., 2023). From a theoretical as well as practical perspective, stability to changes in the environment can also be investigated using random or adversarial perturbations to the transition model or the policy (Nilim and Ghaoui, 2005; Iyengar, 2005; Wiesemann et al., 2013; Pinto et al., 2017). Finally, from an empirical perspective, robustness to hyperparameters (Ceron et al., 2024; Patterson et al., 2024a) and attempts at variance reduction to get more reliable solutions (Anschel et al., 2017; Kuang et al., 2023) can be considered notions of stability.

7.7 Conclusion

Our experiments allow us to conclude that wrong generalization of the value functions to unseen, on-policy actions is indeed a major challenge that prevents stable off-policy RL, both in theory and in practice. Model-Augmented Data for TD learning (MAD-TD) is able to leverage the learning abilities of latent self-prediction models to provide small, yet crucial amounts of on-policy transitions which help stabilize learning across the hardest DeepMind Control suite tasks. With a relatively simple model architecture and learning algorithm, this method proves to be on par with, or even outperform other strong approaches, and does not rely on mechanisms such as value function ensembles or resetting which were previously conjectured to be necessary for stable learning in high UTD regimes.

Our work opens up exciting avenues for future work. The issue of poor generalization in off-policy learning can likely be tackled with other approaches such as diffusion models (Lu et al., 2024) or better pretrained foundation models, and our presented experiments provide an important baseline for such work. Furthermore, while we have purposefully kept our approach as simple as possible to validate our hypothesis, many ideas from the model-based RL community such as uncertainty quantification (Chua et al., 2018; Talvitie et al., 2024), multi-step corrections (Buckman et al., 2018; Hafner et al., 2020), or policy gradient estimation (Amos et al., 2021) can be combined with our approach. Our insight that surprisingly little data is necessary to achieve strong correction can likely be leveraged in these other approaches as well to trade-off model errors and value function errors more carefully. Finally, while we chose the data to roll out in our models at random, our insights can likely be combined with ideas from the area of Dyna search control (Pan et al., 2019; Pan et al., 2020) to select data points on which the correction has the most impact.

Chapter 8

Conclusion

In this thesis, we set out to understand how we can improve value function learning in RL, by incorporating decision-aware and general purpose techniques that improve learned representations and world models. Our goal was to show that by carefully combining decision-aware approaches, such as Q learning or IterVAML, with general-purpose learning methods, such as maximum-likelihood estimation or self-supervised learning, we are able to improve and stabilize DRL.

We started in [Chapter 3](#) by understanding the central role that neural network representations play for stabilizing value learning in DRL. As our initial experiments show, relying purely on a model-free Q learning loss to shape the representation learning in a neural network leads to instability and divergence. This issue worsens as we attempt to increase the sample efficiency of RL algorithms by increasing the number of gradient steps. To alleviate this issue, we discussed a simple yet highly impactful L_2 -norm regularization of the latent feature space via an architectural constraint in the neural network.

To go beyond merely regularizing features, in [Chapter 4](#) we took a look at auxiliary tasks which use additional losses to shape the representation layers of a neural network. Using the tractable surrogate model of a linear two-layer network, we were able to show that there is strong theoretical evidence that latent self-prediction is an advantageous representation learning loss. The learned features are compatible with those learned by temporal-difference based learning, which means the auxiliary loss is able to stabilize and improve the feature space without interfering with the core task of value representation. In addition to mathematically proving this relationship in the surrogate model, we also looked at empirical evidence that these insights transfer beyond tabular environments, and to more complex neural network architectures.

We then took a look at MBRL in [Chapter 5](#) and found a similar issue with decision-aware learning to that of value overestimation. The lack of a good value function approximation

early in training prevents a model based on the IterVAML loss from learning useful predictions. This in turn prevents the agent from improving its policy, and so learning is stuck as no new information is gathered that can improve either model or value function. To break this cycle, we looked at using value function information to shape a simple mean squared loss. The value-gradient aware model learning loss uses the gradient of the estimated value function as a measure of sensitivity to penalize model errors based on their estimated impact on value prediction in each dimension.

In [Chapter 6](#), we started combining the insights into model learning and representation learning. We first introduced the family of (m, b) -VAML losses that encompasses both MuZero and IterVAML. We showed that these losses lead to wrong model and value estimates when used with stochastic environment models and suggested a calibrated version of the losses that shows improved accuracy in model and value learning.. Finally, we highlighted how combining the latent self-prediction losses discussed in [Chapter 4](#) and value-aware model learning leads to reliable and useful model learning architecture.

Finally, in [Chapter 7](#), we revisited the difficult high UTD training regime first discussed in [Chapter 3](#) and used our gained knowledge to build a stable and efficient latent value-aware model learning algorithm called MAD-TD. It is able to smoothly make progress on hard tasks in the standard mujoco locomotion benchmarking suite and does not require previously established workarounds to stabilize training such as resetting. This algorithm is a strong example of the benefits of combining decision-aware and general-purpose learning, and serves as the final empirical example that supports the thesis main claim.

8.1 Future work

As discussed in the very beginning of the thesis, stable reinforcement learning, and especially value function learning as a central problem in RL, will remain a challenging task even after we introduced many new insights and techniques that mitigate existing problems. Therefore we will now briefly survey some enticing open questions that this thesis leaves for future work.

8.1.1 Decision-aware and general purpose learning

While this thesis looks mostly at value functions, self-supervised representation learning, and value-aware model learning, there are many other paradigms in RL and planning that can be treated in a similar manner. Other ideas in the space of decision-aware model learning have also been considered in the literature. For example, Abachi et al. (2020) present Policy-gradient Aware Model Learning (PAML), a policy-gradient aware model learning algorithm. However, similar to VAML and IterVAML, PAML has so far not been extensively used in

the context of Deep Reinforcement Learning. It is an exciting question whether similar techniques to those presented in this thesis could be considered for stabilizing PAML.

Beyond already published techniques for decision-aware learning, another important question is whether the underlying idea can be extended to other model-based algorithms. One promising candidate is Stochastic Value Gradient (SVG) (Heess et al., 2015; Amos et al., 2021), a technique that uses the end-to-end differentiability of modern neural network architectures to obtain first-order policy gradients. This technique has been used to obtain good policies for example as part of the Dreamer architecture and algorithm (Hafner et al., 2020). The difference between a zeroth-order and simulator-based first order policy gradient estimation has been receiving increased attention over the last years (Xu et al., 2022; Suh et al., 2022; Antonova et al., 2022; Georgiev et al., 2024), with some works investigating learned models (Ma et al., 2024; Georgiev et al., 2024). Extending gradient-aware model learning to account for first-order gradient estimation is an important topic for future work.

8.1.2 Explicit and implicit uses of model learning

In our work, we have shown explicit uses of world models to improve value function learning (Chapter 5), implicit uses of model-based losses to stabilize representation learning (Chapter 4), and combinations of both (Chapter 6 and Chapter 7). While we conclude in Chapter 7 that explicitly using world models provides an important stabilization for value function learning, other recent works obtain strong results without using learned models explicitly (Fujimoto et al., 2025). As we discuss in Chapter 7, using a world model to improve value learning directly effectively trades off the generalization capabilities of the learned model with those of the value and policy. Therefore, an important avenue for future work is to build adaptive algorithms which can actively monitor this trade-off and uses the model and value function dynamically based on their relative performance.

8.1.3 Environment understanding

When discussing optimal ways of representing the information an agent gathers, a central question that we have not tackled so far is what kind of task an agent is attempting to learn in the first place. Even though reinforcement learning should ostensibly be able to learn any task in any environment, practical necessity forces us to make many explicit choices in terms of architecture, loss, observation representation, and even rewards. As the theory and experiments in Chapter 4 show, the structure of the environment is relevant for choosing proper representation learning methods, and the same is true for any other algorithmic design decisions as well. While this thesis and other works partially formalize and use concepts such as distracting dimensions, sparse rewards, and observation representations, much more work on studying the intersection between RL algorithm design and environment structure is necessary. A preliminary proposal for environment-centered research was outlined in

Voelcker et al. (2024b), where we discuss the necessity of studying environments. We propose to bring decision-making environments themselves into the focus of the reinforcement learning literature as an area of research equal to the algorithms which are traditionally centered in the community. This research should be structured around the goal of identifying formalizeable, testable, and impactful characteristics of environments. This will allow researchers, especially those focused on empirical methodologies, to test their algorithmic ideas more carefully, and give practitioners actionable guidelines for choosing algorithms based on the properties of their intended application.

8.1.4 Multi-task learning

All techniques discussed in this thesis focus on a single-task setting in which the agent is solely aiming to optimize its cumulative reward from a single reward function. While this is the most common formalization of reinforcement learning, other ideas such as goal-conditioned reinforcement learning exist (Kaelbling, 1993; Schaul et al., 2015). Goal-conditioned reinforcement learning has recently incorporated many ideas from representation learning, such as learning features and value functions via contrastive objectives from offline datasets (Eysenbach et al., 2022b). These ideas provide an alternative to the model-based approaches presented in this thesis, or could also be combined with them. Goal-conditioned learning provides another way to bridge the gap between general-purpose and task-driven learning, as practitioners can provide distributions over relevant goals or tasks naturally.

8.2 Final remarks

Beyond specific algorithms and approaches, which will likely soon be overtaken by better ones, the more important contribution of a thesis is an overarching idea. I hope this thesis will leave readers with the following: while decision-aware and general-purpose learning methods might seem like opposing ideas, in truth, they supplement each other. The most promising direction forward is therefore to see both as two ends of a spectrum and ask what algorithms are able to synthesize the best of both worlds. We have shown that both finding the right losses that provide synergy with each other and using both, as well as combining losses of either type to create novel loss functions are viable strategies. Recognizing this as an important direction for algorithm design will allow us to move closer and closer to the overarching dream of truly intelligent and autonomous agents acting in the messy complexity of the real world.

Bibliography

- Romina Abachi, Mohammad Ghavamzadeh, and Amir-massoud Farahmand (2020). “Policy-Aware Model Learning for Policy Gradient Methods”. In: *ArXiv*. Vol. abs/2003.00030.
- Romina Abachi, Claas Voelcker, Animesh Garg, and Amir-massoud Farahmand (2022). “VIPer: Iterative Value-Aware Model Learning on the Value Improvement Path”. In: *Decision Awareness in Reinforcement Learning Workshop ICML*.
- Zaheer Abbas, Samuel Sokota, Erin Talvitie, and Martha White (2020). “Selective Dyna-Style Planning Under Limited Model Capacity”. In: *Proceedings of the International Conference on Machine Learning*.
- Zaheer Abbas, Rosie Zhao, Joseph Modayil, Adam White, and Marlos C. Machado (2023). “Loss of Plasticity in Continual Deep Reinforcement Learning”. In: *Proceedings of the Conference on Lifelong Learning Agents*.
- David Abel (2020). *A Theory of Abstraction in Reinforcement Learning*. Amherst, USA: PhD Thesis.
- Alekh Agarwal, Sham Kakade, and Lin F Yang (2020). “Model-based reinforcement learning with a generative model is minimax optimal”. In: *Conference on Learning Theory*.
- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G Bellemare (2021). “Deep Reinforcement Learning at the Edge of the Statistical Precipice”. In: *Advances in Neural Information Processing Systems*.
- (2022). “Reincarnating Reinforcement Learning: Reusing Prior Computation to Accelerate Progress”. In: *Advances in Neural Information Processing Systems*.
- Brandon Amos, Samuel Stanton, Denis Yarats, and Andrew Gordon Wilson (2021). “On the model-based stochastic value gradient for continuous reinforcement learning”. In: *Learning for Dynamics and Control*.
- Charles Anderson (1992). “Q-Learning with Hidden-Unit Restarting”. In: *Advances in Neural Information Processing Systems*.
- Oron Anschel, Nir Baram, and Nahum Shimkin (2017). “Averaged-DQN: Variance Reduction and Stabilization for Deep Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.

- Ioannis Antonoglou, Julian Schrittwieser, Sherjil Ozair, Thomas K Hubert, and David Silver (2022). “Planning in Stochastic Environments with a Learned Model”. In: *Proceedings of the International Conference on Learning Representations*.
- Rika Antonova, Jingyun Yang, Krishna Murthy Jatavallabhula, and Jeannette Bohg (2022). “Rethinking Optimization with Differentiable Simulation from a Global Perspective”. In: *Proceedings of the Conference on Robot Learning*.
- András Antos, Csaba Szepesvári, and Rémi Munos (2008). “Learning near-optimal policies with Bellman-residual minimization based fitted policy iteration and a single sample path”. In: *Machine Learning*. Vol. 71. Springer.
- Kavosh Asadi, Evan Cater, Dipendra Misra, and Michael L Littman (2018). “Equivalence Between Wasserstein and Value-Aware Loss for Model-based Reinforcement Learning”. In: *Prediction and Generative Modeling in Reinforcement Learning Workshop at ICML*.
- Kavosh Asadi, Rasool Fakoor, and Shoham Sabach (2023). “Resetting the Optimizer in Deep RL: An Empirical Study”. In: *Advances in Neural Information Processing Systems*.
- Christopher G. Atkeson and Stefan Schaal (1997). “Robot Learning From Demonstration”. In: *Proceedings of the International Conference on Machine Learning*.
- Alex Ayoub, Zeyu Jia, Csaba Szepesvari, Mengdi Wang, and Lin Yang (2020). “Model-based reinforcement learning with value-targeted regression”. In: *Proceedings of the International Conference on Machine Learning*.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016). “Layer Normalization”. In: *ArXiv*. Vol. abs/1607.06450.
- Leemon Baird (1995). “Residual algorithms: Reinforcement learning with function approximation”. In: *Machine Learning*. Springer.
- Pierre Baldi and Kurt Hornik (1989). “Neural networks and principal component analysis: Learning from examples without local minima”. In: *Neural networks*. Vol. 2. Elsevier.
- Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine (2023). “Efficient online reinforcement learning with offline data”. In: *Proceedings of the International Conference on Machine Learning*.
- Xuchan Bao, James Lucas, Sushant Sachdeva, and Roger B Grosse (2020). “Regularized linear autoencoders recover the principal components, eventually”. In: *Advances in Neural Information Processing Systems*.
- André Barreto, Will Dabney, Rémi Munos, Jonathan J Hunt, Tom Schaul, Hado P Van Hasselt, and David Silver (2017). “Successor features for transfer in reinforcement learning”. In: *Advances in Neural Information Processing Systems*.
- Bahram Behzadian, Soheil Gharatappeh, and Marek Petrik (2019). “Fast feature selection for linear value function approximation”. In: *Proceedings of the International Conference on Automated Planning and Scheduling*.

- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal (2019). “Reconciling modern machine-learning practice and the classical bias–variance trade-off”. In: vol. 116. 32. National Academy of Sciences.
- Marc G. Bellemare, Will Dabney, Robert Dadashi, Adrien Ali Taiga, Pablo Samuel Castro, Nicolas Le Roux, Dale Schuurmans, Tor Lattimore, and Clare Lyle (2019). “A geometric perspective on optimal representations for reinforcement learning”. In: *Advances in Neural Information Processing systems*.
- Marc G. Bellemare, Will Dabney, and Rémi Munos (2017). “A Distributional Perspective on Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling (2013). “The Arcade Learning Environment: An Evaluation Platform for General Agents”. In: *Journal of Artificial Intelligence Research*. Vol. 47.
- Richard Ernest Bellman (1953). *An Introduction to the Theory of Dynamic Programming*. Santa Monica, CA: RAND Corporation.
- Yoshua Bengio, Aaron Courville, and Pascal Vincent (2013). “Representation Learning: A Review and New Perspectives”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*. Vol. 35. IEEE Computer Society.
- Dimitri P. Bertsekas and Steven E. Shreve (1978). *Stochastic Optimal Control: The Discrete-Time Case*. Academic Press.
- Shalabh Bhatnagar, Mohammad Ghavamzadeh, Mark Lee, and Richard S Sutton (2007). “Incremental natural actor-critic algorithms”. In: *Advances in Neural Information Processing Systems*.
- Aditya Bhatt, Daniel Palenicek, Boris Belousov, Max Argus, Artemij Amiranashvili, Thomas Brox, and Jan Peters (2024). “CrossQ: Batch Normalization in Deep Reinforcement Learning for Greater Sample Efficiency and Simplicity”. In: *Proceedings of the International Conference on Learning Representations*.
- Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. (2024). “Deepseek llm: Scaling open-source language models with longtermism”. In: *arXiv*. Vol. abs/2401.02954.
- Christopher M. Bishop (2006). *Pattern Recognition and Machine Learning*. Berlin, Heidelberg: Springer-Verlag.
- Johan Bjorck, Carla P Gomes, and Kilian Q Weinberger (2022). “Is High Variance Unavoidable in RL? A Case Study in Continuous Control”. In: *Proceedings of the International Conference on Learning Representations*.
- Diana Borsa, André Barreto, John Quan, Daniel J. Mankowitz, Hado Van Hasselt, Rémi Munos, David Silver, and Tom Schaul (2019). “Universal Successor Features Approximators”. In: *Proceedings of the International Conference on Learning Representations*.

- Craig Boutilier, Richard Dearden, and Moisés Goldszmidt (2000). “Stochastic dynamic programming with factored representations”. In: *Artificial intelligence*. Elsevier.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba (2016). “OpenAI gym”. In: *ArXiv*. Vol. abs/1606.01540.
- Jacob Buckman, Danijar Hafner, George Tucker, Eugene Brevdo, and Honglak Lee (2018). “Sample-efficient reinforcement learning with stochastic ensemble value expansion”. In: *Advances in Neural Information Processing systems*.
- Johan Samir Obando Ceron, João Guilherme Madeira Araújo, Aaron Courville, and Pablo Samuel Castro (2024). “On the consistency of hyper-parameter selection in value-based deep reinforcement learning”. In: *Reinforcement Learning Conference*.
- Xinyue Chen, Che Wang, Zijian Zhou, and Keith W. Ross (2021). “Randomized Ensembled Double Q-Learning: Learning Fast Without a Model”. In: *Proceedings of the International Conference on Learning Representations*.
- Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine (2018). “Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models”. In: *Advances in Neural Information Processing Systems*.
- Brandon Cui, Yinlam Chow, and Mohammad Ghavamzadeh (2021). “Control-Aware Representations for Model-based Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.
- Pierluca D’Oro, Alberto Maria Metelli, Andrea Tirinzoni, Matteo Papini, and Marcello Restelli (2020). “Gradient-aware model-based policy search”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Pierluca D’Oro, Max Schwarzer, Evgenii Nikishin, Pierre-Luc Bacon, Marc G Bellemare, and Aaron Courville (2023). “Sample-Efficient Reinforcement Learning by Breaking the Replay Ratio Barrier”. In: *Proceedings of the International Conference on Learning Representations*.
- William C Dabney (2014). *Adaptive step-sizes for reinforcement learning*. Amherst, USA: PhD Thesis.
- Jonas Degraeve, Federico Felici, Jonas Buchli, Michael Neunert, Brendan Tracey, Francesco Carpanese, Timo Ewalds, Roland Hafner, Abbas Abdolmaleki, Diego de Las Casas, et al. (2022). “Magnetic control of tokamak plasmas through deep reinforcement learning”. In: *Nature*. Vol. 602. Nature Publishing Group.
- Marc Deisenroth and Carl E Rasmussen (2011). “PILCO: A model-based and data-efficient approach to policy search”. In: *Proceedings of the International Conference on Machine Learning*.
- Shibhansh Dohare, J Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A Rupam Mahmood, and Richard S Sutton (2024). “Loss of plasticity in deep continual learning”. In: *Nature* 632.

- Eric Eaton, Marcel Hussing, Michael Kearns, and Jessica Sorrell (2023). “Replicable Reinforcement Learning”. In: *Conference on Neural Information Processing Systems*.
- Mohamed Elsayed, Qingfeng Lan, Clare Lyle, and A Rupam Mahmood (2024). “Weight clipping for deep continual and reinforcement learning”. In: *Reinforcement Learning Conference*.
- Damien Ernst, Pierre Geurts, and Louis Wehenkel (2005). “Tree-based batch mode reinforcement learning”. In: *Journal of Machine Learning Research*. Vol. 6. Microtome Publishing.
- Benjamin Eysenbach, Alexander Khazatsky, Sergey Levine, and Russ R Salakhutdinov (2022a). “Mismatched No More: Joint Model-Policy Optimization for Model-Based RL”. In: *Advances in Neural Information Processing Systems*.
- Benjamin Eysenbach, Tianjun Zhang, Sergey Levine, and Ruslan Salakhutdinov (2022b). “Contrastive Learning as Goal-Conditioned Reinforcement Learning”. In: *Advances in Neural Information Processing Systems*.
- Amir-massoud Farahmand (2011). *Regularization in Reinforcement Learning*. Edmonton, Canada: PhD Thesis.
- (2018). “Iterative Value-Aware Model Learning”. In: *Advances in Neural Information Processing Systems*.
- (2021). *Lecture Notes on Reinforcement Learning*.
- Amir-massoud Farahmand, André Barreto, and Daniel Nikovski (2017). “Value-Aware Loss Function for Model-based Reinforcement Learning”. In: *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Amir-massoud Farahmand and Mohammad Ghavamzadeh (2021). “PID accelerated value iteration algorithm”. In: *Proceedings of the International Conference on Machine Learning*.
- Amir-massoud Farahmand, Csaba Szepesvári, and Rémi Munos (2010). “Error propagation for approximate policy and value iteration”. In: *Advances in Neural Information Processing Systems*.
- Jesse Farebrother, Joshua Greaves, Rishabh Agarwal, Charline Le Lan, Ross Goroshin, Pablo Samuel Castro, and Marc G Bellemare (2023). “Proto-Value Networks: Scaling Representation Learning with Auxiliary Tasks”. In: *Proceedings of the International Conference on Learning Representations*.
- Jesse Farebrother, Marlos C. Machado, and Michael Bowling (2018). “Generalization and Regularization in DQN”. In: *NeurIPS Workshop on Deep Reinforcement Learning*.
- Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taiga, Yevgen Chebotar, Ted Xiao, Alex Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, Aviral Kumar, and Rishabh Agarwal (2024). “Stop Regressing: Training Value Functions via Classification for Scalable Deep RL”. In: *Proceedings of the International Conference on Machine Learning*.

- William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Will Dabney (2020). “Revisiting fundamentals of experience replay”. In: *Proceedings of the International Conference on Machine Learning*.
- Norm Ferns, Prakash Panangaden, and Doina Precup (2004). “Metrics for Finite Markov Decision Processes.” In: *Uncertainty in AI*.
- (2011). “Bisimulation metrics for continuous Markov decision processes”. In: *SIAM Journal on Computing*. Vol. 40. 6.
- Roy Fox, Ari Pakman, and Naftali Tishby (2016). “Taming the noise in reinforcement learning via soft updates”. In: *Conference on Uncertainty in Artificial Intelligence*.
- Scott Fujimoto, Wei-Di Chang, Edward Smith, Shixiang Shane Gu, Doina Precup, and David Meger (2024). “For sale: State-action representation learning for deep reinforcement learning”. In: *Advances in Neural Information Processing Systems*.
- Scott Fujimoto, Pierluca D’Oro, Amy Zhang, Yuandong Tian, and Michael Rabbat (2025). “Towards General-Purpose Model-Free Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.
- Scott Fujimoto and Shixiang Gu (2021). “A Minimalist Approach to Offline Reinforcement Learning”. In: *Advances in Neural Information Processing Systems*.
- Scott Fujimoto, Herke Hoof, and David Meger (2018a). “Addressing function approximation error in actor-critic methods”. In: *Proceedings of the International Conference on Machine Learning*.
- Scott Fujimoto, David Meger, and Doina Precup (2019). “Off-Policy Deep Reinforcement Learning without Exploration”. In: *Proceedings of the International Conference on Machine Learning*.
- Scott Fujimoto, Herke van Hoof, and David Meger (2018b). “Addressing Function Approximation Error in Actor-Critic Methods”. In: *Proceedings of the International Conference on Machine Learning*.
- Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G Bellemare (2019). “Deepmdp: Learning continuous latent space models for representation learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Ignat Georgiev, Krishnan Srinivasan, Jie Xu, Eric Heiden, and Animesh Garg (2024). “Adaptive Horizon Actor-Critic for Policy Learning in Contact-Rich Differentiable Simulation”. In: *Proceedings of the International Conference on Machine Learning*.
- Dibya Ghosh and Marc G Bellemare (2020). “Representations for stable off-policy reinforcement learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Raj Ghugare, Homanga Bharadhwaj, Benjamin Eysenbach, Sergey Levine, and Russ Salakhutdinov (2023). “Simplifying Model-based RL: Learning Representations, Latent-space Models, and Policies with One Objective”. In: *Proceedings of the International Conference on Learning Representations*.

- Xavier Glorot, Antoine Bordes, and Yoshua Bengio (2011). “Deep Sparse Rectifier Neural Networks”. In: *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Yoav Goldberg and Omer Levy (2014). “word2vec Explained: Deriving Mikolov et al.’s negative-sampling word-embedding method”. In: *arXiv*. Vol. abs/1402.3722.
- Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio (2016). *Deep learning*. Vol. 1. MIT Press.
- Geoffrey J Gordon (1995). “Stable Function Approximation in Dynamic Programming”. In: *Proceedings of the International Conference on Machine Learning*.
- Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. (2020). “Bootstrap your own latent-a new approach to self-supervised learning”. In: *Advances in Neural Information Processing systems*.
- Christopher Grimm, André Barreto, Gregory Farquhar, David Silver, and Satinder Singh (2021). “Proper Value Equivalence”. In: *Advances in Neural Information Processing Systems*.
- Christopher Grimm, André Barreto, Satinder Singh, and David Silver (2020). “The Value Equivalence Principle for Model-Based Reinforcement Learning”. In: *Advances in Neural Information Processing Systems*.
- Jonas Guan, Shon Eduard Verch, Claas Voelcker, Ethan C Jackson, Nicolas Papernot, and William A Cunningham (2024). “Temporal-Difference Learning Using Distributed Error Signals”. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Zhaohan Daniel Guo, Shantanu Thakoor, Miruna Pislari, Bernardo Avila Pires, Florent Altché, Corentin Tallec, Alaa Saade, Daniele Calandriello, Jean-Bastien Grill, Yunhao Tang, Michal Valko, Remi Munos, Mohammad Gheshlaghi Azar, and Bilal Piot (2022). “BYOL-Explore: Exploration by Bootstrapped Prediction”. In: *Advances in Neural Information Processing Systems*.
- László Györfi, Michael Kohler, Adam Krzyżak, and Harro Walk (2002). “A Distribution-Free Theory of Nonparametric Regression”. In: *Springer Series in Statistics*.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine (2018). “Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor”. In: *Proceedings of the International Conference on Machine Learning*.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi (2020). “Dream to Control: Learning Behaviors by Latent Imagination”. In: *Proceedings of the International Conference on Learning Representations*.
- Danijar Hafner, Timothy P Lillicrap, Mohammad Norouzi, and Jimmy Ba (2021). “Mastering Atari with Discrete World Models”. In: *Proceedings of the International Conference on Learning Representations*.

- Nicklas Hansen, Hao Su, and Xiaolong Wang (2024). “TD-MPC2: Scalable, Robust World Models for Continuous Control”. In: *The Twelfth Proceedings of the International Conference on Learning Representations*.
- Nicklas A Hansen, Hao Su, and Xiaolong Wang (2022). “Temporal Difference Learning for Model Predictive Control”. In: *Proceedings of the International Conference on Machine Learning*.
- Nicolas Heess, Gregory Wayne, David Silver, Timothy Lillicrap, Tom Erez, and Yuval Tassa (2015). “Learning continuous control policies by stochastic value gradients”. In: *Advances in Neural Information Processing systems*.
- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger (2018). “Deep reinforcement learning that matters”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Takuya Hiraoka, Takahisa Imagawa, Taisei Hashimoto, Takashi Onishi, and Yoshimasa Tsuruoka (2022). “Dropout Q-Functions for Doubly Efficient Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.
- Marcel Hushing, Claas Voelcker, Igor Gilitschenski, Amir-massoud Farahmand, and Eric Eaton (2024). “Dissecting Deep RL with High Update Ratios: Combatting Value Divergence”. In: *Reinforcement Learning Conference*.
- Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter (2019). “Learning agile and dynamic motor skills for legged robots”. In: *Science Robotics* 4.
- Maximilian Igl, Gregory Farquhar, Jelena Luketina, Wendelin Boehmer, and Shimon Whiteson (2021). “Transient Non-stationarity and Generalisation in Deep Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.
- Alex Irpan (2018). *Deep Reinforcement Learning Doesn’t Work Yet*. <https://www.alexirpan.com/2018/02/14/rl-hard.html>.
- Garud N. Iyengar (2005). “Robust Dynamic Programming”. In: *Mathematics of Operations Research*. Institute for Operations Research and the Management Sciences.
- Alan Julian Izenman (1975). “Reduced-rank regression for the multivariate linear model”. In: *Journal of Multivariate Analysis*.
- Max Jaderberg, Volodymyr Mnih, Wojciech Marian Czarnecki, Tom Schaul, Joel Z Leibo, David Silver, and Koray Kavukcuoglu (2017). “Reinforcement Learning with Unsupervised Auxiliary Tasks”. In: *Proceedings of the International Conference on Learning Representations*.
- Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine (2019). “When to Trust Your Model: Model-Based Policy Optimization”. In: *Advances in Neural Information Processing Systems*.

- Chi Jin, Zhuoran Yang, Zhaoran Wang, and Michael I Jordan (2020). “Provably efficient reinforcement learning with linear function approximation”. In: *Conference on Learning Theory*.
- Ying Jin, Zhuoran Yang, and Zhaoran Wang (2021). “Is pessimism provably efficient for offline rl?”. In: *Proceedings of the International Conference on Machine Learning*.
- Joshua Joseph, Alborz Geramifard, John W Roberts, Jonathan P How, and Nicholas Roy (2013). “Reinforcement learning with misspecified model classes”. In: *IEEE Proceedings of the International Proceedings of the Conference on Robotics and Automation*.
- Leslie Pack Kaelbling (1993). “Learning to achieve goals”. In: *Proceedings of the International Joint Conference on AI*.
- Tyler Kastner, Murat A Erdogdu, and Amir-massoud Farahmand (2023). “Distributional model equivalence for risk-sensitive reinforcement learning”. In: *Advances in Neural Information Processing Systems*.
- Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza (2023). “Champion-level drone racing using deep reinforcement learning”. In: *Nature* 620.
- Michael Kearns and Satinder Singh (2002). “Near-optimal reinforcement learning in polynomial time”. In: *Machine learning*. Vol. 49. Springer.
- Mete Kemertas and Tristan Ty Aumentado-Armstrong (2021). “Towards Robust Bisimulation Metric Learning”. In: *Advances in Neural Information Processing Systems*.
- Mete Kemertas and Allan Douglas Jepson (2022). “Approximate Policy Iteration with Bisimulation Metrics”. In: *Transactions on Machine Learning Research*.
- Woojun Kim, Yongjae Shin, Jongeui Park, and Youngchul Sung (2023). “Sample-Efficient and Safe Deep Reinforcement Learning via Reset Deep Ensemble Agents”. In: *Advances in Neural Information Processing Systems*.
- Diederik Kingma and Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. In: *Proceedings of the International Conference on Learning Representations*.
- Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel (2023). “A survey of zero-shot generalisation in deep reinforcement learning”. In: *Journal of Artificial Intelligence Research* 76.
- Anders Krogh and John Hertz (1991). “A Simple Weight Decay Can Improve Generalization”. In: *Advances in Neural Information Processing Systems*.
- Qi Kuang, Zhoufan Zhu, Liwen Zhang, and Fan Zhou (2023). “Variance Control for Distributional Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Aviral Kumar, Rishabh Agarwal, Dibya Ghosh, and Sergey Levine (2021). “Implicit Under-Parameterization Inhibits Data-Efficient Deep Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.

- Arsenii Kuznetsov, Pavel Shvechikov, Alexander Grishin, and Dmitry Vetrov (2020). “Controlling Overestimation Bias with Truncated Mixture of Continuous Distributional Quantile Critics”. In: *Proceedings of the International Conference on Machine Learning*.
- Cassidy Laidlaw, Stuart Russell, and Anca Dragan (2023). “Bridging RL Theory and Practice with the Effective Horizon”. In: *Thirty-seventh Conference on Neural Information Processing Systems*.
- Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra (2020). “Objective Mismatch in Model-based Reinforcement Learning”. In: *Conference on Learning for Dynamics and Control*.
- Qingfeng Lan, Yangchen Pan, Alona Fyshe, and Martha White (2020). “Maxmin Q-learning: Controlling the Estimation Bias of Q-learning”. In: *Proceedings of the International Conference on Learning Representations*.
- Sascha Lange, Thomas Gabel, and Martin Riedmiller (2012). “Batch reinforcement learning”. In: *Reinforcement learning: State-of-the-art*. Springer.
- Michael Laskin, Aravind Srinivas, and Pieter Abbeel (2020). “CURL: Contrastive Unsupervised Representations for Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Samuel Lavoie, Christos Tsirigotis, Max Schwarzer, Ankit Vani, Michael Noukhovitch, Kenji Kawaguchi, and Aaron Courville (2023). “Simplicial Embeddings in Self-Supervised Learning and Downstream Classification”. In: *Proceedings of the International Conference on Learning Representations*.
- Charline Le Lan, Marc G Bellemare, and Pablo Samuel Castro (2021). “Metrics and continuity in reinforcement learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Charline Le Lan, Stephen Tu, Adam Oberman, Rishabh Agarwal, and Marc G. Bellemare (2022). “On the Generalization of Representations in Reinforcement Learning”. In: *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Charline Le Lan, Stephen Tu, Mark Rowland, Anna Harutyunyan, Rishabh Agarwal, Marc G Bellemare, and Will Dabney (2023). “Bootstrapped Representations in Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Donghun Lee, Boris Defourny, and Warren Buckler Powell (2013). “Bias-corrected Q-learning to control max-operator bias in Q-learning”. In: *IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning*.
- Hojoon Lee, Hanseul Cho, HYUNSEUNG KIM, DAEHOON GWAK, Joonkee Kim, Jaegul Choo, Se-Young Yun, and Chulhee Yun (2023). “PLASTIC: Improving Input and Label Plasticity for Sample Efficient Reinforcement Learning”. In: *Advances in Neural Information Processing Systems*.
- Hojoon Lee, Hyeonseo Cho, Hyunseung Kim, Donghu Kim, Dugki Min, Jaegul Choo, and Clare Lyle (2024). “Slow and Steady Wins the Race: Maintaining Plasticity with Hare

- and Tortoise Networks”. In: *Proceedings of the International Conference on Machine Learning*.
- Kimin Lee, Michael Laskin, Aravind Srinivas, and Pieter Abbeel (2021). “Sunrise: A simple unified framework for ensemble learning in deep reinforcement learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Nir Levine, Yinlam Chow, Rui Shu, Ang Li, Mohammad Ghavamzadeh, and Hung Bui (2020). “Prediction, Consistency, Curvature: Representation Learning for Locally-Linear Control”. In: *Proceedings of the International Conference on Learning Representations*.
- Sergey Levine (2018). “Reinforcement learning and control as probabilistic inference: Tutorial and review”. In: *arXiv preprint arXiv:1805.00909*.
- Sergey Levine and Vladlen Koltun (2013). “Guided policy search”. In: *Proceedings of the International Conference on Machine Learning*.
- Qiyang Li, Aviral Kumar, Ilya Kostrikov, and Sergey Levine (2023). “Efficient Deep Reinforcement Learning Requires Regulating Overfitting”. In: *Proceedings of the International Conference on Learning Representations*.
- Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra (2016). “Continuous control with deep reinforcement learning”. In: *Proceedings of the International Conference on Learning Representations*.
- Zhuang Liu, Xuanlin Li, Bingyi Kang, and Trevor Darrell (2021). “Regularization Matters in Policy Optimization - An Empirical Study on Continuous Control”. In: *Proceedings of the International Conference on Learning Representations*.
- Ângelo G. Lovatto, Thiago P. Bueno, Denis D. Mauá, and Leliane N. de Barros (2020). “Decision-Aware Model Learning for Actor-Critic Methods: When Theory Does Not Meet Practice”. In: *”I Can’t Believe It’s Not Better!” Workshop at NeurIPS*.
- Cong Lu, Philip Ball, Yee Whye Teh, and Jack Parker-Holder (2024). “Synthetic experience replay”. In: *Advances in Neural Information Processing Systems*.
- Yuping Luo, Huazhe Xu, Yuanzhi Li, Yuandong Tian, Trevor Darrell, and Tengyu Ma (2019). “Algorithmic Framework for Model-based Deep Reinforcement Learning with Theoretical Guarantees”. In: *Proceedings of the International Conference on Learning Representations*.
- Michael Lutter, Leonard Hasenclever, Arunkumar Byravan, Gabriel Dulac-Arnold, Piotr Trochim, Nicolas Heess, Josh Merel, and Yuval Tassa (2021). “Learning Dynamics Models for Model Predictive Agents”. In: *ArXiv*. Vol. abs/2109.14311.
- Clare Lyle, Mark Rowland, and Will Dabney (2022a). “Understanding and Preventing Capacity Loss in Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.

- Clare Lyle, Mark Rowland, Will Dabney, Marta Kwiatkowska, and Yarin Gal (2022b). “Learning dynamics and generalization in deep reinforcement learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Clare Lyle, Mark Rowland, Georg Ostrovski, and Will Dabney (2021). “On the effect of auxiliary tasks on representation dynamics”. In: *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Clare Lyle, Zeyu Zheng, Khimya Khetarpal, Hado Van Hasselt, Razvan Pascanu, James Martens, and Will Dabney (2024). *Disentangling the Causes of Plasticity Loss in Neural Networks*.
- Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney (2023). “Understanding Plasticity in Neural Networks”. In: *Proceedings of the International Conference on Machine Learning*.
- Michel Ma, Tianwei Ni, Clement Gehring, Pierluca D’Oro, and Pierre-Luc Bacon (2024). “Do transformer world models give better policy gradients?” In: *Proceedings of the International Conference on Machine Learning*.
- Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu (2018). “Towards Deep Learning Models Resistant to Adversarial Attacks”. In: *Proceedings of the International Conference on Learning Representations*.
- Hamid Maei, Csaba Szepesvári, Shalabh Bhatnagar, Doina Precup, David Silver, and Richard S Sutton (2009). “Convergent temporal-difference learning with arbitrary smooth function approximation”. In: *Advances in Neural Information Processing systems*.
- Sridhar Mahadevan (2005). “Proto-value functions: Developmental reinforcement learning”. In: *Proceedings of the International Conference on Machine learning*.
- (2009). “Learning representation and control in Markov decision processes: New frontiers”. In: *Foundations and Trends® in Machine Learning*.
- Sridhar Mahadevan and Mauro Maggioni (2007). “Proto-value Functions: A Laplacian Framework for Learning Representation and Control in Markov Decision Processes.” In: *Journal of Machine Learning Research*. Vol. 8. 10.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean (2013). “Distributed Representations of Words and Phrases and their Compositionality”. In: *Advances in Neural Information Processing Systems*.
- Diganta Misra (2020). “Mish: A self regularized non-monotonic activation function”. In: *British Machine Vision Conference*.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller (2013). “Playing Atari With Deep Reinforcement Learning”. In: *NeurIPS Deep Learning Workshop*.
- Nirbhay Modhe, Harish Kamath, Dhruv Batra, and A. Kalyan (2021). “Model-Advantage Optimization for Model-Based Reinforcement Learning”. In: *ArXiv*. Vol. abs/2106.14080.

- Aditya Modi, Nan Jiang, Ambuj Tewari, and Satinder Singh (2020). “Sample Complexity of Reinforcement Learning using Linearly Combined Model Ensembles”. In: *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker (2023). “Model-based Reinforcement Learning: A Survey”. In: *Foundations and Trends in Machine Learning*. Vol. 16. Now Publishers.
- Ted Moskovitz, Jack Parker-Holder, Aldo Pacchiano, Michael Arbel, and Michael Jordan (2021). “Tactical optimism and pessimism for deep reinforcement learning”. In: *Advances in Neural Information Processing Systems*.
- James R. Munkres (2018). *Topology*. 2nd. Pearson Modern Classic.
- Suraj Nair, Silvio Savarese, and Chelsea Finn (2020). “Goal-aware prediction: Learning to model what matters”. In: *Proceedings of the International Conference on Machine Learning*.
- Vinod Nair and Geoffrey E Hinton (2010). “Rectified linear units improve restricted boltzmann machines”. In: *Proceedings of the International Conference on Machine Learning*.
- Michał Nauman, Michał Bortkiewicz, Piotr Miłoś, Tomasz Trzcinski, Mateusz Ostaszewski, and Marek Cygan (2024a). “Overestimation, Overfitting, and Plasticity in Actor-Critic: the Bitter Lesson of Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Michał Nauman, Mateusz Ostaszewski, Krzysztof Jankowski, Piotr Miłoś, and Marek Cygan (2024b). “Bigger, Regularized, Optimistic: scaling for compute and sample-efficient continuous control”. In: *Advances in Neural Information Processing Systems*.
- Tianwei Ni, Benjamin Eysenbach, Erfan Seyedsalehi, Michel Ma, Clement Gehring, Aditya Mahajan, and Pierre-Luc Bacon (2024). “Bridging State and History Representations: Understanding Self-Predictive RL”. In: *Proceedings of the International Conference on Learning Representations*.
- Evgenii Nikishin, Romina Abachi, Rishabh Agarwal, and Pierre-Luc Bacon (2022a). “Control-Oriented Model-Based Reinforcement Learning with Implicit Differentiation”. In: *Proceedings of AAAI Conference on Artificial Intelligence*.
- Evgenii Nikishin, Junhyuk Oh, Georg Ostrovski, Clare Lyle, Razvan Pascanu, Will Dabney, and André Barreto (2024). “Deep reinforcement learning with plasticity injection”. In: *Advances in Neural Information Processing Systems*.
- Evgenii Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon, and Aaron Courville (2022b). “The Primacy Bias in Deep Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Arnab Nilim and Laurent El Ghaoui (2005). “Robust Control of Markov Decision Processes with Uncertain Transition Matrices.” In: *Operations Research*.
- Junhyuk Oh, Satinder Singh, and Honglak Lee (2017). “Value prediction network”. In: *Advances in Neural Information Processing systems*. Vol. 30.

- Georg Ostrovski, Pablo Samuel Castro, and Will Dabney (2021). “The difficulty of passive learning in deep reinforcement learning”. In: *Advances in Neural Information Processing Systems*.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. (2022). “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems*.
- Yangchen Pan, Jincheng Mei, and Amir-massoud Farahmand (2020). “Frequency-based Search-control in Dyna”. In: *Proceedings of the International Conference on Learning Representations*.
- Yangchen Pan, Hengshuai Yao, Amir-massoud Farahmand, and Martha White (2019). “Hill climbing on value estimates for search-control in Dyna”. In: *Proceedings of the International Joint Conference on Artificial Intelligence*.
- German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter (2019). “Continual lifelong learning with neural networks: A review”. In: *Neural networks* 113.
- Keiran Paster, Lev E McKinney, Sheila A. McIlraith, and Jimmy Ba (2021). “BLAST: Latent Dynamics Models from Bootstrapping”. In: *NeurIPS Deep Learning Workshop*.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala (2019). “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems* 32.
- Andrew Patterson, Samuel Neumann, Raksha Kumaraswamy, Martha White, and Adam White (2024a). “Cross-environment Hyperparameter Tuning for Reinforcement Learning”. In: *Reinforcement Learning Journal*.
- Andrew Patterson, Samuel Neumann, Martha White, and Adam White (2024b). “Empirical design in reinforcement learning”. In: *Journal of Machine Learning Research* 25.
- Oren Peer, Chen Tessler, Nadav Merlis, and Ron Meir (2021). “Ensemble Bootstrapping for Q-Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Jan Peters and Stefan Schaal (2008). “Natural actor-critic”. In: *Neurocomputing*. Vol. 71. Elsevier.
- Luis Pineda, Brandon Amos, Amy Zhang, Nathan O. Lambert, and Roberto Calandra (2021). “MBRL-Lib: A Modular Library for Model-based Reinforcement Learning”. In: *ArXiv*. Vol. abs/2104.10159.
- Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta (2017). “Robust Adversarial Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.

- Dean A. Pomerleau (1988). “ALVINN: An Autonomous Land Vehicle in a Neural Network”. In: *Advances in Neural Information Processing Systems*.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra (2022). “Grokking: Generalization beyond overfitting on small algorithmic datasets”. In: *arXiv*. Vol. abs/2201.02177.
- Doina Precup, Richard S. Sutton, and Sanjoy Dasgupta (2001). “Off-Policy Temporal Difference Learning with Function Approximation”. In: *Proceedings of the International Conference on Machine Learning*.
- Arnu Pretorius, Steve Kroon, and Herman Kamper (2018). “Learning dynamics of linear denoising autoencoders”. In: *Proceedings of the International Conference on Machine Learning*.
- Martin L. Puterman (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. 1st. USA: John Wiley & Sons, Inc.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn (2023). “Direct Preference Optimization: Your Language Model is Secretly a Reward Model”. In: *Advances in Neural Information Processing Systems*.
- Amin Rakhsha, Mete Kemertas, Mohammad Ghavamzadeh, and Amir-massoud Farahmand (2024). “Maximum Entropy Model Correction in Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.
- Amin Rakhsha, Andrew Wang, Mohammad Ghavamzadeh, and Amir-massoud Farahmand (2022). “Operator Splitting Value Iteration”. In: *Advances in Neural Information Processing Systems*.
- Stéphane Ross and Drew Bagnell (2012). “Agnostic System Identification for Model-Based Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Baturay Saglam, Enes Duran, Dogan C. Cicek, Furkan B. Mutlu, and Suleyman S. Kozat (2021). “Estimation Error Correction in Deep Reinforcement Learning for Deterministic Actor-Critic Methods”. In: *Proceedings of the International Conference on Tools with Artificial Intelligence*.
- Andrew M. Saxe, James L. McClelland, and Surya Ganguli (2014). “Exact solutions to the nonlinear dynamics of learning in deep linear neural networks”. In: *Proceedings of the International Conference on Learning Representations*.
- Tom Schaul, André Barreto, John Quan, and Georg Ostrovski (2022). “The Phenomenon of Policy Churn”. In: *Advances in Neural Information Processing Systems*.
- Tom Schaul, Daniel Horgan, Karol Gregor, and David Silver (2015). “Universal Value Function Approximators”. In: *Proceedings of the International Conference on Machine Learning*.
- Jeff G Schneider (1997). “Exploiting model uncertainty estimates for safe dynamic control learning”. In: *Advances in Neural Information Processing Systems*.

- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. (2020). “Mastering atari, go, chess and shogi by planning with a learned model”. In: *Nature*. Vol. 588. Nature Publishing Group.
- Max Schwarzer, Ankesh Anand, Rishab Goel, R Devon Hjelm, Aaron Courville, and Philip Bachman (2021a). “Data-Efficient Reinforcement Learning with Self-Predictive Representations”. In: *Proceedings of the International Conference on Learning Representations*.
- Max Schwarzer, Johan Samir Obando Ceron, Aaron Courville, Marc G Bellemare, Rishabh Agarwal, and Pablo Samuel Castro (2023). “Bigger, Better, Faster: Human-level Atari with human-level efficiency”. In: *Proceedings of the International Conference on Machine Learning*.
- Max Schwarzer, Nitarshan Rajkumar, Michael Noukhovitch, Ankesh Anand, Laurent Charlin, R Devon Hjelm, Philip Bachman, and Aaron C Courville (2021b). “Pretraining representations for data-efficient reinforcement learning”. In: *Advances in Neural Information Processing Systems*.
- David Silver, Aja Huang, ”Chris J.” Maddison, Arthur Guez, Laurent Sifre, George Van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, and Marc Lanctot (2016). “Mastering the game of Go with deep neural networks and tree search”. In: *nature*. Vol. 529.
- David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller (2014). “Deterministic policy gradient algorithms”. In: *Proceedings of the International Conference on Machine Learning*.
- David Silver, Hado Van Hasselt, Matteo Hessel, Tom Schaul, Arthur Guez, Tim Harley, Gabriel Dulac-Arnold, David Reichert, Neil Rabinowitz, André Barreto, et al. (2017). “The predictron: end-to-end learning and planning”. In: *Proceedings of the International Conference on Machine Learning*.
- Laura Smith, Ilya Kostrikov, and Sergey Levine (2023). “Demonstrating a walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning”. In.
- Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci (2023). “The dormant neuron phenomenon in deep reinforcement learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Xingyou Song, Yiding Jiang, Stephen Tu, Yilun Du, and Behnam Neyshabur (2020). “Observational Overfitting in Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov (2014). “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research*. Vol. 15. MIT Press.

- Ingo Steinwart and Andreas Christmann (2008). *Support vector machines: Information Science and Statistics*. New York, USA: Springer.
- Austin Stone, Oscar Ramirez, Kurt Konolige, and Rico Jonschkowski (2021). “The Distracting Control Suite - A Challenging Benchmark for Reinforcement Learning from Pixels”. In: *ArXiv*. Vol. abs/2101.02722.
- Hyung Ju Suh, Max Simchowitz, Kaiqing Zhang, and Russ Tedrake (2022). “Do differentiable simulators give better policy gradients?” In: *Proceedings of the International Conference on Machine Learning*.
- Richard S Sutton (1988). “Learning to predict by the methods of temporal differences”. In: *Machine learning*. Vol. 3. Springer.
- (1990). “Integrated architectures for learning, planning, and reacting based on approximating dynamic programming”. In: *Machine Learning*. Vol. 5. Springer.
- Richard S Sutton, A Rupam Mahmood, and Martha White (2016). “An emphatic approach to the problem of off-policy temporal-difference learning”. In: *Journal of Machine Learning Research*. Vol. 17. MIT Press.
- Richard S. Sutton and Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA, USA: A Bradford Book.
- Erin Talvitie (2017). “Self-correcting models for model-based reinforcement learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Erin J Talvitie, Zilei Shao, Huiying Li, Jinghan Hu, Jacob Boerma, Rory Zhao, and Xintong Wang (2024). “Bounding-Box Inference for Error-Aware Model-Based Reinforcement Learning”. In: *Reinforcement Learning Journal*.
- Yunhao Tang, Zhaohan Daniel Guo, Pierre Harvey Richemond, Bernardo Ávila Pires, Yash Chandak, Rémi Munos, Mark Rowland, Mohammad Gheshlaghi Azar, Charline Le Lan, Clare Lyle, et al. (2023). “Understanding Self-Predictive Learning for Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Yunhao Tang and Rémi Munos (2023). “Towards a better understanding of representation dynamics under TD-learning”. In: *Proceedings of the International Conference on Machine Learning*.
- Denis Tarasov, Vladislav Kurenkov, Alexander Nikulin, and Sergey Kolesnikov (2023). “Revisiting the Minimalist Approach to Offline Reinforcement Learning”. In: *Advances in Neural Information Processing Systems*.
- Sebastian Thrun and Anton Schwartz (1993). “Issues in Using Function Approximation for Reinforcement Learning”. In: *Connectionist Models Summer School*.
- Yuandong Tian, Xinlei Chen, and Surya Ganguli (2021). “Understanding self-supervised learning dynamics without contrastive pairs”. In: *Proceedings of the International Conference on Machine Learning*.
- Dhruva Tirumala, Thomas Lampe, Jose Enrique Chen, Tuomas Haarnoja, Sandy Huang, Guy Lever, Ben Moran, Tim Hertweck, Leonard Hasenclever, Martin Riedmiller, Nicolas

- Heess, and Markus Wulfmeier (2024). “Replay across Experiments: A Natural Extension of Off-Policy RL”. In: *Proceedings of the International Conference on Learning Representations*.
- Manan Tomar, Utkarsh Aashu Mishra, Amy Zhang, and Matthew E. Taylor (2023). “Learning Representations for Pixel-based Control: What Matters and Why?” In: *Transactions on Machine Learning Research*.
- John Tsitsiklis and Benjamin Van Roy (1996). “Analysis of Temporal-Difference Learning with Function Approximation”. In: *Advances in Neural Information Processing Systems*.
- Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqu Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa (2020). “dm_control: Software and tasks for continuous control”. In: *Software Impacts*. Vol. 6.
- Hado Van Hasselt (2010). “Double Q-learning”. In: *Advances in Neural Information Processing Systems*.
- Hado Van Hasselt, Arthur Guez, and David Silver (2016a). “Deep reinforcement learning with double Q-Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*.
- (2016b). “Deep reinforcement learning with double q-learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Nino Vieillard, Bruno Scherrer, Olivier Pietquin, and Matthieu Geist (2020). “Momentum in Reinforcement Learning”. In: *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Giuseppe Vietri, Borja Balle, Akshay Krishnamurthy, and Steven Wu (2020). “Private Reinforcement Learning with PAC and Regret Guarantees”. In: *Proceedings of the International Conference on Machine Learning*.
- Claas Voelcker, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-massoud Farahmand (2024a). λ -models: Effective Decision-Aware Reinforcement Learning with Latent Models.
- Claas Voelcker, Marcel Hussing, and Eric Eaton (2024b). “Can we hop in general? A discussion of benchmark selection and design using the Hopper environment”. In: *Finding the Frame: An RLC Workshop for Examining Conceptual Frameworks*.
- Claas Voelcker, Marcel Hussing, Eric Eaton, Amir-massoud Farahmand, and Igor Gilitschenski (2025a). “MAD-TD: Model-Augmented Data stabilizes High Update Ratio RL”. In: *Proceedings of the International Conference on Learning Representations*.
- Claas Voelcker, Tyler Kastner, Igor Gilitschenski, and Amir-massoud Farahmand (2024c). “When does self-prediction help? Understanding Auxiliary Tasks in Reinforcement Learning”. In: *Reinforcement Learning Conference*.
- Claas Voelcker, Victor Liao, Animesh Garg, and Amir-massoud Farahmand (2022). “Value Gradient weighted Model-Based Reinforcement Learning”. In: *Proceedings of the International Conference on Learning Representations*.

- Claas Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-massoud Farahmand (2025b). “Calibrated value-aware loss functions with stochastic environment models”. In: *Proceedings of the International Conference on Machine Learning*.
- Yi Wan, Zaheer Abbas, Adam White, Martha White, and Richard S Sutton (2019). “Planning with expectation models”. In: *Proceedings of the International Joint Conference on Artificial Intelligence*.
- Che Wang, Yanqiu Wu, Quan Vuong, and Keith Ross (2020). “Striving for Simplicity and Performance in Off-Policy DRL: Output Normalization and Non-Uniform Sampling”. In: *Proceedings of the International Conference on Machine Learning*.
- Manuel Watter, Jost Tobias Springenberg, Joschka Boedecker, and Martin Riedmiller (2015). “Embed to Control: A Locally Linear Latent Dynamics Model for Control from Raw Images”. In: *Advances in Neural Information Processing Systems*.
- Ran Wei, Nathan Lambert, Anthony D McDonald, Alfredo Garcia, and Roberto Calandra (2024). “A Unified View on Solving Objective Mismatch in Model-Based Reinforcement Learning”. In: *Transactions on Machine Learning Research*.
- Wolfram Wiesemann, Daniel Kuhn, and Breç Rustem (2013). “Robust Markov Decision Processes”. In: *Mathematics of Operations Research*. Vol. 38. Institute for Operations Research and the Management Sciences.
- Dongming Wu, Xingping Dong, Jianbing Shen, and Steven C. H. Hoi (2020). “Reducing Estimation Bias via Triplet-Average Deep Deterministic Policy Gradient”. In: *IEEE Transactions on Neural Networks and Learning Systems*.
- Guowei Xu, Ruijie Zheng, Yongyuan Liang, Xiyao Wang, Zhecheng Yuan, Tianying Ji, Yu Luo, Xiaoyu Liu, Jiaxin Yuan, Pu Hua, Shuzhen Li, Yanjie Ze, Hal Daumé III, Furong Huang, and Huazhe Xu (2024). “DrM: Mastering Visual Reinforcement Learning through Dormant Ratio Minimization”. In: *Proceedings of the International Conference on Learning Representations*.
- Jie Xu, Miles Macklin, Viktor Makoviychuk, Yashraj Narang, Animesh Garg, Fabio Ramos, and Wojciech Matusik (2022). “Accelerated Policy Learning with Parallel Differentiable Simulation”. In: *Proceedings of the International Conference on Learning Representations*.
- Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin (2019). “Understanding and Improving Layer Normalization”. In: *Advances in Neural Information Processing Systems*.
- Denis Yarats, Ilya Kostrikov, and Rob Fergus (2021). “Image Augmentation Is All You Need: Regularizing Deep Reinforcement Learning from Pixels”. In: *Proceedings of the International Conference on Learning Representations*.

- Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao (2021). “Mastering atari games with limited data”. In: *Advances in Neural Information Processing Systems*.
- Kenny Young and Tian Tian (2019). “MinAtar: An Atari-Inspired Testbed for Thorough and Reproducible Reinforcement Learning Experiments”. In: *arXiv*. Vol. abs/1903.03176.
- Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Y Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma (2020). “Mopo: Model-based offline policy optimization”. In: *Advances in Neural Information Processing Systems*.
- Amy Zhang, Rowan Thomas McAllister, Roberto Calandra, Yarín Gal, and Sergey Levine (2021). “Learning Invariant Representations for Reinforcement Learning without Reconstruction”. In: *Proceedings of the International Conference on Learning Representations*.
- Biao Zhang and Rico Sennrich (2019). “Root Mean Square Layer Normalization”. In: *Advances in Neural Information Processing Systems*.
- Zongzhang Zhang, Zhiyuan Pan, and Mykel J. Kochenderfer (2017). “Weighted Double Q-learning”. In: *Proceedings of the International Joint Conference on Artificial Intelligence*.
- Yi Zhao, Wenshuai Zhao, Rinu Boney, Juho Kannala, and Joni Pajarinen (2023). “Simplified Temporal Consistency Reinforcement Learning”. In: *Proceedings of the International Conference on Machine Learning*.

Appendix A

Auxiliary Lemmata

A.1 Chapter 4: Helpful definitions and lemmata

This section contains helpful lemmata that are used for our proofs. Where we took these from existing work, we provide references; otherwise the proofs are our own, although probably also known in the literature.

A.1.1 Linear Algebra

As before, for matrices V we write $\text{span}(V)$ to denote the span of their column vectors.

Definition A.1.1.1 (Top- k singular vectors). *Let $P^\pi = U^\top \Sigma V$ be the singular value decomposition, and assume that the diagonal of Σ is arranged in decreasing order. The first k rows of $U^\top$ and the first k columns of V are called the top- k left and right singular vectors, respectively.*

Lemma A.1.1.1 (Spectrum of Kronecker product matrix). *Consider two non-singular matrices M and N . Let λ_i be the eigenvalues of M and μ_j be the eigenvalues of N , with eigenvectors u_i and v_j respectively. Then the eigenvalues of $M \otimes N$ are $\lambda_i \mu_j$ with eigenvectors $u_i \otimes v_j$ respectively.*

Proof. For any eigenvector u_i of M and v_j of N , we have

$$(M \otimes N)(u_i \otimes v_j) = (Mu_i) \otimes (Nv_j) = (\lambda_i u_i) \otimes (\mu_j v_j) = \lambda_i \mu_j (u_i \otimes v_j).$$

So $u_i \otimes v_j$ is an eigenvector of $M \otimes N$ with eigenvalue $\lambda_i \mu_j$ □

Lemma A.1.1.2 (Orthogonalized bases and the Kronecker product). *Let $V \in \mathbb{R}^{n \times m}$ be a matrix. Let $\text{orth}(V)$ be a matrix of any orthonormal basis vectors for the column vectors of*

V . Then $\text{span}(V \otimes \mathbf{1}) = \text{span}(\text{orth}(V) \otimes \mathbf{1})$. Furthermore $(1/k)\text{orth}(V) \otimes \mathbf{1}$ is an orthonormal basis of $\text{span}(V \otimes \mathbf{1})$.

Proof. Let $V_i \otimes \mathbf{1}^k$ be any column vector of $V \otimes \mathbf{1}^k$. Let $\alpha_1, \dots, \alpha_m$ be coefficients so that $V_i = \sum_{j=1}^m \alpha_j \text{orth}(V)_j$. Then

$$\sum_{j=1}^m \alpha_j \left(\text{orth}(V)_j \otimes \mathbf{1}^k \right) = \left(\sum_{j=1}^m \alpha_j \text{orth}(V)_j \right) \otimes \mathbf{1}^k = V_i \otimes \mathbf{1}^k,$$

following the standard associative and distributive properties of the Kronecker product.

As every vector in the original span can be represented in the orthogonalized span, the two are equivalent.

Finally note that

$$\frac{1}{k} (\text{orth}(V) \otimes \mathbf{1})_i^\top (\text{orth}(V) \otimes \mathbf{1})_i = \frac{1}{k} \sum_{j=1}^k \left(\text{orth}(V)_i^\top \text{orth}(V)_j \right) = 1, \text{ and}$$

$$\frac{1}{k} (\text{orth}(V) \otimes \mathbf{1})_j^\top (\text{orth}(V) \otimes \mathbf{1})_i = \frac{1}{k} \sum_{t=1}^k \left(\text{orth}(V)_j^\top \text{orth}(V)_t \right) = 0.$$

□

Lemma A.1.1.3 (Reduced rank regression). *Let $C, D \in \mathbb{R}^{n \times n}$, $A \in \mathbb{R}^{n \times k}$, and $B \in \mathbb{R}^{k \times n}$ be full rank matrices with $n \geq k$. Let $A^*, B^* = \min_{A, B} \|CAB - DC\|_F^2$. Let $u_1, \dots, u_k$ and $v_1, \dots, v_k$ be the top- k singular vectors of $C^{-1}DC$ according to [Definition A.1.1.1](#). Then $\text{span}(A^*) = \text{span}(u_1, \dots, u_k)$ and $\text{span}(B) = \text{span}(v_1, \dots, v_k)$.*

This is a reduced rank regression problem (Izenman, 1975) or low-rank matrix approximation problem.

The unconstrained solution to the problem is given by $\hat{A}\hat{B} = C^{-1}DC$. From the Eckhart-Young theorem, we know that the optimal low-rank approximation to $C^{-1}DC$ is given by the top- k singular vectors. Therefore A and B span top- k left and right singular vectors respectively. For a more extensive proof, please refer to Izenman (1975).

A.1.2 Stochastic matrices

Lemma A.1.2.1 (Spectrum of a resolvent matrix). *Let A be an invertible matrix with unique real eigenvalues and $-1 \leq \lambda_{\min} \leq \lambda_{\max} \leq 1$. Let $\gamma \in (0, 1)$. The matrices A and $(I - \gamma A)^{-1}$ have the same eigenvectors and the ordering of the corresponding eigenvalues remains the same.*

Proof. Let e be an eigenvector of A and λ the corresponding eigenvalue. Then

$$(I - \gamma A)^{-1}e = \sum_{n=0}^{\infty} \gamma^n A^n e = \sum_{n=0}^{\infty} (\gamma \lambda)^n e = \frac{1}{1 - \gamma \lambda} e.$$

As $(1 - \gamma \lambda)^{-1}$ is a monotonic function for $-1 \leq \lambda \leq 1$ and $\gamma \in [0, 1]$, the ordering of the eigenvalues remains the same. $\square$

Lemma A.1.2.2 (Basis equivalence of linear reward and value function). *Let r be the vector representation of the reward function and V of the value function respectively for an MDP with fixed policy and transition matrix P^π . Let $U = \{u_1, \dots, u_n\}$ be the set of eigenvectors of P^π , and let $U^r \subseteq U$ be a minimal set of eigenvectors so that $r \in \text{span}(U^r)$. Then $V \in \text{span}(U^r)$.*

Proof. Let $r = \sum_{i=1}^k \alpha_i u_i^r$ be the reward representation in the basis U^r . Then, by Lemma A.1.2.1

$$V^\pi = (I - \gamma P^\pi)^{-1} r = \sum_{i=1}^k \frac{\alpha_i}{1 - \lambda_i \gamma} u_i^r.$$

$\square$

A.1.3 Ordinary differential equations

For describing the stability of ODEs, we use the notion of *asymptotic stability*, with the condition $\Re(\lambda_i) < 0$ for all eigenvalues λ_i of the Jacobian at a critical point.

Lemma A.1.3.1 (Linear reparameterization of an autonomous ODE). *Let $y' = f(y)$ be an autonomous ordinary differential equation. Let y^* be any critical point for which $f(y^*) = 0$. Let furthermore $x' = f(Ax)$ be a reparameterized autonomous ODE for any invertible matrix A . Then the x^* is a critical point with $f(Ax) = 0$ iff $x^* = A^{-1}y^*$. Furthermore, the eigenvalues of the Jacobian of y' at y^* are equal to the eigenvalues of the Jacobian of x' at $x^* = A^{-1}y^*$.*

Proof. The direction $x = A^{-1}y^* \Rightarrow f(Ax) = 0$ is clear by direct evaluation. We now focus on the direction $f(Ax) = 0 \Rightarrow x = A^{-1}y^*$. Assume that $f(Ax)$ is 0 and $x = A^{-1}y$ for a y which is not a point satisfying $f(y) = 0$. But then $0 = f(AA^{-1}y) = f(y) \neq 0$, a contradiction.

For stability, note that

$$\begin{aligned} \frac{d}{dt}y &= f(y) \\ \Rightarrow \frac{d}{dt}x &= \frac{d}{dy}x \frac{d}{dt}y \end{aligned}$$

$$\begin{aligned}
&= A^{-\top} f(y) \\
\Rightarrow \quad \frac{d}{dx} \frac{d}{dt} x &= A^{-\top} \frac{d}{dx} f(Ax) \\
&= A^{-\top} \frac{d}{dy} f(y) \frac{d}{dx} Ax \\
&= A^{-\top} \frac{d}{dy} f(y) A^{\top}.
\end{aligned}$$

This shows that the Jacobian $\frac{d}{dx} f(Ax)|_{x=A^{-1}y_0}$ is similar to the Jacobian $\frac{d}{dy} f(y)|_{y=y_0}$, which means their spectra are identical. $\square$

A.1.4 MDP representation and TD learning

Lemma A.1.4.1 (Lemma 5 of Tang et al. 2023). *Suppose P^π is real-diagonalizable, and write $u_1, \dots, u_n$ for its eigenvectors. Then any orthonormal matrix Φ which has the same span as a set of k eigenvectors is a minimizer of L_{lat} .*

The next three statements are taken from Ghosh and Bellemare (2020). They address the TD loss wrt to the learned weights $\hat{V}$ and fixed Φ (compare Section 4.2). We changed the notation of the statements to fit our notation here, we have denoted the diagonal matrix of the state distribution as D and assume that $D = I$ in Assumption 4.2.1.1, while Ghosh and Bellemare (2020) uses Ξ . They use θ for the value function weights while we use $\hat{V}$. They also uses $\text{Spec}(A)$ to denote the spectrum, the set of all eigenvalues of a matrix A .

The notion of stability used in Ghosh and Bellemare (2020) is that of convergence to the unique fixed point of the projected Bellman update of the linear ODE induced by the L_{TD} loss when fixing Φ . In the two-timescale scenario considered in Chapter 4, this corresponds to the “inner” ODE over $\hat{V}$.

Lemma A.1.4.2 (Proposition 3.1 of Ghosh and Bellemare 2020). *TD(0) is stable if and only if the eigenvalues of the implied iteration matrix $A_\Phi = \Phi^\top D(I - \gamma P^\pi)\Phi$ have positive real components, that is*

$$\text{Spec}(A_\Phi) \subset \mathbb{C}_+ := \{z : \text{Re}(z) > 0\}.$$

We say that a particular choice of representation Φ is stable for (P^π, γ, D) when A_Φ satisfies the above condition.

Lemma A.1.4.3 (Proposition 3.2 of Ghosh and Bellemare 2020). *An orthogonal representation Φ is stable if and only if the real part of the eigenvalues of the induced transition matrix $\Pi P^\pi \Pi$ where $\Pi = \Phi \Phi^\top$ is bounded above, according to*

$$\text{Spec}(\Pi P^\pi \Pi) \subset \{z \in \mathbb{C} : \text{Re}(z) < \frac{1}{\gamma}\}.$$

In particular, Φ is stable if $\rho(\Pi P^\pi \Pi) < \frac{1}{\gamma}$.

Lemma A.1.4.4 (Theorem 4.1 of Ghosh and Bellemare 2020). *An orthogonal invariant representation Φ (meaning $\text{span}(P^\pi \Phi) \subseteq \text{span}(\Phi)$) satisfies*

$$\text{Spec}(\Pi P^\pi \Pi) \subseteq \text{Spec}(P^\pi) \cup \{0\}$$

and is therefore stable.

As a corollary of their proof we have that

Lemma A.1.4.5 (Corollary of Ghosh and Bellemare 2020). *Let Φ be an orthogonal (but not necessarily square) invariant representation of P^π . Then the spectral radius $\rho(\Phi^\top P^\pi \Phi) \leq 1$.*

The proof follows directly from Lemma A.1.4.4 by the cyclical property of the spectrum.

The following two results are our own, although closely related results exist in the literature.

Lemma A.1.4.6 (Lossless approximation of V). *Let Φ be an orthonormal basis of an invariant subspace of P^π and let Assumption 4.5.2.1 hold. Let $V = (I - \gamma P^\pi)^{-1} r$ be the value function of P^π and r^π . Then*

$$\Phi(I - \gamma \Phi^\top P^\pi \Phi)^{-1} \Phi^\top r^\pi = V.$$

Proof. By Assumption 4.5.2.1 we can find a matrix A so that $r^\pi = \Phi A$ and $\Phi \Phi^\top r^\pi = r^\pi$, and by the invariant subspace assumption, we can find a matrix B so that $P^\pi \Phi = \Phi B$. Writing the inverted matrix as an infinite sum, which is valid as the spectrum of $\Phi^\top P^\pi \Phi$ is bounded by 1 following from Lemma A.1.4.5 and Carl Neumann's theorem over power series, we obtain

$$\Phi(I - \gamma \Phi^\top P^\pi \Phi)^{-1} \Phi^\top r^\pi = \Phi \sum_{n=0}^{\infty} \gamma^n (\Phi^\top P^\pi \Phi)^n \Phi^\top r^\pi \quad (\text{A.1})$$

$$= \Phi \sum_{n=0}^{\infty} \gamma^n B^n \Phi^\top r^\pi \quad (P^\pi \Phi = \Phi B) \text{ and } (\Phi^\top \Phi = I) \quad (\text{A.2})$$

$$= \sum_{n=0}^{\infty} \gamma^n \Phi B^n \Phi^\top r^\pi \quad (\text{A.3})$$

$$= \sum_{n=0}^{\infty} \gamma^n P^{\pi n} \Phi \Phi^\top r^\pi \quad (\Phi B = P^\pi \Phi) \text{ iterated} \quad (\text{A.4})$$

$$= \sum_{n=0}^{\infty} \gamma^n P^{\pi n} r^\pi \quad (\Phi \Phi^\top r^\pi = r^\pi) \quad (\text{A.5})$$

$$= V \quad (\text{A.6})$$

□

A.2 Chapter 6: Helpful definitions and lemmata

We provide helper lemmata for [Section 6.4](#) and [Subsection 6.6.1](#) in this section.

All of our proofs rely heavily on a standard expansion technique which is used to prove that sample-based losses correctly approximate population losses. This is found in standard textbooks on learning theory such as Györfi et al. (2002).

It proceeds by expanding a loss $\mathbb{E}_{x \sim p, y \sim q(\cdot|x)} [|f(x) - y|^2]$ with an expected target $f^*(x)$. In our case, this is mostly the minimizer of the target loss when evaluating surrogate losses. Then we obtain

$$\mathbb{E}_{x \sim p, y \sim q(\cdot|x)} [|f(x) - y|^2] = \mathbb{E}_{x \sim p, y \sim q(\cdot|x)} [|f(x) - f^*(x) + f^*(x) - y|^2],$$

and continue expanding from there. Issues generally arise when y depends on $f(x)$ in some way, or when $f(x)$ itself involves a sampling procedure.

A.2.1 Section 6.4 Lemmata

For the following results, we will use the following notation. Let $\mathcal{X}$ be a discrete sample space. Let $f : \mathcal{X} \rightarrow \mathbb{R}$ be a random variable. Let $\mathcal{P}^*$ be the set of distributions for which both $\mathcal{P}^* = \text{Arg min}_{p'} \text{Var}_{p'}[f(x)]$, and for all $p^* \in \mathcal{P}^*$ $\mathbb{E}_{p^*}[f(x)] = \mathbb{E}_p[f(x)]$. Let k be an integer. Finally, let $g(\xi) = \mathbb{E}_{x \sim p} [(\mathbb{E}_{y \sim \xi}[f(y)] - f(x))^2] + \frac{1}{k} \text{Var}_{\xi}[f(x)]$.

We now obtain three simple lemmata about the minima of the function g .

Lemma A.2.1.1. *There does not exist a distribution p' such that $p' \notin \mathcal{P}^*$, $\mathbb{E}_{p'}[f(x)] = \mathbb{E}_p[f(x)]$, and $g(p') \leq g(p^*)$ for any $p^* \in \mathcal{P}^*$.*

Proof. To prove the lemma, we first evaluate g for any distribution ξ^* with $\mathbb{E}_{\xi^*}[f(x)] = \mathbb{E}_p[f(x)]$

$$g(\xi^*) = \mathbb{E}_{x \sim p} [(\mathbb{E}_{y \sim \xi^*}[f(y)] - f(x))^2] + \frac{1}{k} \text{Var}_{\xi^*}[f(x)] \quad (\text{A.7})$$

$$= \mathbb{E}_{x \sim p} [(\mathbb{E}_{y \sim p}[f(y)] - f(x))^2] + \frac{1}{k} \text{Var}_{\xi^*}[f(x)] \quad (\text{A.8})$$

$$= \text{Var}_p(f(x)) + \frac{1}{k} \text{Var}_{\xi^*}[f(x)] \quad (\text{A.9})$$

As we constructed the set $\mathcal{P}^*$ so that all members have equal (minimum) variance, we obtain the same $g(p^*)$ for all $p^* \in \mathcal{P}^*$.

We now set up a contradiction by assuming p' exists. By this assumption

$$g(p') < g(p^*) \quad (\text{A.10})$$

$$\text{Var}_p(f(x)) + \frac{1}{k} \text{Var}_{p'}[f(x)] < \text{Var}_p(f(x)) + \frac{1}{k} \text{Var}_{p^*}[f(x)] \quad (\text{A.11})$$

$$\text{Var}_{p'}[f(x)] < \text{Var}_{p^*}[f(x)] \quad (\text{A.12})$$

For the function p' to exist so that $g(p') < g(p^*)$, we would therefore require $\text{Var}_{p'}[f(x)] < \text{Var}_{p^*}[f(x)]$, which is impossible by the definition of $\mathcal{P}^*$. This is a contradiction with the requirements for p' in the lemma, which concludes our proof. $\square$

In the following lemma, we show that under some conditions a distribution q exists which has $g(q) < g(p^*)$ and for which $\mathbb{E}_p[f(x)] \neq \mathbb{E}_q[f(x)]$. We can rewrite g as

$$g(q) = \mathbb{E}_p \left[(\mathbb{E}_q[f(y)] - f(x))^2 \right] + \frac{1}{k} \text{Var}_q[f(x)] \quad (\text{A.13})$$

$$= \mathbb{E}_p [f(x)^2] - 2\mathbb{E}_p[f(x)] \mathbb{E}_q[f(x)] + \mathbb{E}_q[f(x)]^2 + \frac{1}{k} \text{Var}_q[f(x)] \quad (\text{A.14})$$

$$\leq \mathbb{E}_p [f(x)^2] - 2\mathbb{E}_p[f(x)] \mathbb{E}_q[f(x)] + \mathbb{E}_q[f(x)]^2 + \frac{1}{k} \text{Var}_q[f(x)] \quad (\text{A.15})$$

$$= \text{Var}_p[f(x)] + \frac{k+1}{k} \text{Var}_q[f(x)] + (\mathbb{E}_p[f(x)] - \mathbb{E}_q[f(x)])^2 \quad (\text{A.16})$$

Equation A.15 follows from Jensen's inequality. Intuitively, the function g depends on the squared deviation of the expectation and the variance of both p and q . If the variance of q can be reduced more than the squared deviation of the means, then $\mathcal{P}^*$ will not contain the minimum of g .

Note that the conditions on q are sufficient but not necessary, as we use Jensen's inequality to obtain our bound. In addition, to simplify the proof, we assume that q is a distribution with zero variance. In practice, any distribution with $\frac{k+1}{k} \text{Var}_q[f(x)] + (\mathbb{E}_p[f(x)] - \mathbb{E}_q[f(x)])^2 < \text{Var}_{p^*}[f(x)]$ and $\mathbb{E}_q[f(x)] \neq \mathbb{E}_p[f(x)]$ will suffice, but this requirement is somewhat less intuitive to grasp. As our definition of calibration does not require us to exhaustively characterize all cases for p , p^* , and q , we have chosen this set of conditions which simplifies the proof. As our Garnet experiments show, many randomly generated transition distributions admit distributions where the minimizer of g does not match p in expectation.

Lemma A.2.1.2. *If there exists a distribution q with $(\mathbb{E}_q[f(x)] - \mathbb{E}_p[f(x)])^2 < \frac{1}{k} \text{Var}_{p^*}[f(x)]$, and $\text{Var}_q[f(x)] = 0$, then $g(q) < g(p^*)$ for all p^* and, by the assumptions on p and f , $\mathbb{E}_{x \sim q}[f(x)] \neq \mathbb{E}_{x \sim p}[f(x)]$.*

Proof. Choose any $p^* \in \mathcal{P}^*$. As $\text{Var}_q[f(x)] = 0$, $g(q) = \mathbb{E}_{x \sim p}[(\mathbb{E}_{y \sim q}[f(y)] - f(x))]$. We can now decompose $g(q)$ as

$$g(q) = \mathbb{E}_p \left[(\mathbb{E}_q[f(y)] - f(x))^2 \right] = \mathbb{E}_p [f(x)^2] - 2\mathbb{E}_p[f(x)] \mathbb{E}_q[f(x)] + \mathbb{E}_q[f(x)]^2 \quad (\text{A.17})$$

$$= \text{Var}_p[f(x)] + (\mathbb{E}_p[f(x)] - \mathbb{E}_q[f(x)])^2 \quad (\text{A.18})$$

$$< \text{Var}_p[f(x)] + \frac{1}{k} \text{Var}_{p^*}[f(x)] = g(p^*) \quad (\text{A.19})$$

Equation A.19 follows from the assumption on q .

By [Assumption 6.4.2.1](#) there does not exist a $x \in \mathcal{X}$ so that $f(x) = \mathbb{E}_p[f(x)]$. However, as $\text{Var}_q[f(x)] = 0$, $\mathbb{E}_q[f(x)] = f(x_i)$ for some $x_i \in \mathcal{X}$. Therefore, $\mathbb{E}_q[f(x)] \neq \mathbb{E}_p[f(x)]$, which concludes the proof. $\square$

As a consequence, we obtain the following, final lemma.

Lemma A.2.1.3. *Let $\mathcal{P}^{\mathbb{E}}$ be the set of all distributions ξ so that $\mathbb{E}_p[f(x)] = \mathbb{E}_\xi[f(x)]$. Assume q satisfying [Lemma A.2.1.2](#) exists.*

Then $\arg \min_\xi g(\xi) \not\subseteq \mathcal{P}^{\mathbb{E}}$.

Proof. The lemma is a direct consequence of [Lemma A.2.1.1](#) and [Lemma A.2.1.2](#). By [Lemma A.2.1.1](#), for all $p' \in \mathcal{P}^{\mathbb{E}}$ $g(p') \geq g(p^*)$. As a consequence of [Lemma A.2.1.2](#), there exists at least one q with $g(q) < g(p^*)$ and that q has $\mathbb{E}_q[f(x)] \neq \mathbb{E}_p[f(x)]$. Then the statement follows directly as $q \notin \mathcal{P}^{\mathbb{E}}$. $\square$

The following lemma shows that a function that minimizes a quadratic and a variance term cannot be the minimum function of the quadratic. This is used to show that the minimum of the MuZero value function learning term is not the same as applying the model-based Bellman operator.

Lemma A.2.1.4. *Let $g : \mathcal{X} \rightarrow \mathbb{R}$ be a function that is not constant and let μ be a probability distribution over $\mathcal{X}$. Let $\mathcal{L}(f) = \mathbb{E}_{x \sim \mu} [(f(x) - g(x))^2] + \mathbb{E}_{x \sim \mu} [f(x)g(x)] - \mathbb{E}_{x \sim \mu} [f(x)] \mathbb{E}_\mu [g(x)]$. There exists a function space $\mathcal{F}$ with $g \in \mathcal{F}$ so that $g \notin \arg \min_{f \in \mathcal{F}} \mathcal{L}(f)$.*

Proof. The proof follows by showing that there is a descent direction from g that improves upon $\mathcal{L}$. For this, we construct the auxiliary function $\hat{g}(x) = g(x) - \varepsilon g(x)$. Evaluating $\mathcal{L}(\hat{g})$ yields

$$\varepsilon^2 \mathbb{E}_\mu [g(x)^2] + \mathbb{E}_\mu [(g(x) - \varepsilon g(x)) g(x)] \quad (\text{A.20})$$

$$- \mathbb{E}_\mu [(g(x) - \varepsilon g(x))] \mathbb{E}_\mu [g(x)] \quad (\text{A.21})$$

$$= \varepsilon^2 \mathbb{E}_\mu [g(x)^2] + (1 - \varepsilon) \mathbb{E}_\mu [g(x)^2] - (1 - \varepsilon) \mathbb{E}_\mu [g(x)]^2. \quad (\text{A.22})$$

Taking the derivative of this function wrt to ε yields

$$\frac{d}{d\varepsilon} \varepsilon^2 \mathbb{E}_\mu [g(x)^2] + (1 - \varepsilon) \mathbb{E}_\mu [g(x)^2] - (1 - \varepsilon) \mathbb{E}_\mu [g(x)]^2 \quad (\text{A.23})$$

$$= 2\varepsilon \mathbb{E}_\mu [g(x)^2] - \mathbb{E}_\mu [g(x)^2] + \mathbb{E}_\mu [g(x)]^2. \quad (\text{A.24})$$

Setting ε to 0, we obtain

$$\mathbb{E}_\mu [g(x)]^2 - \mathbb{E}_\mu [g(x)^2] = \text{Var}_\mu [g(x)] \quad (\text{A.25})$$

By the Cauchy-Schwartz inequality, the variance is only 0 for a $g(x)$ constant almost everywhere. However, this violates the assumption. Therefore there exists an $\varepsilon > 0$ so that $\mathcal{L}(\hat{g}) \leq \mathcal{L}(g)$. We now can construct the function space $\mathcal{F}$ so that it includes at least g and $g + \varepsilon g$.

□

A.2.2 Propositions from Bertsekas & Shreve

For convenience, we quote some results from Bertsekas and Shreve (1978). These are used in the proof of Lemma 6.6.1.1. While some of the definitions are rather technical, it is mostly sufficient to see a *stochastic kernel* as the continuous generalization of the transition matrix in finite MDPs. The projection used in Proposition A.2.2.2 is simply a restriction of a set of tuples (x, y) on the x values. Other topological statements are standard and can be found in textbooks such as Munkres (2018).

Proposition A.2.2.1 (Proposition 7.30). *Let $\mathcal{X}$ and $\mathcal{Y}$ be separable metrizable spaces and let $q(\text{d}y|x)$ be a continuous stochastic kernel on $\mathcal{Y}$ given $\mathcal{X}$. If $f \in \mathcal{C}(\mathcal{X} \times \mathcal{Y})$, the function $\lambda : \mathcal{X} \rightarrow \mathbb{R}$ defined by*

$$\lambda(x) = \int f(x, y)q(\text{d}y|x)$$

is continuous.

Proposition A.2.2.2 (Proposition 7.33). *Let $\mathcal{X}$ be a metrizable space, $\mathcal{Y}$ a compact metrizable space, $\mathcal{D}$ a closed subset of $\mathcal{X} \times \mathcal{Y}$, $\mathcal{D}_x = \{y|(x, y) \in \mathcal{D}\}$, and let $f : \mathcal{D} \rightarrow \mathbb{R}^*$ be lower semi-continuous. Let $f^* : \text{proj}_{\mathcal{X}}(\mathcal{D}) \rightarrow \mathbb{R}^*$ be given by*

$$f^*(x) = \min_{y \in \mathcal{D}_x} f(x, y).$$

Then $\text{proj}_{\mathcal{X}}(\mathcal{D})$ is closed in $\mathcal{X}$, f^ is lower semi-continuous, and there exists a Borel-measurable function $\varphi : \text{proj}_{\mathcal{X}}(\mathcal{D}) \rightarrow \mathcal{Y}$ such that $\text{range}(\varphi) \subset \mathcal{D}$ and*

$$f[x, \varphi(x)] = f^*(x), \quad \forall x \in \text{proj}_{\mathcal{X}}(\mathcal{D}).$$

In our proof, we construct f^* as the minimum of an IterVAML style loss and equate φ with the function we call f in our proof. The change in notation is chosen to reflect the modern notation in MBRL – in the textbook, older notation is used.

Appendix B

Appendix – Instability of Value Function Learning

B.1 Implementation details and hyperparameters

We employ two commonly used implementations, one for fast iterations on priming experiments (https://github.com/denisyarats/pytorch_sac) and one for scaling up our experiments to high update ratios (https://github.com/proceduralia/high_replay_ratio_continuous_control). All experiments in the main sections use default hyperparameters of the high update ratio codebase unless otherwise specified with minor exceptions.

Table B.1: Shared hyperparameters between priming and high-UTD implementations
 Table B.2: Differing hyperparameters between priming and high-UTD implementations

Optimizer	Adam
Adam β_1	0.9
Adam β_2	0.999
Adam ε	1×10^{-8}
Actor Learning Rate	4×10^{-3}
Critic Learning Rate	4×10^{-3}
Temp. Learning Rate	3×10^{-4}
Batch Size	256
γ	0.99
τ	0.005
# critics	2
# critic layers	2
# actor layers	2
critic hidden dim	256
actor hidden dim	256

	Priming	High UTD
Initial temperature	0.1	1.0
Target entropy	-action_dim	-action_dim / 2
actor log std bounds	[-5, 2]	[-10, 2]

B.2 Additional experimental results

B.2.1 Returns on all environments

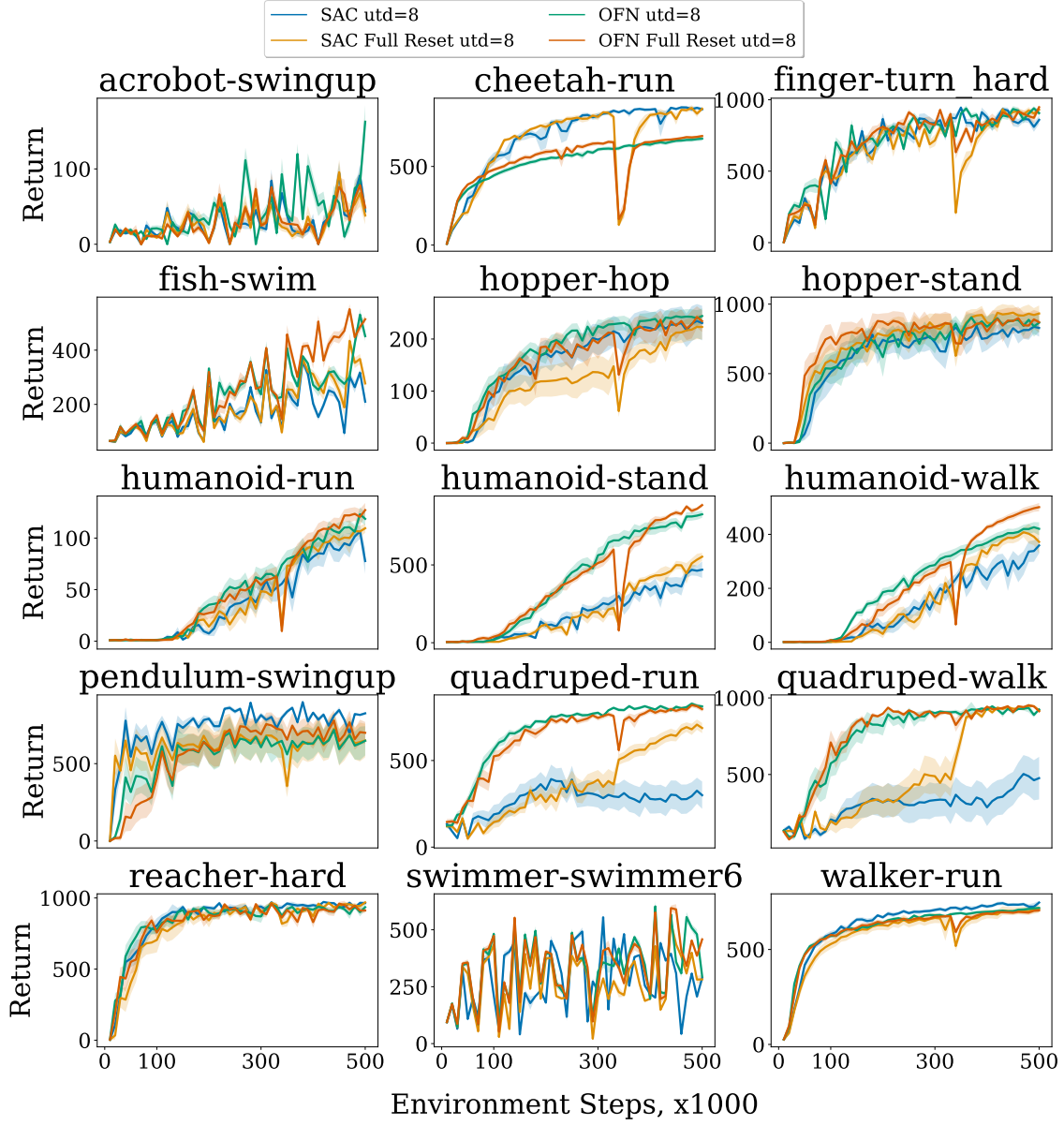


Figure B.1: UTD8 Returns on Full DMC15-500K.

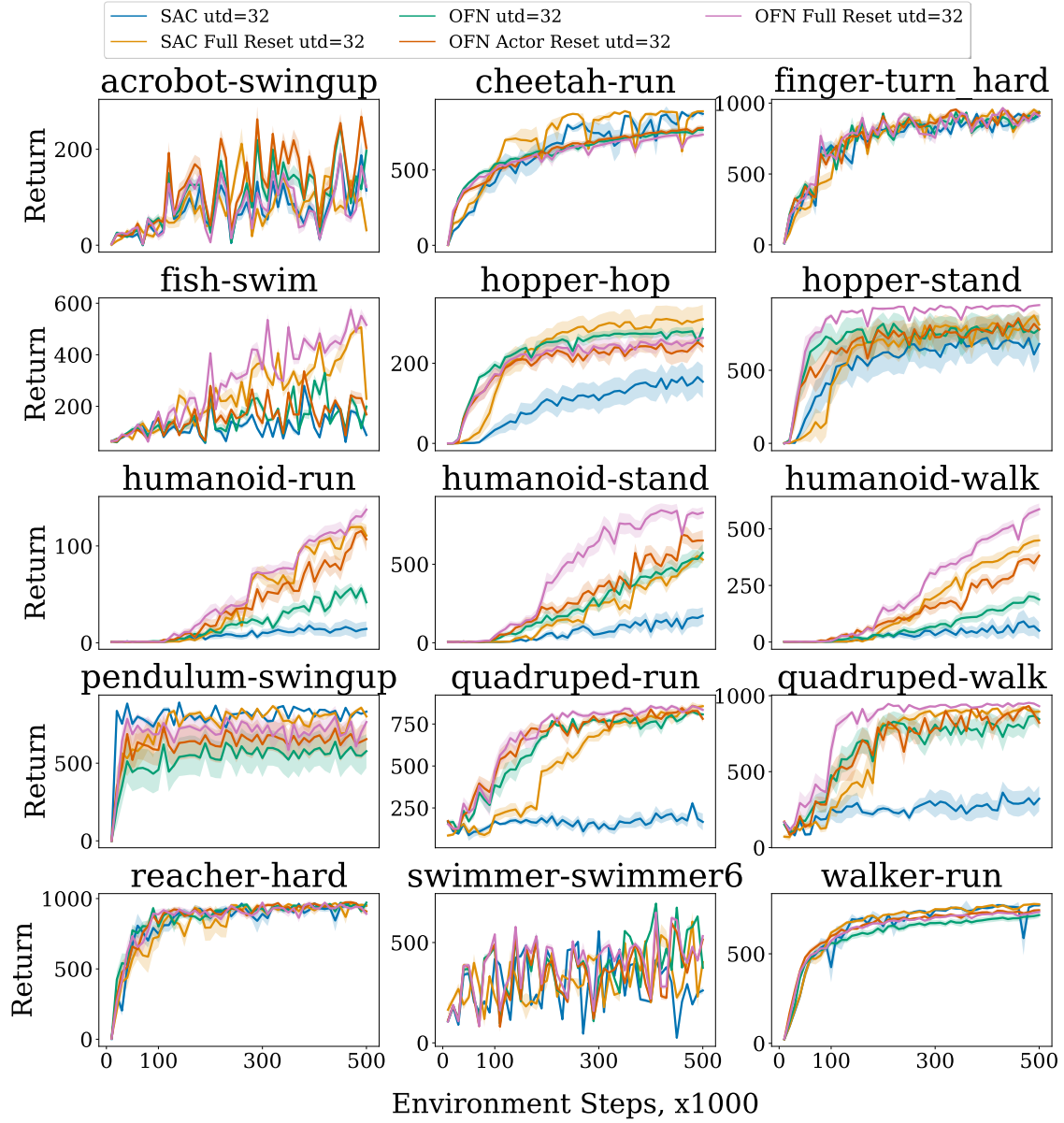


Figure B.2: UTD32 Returns on Full DMC15-500K.

B.2.2 Q-values on all environments

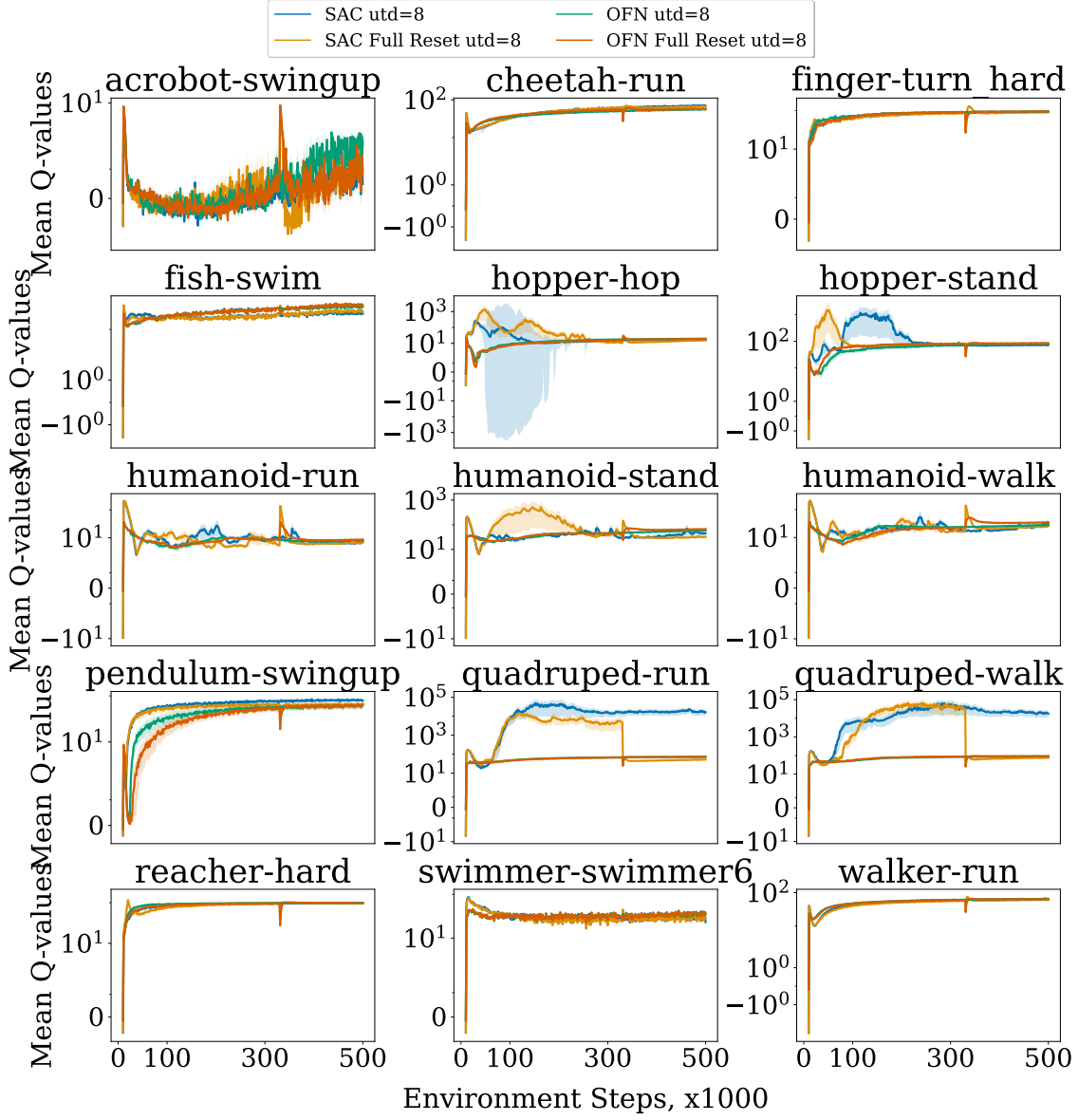


Figure B.3: UTD8 Q-values on Full DMC15-500K. Resetting often works when Q-values diverge. ONF mitigates divergence.

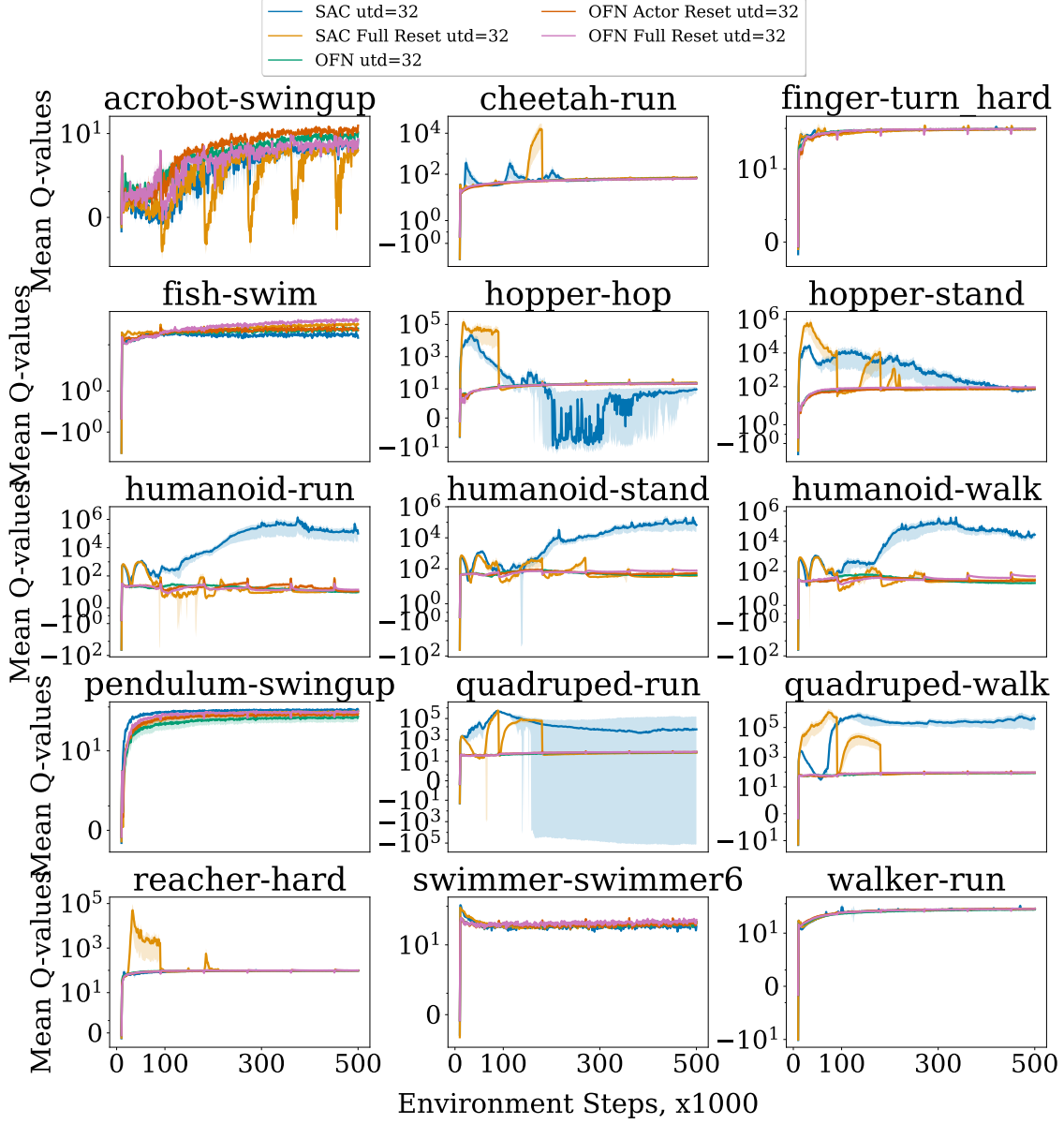


Figure B.4: UTD32 Q-values on Full DMC15-500K. Resetting often works when Q-values diverge. ONF mitigates divergence.

B.3 Unit norm gradient derivation

Here, we take a look at the gradient of the unit norm projection.

Let $i \in 1, \dots, n$, for all $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n \setminus \{0\}$. Suppose $f(\mathbf{x}) = \frac{\mathbf{x}}{\|\mathbf{x}\|}$.

Then,

$$\partial_i f(\mathbf{x}) = \frac{\|\mathbf{x}\|e_i - \frac{x_i}{\|\mathbf{x}\|}\mathbf{x}}{\|\mathbf{x}\|^2} \quad (\text{B.1})$$

$$= \frac{1}{\|\mathbf{x}\|}e_i - \frac{x_i}{\|\mathbf{x}\|^3}\mathbf{x}. \quad (\text{B.2})$$

Note that the second term can grow quite large if the norm of $\mathbf{x}$ is relatively small. Despite this, we are able to remedy the exploding gradients using unit norm projection, likely because gradients are small when the norm is small.

B.4 Open Problems and Limitations

Feature divergence without regularization is an important problem that contributes substantially to the issues facing high-UTD learning. However, as our experiments show, there are many additional open problems that introducing normalization does not address.

Understanding actor issues The resetting experiments in Figure 3.8 highlight that a part of the performance impact of high UTD comes from the actor optimization, not the critic optimization, as resetting the actor can boost performance without changing the critic. Our work does not address this issue, and to the best of our knowledge there are no specific attempts to investigate the actor optimization process in deep actor-critic reinforcement learning.

RL Optimizer The update dynamics introduced by modern optimizers can exacerbate existing overestimation problems. Dabney (2014) derives adaptive step-sizes for reinforcement learning from a theoretical perspective, but the resulting optimization rules have not been adapted to Deep Reinforcement Learning to the best of our knowledge. A recent study by Asadi et al. (2023) shows that resetting the optimizer can have some benefit in the DQN setting, where it can be tied to the hard updates of the target Q network. In addition, Lyle et al. (2023) show that optimizers like Adam can lead to reduced plasticity of neural networks. However, our experiments also highlight that without the accelerated optimization of modern optimizers, convergence of the Q value can be prohibitively slow, highlighting the urgent need for stable and fast optimization in RL.

Conservative Learning for Online RL Most current actor-critic methods use some form of pessimistic value estimate to combat the overestimation bias inherent in off-policy Q learning. i.e. via the use of a twinned Q network (Fujimoto et al., 2018b). However, this can lead to pessimistic under-exploration (Lan et al., 2020). To address this, Moskovitz et al. (2021) propose to tune the relative impact of pessimistic and optimistic exploration

for the environments, while Lee et al. (2021) show that by combining independent critic estimates from ensembles, a UBC like exploration bound can be computed. These changes could be combined with the mitigation strategies for the feature layer divergence in future work to mitigate the harmful effects of under-exploration further.

As our work shows, some of the previous problems with overestimation might not emerge from the bias introduced by off-policy actions, but from the learning dynamics of neural network updates. This suggests that more work on the exact causes of overestimation might allow us to move beyond the overly pessimistic twinned network minimization trick without needing costly solutions like ensemble methods.

Tau The rate of the target network updates is an important hyperparameter in online RL, either through periodic hard copies (Mnih et al., 2013) or the use of a Polyak averaging scheme (Lillicrap et al., 2016). Updating the network too fast can exacerbate the impact of value divergence, while updating too slowly can delay learning. Preliminary experiments show a relationship between value divergence and target update speed that requires further investigation.

There have also been attempts to accelerate optimization not via the neural network optimization, but through adapting the updates of the target networks (Vieillard et al., 2020; Farahmand and Ghavamzadeh, 2021). This is an orthogonal direction to the one presented here, and the interplay between target network updates and neural network optimization steps are an important topic for future work.

Reward Shaping Impact In several environments, we observe almost no detrimental effects due to high update ratios, while in others the Q-values diverge even without moving beyond one update per sample collected. A closer inspection suggests that environments in which the initial reward is small and uninformative are much more prone to lead to catastrophic divergence, suggesting a close connection between reward shaping and divergence. While sparse reward problems have received much attention in the context of exploration, our findings suggests that they also present a challenge for efficient optimization. Beyond this phenomenon, the interactions between optimization and explorations have been hypothesized to be a strong contributing factor to the good performance of some algorithms (Schaul et al., 2022), but the role diverging Q-values play in this phenomenon is to the best of our knowledge mostly unexplored.

Appendix C

Appendix – Understanding Auxiliary Tasks

C.1 Implementation details for the experiments

For the Minatar experiments, we use a simple Double DQN architecture (Van Hasselt et al., 2016b). We find that our implementation performs roughly on par with those reported by Young and Tian (2019), however we changed the network architecture slightly to allow a clear “encoder” and “prediction head” split.

We implement the latent self-prediction loss using a periodically updated copy of the encoder network. This hard update of the encoder target is synchronized with the Q target network update.

Our networks are parameterized as presented in Table C.2

Relevant hyper-parameters are shown in Table C.1.

The random noise matrix is sampled from a Bernoulli distribution with $p(1) = 0.1$. The distortion matrix was created using a random matrix of size $10 \cdot 10 \cdot \text{channels} \times 10 \cdot 10 \cdot \text{channels}$ with entries independently sampled from a Bernoulli distribution with $p(1) = 0.2$. This was chosen to have a similar sparsity in the distraction as in the main observation channels. Additionally, we verified that the matrix was invertible and $\|\mathcal{O}\|_1 < 255$ to ensure that the replay buffer implementation would not overflow.

The DMC results were obtained using the TD3 algorithm (Fujimoto et al., 2018a) with constant Gaussian noise with standard deviation of 0.01 added for exploration following Yarats et al. (2021). In addition to the Q function network, TD3 also requires an actor

Table C.1: Hyper-parameters the RL experiments

Parameter	MinAtar	DMC
Initial steps (Random policy)	5000	5000
Env steps per update step	4	1
Batch size	512	512
Exploration ϵ	0.05	0.01
RL learning rate	0.0003	0.0003
Model/decoder learning rate	0.0003	0.0003
Encoder learning rate	0.0001	0.0001
Target network update interval	1000	n/a
Soft update τ	n/a	0.995
Discount factor γ	0.99	0.99
Model forward prediction steps	4	4

Table C.2: Network architecture for the MinAtar experiments.

Encoder Φ	ConvLayer	channels=16, kernel=(3, 3), padding=0, stride=(1, 1)
	ELU activation	–
	Dense Layer	out_size=100
	ELU activation	–
Latent Model F	Dense Layer	out_size=256
	ELU activation	–
	Dense Layer	out_size=100
Decoder Ψ	Dense Layer	out_size=10 · 10 · 16
	ELU activation	–
	ConvTranspose Layer	kernel=(3, 3), padding=1, stride=(1, 1)
Q head $\hat{V}$	Dense Layer	out_size=256
	ELU activation	–
	Dense Layer	out_size=action_space

network. We do not propagate any gradients from the actor network into the encoder, following standard practice in actor-critic learning.

Networks for the DMC implementation are shown in Table C.3.

In the DMC experiments, we used isotropic Gaussian noise for the distracting noise, and a copy of the *humanoid* environment as the distraction.

Full code is available at https://github.com/adaptive-agents-lab/understading_auxilliary_tasks.

Encoder Φ	Dense Layer	out_size=256
	ELU activation	–
	Dense Layer	out_size=256
	ELU activation	–
Latent Model F	Dense Layer	out_size=256
	ELU activation	–
	Dense Layer	out_size=100
Decoder Ψ	Dense Layer	out_size=10 · 10 · 16
	ELU activation	–
	Dense Layer	out_size=obs_dim
Q head $\hat{V}$	Dense Layer	out_size=256
	ELU activation	–
	Dense Layer	out_size=1
Actor head	Dense Layer	out_size=256
	ELU activation	–
	Dense Layer	out_size=action_dim

Table C.3: Network architecture for the DMC experiments.

C.2 DMC results

The DMC environments have a very different observation structure and transition dynamics compared to those of the MinAtar games. The observation spaces are dense and often contain topological discontinuities such as those outlined in [Section 2.2](#).

All experiments are repeated across 10 seeds.

We find slightly different results in these environments compared to MinAtar, especially about the efficacy of the latent self-prediction and observation reconstructions in the stand-alone setting ([Figure C.2](#) and [Figure C.3](#)). Overall, latent self-prediction performs much more strongly in these environments compared to the MinAtar experiments, especially when using it as a stand-alone loss. Curiously, in the only environment where observation prediction shines (quadruped-walk), stand-alone latent self-prediction and reward prediction seems to outperform all other test settings. This highlights the second important difference between the MinAtar games and the DMC suite: dense rewards. We conjecture that most of the differences between observation prediction and latent self-prediction in the DMC suite comes from dense rewards and different topological continuity, but a precise investigation is needed in future work.

Finally, our assumptions about the impact of noise distortions on the efficacy of different loss functions seems to be much more clearly apparent in DMC than in MinAtar. This suggests that the differing primary observation spaces also change how the learning process interacts with the noise, i.e. because the spectral properties of the underlying environments might differ. Especially in the challenging distraction setting of structured noise, non trivial

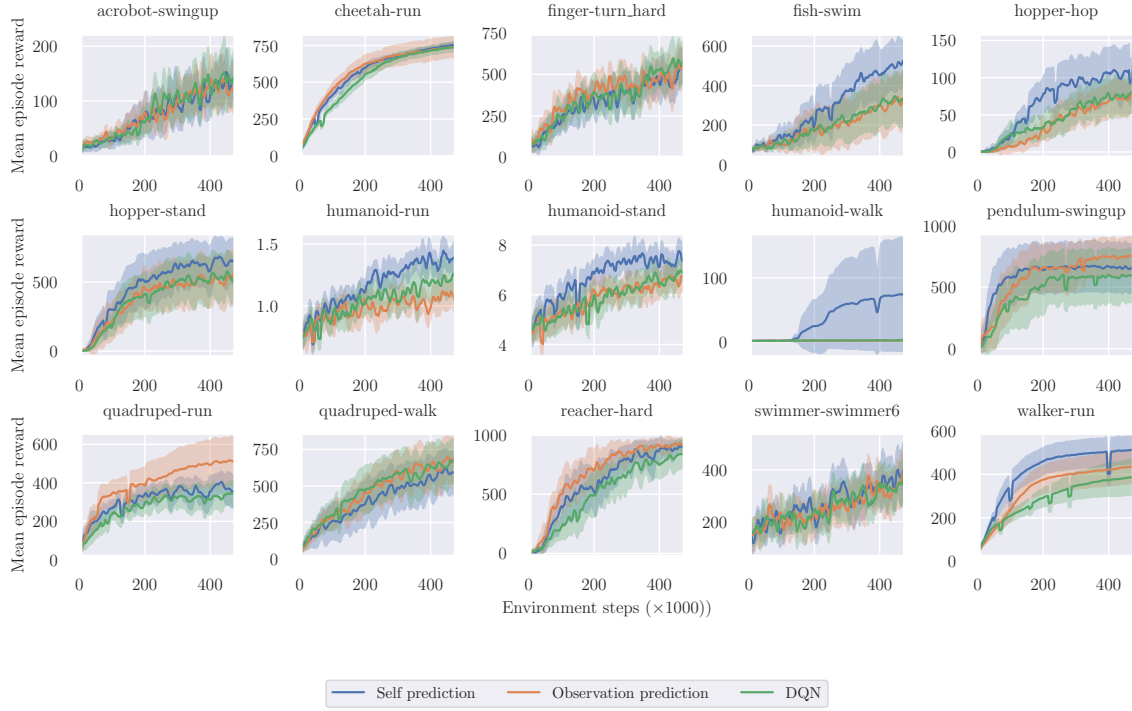


Figure C.1: DMC: Auxiliary task scenario

policies (as measured by a strong performance improvement over a random baseline) can only be observed in 7 out of the 15 environments. Of these 7, latent self prediction seems to outperform the other baselines in 3 cases, only falling behind in 1. However, we observed that the pendulum environment has a strong bimodal distribution with some runs completely failing to perform for each experimental scenario, so the number of seeds might be insufficient to disambiguate performance here, as evidenced by the large confidence interval at the 95% level.

Overall, 10 seeds, even though widely used in the community, might not be sufficient to fully represent the performance in the DMC suite, suggesting that more work is needed on the inherent variations of these environments. We do not present aggregate performance over the environments, as the different reward scales makes a comparison prone to be driven by outliers such as the humanoid environments, in which several algorithms fail to learn at all at 10 seeds.

C.3 Limitations

While Chapter 4 aims to minimize the theory-practice gap with careful experimentation, we nonetheless need to make several assumptions that are both limitations and also potential for additional analysis in future work. As our aim is to make theory useful and accessible for practitioners, we aim to be very open and clear about our limitations here.

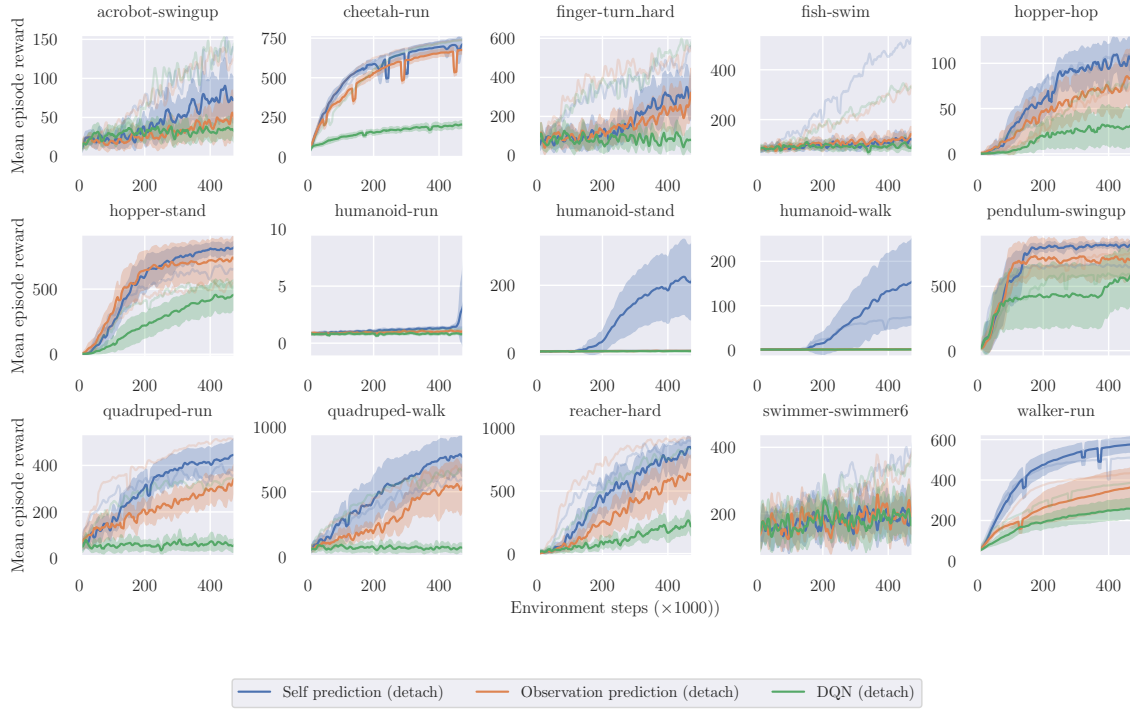


Figure C.2: DMC: Stand alone scenario

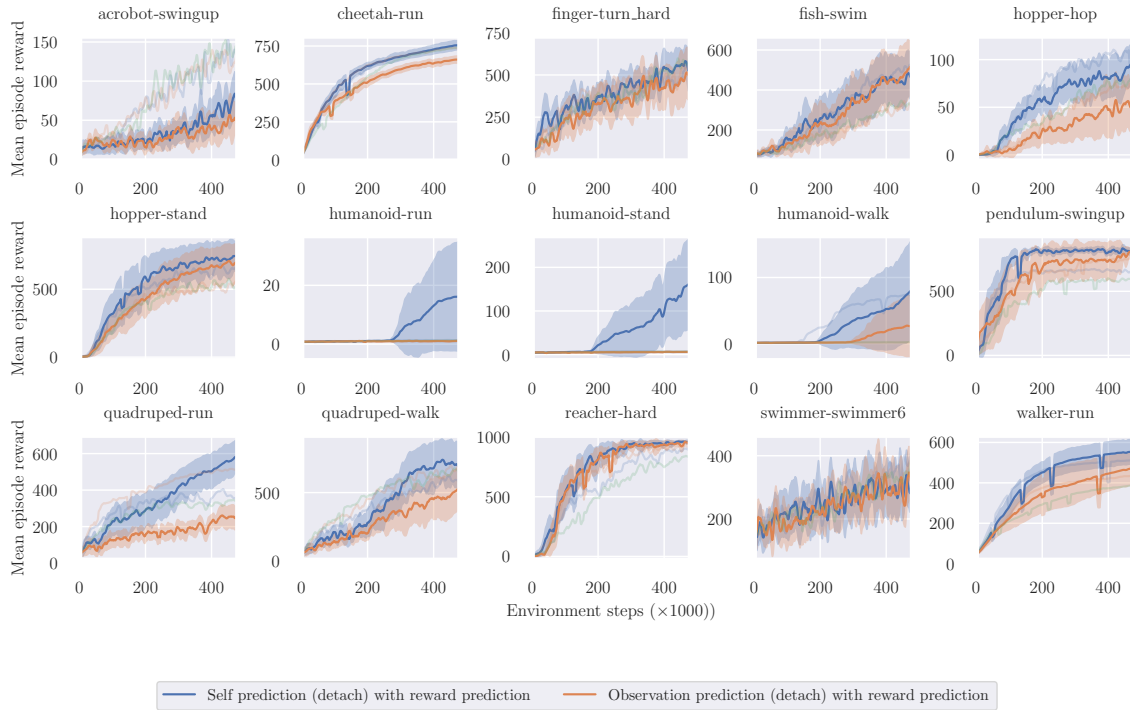


Figure C.3: DMC: Auxiliary loss + reward prediction, not TD

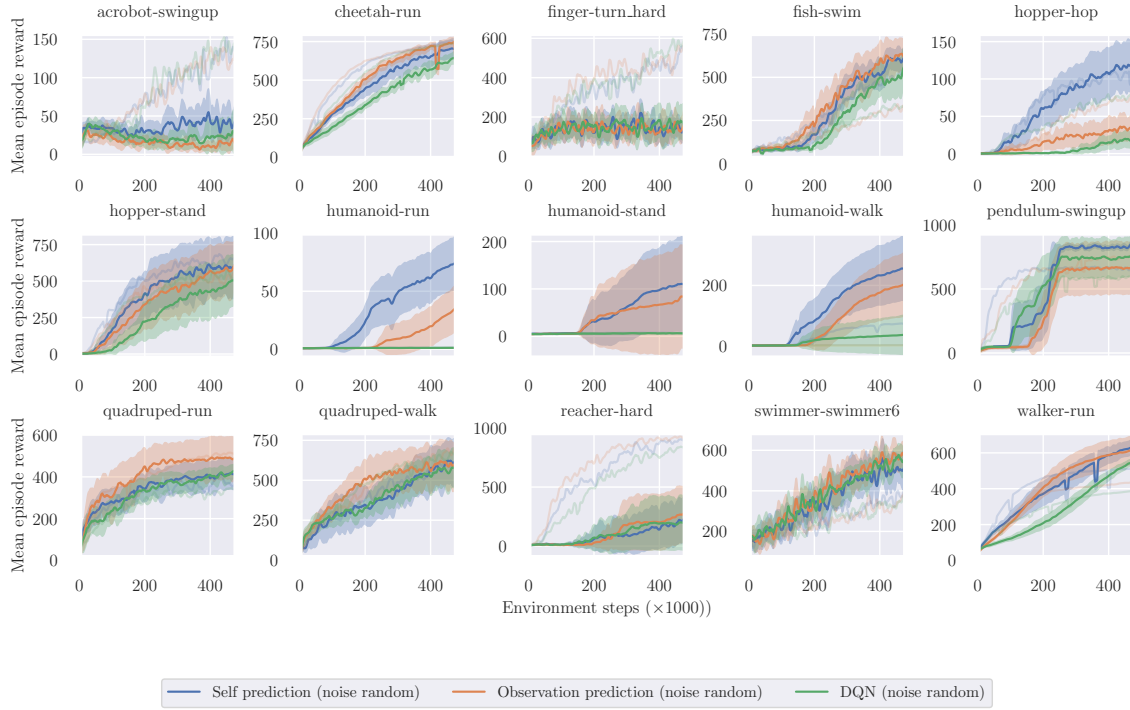


Figure C.4: DMC: Random noise distraction

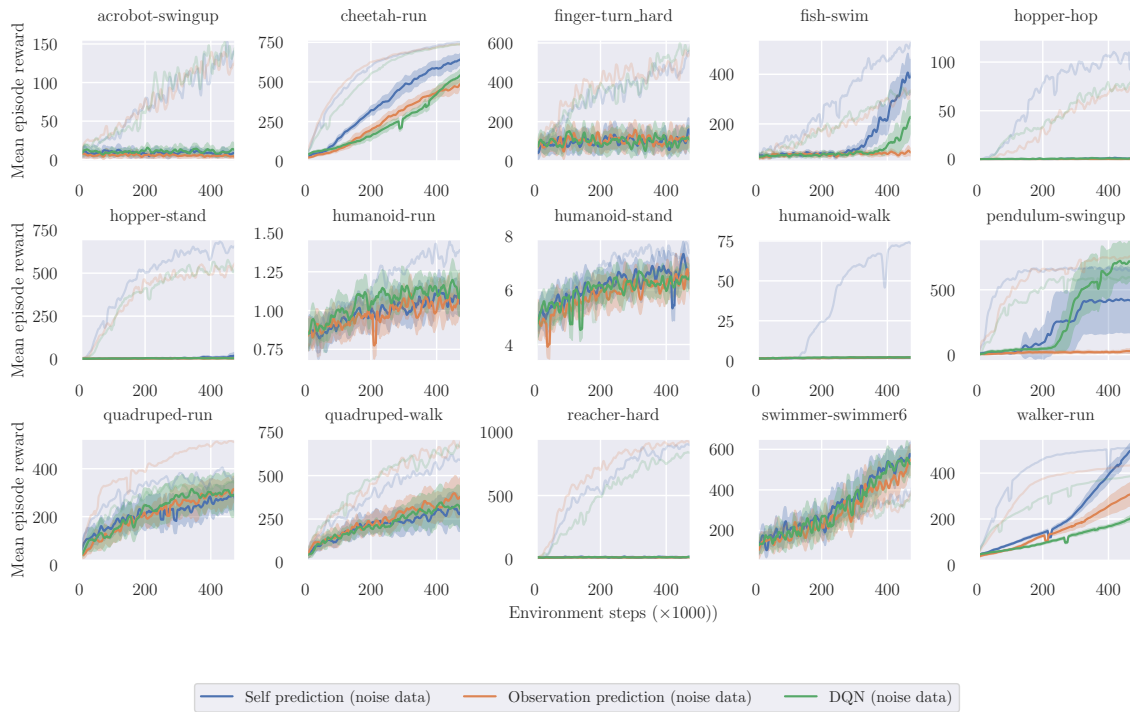


Figure C.5: DMC: Structured distraction

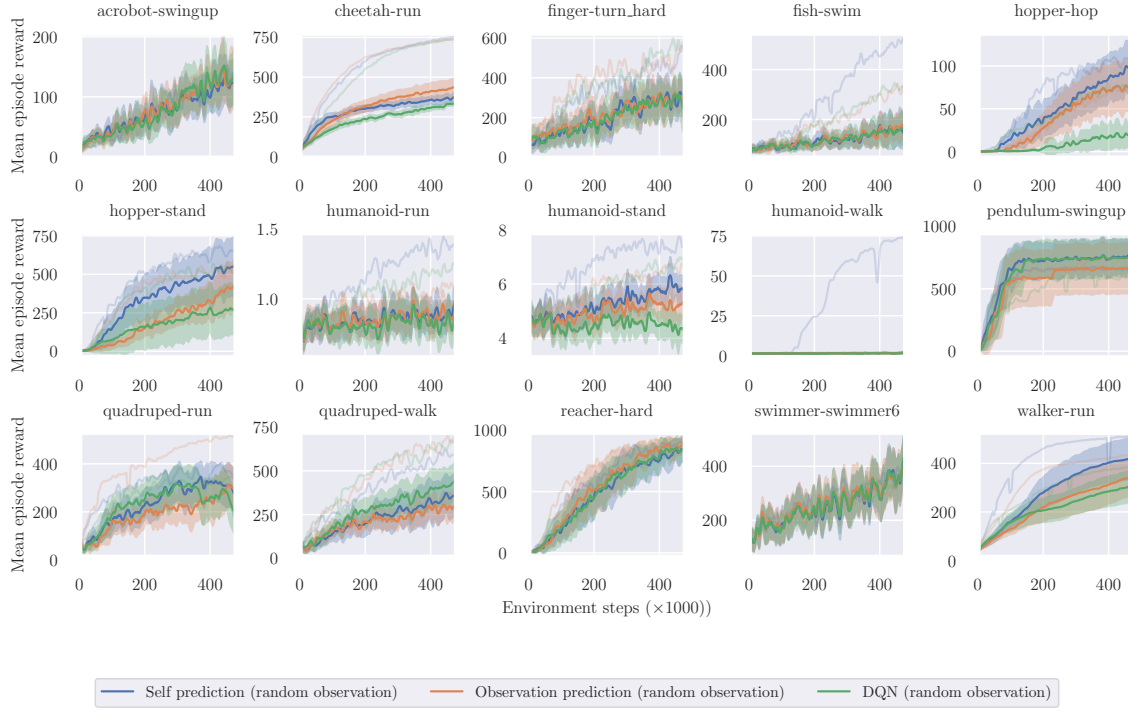


Figure C.6: DMC: Observation space distortion

Limitations of the analytical framework: From previous work (Tang et al., 2023; Le Lan et al., 2023) we inherit the limitation of studying deterministic models in potentially stochastic environments. This is a necessary limitation, as considering the stochastic equivalents of e.g. the observation prediction loss would render the model and their gradients non-linear due to the introduction of a softmax or similar constraint. As other works carry the same limitation, we believe that this does not render our work inapplicable, but it does suggest the need for more powerful mathematical tools in future work.

We also conduct all of our theoretical work in the on-policy policy evaluation regime, while our empirical study includes both off-policy policy estimation and policy improvement. Again, this is a limitation inherited from all related work. As our theoretical predictions are still validated, we consider this an acceptable limitation, but studying the impact of off-policy samples and shifting policies is an important step for future work.

Limitations of the formalism: Our notion of observation distortion requires unnaturally large observation spaces. Again, this stems from our adherence to a linear framework. While the observation space of e.g. the MinAtar games is relatively large, depending on the game a 300-1000 dimensional vector, this is still substantially smaller than the total number of states. Studying these kinds of nonlinear image transformations in more detail would require deviating farther from the previous literature.

We aim to (re-)introduce the notion of distraction into the learning dynamics literature, and so we use a relatively simplistic notion of distraction. Going beyond the independence assumption in the distraction model (due to the Kronecker formulation) and analyzing more complex forms of distractions e.g. processes in which the reward-relevant process causally influences the distracting process but not vice versa is an exciting direction for future work.

Appendix D

Appendix – Value-gradient Aware Model Learning

D.1 Full VAML for SAC

The formulation of VAML which we derive for SAC is a direct extension of the VAML loss to the SAC soft Bellman backup. Specifically, given some distribution $\mathcal{D}$ over the state-action space, state-action value function Q and policy π , we can define a SAC-aware loss.

$$\mathcal{L}_{Q,\pi}(\hat{p}, p, \mathcal{D}) = \int \left| \int (\hat{p}(x'|s, a) - p(x'|s, a)) \mathbb{E}_{a' \sim \pi(\cdot|x')} (Q(x', a') - \log \pi(a'|x')) dx' \right|^2 d\mathcal{D}(x, a) \quad (\text{D.1})$$

$$= \int \left| \int (\hat{p}(x'|x, a) - p(x'|x, a)) V^\pi(x', a') dx' - \int (\hat{p}(x'|x, a) - p(x'|x, a)) \mathbb{E}_{a' \sim \pi(\cdot|x')} [\log \pi(a'|x')] dx' \right|^2 d\mathcal{D}(x, a) \quad (\text{D.2})$$

as well as its sample-based version:

$$\hat{\mathcal{L}}_{Q,\pi}(\hat{p}, p, \mathcal{D}) = \sum_i \left| V^\pi(x'_i) - \int \hat{p}(x'|x_i, a_i) V^\pi(x') dx' - \left(\int \pi(a'|x'_i) \log \pi(a'|x'_i) da' - \int \int \hat{p}(x'|x_i, a_i) \pi(a'|x') \log \pi(a'|x') da' dx' \right) \right|^2 \quad (\text{D.3})$$

We find that in most of our experiments, the entropy terms did not significantly contribute to the loss. Therefore we dropped it in our experiments and directly derived VAGraM for

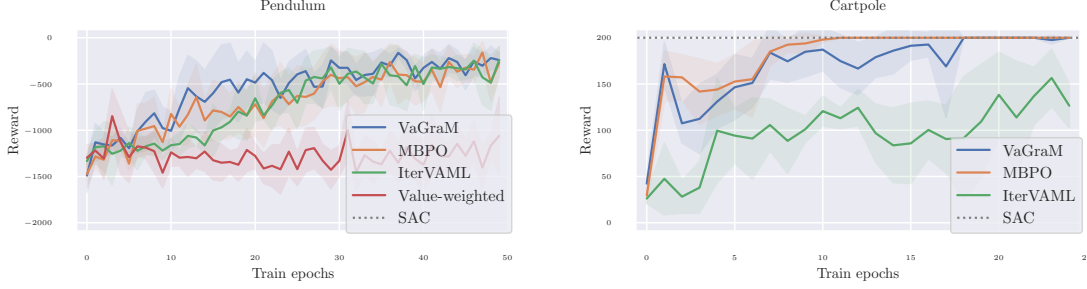


Figure D.1: Comparison of VaGraM, MLE MBPO baseline), IterVAML Farahmand (2018) and a value-weighting ablation on two simple continuous control tasks. The shaded area represents standard error estimated over 8 runs. While VaGraM, IterVAML and MBPO are able to achieve satisfactory performance on the Pendulum Swingup task, IterVAML fails to stabilize the more difficult Cartpole balancing task.

the value function prediction error only. This makes our loss applicable in all situations where the model is used to estimate the value function of a policy.

Finally, we do not account for the dependency of the policy function update on the model. Since SAC aims to minimize the KL divergence between the action distribution and the Gibbs distribution defined by the value function directly, the only way the model influences this update is via the induced state space distribution. We do not address this matching explicitly in this work, but a hybrid policy-aware and value-aware model is an enticing direction for future research.

D.2 Additional experiments

D.2.1 Ablations

To test the performance of VaGraM and MBPO against alternative models, we used the classic Pendulum swingup and Cartpole benchmarks. As points of comparison, we used IterVAML (Farahmand, 2018) and a simple value-weighted regression similar to Nair et al. (2020), where the MSE error is multiplied by the inverse of the value function of the sample. The results are visualized in Figure D.1. Hyperparameters follow the Cartpole task baseline in Pineda et al. (2021).

Since IterVAML suffers from strong destabilization on the Pendulum model learning problem, as discussed in Section 5.3, we added an MSE loss to the original formulation $\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{IterVAML}} + \lambda \mathcal{L}_{\text{MSE}}$ with a tradeoff factor of $\lambda = 0.01$. We see that this is sufficient to stabilize the performance in the Pendulum swingup task and prevent catastrophic divergence, but the model still performs significantly below the MBPO and VaGraM results in the Cartpole stabilization task. This is evidence that even with additional stabilization



Figure D.2: Comparison of VaGraM and MBPO on the Mujoco tasks presented in Janner et al. (2019). Lines mark average performance over 8 seeds and shaded areas mark the standard error of the mean. We see that VaGraM is able to perform on par with the MBPO implementation on the HalfCheetah and Walker tasks. It is unstable on the Ant task however.

mechanisms, the issues presented in Section 3.2 prevents the easy adoption of IterVAML to complex domains.

The value-weighting baseline is unable to achieve satisfactory performance even on the simple pendulum task, leading us to conclude that more complex reweighing schemes such as VaGraM are indeed necessary.

D.2.2 Performance in Mujoco benchmark suite

The results of VaGraM and MBPO are presented in Figure D.2. We find that VaGraM is able to perform on par with the MBPO baseline on two of the remaining tasks, while not performing well on Ant. Due to stability issues, the Humanoid-v2 task is excluded, no algorithm achieved satisfactory performance (compare Pineda et al. (2021) for a discussion).

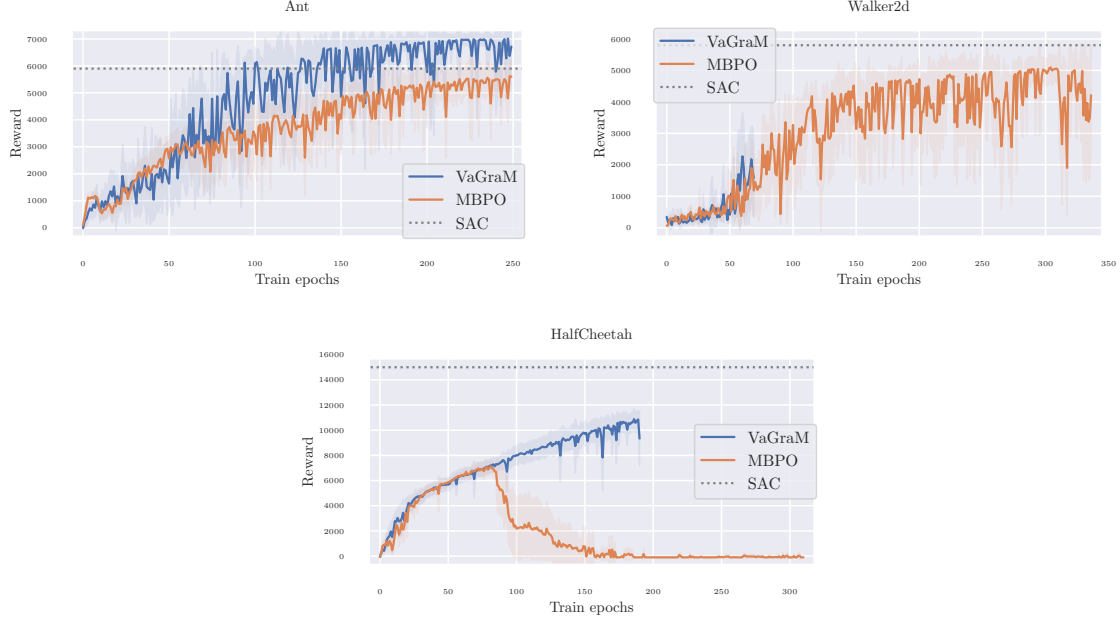


Figure D.3: Comparison of VaGraM and MBPO on the Mujoco tasks presented in Janner et al. (2019) using normalization (Bjorck et al., 2022). The instability on the Ant task is solved using the normalization, but the performance on HalfCheetah and Walker2d suffers.

On the Ant task, a manual inspection of the gradients of the value function reveals that they vary across different data points very strongly. We therefore hypothesize that the VaGraM objective can destabilize if confronted with very sharp value function gradients and requires additional regularization in challenging tasks. To test this hypothesis, we adapt the normalization method discussed in Chapter 3. The results are visualized in Figure D.3.

As is apparent from the results, the issue of destabilization on the Ant task is solved using the normalization, but it is not a panacea, as the performance on HalfCheetah suffers heavily. It is likely that the HalfCheetah environment is now over-regularized which would explain the lack of performance with MBPO as well. Overall these additional experiments suggest that there is a non-trivial relationship between the observation space, value function regularization and model-based RL.

D.3 Experiment details

As described in Chapter 5, we conducted our experiment in two domains from the OpenAI benchmark OpenAI Gym (Brockman et al., 2016), Pendulum-v2 and Hopper-v0. We used both environments as provided by the framework without further modification in our capacity tests. Our algorithm is shown in pseudocode in Algorithm D.1.

Expanded Hopper environment To test the performance of VaGraM in a setting with additional unnecessary state observations, we created a random dynamical system that evolves according to randomly initialized dynamics function, where A is a fixed matrix with entries randomly drawn from $\mathcal{N}(0, 10)$, and a fixed initial state s with components randomly drawn from $\mathcal{N}(0, 0.1)$. We do not resample the fixed components at reset.

The transitions are then described by the following deterministic function:

$$f(x_t, a_t) = \begin{cases} x_t + \sin(Ax) & \text{if } |f(x_t, a_t)| < 20 \\ x_0 & \text{else} \end{cases} \quad (\text{D.4})$$

This dynamics function contains two attributes which are hard for neural networks to model: discontinuity and non-linear dynamics. We find that this is sufficient to provides a very challenging environment for the MLE trained neural network models. To account for varying difficulty over different random initialization of the environments, we made sure to test the comparison runs with the same seeds over the different loss functions, but we did not find that the inter-seed variance was larger than the observed difference between the different loss functions.

D.3.1 Architecture and hyperparameters

For the Pendulum experiments, we use a simple fully connected neural network with a single layer, and a linear model without feature transformations as architectures. The used non-linearity is ReLU. All experiments were implemented in PyTorch (Paszke et al., 2019) and were not specified, the standard initialization of the library were kept. The exact versions of all used libraries are documented in the source code¹. A modern re-implementation using the jax library can be found under https://github.com/cvoelcker/jax_mbpo. REsults differ slightly from those presented here due to library and environment version differences.

To assure a fair comparison we used the hyperparameters provided by Janner et al. (2019) for all experiments with our approach and the NLL loss function used for the baseline. All models used were fully connected neural networks with SiLU non-linearities and standard initialization. For all experiments with full model size we followed MBPO and used seven ensemble members with four layers and 200 neurons per layer.

Even though our loss derivation does not support rolling out the model for more than a single step without accounting for this in the training setup, we find that VaGraM is still stable over the short rollout horizons used by MBPO. Therefore we used the rollout scheme from MBPO.

¹<https://github.com/pairlab/vagram/tree/main>

Algorithm D.1: Value-Gradient weighted Model learning (VaGraM)

```

1 Initialize policy  $\pi_\phi$ , value function  $v_\psi$ , model  $\hat{f}_\theta$ , environment dataset  $\mathcal{D}_{\text{env}}$ , model
  dataset  $\mathcal{D}_{\text{model}}$ ;
2 for  $N$  epochs do
3   while  $\hat{p}_\theta$  not converged do
4     Sample batch  $(x, a, r, s')$  from  $\mathcal{D}_{\text{env}}$ ;
5      $\mathcal{L}_{v_\psi} = \left( (x' - \hat{f}_\theta(x, a))^\top \text{diag} \left( \frac{d}{ds} v_\psi(x) |_{x'} \right)^2 (x' - \hat{f}_\theta(x, a)) \right)$ ;
6      $\theta \leftarrow \theta - \alpha \frac{d}{d\theta} \mathcal{L}_{v_\psi}$ ;
7     Train reward model
8   for  $E$  steps do
9     Take action in env according to  $\pi_\phi$ ; add to  $\mathcal{D}_{\text{env}}$ ;
10    for  $M$  model rollouts do
11      Sample  $s$  from  $\mathcal{D}_{\text{env}}$ ; sample  $a \sim \pi_\phi(s)$ ;
12       $s', r = \hat{f}_\theta(x, a)$ ;
13      Add  $(x, a, r, x')$  to  $\mathcal{D}_{\text{model}}$ 
14    for  $G$  policy gradient updates do
15      Sample batch  $(s, a, r, s')$  from  $\mathcal{D}_{\text{env}} \cup \mathcal{D}_{\text{model}}$ ;
16       $\psi \leftarrow \psi - \beta \frac{d}{d\psi} (v_\psi(x) - (r + \gamma v'(x'))^2)$ ;
17       $\phi \leftarrow \phi - \lambda \hat{\nabla}_\phi J(\pi_\phi, v_\psi, (x, a, r, x'))$ 

```

However, it was necessary to make a small alteration to the training setup: in the provided implementation, the value function is solely trained on model samples. Since our model is directly dependent on the value function, we need to break the inter-dependency between model and value function in the early training iterations. Hence, we used both real environment data and model data to train the value function, linearly increasing the amount of model samples from 0 to 95% of the SAC replay buffer over the first 40 epochs of training (40.000 real environment steps). We did not transition to fully using model data to ensure that real environment samples are still able to inform the value function learning. We found that this did not diminish the returns of MBPO compared to solely using model samples and so used this approach for both VaGraM and MLE.

To estimate the gradient of the value function, we used four empirical value functions, two direct estimates and two target value functions used in the SAC algorithm and summed the losses using each data tuple and value function gradient independently. Furthermore we calculated the L_2 norm of all value function gradients and clipped these at the 95-th percentile. We found that this was necessary since the empirical value function gradients can become very large, which in some cases leads to a destabilization of the gradient descent algorithm used to update the model. Even then, in the Ant environment this is insufficient to prevent destabilization.

D.4 Deterministic vs probabilistic models

Our derivation of VaGraM lead us to the conclusion that a deterministic model was sufficient to achieve the goal of value-aware model learning in environments with small transition noise. This insight stands in contrast to the current literature, which often claims that probabilistic models are needed to achieve optimal performance in model-based reinforcement learning. However, there is no clear consensus among different authors whether probabilistic models are needed or if a deterministic model can be sufficient for MBRL (compare Lutter et al. (2021)).

Our assumption that the model can be replaced with a deterministic one relies on the assumption that the underlying model is not dominated by stochastic transitions, but is deterministic or near deterministic and unimodal. This follows from the requirement that the model prediction admits a small error in the mean squared error sense, otherwise the Taylor approximation does not properly capture the behavior of the value function, as a model prediction might have high likelihood under the environment and still be far away from the environment sample in a mean squared sense otherwise. In domains where capturing the stochasticity is crucial, we have to revisit this requirement in follow up work.

Furthermore, we are operating under the assumption that the mean value function of each distribution over states can be represented as the value function of a single state prediction. Since value function approximations represented by neural networks are continuous, this is true in our setting due to the mean value theorem for integrals. Implicitly due to the Taylor approximation, we also assume that this mean value lies close to or on the environment sample we obtained. If other approximation schemes are used, this property needs to be checked and potentially probabilistic models are needed to represent the expectation over all possible value functions and transition dynamics.

D.4.1 Ablation experiment with deterministic models

In our experiments in the Hopper domain, we used probabilistic models following Janner et al. (2019). To verify that the improvement in performance did not result from using a deterministic model instead of a probabilistic model, we repeated the experiment showing the impact of smaller model sizes, and replaced the probabilistic Gaussian ensemble with an ensemble of deterministic functions trained with the mean squared error loss between sample and state prediction. All other implementation details, architecture and hyperparameters were kept fixed. The results are shown in Figure D.4. In this ablation, we do indeed see that a mean squared trained model capture the dynamics information better than a model that is trained using negative log likelihood. To the best of our knowledge, this phenomenon has not received attention in the literature, but we hypothesize that it is an artifact of training

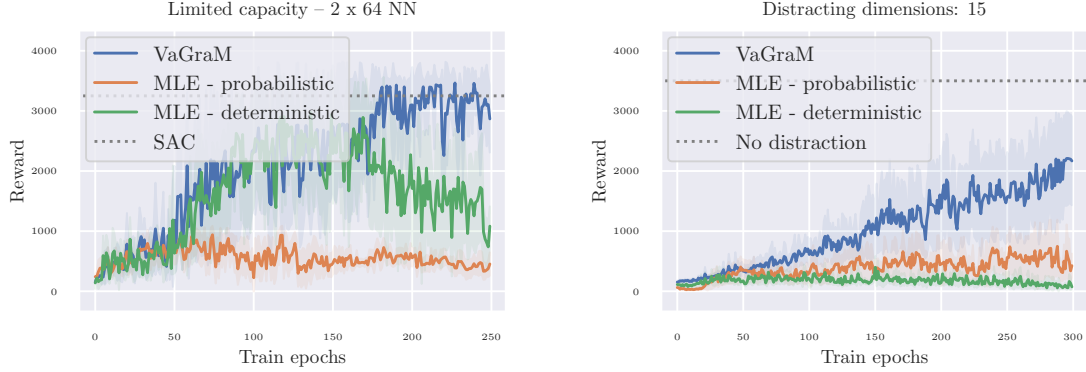


Figure D.4: **Comparison of the empirical performance of deterministic and probabilistic MLE models vs VaGraM.** Thick lines denote the mean and shaded area the standard error over 8 runs. In the limited capacity setting, the deterministic model is able to achieve significantly higher returns than the probabilistic baseline, however with stability issues during longer training. On the distracting benchmark, the deterministic model is not able to achieve any significant returns.

a probabilistic model with gradient descent instead of natural gradient descent (compare Figure 1 in Peters and Schaal (2008) for a visual intuition).

Nonetheless, VaGraM is still consistently able to achieve higher cumulative reward than the MSE model. Especially on the distraction task we find that a deterministic MSE model performs on par with the probabilistic model and fails to achieve any reward when faced with a challenging number of distractions. This validates our hypothesis that value awareness is important in settings with insufficient model capacity, but crucial in cases where the environment observations are not aligned with the control problem.

Appendix E

Appendix – Calibrated Models for VAML

E.1 Implementation Details – Section 6.5: Garnet

Each Garnet is defined by a randomly generated transition matrix and a random reward function. Across all experiments we use 50 dimensional state spaces. For each state x_i , we sample $k = 10$ successor states at random without replacement. Then we assign each successor state x_j a weight $\omega_{ij} \sim \mathcal{N}(0, 1)$ and set the weight of all non-successor states to the floating point minimum. The transition matrix is computed via the softmax function. The reward is sampled as $r(x_i) \sim \mathcal{N}(0, 1)$.

This setup allows us to vary the stochasticity of the environment naturally. The softmax temperature parameter τ controls the stochasticity. As $\tau \rightarrow \infty$, the problem becomes deterministic with $p(x_j|x_i) = 1$ for the state x_j with $\omega_{ij} = \arg \max_j \omega_{i,j}$. For $\tau \rightarrow 0$, the probabilities for all successor states become equal $p(x_j|x_i) = \frac{1}{k}$. In our experiments, we vary τ such that for almost all Garnets we obtain full determinism and equal transition probabilities, which empirically happens in the range $\tau \in [0.001, 10.0]$.

E.2 Implementation Details – Subsection 6.7.3: DM Control

E.2.1 Architecture

The original MuZero paper (Schrittwieser et al., 2020) provides large-scale experiments on the ALE benchmark (Bellemare et al., 2013). However, the original algorithm and the ALE suite are computationally extremely expensive to run. In addition, to the best of our knowledge, no reliable open-source replication of the closed-source MuZero and Stochastic

Hyperparameter	Value		
Batch size	128	Model rollout depth m	1
Discount γ	0.99	Model bootstrap depth b	Varied (0 and 1)
Actor learning rate α_π	0.0003	Model samples k	Varied (1 and 4)
Critic learning rate α_Q	0.0003	Proportion real ρ	0.9
Model learning rate $\alpha_{\hat{p}}$	0.0003	Latent dimension	512
Encoder learning rate α_ϕ	0.0001	Gradient clipping	10

Table E.1: Hyperparameter of the deep learning experiments. Aside from the (m, b) -(C)VAML relevant hyperparameters, we keep all others consistent across environments and algorithms.

MuZero models and algorithms exists. Instead, we use the TD-MPC architecture and algorithm (Hansen et al., 2022) as our main reference. This implementation uses the MuZero loss but is significantly cheaper and faster to run in a small-scale experimental setting, while still solving non-trivial control tasks.

We adopt the architecture and model-based MPC search procedure of Hansen et al. (2022) as our baseline model. In addition to the L_2 based deterministic latent setup, we also use a Gaussian latent model as our stochastic environment model. This implementation follows similar ideas used in observation-space models, such as Chua et al. (2018) and Janner et al. (2019). While the stochastic environment models used in Dreamer (Hafner et al., 2021) would have been an alternative, it has not shown strong performance in continuous control tasks and is significantly more expensive to run. In addition, Voelcker et al. (2024c) shows that observation-space reconstruction can be a suboptimal auxiliary task for aligning a latent space with value function prediction. Finally, we adopt the joint model-free and model-based training proposed in Voelcker et al. (2025a). For the MuZero setting ($b = 1$) we update the value function both for the state and the predicted latent state. In the IterVAML setting, there is no meaningful way to update the model’s predicted value function, and therefore we only update the current state’s value function.

Hyperparameters can be found in Table E.1. Our source code is available under <https://github.com/adaptive-agents-lab/CVAML>.

E.2.2 Environment

As several recent papers have shown, the vast majority of environments in the DM Control suite are effectively saturated. We, therefore, restrict our experiments to the most difficult domains, *humanoid* and *dog*, on which large performance differences still appear (Nauman et al., 2024b; Voelcker et al., 2025a; Fujimoto et al., 2025).

E.2.3 Additional results

In addition to the raw performance graphs in Subsection 6.7.3 we present the average model entropy and VAML loss curves in Figure E.1. As expected, we find that the CVAML loss leads to a higher entropy model compared to the VAML loss. This however does not lead to a noticeable difference in the VAML error itself.

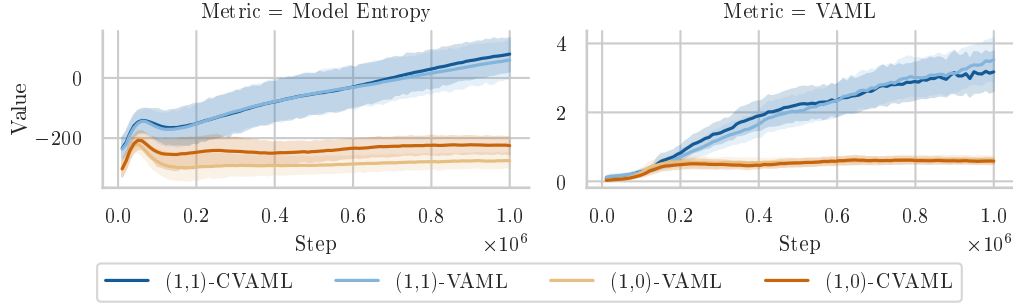


Figure E.1: Model entropy (left) and loss value for the VAML-losses (right) aggregated across all environments. The shaded area represents a 95% stratified bootstrap interval. We see that the model entropy of the corrected (1,0)-VAML differs significantly, as we expect. However, this does not translate to a pronounced difference in the VAML error itself. Therefore, while the calibration term does indeed lead to a higher variance, this does not directly translate into an improvement in performance. In (1,1)-VAML, the model entropy does not differ significantly, however, without calibration, the VAML error seems to be growing slightly stronger (although not outside of overlapping confidence intervals). Given the large deviation in performance we observe in the main results for (1,1)-VAML, we conjecture that correcting the value function learning term is significantly more important for performance.

Appendix F

Appendix – Value Function Learning with Model-Generated Data

F.1 Implementation

Our experiments are implemented in the jax library to allow for easy parallelization of multiple experiments across seeds. All networks follow the standard architecture from Hansen et al. (2024) with two changes: instead of using an ensemble of critics, we opt for a single double critic pair. We also do not use a stochastic policy, instead simply using a deterministic network with a tanh activation as used in Lillicrap et al. (2016) and Fujimoto et al. (2018b). Full hyperparameters are presented in Table F.2 and the architecture can be found in Table F.1. We use mish activation functions (Misra, 2020) and the adam optimizer to train our models (Kingma and Ba, 2015). The reference implementation is provided under `j`

Loss functions: As we use the HL-Gauss representation (Farebrother et al., 2024) for the critic, the loss is the cross-entropy between the estimated Q function’s categorical representation Q_{rep} and the bootstrapped TD estimate,

$$\mathcal{L}_Q = \sum_{i=1}^m \text{TD}(\hat{Q}_{\text{rep}})_i \log \hat{Q}_{\text{rep}_i} ,$$

where the indices i denote the positions of the categorical vector representation used by HL-Gauss. This is the same loss that is used for the two-hot encoding in Hansen et al. (2024),

Encoder Φ	Dense Layer Dense Layer	Mish Simmnorm(8)	in_size= $ \mathcal{X} $, out_size=512 out_size=512
Latent Model F	Dense Layer Dense Layer Dense Layer	Mish Mish Simmnorm(8)	in_size=512 + $ \mathcal{A} $, out_size=512 out_size=512 out_size=512
Q head $\hat{Q}$	Dense Layer Dense Layer Dense Layer	Mish Mish –	in_size=512 + $ \mathcal{A} $, out_size=512 out_size=512 out_size=1
Actor $\hat{\pi}$	Dense Layer Dense Layer Dense Layer	Mish Mish tanh	in_size=512, out_size=512 out_size=512 out_size= $ \mathcal{A} $

Table F.1: Network architecture for MAD-TD.

Parameter		Parameter	
Initial steps (Random policy)	5000	HL-Gauss vmax	$150 \cdot \text{AR}$
Batch size	512	HL-Gauss num bins	151
RL learning rate	0.0003	Model data proportion	0.95
Model learning rate	0.0003	Reset interval (where applicable)	200000
Encoder learning rate	0.0001	Model & encoder update ratio	1
Soft update τ	0.995	Actor & critic update ratio	varying
Discount factor γ	0.99	MPC number of samples	512
Model forward prediction steps	1 (4 for AR=1)	MPC iterations	6
Gradient clipping	10.0	MPC top k	64
HL-Gauss vmin	$-150 \cdot \text{AR}$	MPC temperature	0.5

Table F.2: Hyperparameters. We adapted three parameters to the action repeat = 1 setting, as the magnitude of the reward changes.

the only difference is the target encoding function. For more details, see Farebrother et al. (2024).

We use a latent encoder $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{Z}$ that maps into the simnorm space, the space of n k -dimensional simplices (Lavoie et al., 2023). Writing $\hat{p}$ for the learned world model and $\hat{r}, \hat{x}' \sim \hat{p}(|x, a)$ for reward and next latent-state samples, the loss for our model and encoder is

$$\mathcal{L}_{\text{model}}(x, a, r, x') = \mathcal{L}_{\text{rew}}(x, a, r) + \mathcal{L}_{\text{forward}}(x, a, x') + \mathcal{L}_Q(x, a, r, x') \quad (\text{F.1})$$

$$\mathcal{L}_{\text{rew}}(x, a, r) = (r - \hat{r})^2 \quad (\text{F.2})$$

$$\mathcal{L}_{\text{forward}} = - \sum_{i=1}^{n \cdot k} \phi(x')_i \log \hat{x}'_i, \quad (\text{F.3})$$

where the index i is again element-wise across the simplex representation used for the latent state. Note that we propagate the critic learning gradients into the encoder only for the real data and not the model generated one to prevent instability.

Baseline results We took available results from Nauman et al. (2024b) and Hansen et al. (2024) for all plots where possible, and used the official implementation of BRO to rerun the experiments without resetting and with differing action repeats. Other hyperparameters were left as-is.

F.2 Further results

F.2.1 Q value overestimation

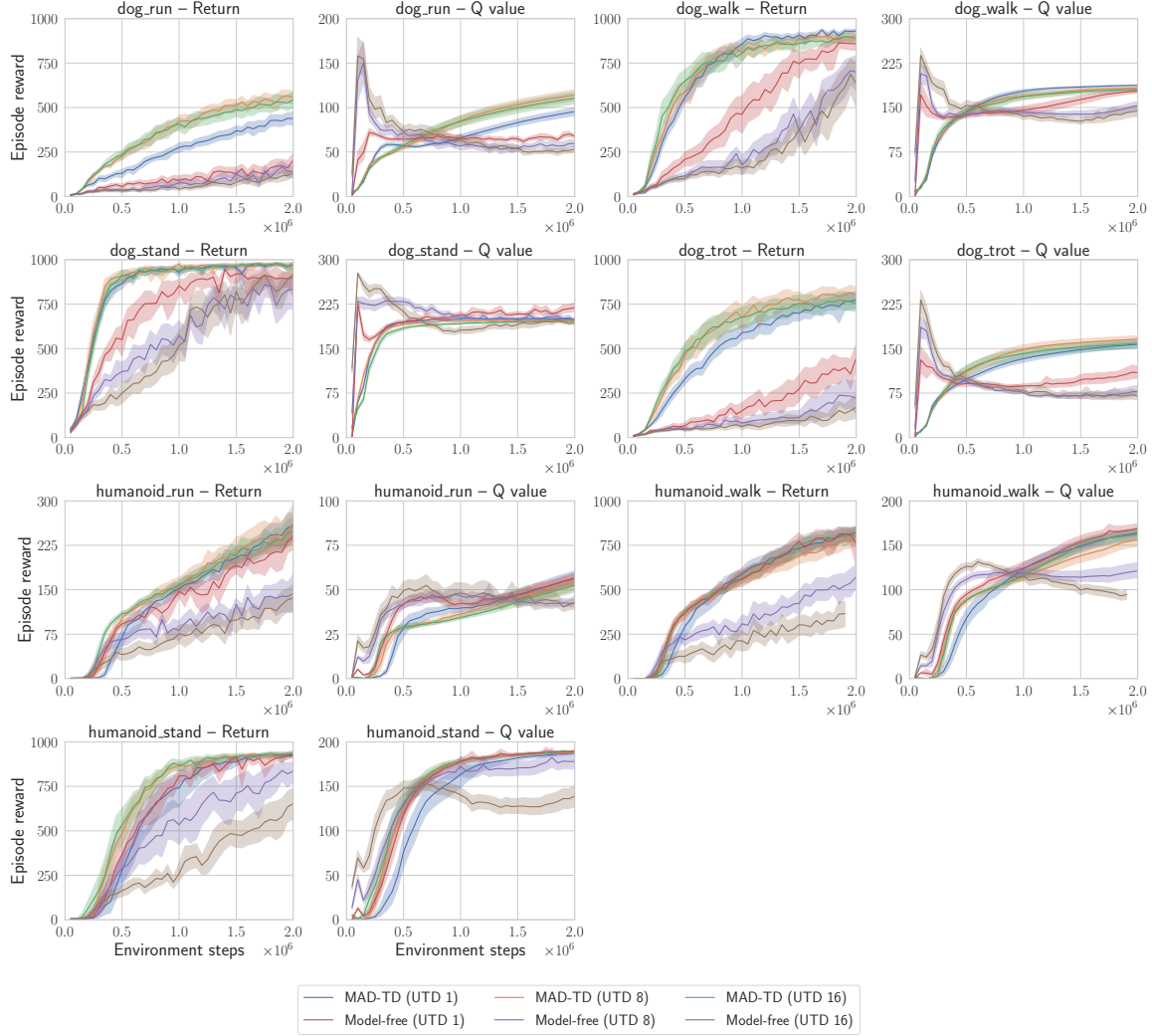


Figure F.1: Return curves and Q values with differing UTD values.

We plot the return curves and corresponding Q estimates for different UTD values and with and without model-generated data on the hard suite. The results are presented in Figure F.1. As we see, across all tasks the model free variant strongly overestimates the Q values, especially in the beginning.

F.2.2 Humanoid results

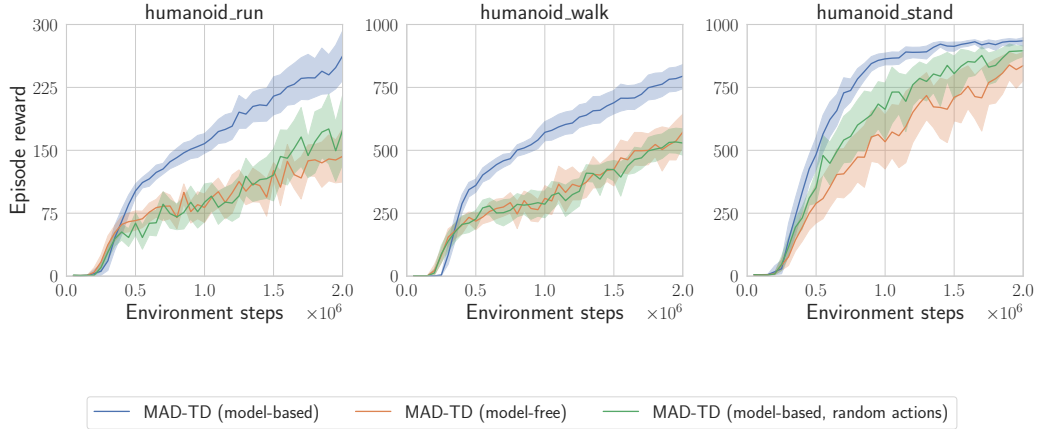


Figure F.2: Return curves for the humanoid tasks when using on-policy (blue), random (green) and no model-generated data (orange). The observed performance impacts are comparable to the dog case.

For several experiments, we only showed the dog results from the main suite to avoid cluttering the main body of the thesis. The corresponding humanoid results are presented in Figure F.1 and Figure F.2, corresponding to Figure 7.4 and Figure 7.10 respectively. As the plots highlight, the main insights transfer across the hard tasks.

F.2.3 TD-MPC2 ablation

As described in Chapter 7, we simplify the base model of TD-MPC2 to improve the computational efficiency of the algorithm. This is necessary to conduct high UTD experiments. Here, we present a direct comparison of the original TD-MPC2 model, and our adapted version (Figure F.3). We compare MAD-TD without any model generated data, at UTD 1, which corresponds to the standard setting of TD-MPC2. As pointed out, all of our changes to the base model boil down to setting different hyperparameters, such as the rollout length, to achieve faster learning.

We find that this does not significantly change the overall results achieved by the base model, and we are therefore confident to attribute performance gains to our presented method.

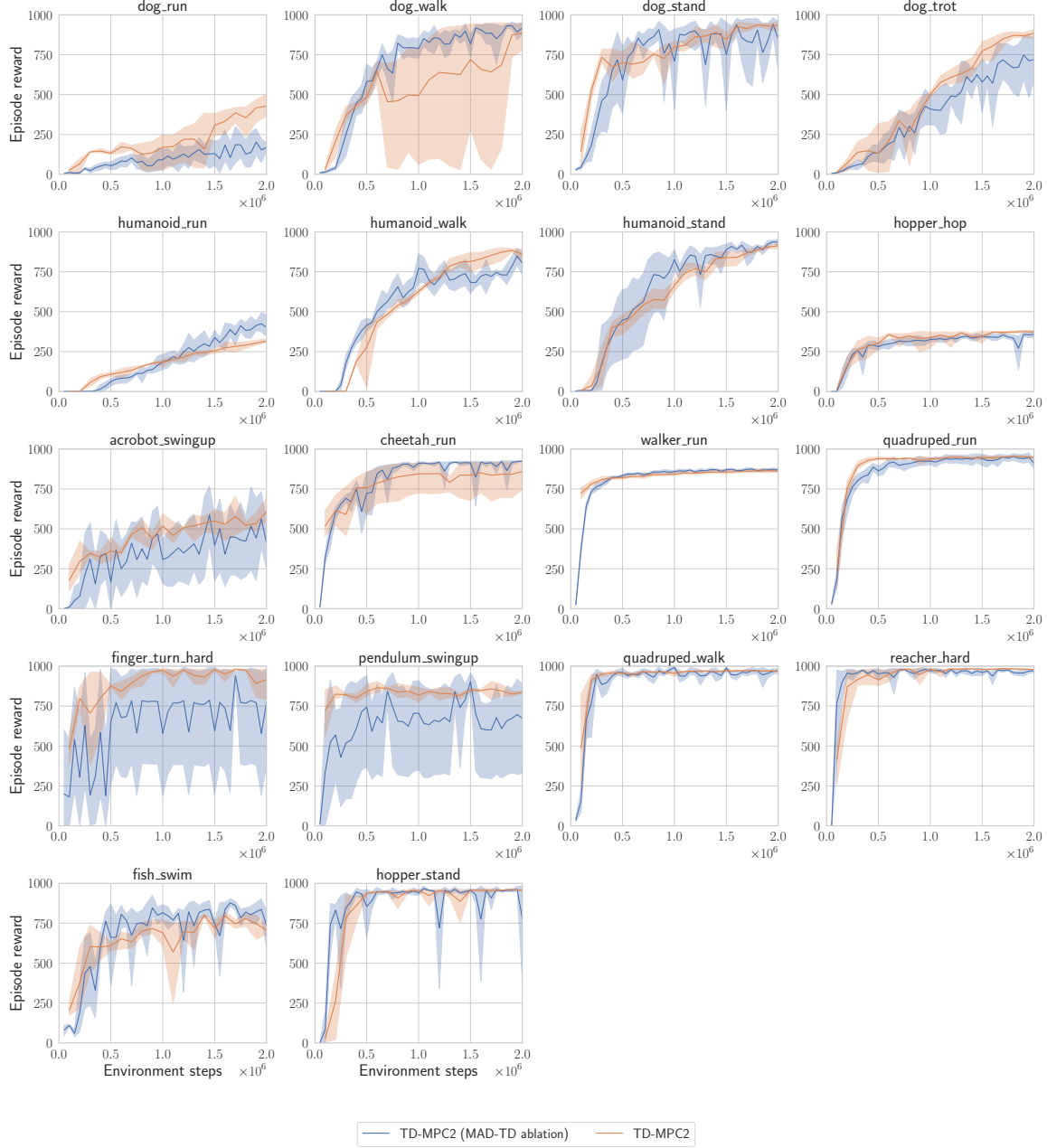


Figure F.3: Performance variation of the base MAD-TD model compared to TD-MPC2. Our changes only very few times lead to lower performance which is acceptable given the large reduction in computational cost.

F.2.4 MAD-TD, BRO, TD-MPC2 per env on the hard suite

We present the return curves for MAD-TD and the baselines per environment on the hard suite. Figure F.4 shows the results with action repeat 2 and Figure F.5 with action repeat 1. Perhaps surprisingly, the results of the algorithms are not fully consistent across this regime. Partially, this can be explained by the fact that our method and TD-MPC2 were

first developed in the regime of action repeat 2, while BRO was only evaluated in the action repeat 1 setting. This suggests that the performance of each method depends in a non-trivial fashion on hyperparameter tuning. Yet, across both action repeat setting MAD-TD outperforms BRO without resetting consistently and only under-performs any previous algorithm on the dog trot task in the action repeat 1 setting.

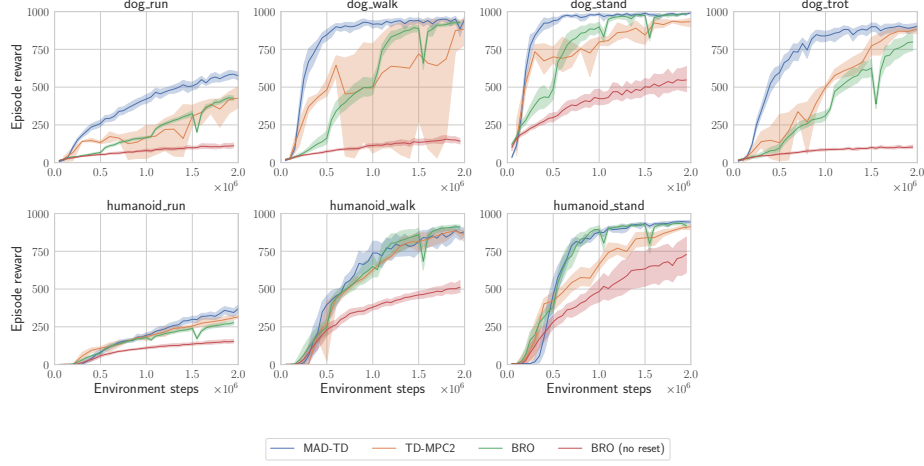


Figure F.4: Return curves with action repeat set to 2.

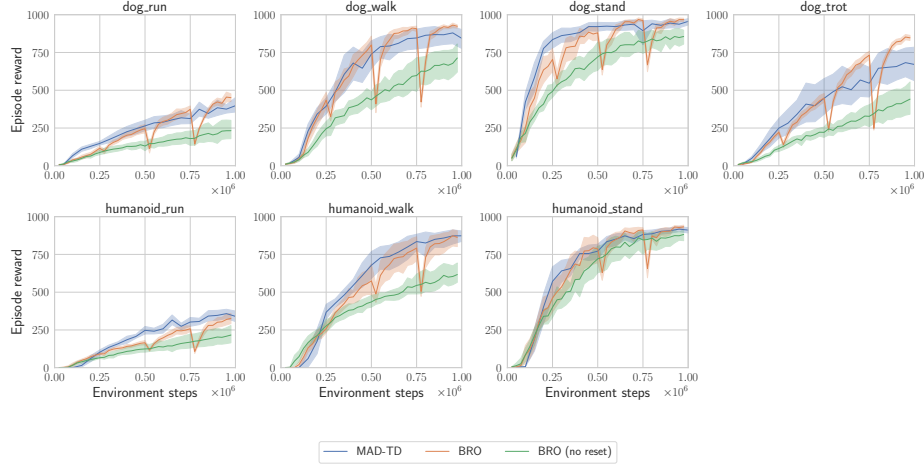


Figure F.5: Return curves with action repeat set to 1.

We conjecture that the remaining gap in performance seems to be most likely attributable to exploration and optimism. While we focus on learning accurate value functions, Bro contains several components which are specifically designed to improve exploration. Investigating the tension between exploration and accurate value function fitting is an important direction for future work.

Bro and TD-MPC2 are explicitly evaluated without their exploration bonuses in separate evaluation roll-outs. We however do not conduct such as separate evaluation as we do not add any additional exploration noise to our training. When plotting training performance, the gap between MAD-TD and Bro further closes, suggesting an important trade-off between test time and training performance.

F.2.5 Results across further DMC environments

We conducted more experiments on all DMC environments which were shown to benefit from the interventions in prior work (D’Oro et al., 2023; Nauman et al., 2024b). These are presented in [Figure F.7](#) and [Figure F.8](#).

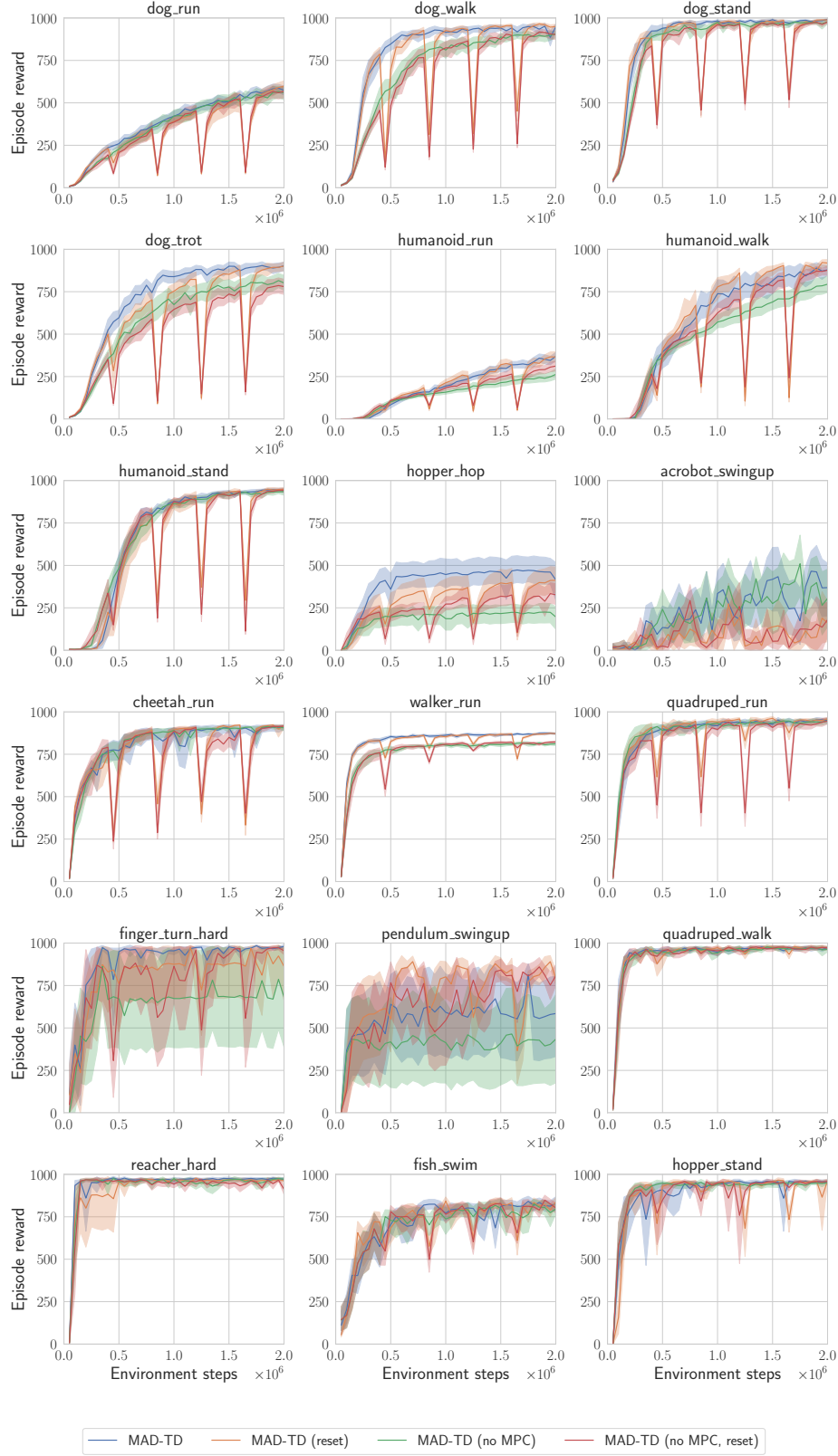


Figure F.7: Return curves for the impact of resetting on MAD-TD with and without MPC. Without MPC, resetting can still improve the performance, but with MPC, we see no significant benefits from resetting across environments except pendulum. The hopper results highlight the importance (and danger) of the reset interval, as seemingly the reset algorithm is not able to recover “in time” to improve performance.

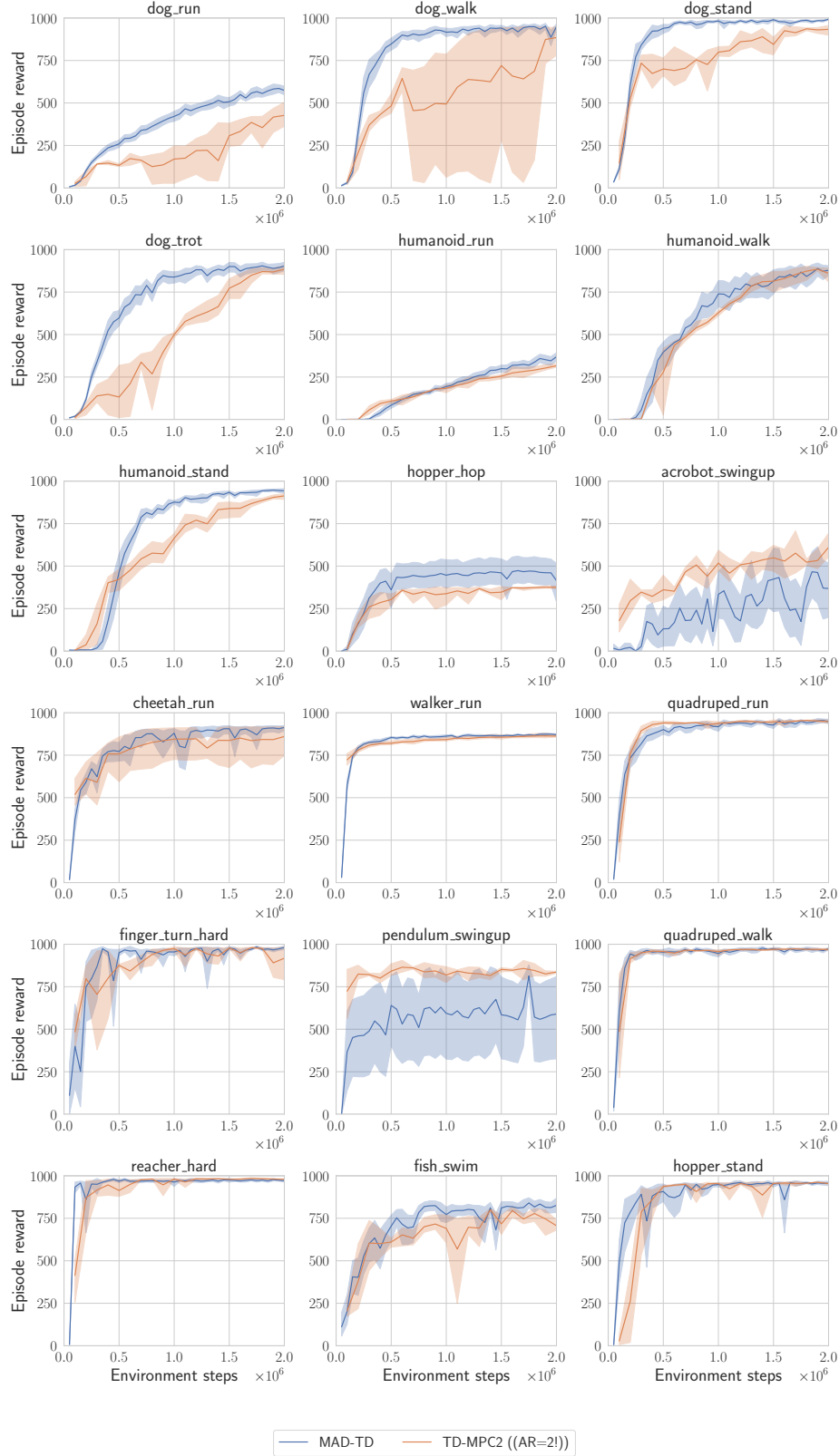


Figure F.8: Comparison of MAD-TD and TD-MPC2 across more environments of the DMC suite. We observe gains compared to TD-MPC2 in the hard tasks, especially in terms of early learning performance, while TD-MPC2 has advantages on the pendulum_swingup and acrobot_swingup tasks. These seem to be exploration and stability issues for which the longer model rollouts of TD-MPC2 seem to help. To prevent confusion, we highlight the choice of AR=2 for TD-MPC2.

F.2.6 Metaworld

To broaden the basis of comparison, we compare our method to BRO and TD-MPC2 on 9 selected environments from the metaworld suite. Results can be found in [Figure F.9](#).

Overall, we observe that our method performs strongly on tasks in which the agent has access to a dense reward, such as *lever-pull* and *button press*. MAD-TD demonstrates the ability to quickly and stably bootstrap reward when available. When exploration is a challenge, learning can take longer with MAD-TD. Strong exploration for high-UTD algorithms is not the focus of MAD-TD and remains an open problem (Hussing et al., 2024). This is consistent with our core hypothesis: high UTD learning benefits in cases where fitting a correct value function is challenging. In tasks such as *pick-place-wall* the core challenge is exploration, as the agent receives no reward signal for the majority of early training. We therefore cannot expect high UTD learning to improve the performance in these tasks.

As pointed out, BRO and to a lesser extent TD-MPC2 have the benefit of exploring with optimism bonuses and ensembled value functions. We removed these from our method to cleanly study the impact of model generated data. However, improvements to exploration are mostly orthogonal to our proposed method and can be freely combined in future work.

Finally, as also shown by Nauman et al. (2024b), there is a curious failure case of TD3 compared to SAC in the case of environments with sparse rewards. In the absence of the entropy penalty from the SAC loss function, the tanh policy of TD3 tends to saturate, which can stymie exploration completely. This is, to the best of our knowledge, not discussed in the literature, and should be investigated in future work.

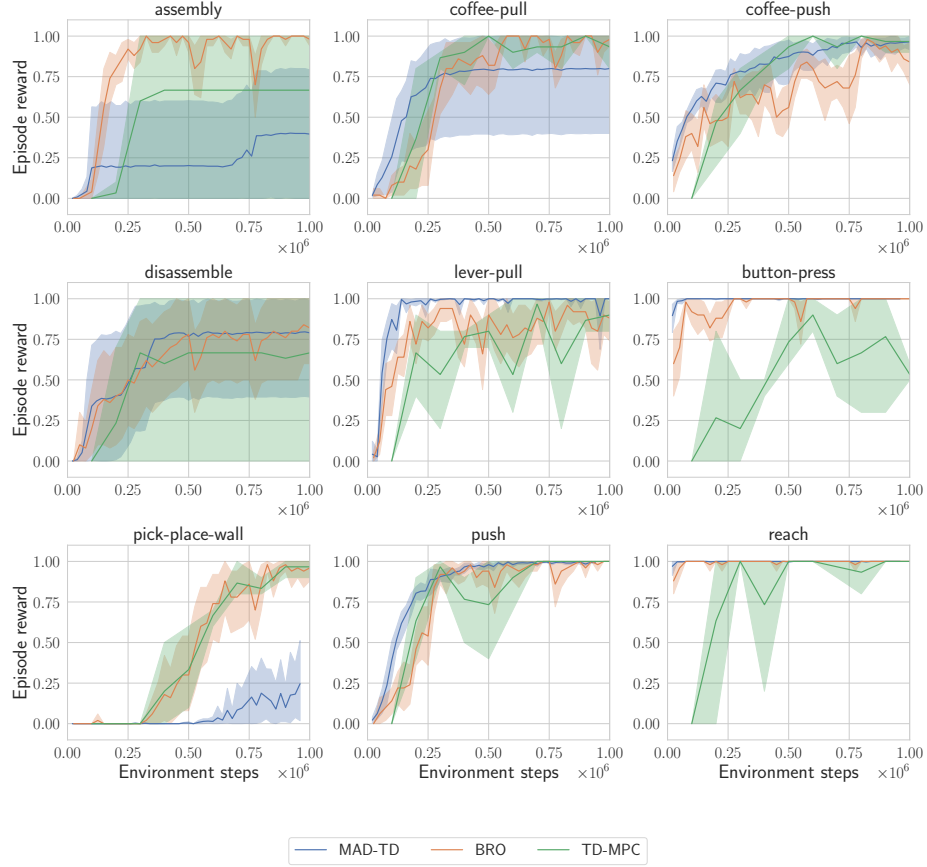


Figure F.9: Performance comparison on Metaworld between MAD-TD, BRO, and TD-MPC2. MAD-TD performs strongly on tasks which provide sufficient reward information to bootstrap the value function quickly, while learning more slowly on sparse reward tasks. This is consistent with the core goal of our algorithm, to stabilize and improve value function learning.

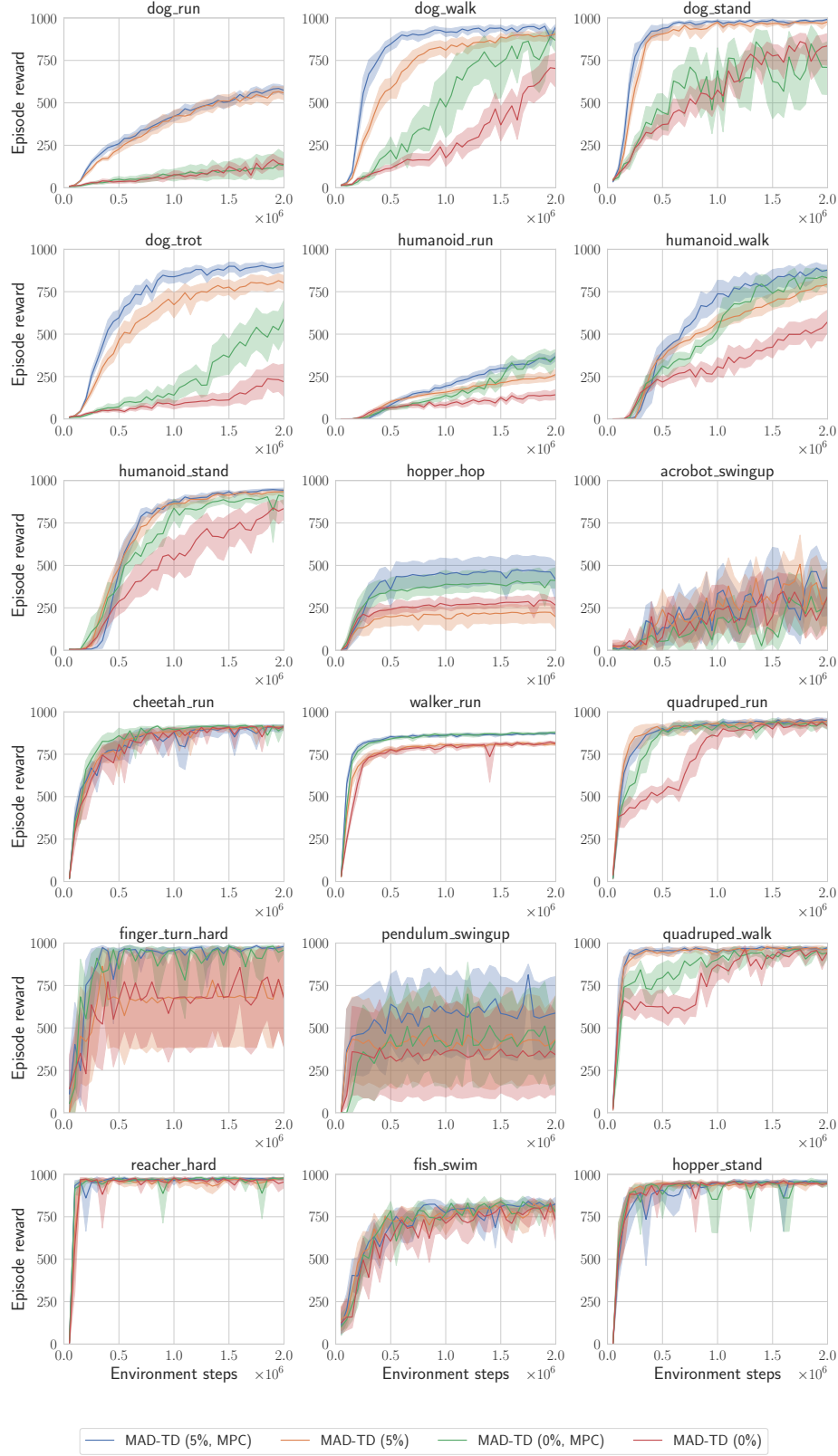


Figure F.6: Return curves evaluating the impact of model-based data for critic learning and MPC. Overall, MPC and model-based critic learning both stabilize the learning process, as conjectured.